Ising_Model_Metropolis_and_Wolff_Monte_Carlo

October 4, 2026

1 Ising Model Main Jupyter Notebook

1.1 This notebook is divided up into several sections, and the sections are designed to investigate the following:

1. Validate the 2D 2-spin ising model using parallelised simulation runs whilst changing the temperature to calculate the onsager critical temperature, and calculate the onsager critical exponents using the extracted simulation data, using both the Metropolis and Wolff Algorithms.
2. Use parallelised simulation runs again whilst changing the lattice size to calculate the value of C_0, the onsager critical amplitude.
3. Visualise the effects of changing the initial conditions of a generic 64x64 Ising lattice using both the Metropolis and Wolff Algorithms. Please note: The Wolff Algorithm was not designed to account for magnetic fields and hence the effects of changing the magnetic induction field are only investigated using the Metropolis Algorithm.

2 Main imports

[1]:
```
### Imports ###

# Calculations
import numpy as np

# Constants
from scipy.constants import mu_0

# Plotting and data fitting
import matplotlib.pyplot as plt
from matplotlib import colors
from matplotlib.ticker import MultipleLocator, AutoMinorLocator
from scipy import stats

# File reading and config
import os
from tqdm.auto import tqdm
```

```
# Multiprocessing
from multiprocessing import Pool
```

3 Sanity check: checking that all CPU cores are available for multiprocessing - Checks that os works properly

[2]:
```
# Defining a global CPU core count
n_cores = os.cpu_count()

if type(n_cores) != int: # Must have an integer number of cores in any CPU

    raise ValueError('Critical error with python environment: os package has␣
 ↪not registered all the CPU cores, please check os is working correctly.')

else:
    print('All CPU cores detected successfully.')
    print(f'Number of available CPU cores for multiprocessing is {n_cores}.')
```

```
All CPU cores detected successfully.
Number of available CPU cores for multiprocessing is 24.
```

4 Sanity check - Check that the helper multiprocessing files are in place

[3]:
```
# Checking that all the parallelisable worker files exist
if os.path.exists('parallelised_lattice_side_length_investigation_functions_MMC.
 ↪py') != True or os.path.
 ↪exists('parallelised_temperature_investigation_functions_MMC.py') != True or␣
 ↪os.path.exists('parallelised_temperature_investigation_functions_WMC.py') !=␣
 ↪True or os.path.
 ↪exists('parallelised_lattice_side_length_investigation_functions_WMC.py') !=␣
 ↪True:

    raise ValueError('Critical error with multiprocessing files: The files␣
 ↪required for multiprocessing are either damaged or not in the same directory␣
 ↪as this code.\nPlease ensure␣
 ↪parallelised_lattice_side_length_investigation_functions_MMC.py and␣
 ↪parallelised_temperature_investigation_functions_MMC.py are both in the same␣
 ↪directory as this notebook before restarting the notebook kernel and␣
 ↪re-running.')

else:
    print('All multiprocessing files successfully detected. Importing␣
 ↪parallelisable functions and proceeding with runs.')
```

```
    # Importing the metropolis algorithm parallelisation functions
    from parallelised_temperature_investigation_functions_MMC import␣
↪run_MMC_temp_investigation_worker
    from parallelised_lattice_side_length_investigation_functions_MMC import␣
↪run_MMC_lattice_side_length_investigation_worker

    # Importing the wolff algorithm parallelisation functions
    from parallelised_lattice_side_length_investigation_functions_WMC import␣
↪run_WMC_lattice_side_length_investigation_worker
    from parallelised_temperature_investigation_functions_WMC import␣
↪run_WMC_temp_investigation_worker
```

```
All multiprocessing files successfully detected. Importing parallelisable
functions and proceeding with runs.
```

5 Function to read in parameter file for single ising model simulation (either wolff or metropolis)

[4]:
```
def read_2D_ising_parameter_file(parameter_file):

    """Reads in the parameter file after checking it is there, and returns the␣
↪parameters in the following formats:

    filename = string
    Lattice side length = int
    Magnetic field strength = float
    temperature = float
    Coupling constant J = float
    k_b (boltzmann constant) = float
    u_b (bohr magneton) = float
    number of steps = int

    This function is borrowed from the previous verlet integrator code, which␣
↪also reads parameter files using the exact same method."""

    # Checking if the file is in the same directory as this notebook
    if os.path.exists(parameter_file) is False:

        raise ValueError('File is not in the same directory as this notebook.␣
↪Please move the file to the same directory as the notebook and re-run.')

    # Empty dictionary to store the parameters
    parameter_storage_dict = {}

    with open(parameter_file, 'r') as f:
```

```
        for txtline in f:

            # Skipping empty lines and lines starting with a hashtag
            txtline = txtline.strip() # Removing spaces - Makes things easier
            if not txtline or txtline.startswith('#'):

                continue

            # Detecting assigned parameters in the file
            if '=' in txtline:
                key, parameter = txtline.split('=', 1) # Split the line based␣
↪on the first occurrence of = character
                key = key.strip()
                value = parameter.strip()

                # Assigning the parameter
                parameter_storage_dict[key] = parameter

    # Sanity checks for dictionary
    for key, parameter in zip(parameter_storage_dict.keys(),␣
↪parameter_storage_dict.values()):

        if key == None or parameter == None:
            raise ValueError('Parameter file is faulty: Conatains variables␣
↪that point to no values or values with no assigned variable.')

        else:
            continue

    # Extracting parameters to return if the file contains no unassigned␣
↪variables
    # Extracting filename
    filename = parameter_storage_dict.get('filename')
    filename = filename.strip() # Important to remove spaces - OneDrive has␣
↪backup issues if the filename has spaces.

    # Extracting simulation numerical parameters
    lattice_side_length = int(parameter_storage_dict['side_length'])
    magnetic_field_B = float(parameter_storage_dict['magnetic_field_B'])
    temperature = float(parameter_storage_dict['temperature'])
    exchange_constant_J = float(parameter_storage_dict['exchange_constant_J'])
    k_b = float(parameter_storage_dict['k_b'])
    u_b = float(parameter_storage_dict['u_b'])
    nsteps = int(parameter_storage_dict['nsteps'])

    # Returning parameters in the same sequence as the run_integrator function␣
↪takes them
```

```
    return lattice_side_length, magnetic_field_B, temperature,␣
↪exchange_constant_J, k_b, u_b, nsteps, filename
```

6 Function to read in parameter file for parallelised critical temperature investigation simulation (either wolff or metropolis)

[5]:
```
def read_tcrit_investigative_file(parameter_file):

    """Reads in the parameter file after checking it is there, and returns the␣
↪parameters in the following formats:

    Standard parameters:

    filename = string
    Lattice side length = int
    Magnetic field strength = float
    temperature = float
    Coupling constant J = float
    k_b (boltzmann constant) = float
    u_b (bohr magneton) = float
    number of steps per run = int

    Investigative parameters for tcrit:

    temperature step between runs = float
    number of runs = int

    This function is borrowed from the previous verlet integrator code, which␣
↪also reads parameter files using the exact same method. """

    # Checking if the file is in the same directory as this notebook
    if os.path.exists(parameter_file) is False:

        raise ValueError('File is not in the same directory as this notebook.␣
↪Please move the file to the same directory as the notebook and re-run.')

    # Empty dictionary to store the parameters
    parameter_storage_dict = {}

    with open(parameter_file, 'r') as f:

        for txtline in f:

            # Skipping empty lines and lines starting with a hashtag
            txtline = txtline.strip() # Removing spaces - Makes things easier
            if not txtline or txtline.startswith('#'):
```

```
                continue

            # Detecting assigned parameters in the file
            if '=' in txtline:
                key, parameter = txtline.split('=', 1) # Split the line based␣
↪on the first occurrence of = character
                key = key.strip()
                value = parameter.strip()

                # Assigning the parameter
                parameter_storage_dict[key] = parameter

    # Sanity checks for dictionary
    for key, parameter in zip(parameter_storage_dict.keys(),␣
↪parameter_storage_dict.values()):

        if key == None or parameter == None:
            raise ValueError('Parameter file is faulty: Conatains variables␣
↪that point to no values or values with no assigned variable.')

        else:
            continue

    # Extracting parameters to return if the file contains no unassigned␣
↪variables
    # Extracting filename
    filename = parameter_storage_dict.get('filename')
    filename = filename.strip() # Important to remove spaces - OneDrive has␣
↪backup issues if the filename has spaces.

    # Extracting simulation numerical parameters
    lattice_side_length = int(parameter_storage_dict['side_length'])
    magnetic_field_B = float(parameter_storage_dict['magnetic_field_B'])
    initial_temperature = float(parameter_storage_dict['initial_temperature'])
    temp_step = float(parameter_storage_dict['temp_step'])
    exchange_constant_J = float(parameter_storage_dict['exchange_constant_J'])
    k_b = float(parameter_storage_dict['k_b'])
    u_b = float(parameter_storage_dict['u_b'])
    nsteps = int(parameter_storage_dict['nsteps'])
    nruns = int(parameter_storage_dict['nruns'])

    # Returning parameters in the same sequence as the run_integrator function␣
↪takes them
    return lattice_side_length, magnetic_field_B, initial_temperature,␣
↪temp_step, exchange_constant_J, k_b, u_b, nsteps, nruns, filename
```

7 Function to read in parameter file for parallelised lattice side length change simulation (either wolff or metropolis)

[6]:
```
def read_lattice_side_length_investigative_file(parameter_file):

    """Reads in the parameter file after checking it is there, and returns the␣
↪parameters in the following formats:

    Standard parameters:

    filename = string
    Lattice side length = int
    Magnetic field strength = float
    temperature = float
    Coupling constant J = float
    k_b (boltzmann constant) = float
    u_b (bohr magneton) = float
    number of steps per run = int

    Investigative parameters for investigating the change of lattice size:

    side length step between runs = int
    number of runs = int

    This function is borrowed from the previous verlet integrator code, which␣
↪also reads parameter files using the exact same method """

    # Checking if the file is in the same directory as this notebook
    if os.path.exists(parameter_file) is False:

        raise ValueError('File is not in the same directory as this notebook.␣
↪Please move the file to the same directory as the notebook and re-run.')

    # Empty dictionary to store the parameters
    parameter_storage_dict = {}

    with open(parameter_file, 'r') as f:

        for txtline in f:

            # Skipping empty lines and lines starting with a hashtag
            txtline = txtline.strip() # Removing spaces - Makes things easier
            if not txtline or txtline.startswith('#'):

                continue
```

```
            # Detecting assigned parameters in the file
            if '=' in txtline:
                key, parameter = txtline.split('=', 1) # Split the line based␣
↪on the first occurrence of = character
                key = key.strip()
                value = parameter.strip()

                # Assigning the parameter
                parameter_storage_dict[key] = parameter

    # Sanity checks for dictionary
    for key, parameter in zip(parameter_storage_dict.keys(),␣
↪parameter_storage_dict.values()):

        if key == None or parameter == None:
            raise ValueError('Parameter file is faulty: Conatains variables␣
↪that point to no values or values with no assigned variable.')

        else:
            continue

    # Extracting parameters to return if the file contains no unassigned␣
↪variables
    # Extracting filename
    filename = parameter_storage_dict.get('filename')
    filename = filename.strip() # Important to remove spaces - OneDrive has␣
↪backup issues if the filename has spaces.

    # Extracting simulation numerical parameters
    initial_lattice_side_length =␣
↪int(parameter_storage_dict['initial_side_length'])
    magnetic_field_B = float(parameter_storage_dict['magnetic_field_B'])
    temperature = float(parameter_storage_dict['temperature'])
    lattice_side_length_step =␣
↪int(parameter_storage_dict['lattice_side_length_step']) # Forcing lattice␣
↪side length step to be an int
    exchange_constant_J = float(parameter_storage_dict['exchange_constant_J'])
    k_b = float(parameter_storage_dict['k_b'])
    u_b = float(parameter_storage_dict['u_b'])
    nsteps = int(parameter_storage_dict['nsteps'])
    nruns = int(parameter_storage_dict['nruns'])

    # Returning parameters in the same sequence as the run_integrator function␣
↪takes them
```

```
    return initial_lattice_side_length, magnetic_field_B, temperature,␣
↪lattice_side_length_step, exchange_constant_J, k_b, u_b, nsteps, nruns,␣
↪filename
```

8 Function to initialise a random 2D lattice. All the ising model simulations start from a random system

[7]:
```
def make_random_2D_lattice(lattice_side_length, generator):

    """Produces a randomly generated 2D lattice of +1 or -1 spins using a␣
↪specific random seed and numpy's random number generator."""

    # PCG64DXSM is the most random number generator available in numpy.
    # True randomness is achieved by passing no seed - Seed is therefore␣
↪sourced from the operating system

    random_2D_lattice = generator.choice([1, -1], size=(lattice_side_length,␣
↪lattice_side_length))

    return random_2D_lattice
```

9 Function to compute system energy change when a spin is flipped - Needed for metropolis algorithm to work. This function is only ever used for the metropolis algorithm function.

[8]:
```
def calculate_energy_MMC(lattice_side_length, random_2D_lattice,␣
↪magnetic_field_B, temperature, exchange_constant_J, u_b, x_coord, y_coord):

    """Function for Metropolis Monte Carlo Algorithm:

    Uses a nearest neighbour approach to calculate the energy change when a␣
↪random spin is flipped.

    DOES NOT SUM OVER THE ENTIRE LATTICE TO CALCULATE ENERGY - NEAREST␣
↪NEIGHBOURS ONLY.

    Requires the lattice, magnetic field, temperature, exchange constant and␣
↪two randomly generated coordinates, generated by the MMC function
    (Metropolis Monte Carlo).

    Periodic boundary conditions are enforced to keep the model physically␣
↪accurate."""
```

```
    ## Ising model hamiltonian for 1 spin and its nearest neighbours using␣
↪periodic boundary conditions ##

    # Nearest neighbours term of the hamiltonian
    nearest_neighbours_term = (-exchange_constant_J) *␣
↪random_2D_lattice[x_coord, y_coord] * np.sum([random_2D_lattice[x_coord,␣
↪(y_coord-1)%lattice_side_length], random_2D_lattice[x_coord,␣
↪(y_coord+1)%lattice_side_length],␣
↪random_2D_lattice[(x_coord-1)%lattice_side_length, y_coord],␣
↪random_2D_lattice[(x_coord+1)%lattice_side_length, y_coord]])

    # Magnetic term of the hamiltonian
    magnetic_term = u_b * random_2D_lattice[x_coord, y_coord] * magnetic_field_B

    # The energy is given by the subtraction of the magnetic term from the␣
↪nearest neighbours term
    spin_flip_energy = nearest_neighbours_term - magnetic_term

    return spin_flip_energy
```

10 Function for performing a single metropolis step

[9]:
```
def MMC_step(lattice_side_length, random_2D_lattice, magnetic_field_B,␣
↪temperature, k_b, u_b, exchange_constant_J, generator):

    """Performs a monte-carlo step - Calculates energies before and after the␣
↪spin change and compares a randomly generated number to
    the maxwell boltzmann distribution"""

    # PCG64DXSM is the most random number generator available in numpy.
    # True randomness is achieved by passing no seed - Seed is therefore␣
↪sourced from the operating system.

    # Set of random coordinates
    x_coord = generator.integers(0, lattice_side_length)
    y_coord = generator.integers(0, lattice_side_length)

    # Calculating initial_energy
    old_energy = calculate_energy_MMC(lattice_side_length, random_2D_lattice,␣
↪magnetic_field_B, temperature, exchange_constant_J, u_b, x_coord, y_coord)

    # Selecting a spin and flipping it
    random_2D_lattice[x_coord, y_coord] *= -1

    # Calculating new energy
```

```
    new_energy = calculate_energy_MMC(lattice_side_length, random_2D_lattice,␣
↪magnetic_field_B, temperature, exchange_constant_J, u_b, x_coord, y_coord)

    # Calculating energy change
    energy_change = new_energy - old_energy

    if energy_change > 0 and np.exp(-energy_change / (k_b * temperature)) <␣
↪generator.random():

        # Undoing the flip if it isnt probable enough to happen
        random_2D_lattice[x_coord, y_coord] *= -1

    else:
        pass # Keep the spin flipped if the condition is not met

    return random_2D_lattice
```

11 Function to compute a Wolff Monte Carlo Step

[10]:
```
def WMC_step(lattice_side_length, random_2D_lattice, magnetic_field_B,␣
↪temperature, k_b, u_b, exchange_constant_J, generator):

    """Performs a Wolff monte carlo step as seen on page 5 of G. T. Barkema and␣
↪M. E. J. Newman 1997.

    Grows a cluster within that lattice, and then decides whether to flip an␣
↪entire cluster or not rather than a singluar spin.

    Copy-pasted text from page 5:

    1) Choose a seed spin at random from the lattice.

    2) Look in turn at each of the neighbours of that spin. If they are pointing
    in the same direction as the seed spin, add them to the cluster with
    probability Padd = 1- e-2 J.

    3) For each spin that was added in the last step, examine each of its neigh-
    bours to find the ones which are pointing in the same direction and add
    each of them to the cluster with the same probability Padd. (Notice that
    as the cluster becomes larger we may find that some of the neighbours
    are already members of the cluster, in which case obviously you don't
    have to consider adding them again. Also, some of the spins may have
    been considered for addition before, as neighbours of other spins in the
    cluster, but rejected. In this case, they get another chance to be added to
    the cluster on this step.) This step is repeated as many times as necessary
    until there are no spins left in the cluster whose neighbours have not been
```

```
    considered for inclusion in the cluster.

    4) Flip all the spins in the cluster."""

    ## Cluster-growth probability from equation 5 in the 1997 paper ##

    # Probability a spin is added to the cluster as it grows in the wolff␣
↪algorithm
    # Note: Magnetic field isnt included in either of these terms because the␣
↪standard wolff algorithm cant account for B-fields
    beta = 1/(k_b*temperature)
    p_add = 1 - np.exp(-2*beta*exchange_constant_J)

    ## Choosing a random spin, re-used code from the MMC_step ##

    # Set of random coordinates
    x_coord = generator.integers(0, lattice_side_length)
    y_coord = generator.integers(0, lattice_side_length)

    # Selecting the spin
    cluster_origin_spin = random_2D_lattice[x_coord, y_coord]

    ## Making the cluster ##

    # Cluster cannot contain duplicates, so a set is used (the set datatype␣
↪CANNOT contain duplicates).
    cluster = set()

    # Adding the initial spin to the cluster
    cluster.add((x_coord, y_coord))

    # Making a list of spins that are to be compared in the cluster to see if␣
↪new spins can be added to it.
    # Spins are checked later to see if they can be added based on the␣
↪selection probability p_add
    cluster_neighbour_comparison_spins_list = [(x_coord, y_coord)]

    # Nearest neighbour boundary conditions
    nearest_neighbours_conditions = [(1, 0), (-1, 0), (0, 1), (0, -1)]

    ## Cluster growth loop ##

    # When this list empties itself at the end of the loop, the cluster is␣
↪fully grown
    while cluster_neighbour_comparison_spins_list:
```

```
        # Removing the last element off the comparison list, so that its␣
↪neighbours can be checked.
        initial_x_coord, initial_y_coord =␣
↪cluster_neighbour_comparison_spins_list.pop()

        # Growing the cluster (adding spins to it based on the positions and␣
↪spin orientation)
        for x_coord_add, y_coord_add in nearest_neighbours_conditions:

            # Coordinates to add to the cluster
            n_x_coord_to_add = (initial_x_coord + x_coord_add) %␣
↪lattice_side_length
            n_y_coord_to_add = (initial_y_coord + y_coord_add) %␣
↪lattice_side_length

            # Growing the cluster based on neighbours if the spins are:
            # 1: Neighbours are not already in the cluster
            # 2: Only has the same z axis orientation as the original
            if (n_x_coord_to_add, n_y_coord_to_add) not in cluster:

                # Only adding spins to the cluster if they are pointing in the␣
↪same direction (so mathematically equal to the seed spin).
                if random_2D_lattice[n_x_coord_to_add, n_y_coord_to_add] ==␣
↪cluster_origin_spin:

                    # Add to cluster with the previously calculated probability␣
↪p_add
                    if generator.random() < p_add:

                        # Adding non-duplicate elements to the cluster
                        cluster.add((n_x_coord_to_add, n_y_coord_to_add))

                        # Adding to neighbour comparison list
                        cluster_neighbour_comparison_spins_list.
↪append((n_x_coord_to_add, n_y_coord_to_add))

    ## Flipping the entire cluster ##

    for x_coord_to_flip, y_coord_to_flip in cluster:

        random_2D_lattice[x_coord_to_flip, y_coord_to_flip] *= -1

    return random_2D_lattice
```

12 Function to calculate total lattice energy

[11]:
```
def calculate_lattice_energy_per_spin(lattice, lattice_side_length,␣
↪exchange_constant_J, magnetic_field_B, u_b):

    """Calculates the total energy of the entire lattice using the same formula␣
↪as the energy used to calculate a spin flip, just
    applied to the entire lattice than a specific set of spin coordinate"""

    # Uses np.roll to get periodic summation - Vectorised method of calculating␣
↪the energy which is much faster than nested for loops
    periodic_boundary_sum = np.roll(lattice, 1, axis=0) + np.roll(lattice, -1,␣
↪axis=0) + np.roll(lattice, 1, axis=1) + np.roll(lattice, -1, axis=1)

    # Nearest neighbour term in the hamiltonian
    nearest_neighbour_term = (-exchange_constant_J/2)*np.
↪sum(lattice*periodic_boundary_sum)

    # Magnetic term in the hamiltonian
    magnetic_term = u_b*np.sum(magnetic_field_B*lattice)

    # Total lattice energy is given by the subtraction of the magnetic term␣
↪from the hamiltonian term
    total_lattice_energy = nearest_neighbour_term - magnetic_term

    # Calculating the energy per spin
    energy_per_spin = total_lattice_energy / (lattice_side_length**2)

    return energy_per_spin
```

13 Function to calculate total lattice magnetisation

[12]:
```
def calculate_magnetisation_per_spin(lattice, lattice_side_length):

    """Calculates the lattice magnetisation for a single 2D lattice."""

    # Using formula from the instructions to calculate the lattice magnetisation
    lattice_magnetisation_per_spin = (1/lattice_side_length**2)*np.sum(lattice)

    return lattice_magnetisation_per_spin
```

14 Function to calculate total lattice internal magnetic field

[13]:
```
def calculate_internal_magnetic_field_per_spin(lattice, lattice_side_length,␣
↪magnetic_field_B):

    """Calculates the internal magentic field strength of a single 2D lattice.␣
↪Calls the lattice magnetisation function
    in order to get the lattice magnetisation."""

    # Extracting the magnetisation using the function to calculate it
    lattice_magnetisation_per_spin = calculate_magnetisation_per_spin(lattice,␣
↪lattice_side_length)

    # Calculating the internal magnetic field using the magnetisation, bohr␣
↪magneton and lattice magnetisation
    lattice_internal_magnetic_field_per_spin = (1/
↪mu_0)*(magnetic_field_B-lattice_magnetisation_per_spin)

    return lattice_internal_magnetic_field_per_spin
```

15 Function to calculate linear regression fit - Needed for straight line diagnostic plots

[14]:
```
def calculate_linear_regress_gradient_fit(xs, ys):

    """Calculates the linear regression fit using scipy's lineregress fit␣
↪library, and returns gradient, intercept and the error on the fit."""

    # Sanity check on the array sizes
    if xs.size != ys.size:

        raise ValueError('Arrays are not of the same size, please check the␣
↪code that makes the arrays that were inputted into this function.')

    else:

        # Running the two numpy arrays through the linear regression library
        result = stats.linregress(xs, ys)

        ## Extracting results to be returned ##

        # Slope and its error
        slope = result.slope
        slope_error = result.stderr
```

```
        # Intercept and its error
        intercept = result.intercept
        intercept_error = result.intercept_stderr

    return slope, slope_error, intercept, intercept_error
```

16 Function to calculate a straight line fit - Needed for plotting lines of best fit

[15]:
```
def straight_line(x, m, c):

    """Function used to calculate the straight line fit"""

    return (m*x)+c
```

17 Function to run a single metropolis algorithm simulation with end system visualisation

[16]:
```
def␣
↪run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file,␣
↪**kwargs):

    """Uses the reading function to read in the parameters,
    and then runs the ising model and returns the final lattice configuration␣
↪along with any plots.

    Stores the lattices in a list of arrays so they can be accessed and␣
↪analysed later."""

    # Function random generator
    generator = np.random.Generator(np.random.PCG64DXSM())

    # Condition to show / hide the diagnostic plots
    plot_mode = kwargs.get('plot_mode', None)

    # Setting up ICS and the lattice from the file

    # Extracting ICS
    lattice_side_length, magnetic_field_B, temperature, exchange_constant_J,␣
↪k_b, u_b, nsteps, filename = read_2D_ising_parameter_file(parameter_file)

    # Initialising a random 2D lattice
    initial_lattice = make_random_2D_lattice(lattice_side_length, generator)
```

```
    # Creating the lattice used for the ising model simulation itself, leaving␣
↪the initial lattice separate and unchanged.
    lattice = initial_lattice.copy()

    # Printing parameters as a sanity check - Make sure the file is read in␣
↪correctly.
    print(f'Running single Metropolis Algorithm simulation with parameters:')
    print(f'temperature={temperature}K,␣
↪lattice_side_length={lattice_side_length},␣
↪magnetic_field={magnetic_field_B},␣
↪exchange_constant_J={exchange_constant_J}, k_b={k_b}, u_b={u_b},␣
↪nsteps={nsteps}\n')

    # Burn-in limit for calculations of variances - This is to account for␣
↪equilibration
    burnin_step_proportion = 0.6
    burnin_equilibration_steps_threshold = int(burnin_step_proportion*nsteps)

    # Lists to store end parameters generated during the main simulation loop
    lattice_energy_per_spin_list = []
    lattice_magnetisation_per_spin_list = []
    lattice_internal_magnetic_field_per_spin_list = []
    lattice_magnetic_susceptibility_per_spin_list = []
    lattice_heat_capacity_per_spin_list = []

    # Number of equilibrated samples
    n_equilibrated = 0

    # Initialising starting variances for the welford algorithm implementation␣
↪in the main simulation loop
    running_mean_magnetisation_per_spin = 0.0
    running_mean_magnetisation_per_spin_2 = 0.0
    running_mean_energy_per_spin = 0.0
    running_mean_energy_per_spin_2 = 0.0

    # Number of equilibrated samples
    n_equilibrated = 0

    ### MAIN SIMULATION LOOP ###
    for i in tqdm(range(nsteps), desc='Running MMC Ising Model:'):

        # Reassigning the lattice with every step, making the ising model run.
        lattice = MMC_step(lattice_side_length, lattice, magnetic_field_B,␣
↪temperature, k_b, u_b, exchange_constant_J, generator)
```

```
        # Calling functions on the lattice every timestep to calculate the␣
↪quantities of interest
        lattice_energy_per_spin = calculate_lattice_energy_per_spin(lattice,␣
↪lattice_side_length, exchange_constant_J, magnetic_field_B, u_b)
        lattice_magnetisation_per_spin =␣
↪calculate_magnetisation_per_spin(lattice, lattice_side_length) # In the␣
↪metropolis algorithm there is no risk of the entire lattice turning into a␣
↪cluster being flipped so the true magnetisation (not the absolute value) can␣
↪be used here
        lattice_internal_magnetic_field_per_spin =␣
↪calculate_internal_magnetic_field_per_spin(lattice, lattice_side_length,␣
↪magnetic_field_B)

        # Appending values to lists
        lattice_energy_per_spin_list.append(lattice_energy_per_spin)
        lattice_magnetisation_per_spin_list.
↪append(lattice_magnetisation_per_spin)
        lattice_internal_magnetic_field_per_spin_list.
↪append(lattice_internal_magnetic_field_per_spin)

        # Only begin the welford loop after equilibration
        if i > burnin_equilibration_steps_threshold:

            # Computing number of equilibrated samples
            n_equilibrated += 1

            ## WELFORD ALGORITHM IMPLEMENTATION - ONLY RUNS AFTER EQUILIBRATION␣
↪HAS BEEN ACHIEVED ##

            # Two staged approach to extracting variances - Magnetisation

            # Stage 1
            magnetisation_diff = lattice_magnetisation_per_spin -␣
↪running_mean_magnetisation_per_spin
            running_mean_magnetisation_per_spin += magnetisation_diff /␣
↪n_equilibrated

            # Stage 2
            magnetisation_diff_2 = lattice_magnetisation_per_spin -␣
↪running_mean_magnetisation_per_spin
            running_mean_magnetisation_per_spin_2 += magnetisation_diff *␣
↪magnetisation_diff_2

            # Two staged approach to extracting variances - Energy

            # Stage 1
```

```
            energy_diff = lattice_energy_per_spin - running_mean_energy_per_spin
            running_mean_energy_per_spin += energy_diff / n_equilibrated

            # Stage 2
            energy_diff_2 = lattice_energy_per_spin - running_mean_energy_per_spin
            running_mean_energy_per_spin_2 += energy_diff * energy_diff_2

            if n_equilibrated > 1:

                # Calculating the variances for magnetisation and energy
                magnetisation_per_spin_running_variance = running_mean_magnetisation_per_spin_2 / n_equilibrated
                energy_per_spin_running_variance = running_mean_energy_per_spin_2 / n_equilibrated

                # Appending magnetic susceptibility and heat capacity
                lattice_magnetic_susceptibility_per_spin_list.append((1/(k_b*temperature)) * magnetisation_per_spin_running_variance)
                lattice_heat_capacity_per_spin_list.append(((1/(k_b*temperature**2)) * energy_per_spin_running_variance))

    # Converting to arrays - useful for calculations later
    lattice_energy_per_spin_array = np.asarray(lattice_energy_per_spin_list)
    lattice_magnetisation_per_spin_array = np.asarray(lattice_magnetisation_per_spin_list)
    lattice_internal_magnetic_field_per_spin_array = np.asarray(lattice_internal_magnetic_field_per_spin_list)
    lattice_magnetic_susceptibility_per_spin_array = np.asarray(lattice_magnetic_susceptibility_per_spin_list)
    lattice_heat_capacity_per_spin_array = np.asarray(lattice_heat_capacity_per_spin_list)

    ## Making main lattice figure ##

    fig, ax = plt.subplots(figsize=(4,4), dpi=300)

    # cmap
    blue_yellow_cmap = colors.ListedColormap(['blue', 'yellow'])

    # Plotting
    ising_2D_plot = ax.imshow(lattice, cmap=blue_yellow_cmap)

    # Formatting
    ax.set_title('2D Ising Model Lattice Plot, MMC')
    ax.set_xlabel('x')
```

```
    ax.set_ylabel('y')

    cbar = fig.colorbar(ising_2D_plot, ax=ax, ticks=[-1, 1], shrink=0.75)
    cbar.ax.set_yticklabels(['Spin = -1', 'Spin = 1'])

    # Saving figure
    fig.savefig(f'End lattice configuration, filename={filename}, MMC.png',␣
↪dpi=300)

    ## Making figures for the quantities of interest against the number of␣
↪timesteps ##

    # Energy per spin vs timestep
    fig1, ax1 = plt.subplots(figsize=(6,4), dpi=300)
    ax1.plot(range(0, nsteps), lattice_energy_per_spin_array)
    ax1.set_title('Lattice energy per spin vs timestep, MMC')
    ax1.set_xlabel('N Monte-Carlo steps')
    ax1.set_ylabel('Energy per spin / J')
    ax1.grid(ls=':', color='0.9')

    # Saving figure
    fig1.savefig(f'Lattice energy per spin vs timestep, filename={filename},␣
↪MMC.png', dpi=300)

    # Magnetisation per spin vs timestep
    fig2, ax2 = plt.subplots(figsize=(6,4), dpi=300)
    ax2.plot(range(0, nsteps), lattice_magnetisation_per_spin_array)
    ax2.set_title('Lattice magnetisation per spin vs timestep, MMC')
    ax2.set_xlabel('N Monte-Carlo steps')
    ax2.set_ylabel('Magnetisation per spin')
    ax2.grid(ls=':', color='0.9')

    # Saving figure
    fig2.savefig(f'Lattice magnetisation per spin vs timestep,␣
↪filename={filename}, MMC.png', dpi=300)

    # Lattice internal magnetic field per spin vs timestep
    fig3, ax3 = plt.subplots(figsize=(6,4), dpi=300)
    ax3.plot(range(0, nsteps), lattice_internal_magnetic_field_per_spin_array)
    ax3.set_title('Lattice internal magnetic field per spin vs timestep, MMC')
    ax3.set_xlabel('N Monte-Carlo steps')
    ax3.set_ylabel('Internal magnetic field per spin')
    ax3.grid(ls=':', color='0.9')

    # Saving figure
    fig3.savefig(f'Internal magnetic field per spin vs timestep,␣
↪filename={filename}, MMC.png', dpi=300)
```

```
    ## Variables that require variances ##

    # Magnetic susceptibility per spin vs timestep
    fig4, ax4 = plt.subplots(figsize=(6,4), dpi=300)
    ax4.plot(range(burnin_equilibration_steps_threshold+2, nsteps),␣
↪lattice_magnetic_susceptibility_per_spin_array) # Skipping the first step,␣
↪since there is no variance between the first lattice and itself.
    ax4.set_title('Lattice magnetic susceptibility per spin vs timestep, MMC')
    ax4.set_xlabel('N Monte-Carlo steps')
    ax4.set_ylabel('Magnetic susceptibility per spin')
    ax4.grid(ls=':', color='0.9')

    # Saving figure
    fig4.savefig(f'Magnetic susceptibility per spin vs timestep,␣
↪filename={filename}, MMC.png', dpi=300)

    # Heat Capacity per spin vs timestep
    fig5, ax5 = plt.subplots(figsize=(6,4), dpi=300)
    ax5.plot(range(burnin_equilibration_steps_threshold+2, nsteps),␣
↪lattice_heat_capacity_per_spin_array) # Skipping the first step, since there␣
↪is no variance between the first lattice and itself.
    ax5.set_title('Lattice heat capacity per spin vs timestep, MMC')
    ax5.set_xlabel('N Monte-Carlo steps')
    ax5.set_ylabel('Heat capacity per spin')
    ax5.grid(ls=':', color='0.9')

    # Saving figure
    fig5.savefig(f'Lattice heat capacity per spin vs timestep,␣
↪filename={filename}, MMC.png', dpi=300)

    # This will be used on the validation plots to conserve memory
    if plot_mode == 'close':
        plt.close('all')

    # If no mode is defined, force the plots to display
    elif plot_mode == None:
        plt.show('all')

    # Return ising pixel plot and the end magnetisation, which is critical for␣
↪the temperature checks.
    return fig, ax, fig1, ax1, fig2, ax2, fig3, ax3, fig4, ax4, fig5, ax5
```

18 Function to run a single wolff algorithm simulation with end system visualisation

[17]:
```
def␣
↪run_single_ising_model_simulation_with_lattice_visualisation_WMC(parameter_file,␣
↪**kwargs):

    """Uses the reading function to read in the parameters,
    and then runs the ising model and returns the final lattice configuration␣
↪along with any plots.

    Stores the lattices in a list of arrays so they can be accessed and␣
↪analysed later.

    Exactly the same as␣
↪run_single_ising_model_simulation_with_lattice_visualisation_MMC, but uses␣
↪the
    Wolff algorithm instead of the met."""

    # Function random generator
    generator = np.random.Generator(np.random.PCG64DXSM())

    # Condition to show / hide the diagnostic plots
    plot_mode = kwargs.get('plot_mode', None)

    # Setting up ICS and the lattice from the file

    # Extracting ICS
    lattice_side_length, magnetic_field_B, temperature, exchange_constant_J,␣
↪k_b, u_b, nsteps, filename = read_2D_ising_parameter_file(parameter_file)

    # Initialising a random 2D lattice
    initial_lattice = make_random_2D_lattice(lattice_side_length, generator)

    # Creating the lattice used for the ising model simulation itself, leaving␣
↪the initial lattice separate and unchanged.
    lattice = initial_lattice.copy()

    # Printing parameters as a sanity check - Make sure the file is read in␣
↪correctly.
    print(f'Running single Wolff Algorithm simulation with parameters:')
    print(f'temperature={temperature}K,␣
↪lattice_side_length={lattice_side_length},␣
↪magnetic_field={magnetic_field_B},␣
↪exchange_constant_J={exchange_constant_J}, k_b={k_b}, u_b={u_b},␣
↪nsteps={nsteps}\n')
```

```
    # Burn-in limit for calculations of variances - This is to account for␣
↪equilibration
    burnin_step_proportion = 0.6
    burnin_equilibration_steps_threshold = int(burnin_step_proportion*nsteps)

    # Lists to store end parameters generated during the main simulation loop
    lattice_energy_per_spin_list = []
    lattice_magnetisation_per_spin_list = []
    lattice_internal_magnetic_field_per_spin_list = []
    lattice_magnetic_susceptibility_per_spin_list = []
    lattice_heat_capacity_per_spin_list = []

    # Number of equilibrated samples
    n_equilibrated = 0

    # Initialising starting variances for the welford algorithm implementation␣
↪in the main simulation loop
    running_mean_magnetisation_per_spin = 0.0
    running_mean_magnetisation_per_spin_2 = 0.0
    running_mean_energy_per_spin = 0.0
    running_mean_energy_per_spin_2 = 0.0

    # Number of equilibrated samples
    n_equilibrated = 0

    ### MAIN SIMULATION LOOP ###
    for i in tqdm(range(nsteps), desc='Running WMC Ising Model:'):

        # Reassigning the lattice with every step, making the ising model run.
        lattice = WMC_step(lattice_side_length, lattice, magnetic_field_B,␣
↪temperature, k_b, u_b, exchange_constant_J, generator)

        ## CHANGE FROM␣
↪run_single_ising_model_simulation_with_lattice_visualisation_MMC ##

        # All magnetic quantities (internal field H and magnetisation M) are␣
↪absolute quantities instead of signed.
        # This is because after it equilibrates, the wolff algorithm will grow␣
↪and flip a cluster
        # as big as the entire lattice, resulting in the plots flaring out and␣
↪becoming unusable.

        # Calling functions on the lattice every timestep to calculate the␣
↪quantities of interest
```

```
        lattice_energy_per_spin = calculate_lattice_energy_per_spin(lattice,␣
↪lattice_side_length, exchange_constant_J, magnetic_field_B, u_b)
        lattice_magnetisation_per_spin = np.
↪abs(calculate_magnetisation_per_spin(lattice, lattice_side_length)) #␣
↪Absolute magnetisation instead of standard magnetisation for wolff algorithm␣
↪to guard against the effects of the entire lattice being turned into a␣
↪cluster and being flipped
        lattice_internal_magnetic_field_per_spin = np.
↪abs(calculate_internal_magnetic_field_per_spin(lattice, lattice_side_length,␣
↪magnetic_field_B)) # Absolute magnetic field instead of standard magnetic␣
↪field for wolff algorithm to guard against the effects of the entire lattice␣
↪being turned into a cluster and being flipped

        # Appending values to lists
        lattice_energy_per_spin_list.append(lattice_energy_per_spin)
        lattice_magnetisation_per_spin_list.
↪append(lattice_magnetisation_per_spin)
        lattice_internal_magnetic_field_per_spin_list.
↪append(lattice_internal_magnetic_field_per_spin)

        # Only begin the welford loop after equilibration
        if i > burnin_equilibration_steps_threshold:

            # Computing number of equilibrated samples
            n_equilibrated += 1

            ## WELFORD ALGORITHM IMPLEMENTATION - ONLY RUNS AFTER EQUILIBRATION␣
↪HAS BEEN ACHIEVED ##

            # Two staged approach to extracting variances - Magnetisation

            # Stage 1
            magnetisation_diff = lattice_magnetisation_per_spin -␣
↪running_mean_magnetisation_per_spin
            running_mean_magnetisation_per_spin += magnetisation_diff /␣
↪n_equilibrated

            # Stage 2
            magnetisation_diff_2 = lattice_magnetisation_per_spin -␣
↪running_mean_magnetisation_per_spin
            running_mean_magnetisation_per_spin_2 += magnetisation_diff *␣
↪magnetisation_diff_2

            # Two staged approach to extracting variances - Energy

            # Stage 1
```

```
            energy_diff = lattice_energy_per_spin - running_mean_energy_per_spin
            running_mean_energy_per_spin += energy_diff / n_equilibrated

            # Stage 2
            energy_diff_2 = lattice_energy_per_spin - running_mean_energy_per_spin
            running_mean_energy_per_spin_2 += energy_diff * energy_diff_2

            if n_equilibrated > 1:

                # Calculating the variances for magnetisation and energy
                magnetisation_per_spin_running_variance = running_mean_magnetisation_per_spin_2 / n_equilibrated
                energy_per_spin_running_variance = running_mean_energy_per_spin_2 / n_equilibrated

                # Appending magnetic susceptibility and heat capacity
                lattice_magnetic_susceptibility_per_spin_list.append((1/(k_b*temperature)) * magnetisation_per_spin_running_variance)
                lattice_heat_capacity_per_spin_list.append(((1/(k_b*temperature**2)) * energy_per_spin_running_variance))

    # Converting to arrays - useful for calculations later
    lattice_energy_per_spin_array = np.asarray(lattice_energy_per_spin_list)
    lattice_magnetisation_per_spin_array = np.asarray(lattice_magnetisation_per_spin_list)
    lattice_internal_magnetic_field_per_spin_array = np.asarray(lattice_internal_magnetic_field_per_spin_list)
    lattice_magnetic_susceptibility_per_spin_array = np.asarray(lattice_magnetic_susceptibility_per_spin_list)
    lattice_heat_capacity_per_spin_array = np.asarray(lattice_heat_capacity_per_spin_list)

    ## Making main lattice figure ##

    fig, ax = plt.subplots(figsize=(4,4), dpi=300)

    # cmap
    blue_yellow_cmap = colors.ListedColormap(['blue', 'yellow'])

    # Plotting
    ising_2D_plot = ax.imshow(lattice, cmap=blue_yellow_cmap)

    # Formatting
    ax.set_title('2D Ising Model Lattice Plot, WMC')
    ax.set_xlabel('x')
```

```
    ax.set_ylabel('y')

    cbar = fig.colorbar(ising_2D_plot, ax=ax, ticks=[-1, 1], shrink=0.75)
    cbar.ax.set_yticklabels(['Spin = -1', 'Spin = 1'])

    # Saving figure
    fig.savefig(f'End lattice configuration, filename={filename}, WMC.png',␣
↪dpi=300)

    ## Making figures for the quantities of interest against the number of␣
↪timesteps ##

    # Energy per spin vs timestep
    fig1, ax1 = plt.subplots(figsize=(6,4), dpi=300)
    ax1.plot(range(0, nsteps), lattice_energy_per_spin_array)
    ax1.set_title('Lattice energy per spin vs timestep, WMC')
    ax1.set_xlabel('N Monte-Carlo steps')
    ax1.set_ylabel('Energy per spin / J')
    ax1.grid(ls=':', color='0.9')

    # Saving figure
    fig1.savefig(f'Lattice energy per spin vs timestep, filename={filename},␣
↪WMC.png', dpi=300)

    # Magnetisation per spin vs timestep
    fig2, ax2 = plt.subplots(figsize=(6,4), dpi=300)
    ax2.plot(range(0, nsteps), lattice_magnetisation_per_spin_array)
    ax2.set_title('Lattice absolute magnetisation per spin vs timestep, WMC')
    ax2.set_xlabel('N Monte-Carlo steps')
    ax2.set_ylabel('Absolute magnetisation per spin')
    ax2.grid(ls=':', color='0.9')

    # Saving figure
    fig2.savefig(f'Lattice magnetisation per spin vs timestep,␣
↪filename={filename}, WMC.png', dpi=300)

    # Lattice internal magnetic field per spin vs timestep
    fig3, ax3 = plt.subplots(figsize=(6,4), dpi=300)
    ax3.plot(range(0, nsteps), lattice_internal_magnetic_field_per_spin_array)
    ax3.set_title('Lattice absolute internal magnetic field per spin vs␣
↪timestep, WMC')
    ax3.set_xlabel('N Monte-Carlo steps')
    ax3.set_ylabel('Absolute internal magnetic field per spin')
    ax3.grid(ls=':', color='0.9')

    # Saving figure
```

```
    fig3.savefig(f'Internal magnetic field per spin vs timestep,␣
↪filename={filename}, WMC.png', dpi=300)

    ## Variables that require variances ##

    # Magnetic susceptibility per spin vs timestep
    fig4, ax4 = plt.subplots(figsize=(6,4), dpi=300)
    ax4.plot(range(burnin_equilibration_steps_threshold+2, nsteps),␣
↪lattice_magnetic_susceptibility_per_spin_array) # Skipping the first step,␣
↪since there is no variance between the first lattice and itself.
    ax4.set_title('Lattice magnetic susceptibility per spin vs timestep, WMC')
    ax4.set_xlabel('N Monte-Carlo steps')
    ax4.set_ylabel('Magnetic susceptibility per spin')
    ax4.grid(ls=':', color='0.9')

    # Saving figure
    fig4.savefig(f'Magnetic susceptibility per spin vs timestep,␣
↪filename={filename}, WMC.png', dpi=300)

    # Heat Capacity per spin vs timestep
    fig5, ax5 = plt.subplots(figsize=(6,4), dpi=300)
    ax5.plot(range(burnin_equilibration_steps_threshold+2, nsteps),␣
↪lattice_heat_capacity_per_spin_array) # Skipping the first step, since there␣
↪is no variance between the first lattice and itself.
    ax5.set_title('Lattice heat capacity per spin vs timestep, WMC')
    ax5.set_xlabel('N Monte-Carlo steps')
    ax5.set_ylabel('Heat capacity per spin')
    ax5.grid(ls=':', color='0.9')

    # Saving figure
    fig5.savefig(f'Lattice heat capacity per spin vs timestep,␣
↪filename={filename}, WMC.png', dpi=300)

    # This will be used on the validation plots to conserve memory
    if plot_mode == 'close':
        plt.close('all')

    # If no mode is defined, force the plots to display
    elif plot_mode == None:
        plt.show('all')

    # Return ising pixel plot and the end magnetisation, which is critical for␣
↪the temperature checks.
    return fig, ax, fig1, ax1, fig2, ax2, fig3, ax3, fig4, ax4, fig5, ax5
```

19 Function for running parallelised critical temperature and exponents investigation using the metropolis algorithm

[18]:
```
def run_parallelised_temp_change_investigation_simulations_MMC(parameter_file,␣
↪**kwargs):

    """Calls the run_single_temp_investigation_simulation(args) function N␣
↪cores times per instance and packages the results up into lists.
    Once enough instances have been called that the full temperature sweep has␣
↪been completed, the lists of mean magnetisations,
    mean magnetic susceptibilities and mean heat capacities are converted into␣
↪arrays and used for critical temperature and critical
    exponent calculations.

    In order to run the run_single_temp_investigation_simulation(args) function␣
↪in parallel, it uses all available cpu cores and the pool module
    from multiprocessing package.

    Returns plots of magnetisations and magnetic susceptibility vs temperatures.
↪"""

    # Getting the plot mode - condition to show / hide the diagnostic plots
    plot_mode = kwargs.get('plot_mode', None)

    # Setting up ICS and the lattice from the file
    lattice_side_length, magnetic_field_B, initial_temperature, temp_step,␣
↪exchange_constant_J, k_b, u_b, nsteps, nruns, filename =␣
↪read_tcrit_investigative_file(parameter_file)

    # Making a list to store temperatures
    run_temperatures_list = []

    # Appending to the list using temp step and the amount of runs
    for i in range(nruns):

        run_temperatures_list.append(initial_temperature + i*temp_step)

    # Turning the list into an array as this will be useful later
    run_temperatures_array = np.asarray(run_temperatures_list)

    ## Setting up parallelised simulation runs ##

    # Runs for worker function defined in the file
    single_temp_run_args = [(temperature, lattice_side_length,␣
↪magnetic_field_B, exchange_constant_J, k_b, u_b, nsteps) for temperature in␣
↪run_temperatures_array]
```

```
    # Printing out initialisation message for parallelised simulations
    print(f'Parallelising {nruns} temperature investigation runs across␣
↪{n_cores} CPU cores using the metropolis algorithm.')
    print(f'Temperature range: {run_temperatures_array[0]} -->␣
↪{run_temperatures_array[-1]}.')
    print(f'Per-simulation parameters: temp_step={temp_step}K,␣
↪lattice_side_length={lattice_side_length},␣
↪magnetic_field_B={magnetic_field_B},␣
↪exchange_constant_J={exchange_constant_J}, k_b={k_b}, u_b={u_b},␣
↪nsteps={nsteps}.')

    # Running multiple simulations in parallel
    with Pool(processes=os.cpu_count()) as pool:

        # Calling the imported parallelisable worker function to run multiple␣
↪simulations at the same time using multiprocessing
        # This completes the runs across multiple temperatures much faster than␣
↪using one core only
        per_run_mean_quantites = list(tqdm(pool.
↪imap(run_MMC_temp_investigation_worker, single_temp_run_args), total=nruns,␣
↪desc='Running parallelised metropolis algorithm temperature sweep:'))

    # Unpacking results from the list generated using multiprocessing
    # This has to be done in the same order as the␣
↪run_single_temp_investigation_simulation(args) returns them in.
    per_run_mean_equilibrated_magnetisations_per_spin_array = np.
↪array([mean_results[0] for mean_results in per_run_mean_quantites])
    per_run_mean_equilibrated_magnetic_susceptibilities_per_spin_array = np.
↪array([mean_results[1] for mean_results in per_run_mean_quantites])
    per_run_mean_equilibrated_heat_capacities_per_spin_array = np.
↪array([mean_results[2] for mean_results in per_run_mean_quantites])

    ## Critical temperature extraction ##

    # Method 1: Steepest drop in absolute magnetisation - Midpoint of the two␣
↪temperatures between where this happens should yield phase transition␣
↪temperature.
    consecutive_magnetisation_diffs = np.
↪diff(per_run_mean_equilibrated_magnetisations_per_spin_array)
    largest_magnetisation_drop_index = np.
↪argmin(consecutive_magnetisation_diffs)

    # Average of 2 values before and after the index for the most negative drop␣
↪in magnetisation should yield the critical temperature
```

```
  tcrit_from_magnetisation =␣
↪(run_temperatures_array[largest_magnetisation_drop_index] +␣
↪run_temperatures_array[largest_magnetisation_drop_index+1])/2

  # Method 2: Index of the temperature where magnetic susceptibility peaks
  largest_magnetic_susceptibility_index = np.
↪argmax(per_run_mean_equilibrated_magnetic_susceptibilities_per_spin_array)
  tcrit_from_magnetic_susceptibility =␣
↪run_temperatures_array[largest_magnetic_susceptibility_index]

  # Method 3: Index of temperature where heat capacity peaks
  largest_heat_capacity_index = np.
↪argmax(per_run_mean_equilibrated_heat_capacities_per_spin_array)
  tcrit_from_heat_capacity =␣
↪run_temperatures_array[largest_heat_capacity_index]

  # Method 3: Average of methods 1, 2 and 3 - Most statistical accuracy
  average_tcrit = (tcrit_from_magnetisation +␣
↪tcrit_from_magnetic_susceptibility + tcrit_from_heat_capacity)/3

  ## Storage of Onsager's calculated critical temperature and critical␣
↪exponents for comparison and error calculations ##

  # Onsager 1944 critical temperature
  tcrit_onsager = 2.269185

  # Storing Onsager 1944 critical exponents

  # Alpha critical exponent
  negative_onsager_critical_exponent = -0.4945

  # Beta critical exponent
  onsager_beta = 1/8

  # Gamma critical exponent
  onsager_gamma = -7/4

  ## Temperature regime filters ##

  # Above tcrit filter with 1% safety threshold.
  above_tcrit_filter = run_temperatures_array > average_tcrit*1.001

  # Below tcrit filter with 1% safety threshold.
  below_tcrit_filter = run_temperatures_array < average_tcrit*0.99
```

```
    ## Using reduced temperatures because they are dimensionless and thus ideal␣
↪for power law scaling ##

    # Reduced temperatures using data above tcrit
    absolute_reduced_temperatures_above_tcrit =␣
↪(run_temperatures_array[above_tcrit_filter] - average_tcrit) / average_tcrit

    # Reduced temperatures using data below tcrit
    absolute_reduced_temperatures_below_tcrit =␣
↪(run_temperatures_array[below_tcrit_filter] - average_tcrit) / average_tcrit

    ## Heat capacity, magnetisation and susceptibility filtering ##

    # Separate heat capacities above and below tcrit
    heat_capacities_above_tcrit =␣
↪per_run_mean_equilibrated_heat_capacities_per_spin_array[above_tcrit_filter]
    heat_capacities_below_tcrit =␣
↪per_run_mean_equilibrated_heat_capacities_per_spin_array[below_tcrit_filter]

    # Separate magnetisations below tcrit. Only applies below tcrit
    magnetisations_below_tcrit =␣
↪per_run_mean_equilibrated_magnetisations_per_spin_array[below_tcrit_filter]

    # Separate magnetic susceptibilities above and below tcrit
    magnetic_susceptibilities_above_tcrit =␣
↪per_run_mean_equilibrated_magnetic_susceptibilities_per_spin_array[above_tcrit_filter]
    magnetic_susceptibilities_below_tcrit =␣
↪per_run_mean_equilibrated_magnetic_susceptibilities_per_spin_array[below_tcrit_filter]

    ## Computing natural logarithms of absolute reduced temperatures above and␣
↪below tcrit ##

    # np.abs is used to prevent the logarithm of a negative quantity from being␣
↪computed.
    ln_absolute_reduced_temperatures_above_tcrit = np.log(np.
↪abs(absolute_reduced_temperatures_above_tcrit))
    ln_absolute_reduced_temperatures_below_tcrit = np.log(np.
↪abs(absolute_reduced_temperatures_below_tcrit))

    ## Computing natural logarithm of magnetisation below tcrit

    # np.abs is used to prevent the logarithm of a negative quantity from being␣
↪computed.
    ln_magnetisations_below_tcrit = np.log(np.abs(magnetisations_below_tcrit))
```

```
   ## Computing natural logarithms of magnetic susceptibilities above and␣
↪below tcrit ##

   # np.abs is used to prevent the logarithm of a negative quantity from being␣
↪computed.
   ln_magnetic_susceptibilities_above_tcrit = np.log(np.
↪abs(magnetic_susceptibilities_above_tcrit))
   ln_magnetic_susceptibilities_below_tcrit = np.log(np.
↪abs(magnetic_susceptibilities_below_tcrit))

   ## Computing the 5 different gradients ##

   # Gradient of ln heat capacity vs ln reduced temperatures above and below␣
↪tcrit
   gradient_heat_capacity_above_tcrit,␣
↪error_gradient_heat_capacity_above_tcrit,␣
↪heat_capacity_intercept_above_tcrit,␣
↪error_heat_capacity_intercept_above_tcrit =␣
↪calculate_linear_regress_gradient_fit(ln_absolute_reduced_temperatures_above_tcrit,␣
↪heat_capacities_above_tcrit)
   gradient_heat_capacity_below_tcrit,␣
↪error_gradient_heat_capacity_below_tcrit,␣
↪heat_capacity_intercept_below_tcrit,␣
↪error_heat_capacity_intercept_below_tcrit =␣
↪calculate_linear_regress_gradient_fit(ln_absolute_reduced_temperatures_below_tcrit,␣
↪heat_capacities_below_tcrit)

   # Gradient of ln magnetisation vs ln reduced temperatures above and below␣
↪tcrit
   gradient_magnetisation_below_tcrit,␣
↪error_gradient_heat_capacity_below_tcrit,␣
↪magnetisation_intercept_below_tcrit,␣
↪error_magnetisation_intercept_below_tcrit =␣
↪calculate_linear_regress_gradient_fit(ln_absolute_reduced_temperatures_below_tcrit,␣
↪ln_magnetisations_below_tcrit)

   # Gradient of ln magnetic susceptibility vs ln reduced temperatures above␣
↪and below tcrit
   gradient_magnetic_susceptibility_above_tcrit,␣
↪error_gradient_magnetic_susceptibility_above_tcrit,␣
↪magnetic_susceptibility_intercept_above_tcrit,␣
↪error_magnetic_susceptibility_intercept_above_tcrit =␣
↪calculate_linear_regress_gradient_fit(ln_absolute_reduced_temperatures_above_tcrit,␣
↪ln_magnetic_susceptibilities_above_tcrit)
```

```
    gradient_magnetic_susceptibility_below_tcrit,␣
↪error_gradient_magnetic_susceptibility_below_tcrit,␣
↪magnetic_susceptibility_intercept_below_tcrit,␣
↪error_magnetic_susceptibility_intercept_below_tcrit =␣
↪calculate_linear_regress_gradient_fit(ln_absolute_reduced_temperatures_below_tcrit,␣
↪ln_magnetic_susceptibilities_below_tcrit)

    ### Making plots ###

    # Generating array of x coordinates for plotting straight line for above␣
↪tcrit
    x_temperatures_straightline_above_tcrit = np.linspace(np.
↪min(ln_absolute_reduced_temperatures_above_tcrit), np.
↪max(ln_absolute_reduced_temperatures_above_tcrit), num=1001)

    # Generating array of x coordinates for plotting straight line for below␣
↪tcrit
    x_temperatures_straightline_below_tcrit = np.linspace(np.
↪min(ln_absolute_reduced_temperatures_below_tcrit), np.
↪max(ln_absolute_reduced_temperatures_below_tcrit), num=1001)

    ## Critical temperates overplotted on graphs of temperature vs␣
↪magnetisation ##

    # Making plot of temperatures vs magnetisation - axvlines for the three␣
↪methods and the onsager critical temperature
    fig, ax = plt.subplots(figsize=(6, 4), dpi=300)
    ax.plot(run_temperatures_array,␣
↪per_run_mean_equilibrated_magnetisations_per_spin_array, '-x')
    ax.axvline(x=tcrit_from_magnetisation, color='red', linestyle='--',␣
↪label='tcrit magnetisation')
    ax.axvline(x=tcrit_from_magnetic_susceptibility, color='green',␣
↪linestyle='--', label='tcrit magnetic susceptibility')
    ax.axvline(x=tcrit_from_heat_capacity, color='orange', linestyle='--',␣
↪label='tcrit heat capacity')
    ax.axvline(x=average_tcrit, color='blue', linestyle='--', label='tcrit␣
↪magnetisation, susceptibility and heat capacity average')
    ax.axvline(x=tcrit_onsager, color='grey', linestyle='--', label='tcrit␣
↪Onsager 1944')
    ax.legend(loc='upper right')
    ax.grid(ls=':', color='0.9')
    ax.set_title('Magnetisation per spin vs temperature MMC')
    ax.set_xlabel('Temperature')
    ax.set_ylabel('Magnetisation per spin')

    # Saving figure
```

```
    fig.savefig(f'Temperature vs magnetisation, filename={filename}, MMC.png',␣
↪dpi=300)

    # Making plot of temperatures vs heat capacity - axvlines for the three␣
↪methods and the onsager critical temperature
    fig0, ax0 = plt.subplots(figsize=(6, 4), dpi=300)
    ax0.plot(run_temperatures_array,␣
↪per_run_mean_equilibrated_heat_capacities_per_spin_array, '-x')
    ax0.axvline(x=tcrit_from_magnetisation, color='red', linestyle='--',␣
↪label='tcrit magnetisation')
    ax0.axvline(x=tcrit_from_magnetic_susceptibility, color='green',␣
↪linestyle='--', label='tcrit magnetic susceptibility')
    ax0.axvline(x=tcrit_from_heat_capacity, color='orange', linestyle='--',␣
↪label='tcrit heat capacity')
    ax0.axvline(x=average_tcrit, color='blue', linestyle='--', label='tcrit␣
↪magnetisation, susceptibility and heat capacity average')
    ax0.axvline(x=tcrit_onsager, color='grey', linestyle='--', label='tcrit␣
↪Onsager 1944')
    ax0.legend(loc='upper right')
    ax0.grid(ls=':', color='0.9')
    ax0.set_title('Heat capacity per spin vs temperature MMC')
    ax0.set_xlabel('Temperature')
    ax0.set_ylabel('Heat capacity per spin')

    # Saving figure
    fig0.savefig(f'Temperature vs heat capacity, filename={filename}, MMC.png',␣
↪dpi=300)

    # Making plot of temperature vs magnetic susceptibility axvlines for the␣
↪three methods and the onsager critical temperature
    fig1, ax1 = plt.subplots(figsize=(6, 4), dpi=300)
    ax1.plot(run_temperatures_array,␣
↪per_run_mean_equilibrated_magnetic_susceptibilities_per_spin_array, '-x')
    ax1.axvline(x=tcrit_from_magnetisation, color='red', linestyle='--',␣
↪label='tcrit magnetisation')
    ax1.axvline(x=tcrit_from_magnetic_susceptibility, color='green',␣
↪linestyle='--', label='tcrit magnetic susceptibility')
    ax1.axvline(x=tcrit_from_heat_capacity, color='orange', linestyle='--',␣
↪label='tcrit heat capacity')
    ax1.axvline(x=average_tcrit, color='blue', linestyle='--', label='tcrit␣
↪magnetisation, susceptibility and heat capacity average')
    ax1.axvline(x=tcrit_onsager, color='grey', linestyle='--', label='tcrit␣
↪Onsager 1944')
    ax1.legend(loc='upper right')
    ax1.grid(ls=':', color='0.9')
    ax1.set_title('Magnetic susceptibility per spin vs temperature MMC')
```

```
    ax1.set_xlabel('Temperature')
    ax1.set_ylabel('Magnetic susceptibility per spin')

    # Saving figure
    fig1.savefig(f'Temperature vs magnetic susceptibility, filename={filename},␣
↪MMC.png', dpi=300)

    ## Error calculations for critical temperature ##

    # Getting absolute errors from the onsager critical temperature
    error_tcrit_magnetisation = np.abs(tcrit_onsager-tcrit_from_magnetisation)
    error_tcrit_magnetic_susceptibility = np.
↪abs(tcrit_onsager-tcrit_from_magnetic_susceptibility)
    error_tcrit_heat_capacity = np.abs(tcrit_onsager-tcrit_from_heat_capacity)
    error_tcrit_average = np.abs(tcrit_onsager-average_tcrit)

    ## 5 Plots for the Onsager exponents in the valid regions - Corresponds to␣
↪the 5 gradients ##

    # 1. Gradient plot for critical exponent alpha: ln(t - tcrit) vs␣
↪ln(heat_capacity) above tcrit
    fig2, ax2 = plt.subplots(figsize=(6, 4), dpi=300)
    ax2.plot(ln_absolute_reduced_temperatures_above_tcrit,␣
↪heat_capacities_above_tcrit, '-x')
    ax2.plot(x_temperatures_straightline_above_tcrit,␣
↪straight_line(x_temperatures_straightline_above_tcrit,␣
↪gradient_heat_capacity_above_tcrit, heat_capacity_intercept_above_tcrit),␣
↪linestyle='--', color='black', label='Calculated Onsager Critical Amplitude␣
↪above tcrit')
    ax2.plot(x_temperatures_straightline_above_tcrit,␣
↪straight_line(x_temperatures_straightline_above_tcrit,␣
↪negative_onsager_critical_exponent, heat_capacity_intercept_above_tcrit),␣
↪linestyle='--', color='red', label='Onsager Critical Exponent')
    ax2.legend(loc='upper right')
    ax2.grid(ls=':', color='0.9')
    ax2.set_title('Onsager critical amplitude above tcrit MMC')
    ax2.set_xlabel('ln(|temps_above_tcrit - tcrit_average|)')
    ax2.set_ylabel('Heat Capacity')

    # Saving figure
    fig2.savefig(f'Onsager critical exponent above tcrit, filename={filename},␣
↪MMC.png', dpi=300)

    # 2. Gradient plot for critical exponent alpha: ln(t - tcrit) vs␣
↪ln(heat_capacity) below tcrit
    fig3, ax3 = plt.subplots(figsize=(6, 4), dpi=300)
```

```
    ax3.plot(ln_absolute_reduced_temperatures_below_tcrit,␣
↪heat_capacities_below_tcrit, '-x')
    ax3.plot(x_temperatures_straightline_below_tcrit,␣
↪straight_line(x_temperatures_straightline_below_tcrit,␣
↪gradient_heat_capacity_below_tcrit, heat_capacity_intercept_below_tcrit),␣
↪linestyle='--', color='black', label='Calculated Onsager Critical Amplitude␣
↪below tcrit')
    ax3.plot(x_temperatures_straightline_below_tcrit,␣
↪straight_line(x_temperatures_straightline_below_tcrit,␣
↪negative_onsager_critical_exponent, heat_capacity_intercept_below_tcrit),␣
↪linestyle='--', color='red', label='Onsager Critical Exponent')
    ax3.legend(loc='upper right')
    ax3.grid(ls=':', color='0.9')
    ax3.set_title('Onsager critical amplitude below tcrit MMC')
    ax3.set_xlabel('ln(|temps_below_tcrit - tcrit_average|)')
    ax3.set_ylabel('Heat Capacity')

    # Saving figure
    fig3.savefig(f'Onsager critical exponent below tcrit, filename={filename},␣
↪MMC.png', dpi=300)

    # 3. Gradient plot for critical exponent beta: ln(t - tcrit) vs␣
↪ln(magnetisation) below tcrit
    fig4, ax4 = plt.subplots(figsize=(6, 4), dpi=300)
    ax4.plot(ln_absolute_reduced_temperatures_below_tcrit,␣
↪ln_magnetisations_below_tcrit, '-x')
    ax4.plot(x_temperatures_straightline_below_tcrit,␣
↪straight_line(x_temperatures_straightline_below_tcrit,␣
↪gradient_magnetisation_below_tcrit, magnetisation_intercept_below_tcrit),␣
↪linestyle='--', color='black', label='Calculated Beta below tcrit')
    ax4.plot(x_temperatures_straightline_below_tcrit,␣
↪straight_line(x_temperatures_straightline_below_tcrit, onsager_beta,␣
↪magnetisation_intercept_below_tcrit), linestyle='--', color='red',␣
↪label='Onsager Beta')
    ax4.legend(loc='upper right')
    ax4.grid(ls=':', color='0.9')
    ax4.set_title('Beta below tcrit MMC')
    ax4.set_xlabel('ln(|temps_below_tcrit - tcrit_average|)')
    ax4.set_ylabel('ln(|magnetisation per spin|)')

    # Saving figure
    fig4.savefig(f'Beta below tcrit, filename={filename}, MMC.png', dpi=300)

    # 4. Gradient plot for critical exponent Gamma: ln(t - tcrit) vs␣
↪ln(heat_capacity) above tcrit
    fig5, ax5 = plt.subplots(figsize=(6, 4), dpi=300)
```

```
    ax5.plot(ln_absolute_reduced_temperatures_above_tcrit,␣
↪ln_magnetic_susceptibilities_above_tcrit, '-x')
    ax5.plot(x_temperatures_straightline_above_tcrit,␣
↪straight_line(x_temperatures_straightline_above_tcrit,␣
↪gradient_magnetic_susceptibility_above_tcrit,␣
↪magnetic_susceptibility_intercept_above_tcrit), linestyle='--',␣
↪color='black', label='Calculated Gamma above tcrit')
    ax5.plot(x_temperatures_straightline_above_tcrit,␣
↪straight_line(x_temperatures_straightline_above_tcrit, onsager_gamma,␣
↪magnetic_susceptibility_intercept_above_tcrit), linestyle='--', color='red',␣
↪label='Onsager Gamma')
    ax5.legend(loc='upper right')
    ax5.grid(ls=':', color='0.9')
    ax5.set_title('Gamma above tcrit MMC')
    ax5.set_xlabel('ln(|temps_above_tcrit - tcrit_average|)')
    ax5.set_ylabel('ln(|magnetic susceptibility per spin|)')

    # Saving figure
    fig5.savefig(f'Gamma above tcrit, filename={filename}, MMC.png', dpi=300)

    # 5. Gradient plot for critical exponent Gamma: ln(t - tcrit) vs␣
↪ln(heat_capacity) below tcrit
    fig6, ax6 = plt.subplots(figsize=(6, 4), dpi=300)
    ax6.plot(ln_absolute_reduced_temperatures_below_tcrit,␣
↪ln_magnetic_susceptibilities_below_tcrit, '-x')
    ax6.plot(x_temperatures_straightline_below_tcrit,␣
↪straight_line(x_temperatures_straightline_below_tcrit,␣
↪gradient_magnetic_susceptibility_below_tcrit,␣
↪magnetic_susceptibility_intercept_below_tcrit), linestyle='--',␣
↪color='black', label='Calculated Gamma below tcrit')
    ax6.plot(x_temperatures_straightline_below_tcrit,␣
↪straight_line(x_temperatures_straightline_below_tcrit, onsager_gamma,␣
↪magnetic_susceptibility_intercept_below_tcrit), linestyle='--', color='red',␣
↪label='Onsager Gamma')
    ax6.legend(loc='upper right')
    ax6.grid(ls=':', color='0.9')
    ax6.set_title('Gamma below tcrit MMC')
    ax6.set_xlabel('ln(|temps_below_tcrit - tcrit_average|)')
    ax6.set_ylabel('ln(|magnetic susceptibility per spin|)')

    # Saving figure
    fig6.savefig(f'Gamma below tcrit, filename={filename}, MMC.png', dpi=300)

    ## Calculating average of the two gradients in each safety region for alpha␣
↪and gamma. Beta doesnt need it as it is only valid below tcrit ##
```

```
    average_negative_onsager_critical_exponent =␣
↪(gradient_heat_capacity_above_tcrit + gradient_heat_capacity_below_tcrit) / 2
    average_gamma = (gradient_magnetic_susceptibility_above_tcrit +␣
↪gradient_magnetic_susceptibility_below_tcrit) / 2

    ## Calculating errors in critical exponents for above and below tcrit ##

    # Errors for alpha in both regimes and the average
    error_negative_onsager_critical_exponent_above_tcrit = np.
↪abs(negative_onsager_critical_exponent - gradient_heat_capacity_above_tcrit)
    error_negative_onsager_critical_exponent_below_tcrit = np.
↪abs(negative_onsager_critical_exponent - gradient_heat_capacity_below_tcrit)
    error_average_negative_onsager_critical_exponent = np.
↪abs(negative_onsager_critical_exponent -␣
↪average_negative_onsager_critical_exponent)

    # Error in beta below tcrit
    error_beta = np.abs(onsager_beta-gradient_magnetisation_below_tcrit)

    # Errors for alpha in both regimes and the average
    error_gamma_above_tcrit = np.abs(onsager_gamma -␣
↪gradient_magnetic_susceptibility_above_tcrit)
    error_gamma_below_tcrit = np.abs(onsager_gamma -␣
↪gradient_magnetic_susceptibility_below_tcrit)
    error_average_gamma = np.abs(onsager_gamma - average_gamma)

    ## Printing out critical temperature results so it is easy to access them ##

    # Critical temperature results
    print('*** Critical Temperature Results ***')
    print(f'The Onsager 1944 result for the critical temperature of an infinite␣
↪lattice is {tcrit_onsager}K.')
    print(f'Calculated critical temperature from magnetisation drop using the␣
↪metropolis algorithm is {tcrit_from_magnetisation}K.')
    print(f'Calculated critical temperature from magnetic susceptibility peak␣
↪using the metropolis algorithm is {tcrit_from_magnetic_susceptibility}K.')
    print(f'Calculated critical temperature from heat capacity peak using the␣
↪metropolis algorithm is {tcrit_from_heat_capacity}K.')
    print(f'Average from the three methods for calculating critical temperature␣
↪using the metropolis algoritm is {average_tcrit}K.')
    print('\n')

    print('*** Critical Temperature Errors ***')
    print(f'Error in critical temperature for the magnetisation drop method␣
↪using the metropolis algorithm is {error_tcrit_magnetisation}K.')
```

```
    print(f'Error in critical temperature for the magnetic susceptibility peak␣
↪method using the metropolis algorithm is␣
↪{error_tcrit_magnetic_susceptibility}K.')
    print(f'Error in critical temperature from heat capacity peak using the␣
↪metropolis algorithm is {error_tcrit_heat_capacity}K')
    print(f'Error in critical temperature for the average between the three␣
↪methods using the metropolis algorithm is {error_tcrit_average}.')
    print('\n')

    ## Printing out critical exponent results and errors so it is easy to␣
↪access them ##

    # Onsager values for critical exponents
    print('*** Onsager Critical Exponents ***')
    print(f'The Onsager 1944 result for the critical amplitude {np.
↪abs(negative_onsager_critical_exponent)}.')
    print('The Onsager 1944 result for the critical exponent alpha is 0')
    print(f'The Onsager 1944 result for the critical exponent beta is␣
↪{onsager_beta}.')
    print(f'The Onsager 1944 result for the critical exponent gamma is␣
↪{onsager_gamma}.')
    print('\n')

    # Results for Onsager critical amplitude
    print('Since alpha is 0, the best way to verify it is a linear plot of␣
↪ln(T) vs the heat capacity.')
    print('If the simulation is working correctly and alpha is behaving as␣
↪Onsager predicted, then the plot will be linear and have a gradient of -0.
↪4945, which is the negative of the Onsager critical amplitude.')
    print('\n')
    print('*** Results for Onsager Critical Amplitude ***')
    print(f'The calculated value of the negative Onsager critical amplitude␣
↪above tcrit is {gradient_heat_capacity_above_tcrit}.')
    print(f'The calculated value of the negative Onsager critical amplitude␣
↪below tcrit is {gradient_heat_capacity_below_tcrit}.')
    print(f'The calculated value of the negative Onsager critical amplitude␣
↪using the average of the two temperature regimes is␣
↪{average_negative_onsager_critical_exponent}.')
    print('\n')

    # Errors for alpha in both temperature regimes and the average value
    print('*** Errors for Onsager critical amplitude ***')
    print(f'Error in Onsager critical amplitude above tcrit is␣
↪{error_negative_onsager_critical_exponent_above_tcrit}')
    print(f'Error in Onsager critical amplitude below tcrit is␣
↪{error_negative_onsager_critical_exponent_below_tcrit}')
```

```
    print(f'Error in Onsager critical amplitude for the average of both␣
↪temperature regimes is {error_average_negative_onsager_critical_exponent}')
    print('\n')

    # Result for beta
    print('*** Results for exponent beta ***')
    print(f'The calculated value of beta below tcrit is␣
↪{gradient_magnetisation_below_tcrit}')
    print('\n')

    # Error for beta
    print('*** Error for exponent beta ***')
    print(f'Error in beta for below tcrit is {error_beta}')
    print('\n')

    # Results for gamma
    print('*** Results for exponent gamma ***')
    print(f'The calculated value of gamma above tcrit is␣
↪{gradient_magnetic_susceptibility_above_tcrit}.')
    print(f'The calculated value of gamma below tcrit is␣
↪{gradient_magnetic_susceptibility_below_tcrit}.')
    print(f'The calculated value of gamma using the average of the two␣
↪temperature regimes is {average_gamma}.')
    print('\n')

    # Errors for gamma in both temperature regimes and the average value
    print('*** Errors for exponent gamma ***')
    print(f'Error in gamma above tcrit is {error_gamma_above_tcrit}')
    print(f'Error in gamma below tcrit is {error_gamma_below_tcrit}')
    print(f'Error in gamma for the average of both temperature regimes is␣
↪{error_average_gamma}')
    print('\n')

    # This will be used on the validation plots to conserve memory
    if plot_mode == 'close':
        plt.close('all')

    # If no mode is defined, force the plots to display
    elif plot_mode == None:
        plt.show('all')

    return fig, ax, fig0, ax0, fig1, ax1, fig2, ax2, fig3, ax3, fig4, ax4,␣
↪fig5, ax5, fig6, ax6
```

20 Function for running parallelised critical temperature and exponents investigation using the wolff algorithm

[19]:
```
def run_parallelised_temp_change_investigation_simulations_WMC(parameter_file,␣
↪**kwargs):

    """Calls the run_single_temp_investigation_simulation(args) function N␣
↪cores times per instance and packages the results up into lists.
    Once enough instances have been called that the full temperature sweep has␣
↪been completed, the lists of mean magnetisations,
    mean magnetic susceptibilities and mean heat capacities are converted into␣
↪arrays and used for critical temperature and critical
    exponent calculations.

    In order to run the run_single_temp_investigation_simulation(args) function␣
↪in parallel, it uses all available cpu cores and the pool module
    from multiprocessing package.

    Returns plots of magnetisations and magnetic susceptibility vs temperatures.
↪"""

    # Getting the plot mode - condition to show / hide the diagnostic plots
    plot_mode = kwargs.get('plot_mode', None)

    # Setting up ICS and the lattice from the file
    lattice_side_length, magnetic_field_B, initial_temperature, temp_step,␣
↪exchange_constant_J, k_b, u_b, nsteps, nruns, filename =␣
↪read_tcrit_investigative_file(parameter_file)

    # Making a list to store temperatures
    run_temperatures_list = []

    # Appending to the list using temp step and the amount of runs
    for i in range(nruns):

        run_temperatures_list.append(initial_temperature + i*temp_step)

    # Turning the list into an array as this will be useful later
    run_temperatures_array = np.asarray(run_temperatures_list)

    ## Setting up parallelised simulation runs ##

    # Runs for worker function defined in the file
    single_temp_run_args = [(temperature, lattice_side_length,␣
↪magnetic_field_B, exchange_constant_J, k_b, u_b, nsteps) for temperature in␣
↪run_temperatures_array]
```

```
    # Printing out initialisation message for parallelised simulations
    print(f'Parallelising {nruns} temperature investigation runs across␣
↪{n_cores} CPU cores using the wolff algorithm.')
    print(f'Temperature range: {run_temperatures_array[0]} -->␣
↪{run_temperatures_array[-1]}.')
    print(f'Per-simulation parameters: temp_step={temp_step}K,␣
↪lattice_side_length={lattice_side_length},␣
↪magnetic_field_B={magnetic_field_B},␣
↪exchange_constant_J={exchange_constant_J}, k_b={k_b}, u_b={u_b},␣
↪nsteps={nsteps}.')

    # Running multiple simulations in parallel
    with Pool(processes=os.cpu_count()) as pool:

        # Calling the imported parallelisable worker function to run multiple␣
↪simulations at the same time using multiprocessing
        # This completes the runs across multiple temperatures much faster than␣
↪using one core only
        per_run_mean_quantites = list(tqdm(pool.
↪imap(run_WMC_temp_investigation_worker, single_temp_run_args), total=nruns,␣
↪desc='Running parallelised wolff algorithm temperature sweep:'))

    # Unpacking results from the list generated using multiprocessing
    # This has to be done in the same order as the␣
↪run_single_temp_investigation_simulation(args) returns them in.
    per_run_mean_equilibrated_magnetisations_per_spin_array = np.
↪array([mean_results[0] for mean_results in per_run_mean_quantites])
    per_run_mean_equilibrated_magnetic_susceptibilities_per_spin_array = np.
↪array([mean_results[1] for mean_results in per_run_mean_quantites])
    per_run_mean_equilibrated_heat_capacities_per_spin_array = np.
↪array([mean_results[2] for mean_results in per_run_mean_quantites])

    ## Critical temperature extraction ##

    # Method 1: Steepest drop in absolute magnetisation - Midpoint of the two␣
↪temperatures between where this happens should yield phase transition␣
↪temperature.
    consecutive_magnetisation_diffs = np.
↪diff(per_run_mean_equilibrated_magnetisations_per_spin_array)
    largest_magnetisation_drop_index = np.
↪argmin(consecutive_magnetisation_diffs)

    # Average of 2 values before and after the index for the most negative drop␣
↪in magnetisation should yield the critical temperature
```

```
    tcrit_from_magnetisation =␣
↪(run_temperatures_array[largest_magnetisation_drop_index] +␣
↪run_temperatures_array[largest_magnetisation_drop_index+1])/2

    # Method 2: Index of the temperature where magnetic susceptibility peaks
    largest_magnetic_susceptibility_index = np.
↪argmax(per_run_mean_equilibrated_magnetic_susceptibilities_per_spin_array)
    tcrit_from_magnetic_susceptibility =␣
↪run_temperatures_array[largest_magnetic_susceptibility_index]

    # Method 3: Index of temperature where heat capacity peaks
    largest_heat_capacity_index = np.
↪argmax(per_run_mean_equilibrated_heat_capacities_per_spin_array)
    tcrit_from_heat_capacity =␣
↪run_temperatures_array[largest_heat_capacity_index]

    # Method 3: Average of methods 1, 2 and 3 - Most statistical accuracy
    average_tcrit = (tcrit_from_magnetisation +␣
↪tcrit_from_magnetic_susceptibility + tcrit_from_heat_capacity)/3

    ## Storage of Onsager's calculated critical temperature and critical␣
↪exponents for comparison and error calculations ##

    # Onsager 1944 critical temperature
    tcrit_onsager = 2.269185

    # Storing Onsager 1944 critical exponents

    # Alpha critical exponent
    negative_onsager_critical_exponent = -0.4945

    # Beta critical exponent
    onsager_beta = 1/8

    # Gamma critical exponent
    onsager_gamma = -7/4

    ## Temperature regime filters ##

    # Above tcrit filter with 1% safety threshold.
    above_tcrit_filter = run_temperatures_array > average_tcrit*1.001

    # Below tcrit filter with 1% safety threshold.
    below_tcrit_filter = run_temperatures_array < average_tcrit*0.99
```

```
    ## Using reduced temperatures because they are dimensionless and thus ideal␣
↪for power law scaling ##

    # Reduced temperatures using data above tcrit
    absolute_reduced_temperatures_above_tcrit =␣
↪(run_temperatures_array[above_tcrit_filter] - average_tcrit) / average_tcrit

    # Reduced temperatures using data below tcrit
    absolute_reduced_temperatures_below_tcrit =␣
↪(run_temperatures_array[below_tcrit_filter] - average_tcrit) / average_tcrit

    ## Heat capacity, magnetisation and susceptibility filtering ##

    # Separate heat capacities above and below tcrit
    heat_capacities_above_tcrit =␣
↪per_run_mean_equilibrated_heat_capacities_per_spin_array[above_tcrit_filter]
    heat_capacities_below_tcrit =␣
↪per_run_mean_equilibrated_heat_capacities_per_spin_array[below_tcrit_filter]

    # Separate magnetisations below tcrit. Only applies below tcrit
    magnetisations_below_tcrit =␣
↪per_run_mean_equilibrated_magnetisations_per_spin_array[below_tcrit_filter]

    # Separate magnetic susceptibilities above and below tcrit
    magnetic_susceptibilities_above_tcrit =␣
↪per_run_mean_equilibrated_magnetic_susceptibilities_per_spin_array[above_tcrit_filter]
    magnetic_susceptibilities_below_tcrit =␣
↪per_run_mean_equilibrated_magnetic_susceptibilities_per_spin_array[below_tcrit_filter]

    ## Computing natural logarithms of absolute reduced temperatures above and␣
↪below tcrit ##

    # np.abs is used to prevent the logarithm of a negative quantity from being␣
↪computed.
    ln_absolute_reduced_temperatures_above_tcrit = np.log(np.
↪abs(absolute_reduced_temperatures_above_tcrit))
    ln_absolute_reduced_temperatures_below_tcrit = np.log(np.
↪abs(absolute_reduced_temperatures_below_tcrit))

    ## Computing natural logarithm of magnetisation below tcrit

    # np.abs is used to prevent the logarithm of a negative quantity from being␣
↪computed.
    ln_magnetisations_below_tcrit = np.log(np.abs(magnetisations_below_tcrit))
```

```
   ## Computing natural logarithms of magnetic susceptibilities above and␣
↪below tcrit ##

   # np.abs is used to prevent the logarithm of a negative quantity from being␣
↪computed.
   ln_magnetic_susceptibilities_above_tcrit = np.log(np.
↪abs(magnetic_susceptibilities_above_tcrit))
   ln_magnetic_susceptibilities_below_tcrit = np.log(np.
↪abs(magnetic_susceptibilities_below_tcrit))

   ## Computing the 5 different gradients ##

   # Gradient of ln heat capacity vs ln reduced temperatures above and below␣
↪tcrit
   gradient_heat_capacity_above_tcrit,␣
↪error_gradient_heat_capacity_above_tcrit,␣
↪heat_capacity_intercept_above_tcrit,␣
↪error_heat_capacity_intercept_above_tcrit =␣
↪calculate_linear_regress_gradient_fit(ln_absolute_reduced_temperatures_above_tcrit,␣
↪heat_capacities_above_tcrit)
   gradient_heat_capacity_below_tcrit,␣
↪error_gradient_heat_capacity_below_tcrit,␣
↪heat_capacity_intercept_below_tcrit,␣
↪error_heat_capacity_intercept_below_tcrit =␣
↪calculate_linear_regress_gradient_fit(ln_absolute_reduced_temperatures_below_tcrit,␣
↪heat_capacities_below_tcrit)

   # Gradient of ln magnetisation vs ln reduced temperatures above and below␣
↪tcrit
   gradient_magnetisation_below_tcrit,␣
↪error_gradient_heat_capacity_below_tcrit,␣
↪magnetisation_intercept_below_tcrit,␣
↪error_magnetisation_intercept_below_tcrit =␣
↪calculate_linear_regress_gradient_fit(ln_absolute_reduced_temperatures_below_tcrit,␣
↪ln_magnetisations_below_tcrit)

   # Gradient of ln magnetic susceptibility vs ln reduced temperatures above␣
↪and below tcrit
   gradient_magnetic_susceptibility_above_tcrit,␣
↪error_gradient_magnetic_susceptibility_above_tcrit,␣
↪magnetic_susceptibility_intercept_above_tcrit,␣
↪error_magnetic_susceptibility_intercept_above_tcrit =␣
↪calculate_linear_regress_gradient_fit(ln_absolute_reduced_temperatures_above_tcrit,␣
↪ln_magnetic_susceptibilities_above_tcrit)
```

```
    gradient_magnetic_susceptibility_below_tcrit,␣
↪error_gradient_magnetic_susceptibility_below_tcrit,␣
↪magnetic_susceptibility_intercept_below_tcrit,␣
↪error_magnetic_susceptibility_intercept_below_tcrit =␣
↪calculate_linear_regress_gradient_fit(ln_absolute_reduced_temperatures_below_tcrit,␣
↪ln_magnetic_susceptibilities_below_tcrit)

    ### Making plots ###

    # Generating array of x coordinates for plotting straight line for above␣
↪tcrit
    x_temperatures_straightline_above_tcrit = np.linspace(np.
↪min(ln_absolute_reduced_temperatures_above_tcrit), np.
↪max(ln_absolute_reduced_temperatures_above_tcrit), num=1001)

    # Generating array of x coordinates for plotting straight line for below␣
↪tcrit
    x_temperatures_straightline_below_tcrit = np.linspace(np.
↪min(ln_absolute_reduced_temperatures_below_tcrit), np.
↪max(ln_absolute_reduced_temperatures_below_tcrit), num=1001)

    ## Critical temperates overplotted on graphs of temperature vs␣
↪magnetisation ##

    # Making plot of temperatures vs magnetisation - axvlines for the three␣
↪methods and the onsager critical temperature
    fig, ax = plt.subplots(figsize=(6, 4), dpi=300)
    ax.plot(run_temperatures_array,␣
↪per_run_mean_equilibrated_magnetisations_per_spin_array, '-x')
    ax.axvline(x=tcrit_from_magnetisation, color='red', linestyle='--',␣
↪label='tcrit magnetisation')
    ax.axvline(x=tcrit_from_magnetic_susceptibility, color='green',␣
↪linestyle='--', label='tcrit magnetic susceptibility')
    ax.axvline(x=tcrit_from_heat_capacity, color='orange', linestyle='--',␣
↪label='tcrit heat capacity')
    ax.axvline(x=average_tcrit, color='blue', linestyle='--', label='tcrit␣
↪magnetisation, susceptibility and heat capacity average')
    ax.axvline(x=tcrit_onsager, color='grey', linestyle='--', label='tcrit␣
↪Onsager 1944')
    ax.legend(loc='upper right')
    ax.grid(ls=':', color='0.9')
    ax.set_title('Magnetisation per spin vs temperature WMC')
    ax.set_xlabel('Temperature')
    ax.set_ylabel('Magnetisation per spin')

    # Saving figure
```

```
    fig.savefig(f'Temperature vs magnetisation, filename={filename}, WMC.png',␣
↪dpi=300)

    # Making plot of temperatures vs heat capacity - axvlines for the three␣
↪methods and the onsager critical temperature
    fig0, ax0 = plt.subplots(figsize=(6, 4), dpi=300)
    ax0.plot(run_temperatures_array,␣
↪per_run_mean_equilibrated_heat_capacities_per_spin_array, '-x')
    ax0.axvline(x=tcrit_from_magnetisation, color='red', linestyle='--',␣
↪label='tcrit magnetisation')
    ax0.axvline(x=tcrit_from_magnetic_susceptibility, color='green',␣
↪linestyle='--', label='tcrit magnetic susceptibility')
    ax0.axvline(x=tcrit_from_heat_capacity, color='orange', linestyle='--',␣
↪label='tcrit heat capacity')
    ax0.axvline(x=average_tcrit, color='blue', linestyle='--', label='tcrit␣
↪magnetisation, susceptibility and heat capacity average')
    ax0.axvline(x=tcrit_onsager, color='grey', linestyle='--', label='tcrit␣
↪Onsager 1944')
    ax0.legend(loc='upper right')
    ax0.grid(ls=':', color='0.9')
    ax0.set_title('Heat capacity per spin vs temperature WMC')
    ax0.set_xlabel('Temperature')
    ax0.set_ylabel('Heat capacity per spin')

    # Saving figure
    fig0.savefig(f'Temperature vs heat capacity, filename={filename}, WMC.png',␣
↪dpi=300)

    # Making plot of temperature vs magnetic susceptibility axvlines for the␣
↪three methods and the onsager critical temperature
    fig1, ax1 = plt.subplots(figsize=(6, 4), dpi=300)
    ax1.plot(run_temperatures_array,␣
↪per_run_mean_equilibrated_magnetic_susceptibilities_per_spin_array, '-x')
    ax1.axvline(x=tcrit_from_magnetisation, color='red', linestyle='--',␣
↪label='tcrit magnetisation')
    ax1.axvline(x=tcrit_from_magnetic_susceptibility, color='green',␣
↪linestyle='--', label='tcrit magnetic susceptibility')
    ax1.axvline(x=tcrit_from_heat_capacity, color='orange', linestyle='--',␣
↪label='tcrit heat capacity')
    ax1.axvline(x=average_tcrit, color='blue', linestyle='--', label='tcrit␣
↪magnetisation, susceptibility and heat capacity average')
    ax1.axvline(x=tcrit_onsager, color='grey', linestyle='--', label='tcrit␣
↪Onsager 1944')
    ax1.legend(loc='upper right')
    ax1.grid(ls=':', color='0.9')
    ax1.set_title('Magnetic susceptibility per spin vs temperature WMC')
```

```
    ax1.set_xlabel('Temperature')
    ax1.set_ylabel('Magnetic susceptibility per spin')

    # Saving figure
    fig1.savefig(f'Temperature vs magnetic susceptibility, filename={filename},␣
↪WMC.png', dpi=300)

    ## Error calculations for critical temperature ##

    # Getting absolute errors from the onsager critical temperature
    error_tcrit_magnetisation = np.abs(tcrit_onsager-tcrit_from_magnetisation)
    error_tcrit_magnetic_susceptibility = np.
↪abs(tcrit_onsager-tcrit_from_magnetic_susceptibility)
    error_tcrit_heat_capacity = np.abs(tcrit_onsager-tcrit_from_heat_capacity)
    error_tcrit_average = np.abs(tcrit_onsager-average_tcrit)

    ## 5 Plots for the Onsager exponents in the valid regions - Corresponds to␣
↪the 5 gradients ##

    # 1. Gradient plot for critical exponent alpha: ln(t - tcrit) vs␣
↪ln(heat_capacity) above tcrit
    fig2, ax2 = plt.subplots(figsize=(6, 4), dpi=300)
    ax2.plot(ln_absolute_reduced_temperatures_above_tcrit,␣
↪heat_capacities_above_tcrit, '-x')
    ax2.plot(x_temperatures_straightline_above_tcrit,␣
↪straight_line(x_temperatures_straightline_above_tcrit,␣
↪gradient_heat_capacity_above_tcrit, heat_capacity_intercept_above_tcrit),␣
↪linestyle='--', color='black', label='Calculated Onsager Critical Amplitude␣
↪above tcrit')
    ax2.plot(x_temperatures_straightline_above_tcrit,␣
↪straight_line(x_temperatures_straightline_above_tcrit,␣
↪negative_onsager_critical_exponent, heat_capacity_intercept_above_tcrit),␣
↪linestyle='--', color='red', label='Onsager Critical Exponent')
    ax2.legend(loc='upper right')
    ax2.grid(ls=':', color='0.9')
    ax2.set_title('Onsager critical amplitude above tcrit WMC')
    ax2.set_xlabel('ln(|temps_above_tcrit - tcrit_average|)')
    ax2.set_ylabel('Heat Capacity')

    # Saving figure
    fig2.savefig(f'Onsager critical exponent above tcrit, filename={filename},␣
↪WMC.png', dpi=300)

    # 2. Gradient plot for critical exponent alpha: ln(t - tcrit) vs␣
↪ln(heat_capacity) below tcrit
    fig3, ax3 = plt.subplots(figsize=(6, 4), dpi=300)
```

```
    ax3.plot(ln_absolute_reduced_temperatures_below_tcrit,␣
↪heat_capacities_below_tcrit, '-x')
    ax3.plot(x_temperatures_straightline_below_tcrit,␣
↪straight_line(x_temperatures_straightline_below_tcrit,␣
↪gradient_heat_capacity_below_tcrit, heat_capacity_intercept_below_tcrit),␣
↪linestyle='--', color='black', label='Calculated Onsager Critical Amplitude␣
↪below tcrit')
    ax3.plot(x_temperatures_straightline_below_tcrit,␣
↪straight_line(x_temperatures_straightline_below_tcrit,␣
↪negative_onsager_critical_exponent, heat_capacity_intercept_below_tcrit),␣
↪linestyle='--', color='red', label='Onsager Critical Exponent')
    ax3.legend(loc='upper right')
    ax3.grid(ls=':', color='0.9')
    ax3.set_title('Onsager critical amplitude below tcrit WMC')
    ax3.set_xlabel('ln(|temps_below_tcrit - tcrit_average|)')
    ax3.set_ylabel('Heat Capacity')

    # Saving figure
    fig3.savefig(f'Onsager critical exponent below tcrit, filename={filename},␣
↪WMC.png', dpi=300)

    # 3. Gradient plot for critical exponent beta: ln(t - tcrit) vs␣
↪ln(magnetisation) below tcrit
    fig4, ax4 = plt.subplots(figsize=(6, 4), dpi=300)
    ax4.plot(ln_absolute_reduced_temperatures_below_tcrit,␣
↪ln_magnetisations_below_tcrit, '-x')
    ax4.plot(x_temperatures_straightline_below_tcrit,␣
↪straight_line(x_temperatures_straightline_below_tcrit,␣
↪gradient_magnetisation_below_tcrit, magnetisation_intercept_below_tcrit),␣
↪linestyle='--', color='black', label='Calculated Beta below tcrit')
    ax4.plot(x_temperatures_straightline_below_tcrit,␣
↪straight_line(x_temperatures_straightline_below_tcrit, onsager_beta,␣
↪magnetisation_intercept_below_tcrit), linestyle='--', color='red',␣
↪label='Onsager Beta')
    ax4.legend(loc='upper right')
    ax4.grid(ls=':', color='0.9')
    ax4.set_title('Beta below tcrit WMC')
    ax4.set_xlabel('ln(|temps_below_tcrit - tcrit_average|)')
    ax4.set_ylabel('ln(|magnetisation per spin|)')

    # Saving figure
    fig4.savefig(f'Beta below tcrit, filename={filename}, WMC.png', dpi=300)

    # 4. Gradient plot for critical exponent Gamma: ln(t - tcrit) vs␣
↪ln(heat_capacity) above tcrit
    fig5, ax5 = plt.subplots(figsize=(6, 4), dpi=300)
```

```
    ax5.plot(ln_absolute_reduced_temperatures_above_tcrit,␣
↪ln_magnetic_susceptibilities_above_tcrit, '-x')
    ax5.plot(x_temperatures_straightline_above_tcrit,␣
↪straight_line(x_temperatures_straightline_above_tcrit,␣
↪gradient_magnetic_susceptibility_above_tcrit,␣
↪magnetic_susceptibility_intercept_above_tcrit), linestyle='--',␣
↪color='black', label='Calculated Gamma above tcrit')
    ax5.plot(x_temperatures_straightline_above_tcrit,␣
↪straight_line(x_temperatures_straightline_above_tcrit, onsager_gamma,␣
↪magnetic_susceptibility_intercept_above_tcrit), linestyle='--', color='red',␣
↪label='Onsager Gamma')
    ax5.legend(loc='upper right')
    ax5.grid(ls=':', color='0.9')
    ax5.set_title('Gamma above tcrit WMC')
    ax5.set_xlabel('ln(|temps_above_tcrit - tcrit_average|)')
    ax5.set_ylabel('ln(|magnetic susceptibility per spin|)')

    # Saving figure
    fig5.savefig(f'Gamma above tcrit, filename={filename}, WMC.png', dpi=300)

    # 5. Gradient plot for critical exponent Gamma: ln(t - tcrit) vs␣
↪ln(heat_capacity) below tcrit
    fig6, ax6 = plt.subplots(figsize=(6, 4), dpi=300)
    ax6.plot(ln_absolute_reduced_temperatures_below_tcrit,␣
↪ln_magnetic_susceptibilities_below_tcrit, '-x')
    ax6.plot(x_temperatures_straightline_below_tcrit,␣
↪straight_line(x_temperatures_straightline_below_tcrit,␣
↪gradient_magnetic_susceptibility_below_tcrit,␣
↪magnetic_susceptibility_intercept_below_tcrit), linestyle='--',␣
↪color='black', label='Calculated Gamma below tcrit')
    ax6.plot(x_temperatures_straightline_below_tcrit,␣
↪straight_line(x_temperatures_straightline_below_tcrit, onsager_gamma,␣
↪magnetic_susceptibility_intercept_below_tcrit), linestyle='--', color='red',␣
↪label='Onsager Gamma')
    ax6.legend(loc='upper right')
    ax6.grid(ls=':', color='0.9')
    ax6.set_title('Gamma below tcrit WMC')
    ax6.set_xlabel('ln(|temps_below_tcrit - tcrit_average|)')
    ax6.set_ylabel('ln(|magnetic susceptibility per spin|)')

    # Saving figure
    fig6.savefig(f'Gamma below tcrit, filename={filename}, WMC.png', dpi=300)

    ## Calculating average of the two gradients in each safety region for alpha␣
↪and gamma. Beta doesnt need it as it is only valid below tcrit ##
```

```
    average_negative_onsager_critical_exponent =␣
↪(gradient_heat_capacity_above_tcrit + gradient_heat_capacity_below_tcrit) / 2
    average_gamma = (gradient_magnetic_susceptibility_above_tcrit +␣
↪gradient_magnetic_susceptibility_below_tcrit) / 2

    ## Calculating errors in critical exponents for above and below tcrit ##

    # Errors for alpha in both regimes and the average
    error_negative_onsager_critical_exponent_above_tcrit = np.
↪abs(negative_onsager_critical_exponent - gradient_heat_capacity_above_tcrit)
    error_negative_onsager_critical_exponent_below_tcrit = np.
↪abs(negative_onsager_critical_exponent - gradient_heat_capacity_below_tcrit)
    error_average_negative_onsager_critical_exponent = np.
↪abs(negative_onsager_critical_exponent -␣
↪average_negative_onsager_critical_exponent)

    # Error in beta below tcrit
    error_beta = np.abs(onsager_beta-gradient_magnetisation_below_tcrit)

    # Errors for alpha in both regimes and the average
    error_gamma_above_tcrit = np.abs(onsager_gamma -␣
↪gradient_magnetic_susceptibility_above_tcrit)
    error_gamma_below_tcrit = np.abs(onsager_gamma -␣
↪gradient_magnetic_susceptibility_below_tcrit)
    error_average_gamma = np.abs(onsager_gamma - average_gamma)

    ## Printing out critical temperature results so it is easy to access them ##

    # Critical temperature results
    print('*** Critical Temperature Results ***')
    print(f'The Onsager 1944 result for the critical temperature of an infinite␣
↪lattice is {tcrit_onsager}K.')
    print(f'Calculated critical temperature from magnetisation drop using the␣
↪wolff algorithm is {tcrit_from_magnetisation}K.')
    print(f'Calculated critical temperature from magnetic susceptibility peak␣
↪using the wolff algorithm is {tcrit_from_magnetic_susceptibility}K.')
    print(f'Calculated critical temperature from heat capacity peak using the␣
↪wolff algorithm is {tcrit_from_heat_capacity}K.')
    print(f'Average from the three methods for calculating critical temperature␣
↪using the wolff algoritm is {average_tcrit}K.')
    print('\n')

    print('*** Critical Temperature Errors ***')
    print(f'Error in critical temperature for the magnetisation drop method␣
↪using the wolff algorithm is {error_tcrit_magnetisation}K.')
```

```
    print(f'Error in critical temperature for the magnetic susceptibility peak␣
↪method using the wolff algorithm is {error_tcrit_magnetic_susceptibility}K.')
    print(f'Error in critical temperature from heat capacity peak using the␣
↪wolff algorithm is {error_tcrit_heat_capacity}K')
    print(f'Error in critical temperature for the average between the three␣
↪methods using the wolff algorithm is {error_tcrit_average}.')
    print('\n')

    ## Printing out critical exponent results and errors so it is easy to␣
↪access them ##

    # Onsager values for critical exponents
    print('*** Onsager Critical Exponents ***')
    print(f'The Onsager 1944 result for the critical amplitude {np.
↪abs(negative_onsager_critical_exponent)}.')
    print('The Onsager 1944 result for the critical exponent alpha is 0')
    print(f'The Onsager 1944 result for the critical exponent beta is␣
↪{onsager_beta}.')
    print(f'The Onsager 1944 result for the critical exponent gamma is␣
↪{onsager_gamma}.')
    print('\n')

    # Results for Onsager critical amplitude
    print('Since alpha is 0, the best way to verify it is a linear plot of␣
↪ln(T) vs the heat capacity.')
    print('If the simulation is working correctly and alpha is behaving as␣
↪Onsager predicted, then the plot will be linear and have a gradient of -0.
↪4945, which is the negative of the Onsager critical amplitude.')
    print('\n')
    print('*** Results for Onsager Critical Amplitude ***')
    print(f'The calculated value of the negative Onsager critical amplitude␣
↪above tcrit is {gradient_heat_capacity_above_tcrit}.')
    print(f'The calculated value of the negative Onsager critical amplitude␣
↪below tcrit is {gradient_heat_capacity_below_tcrit}.')
    print(f'The calculated value of the negative Onsager critical amplitude␣
↪using the average of the two temperature regimes is␣
↪{average_negative_onsager_critical_exponent}.')
    print('\n')

    # Errors for alpha in both temperature regimes and the average value
    print('*** Errors for Onsager critical amplitude ***')
    print(f'Error in Onsager critical amplitude above tcrit is␣
↪{error_negative_onsager_critical_exponent_above_tcrit}')
    print(f'Error in Onsager critical amplitude below tcrit is␣
↪{error_negative_onsager_critical_exponent_below_tcrit}')
```

```
    print(f'Error in Onsager critical amplitude for the average of both␣
↪temperature regimes is {error_average_negative_onsager_critical_exponent}')
    print('\n')

    # Result for beta
    print('*** Results for exponent beta ***')
    print(f'The calculated value of beta below tcrit is␣
↪{gradient_magnetisation_below_tcrit}')
    print('\n')

    # Error for beta
    print('*** Error for exponent beta ***')
    print(f'Error in beta for below tcrit is {error_beta}')
    print('\n')

    # Results for gamma
    print('*** Results for exponent gamma ***')
    print(f'The calculated value of gamma above tcrit is␣
↪{gradient_magnetic_susceptibility_above_tcrit}.')
    print(f'The calculated value of gamma below tcrit is␣
↪{gradient_magnetic_susceptibility_below_tcrit}.')
    print(f'The calculated value of gamma using the average of the two␣
↪temperature regimes is {average_gamma}.')
    print('\n')

    # Errors for gamma in both temperature regimes and the average value
    print('*** Errors for exponent gamma ***')
    print(f'Error in gamma above tcrit is {error_gamma_above_tcrit}')
    print(f'Error in gamma below tcrit is {error_gamma_below_tcrit}')
    print(f'Error in gamma for the average of both temperature regimes is␣
↪{error_average_gamma}')
    print('\n')

    # This will be used on the validation plots to conserve memory
    if plot_mode == 'close':
        plt.close('all')

    # If no mode is defined, force the plots to display
    elif plot_mode == None:
        plt.show('all')

    return fig, ax, fig0, ax0, fig1, ax1, fig2, ax2, fig3, ax3, fig4, ax4,␣
↪fig5, ax5, fig6, ax6
```

21 Function to run parallelised lattice side length change investigation (calculating the Onsager critical exponent from lattice size changes) using the metropolis algorithm

[20]:
```
def␣
↪run_parallelised_lattice_side_length_change_investigation_simulations_MMC(parameter_file,␣
↪**kwargs):

    """Calls the run_single_temp_investigation_simulation(args) function N␣
↪cores times per instance and packages the results up into lists.
    Once enough instances have been called that the full temperature sweep has␣
↪been completed, the lists of median magnetisations,
    median magnetic susceptibilities and median heat capacities are converted␣
↪into arrays and used for critical temperature and critical
    exponent calculations.

    In order to run the run_single_temp_investigation_simulation(args) function␣
↪in parallel, it uses all available cpu cores and the pool module
    from multiprocessing package.

    Returns plots of magnetisations and magnetic susceptibility vs temperatures.
↪"""

    # Getting the plot mode - condition to show / hide the diagnostic plots
    plot_mode = kwargs.get('plot_mode', None)

    # Setting up ICS and the lattice from the file
    initial_lattice_side_length, magnetic_field_B, temperature,␣
↪lattice_side_length_step, exchange_constant_J, k_b, u_b, nsteps, nruns,␣
↪filename = read_lattice_side_length_investigative_file(parameter_file)

    # Making a list to store temperatures
    run_lattice_side_lenths_list = []

    # Appending to the list using lattice side length step and the amount of␣
↪runs
    for i in range(nruns):

        run_lattice_side_lenths_list.append(initial_lattice_side_length +␣
↪i*lattice_side_length_step)

    # Turning the list into an array as this will be useful later
    run_lattice_side_lengths_array = np.asarray(run_lattice_side_lenths_list)

    ## Setting up parallelised simulation runs ##
```

```
    # Runs for worker function defined in the file
    single_lattice_size_run_args = [(temperature, lattice_side_length,␣
↪magnetic_field_B, exchange_constant_J, k_b, u_b, nsteps) for␣
↪lattice_side_length in run_lattice_side_lengths_array]

    # Printing out initialisation message for parallelised simulations
    print(f'Parallelising {nruns} lattice size runs across {n_cores} CPU cores␣
↪using the metropolis algorithm.')
    print(f'Lattice size range: {(run_lattice_side_lengths_array[0])**2} -->␣
↪{(run_lattice_side_lengths_array[-1])**2}.')
    print(f'Per-simulation parameters: temperature={temperature}, lattice side␣
↪length step={lattice_side_length_step}, magnetic_field_B={magnetic_field_B},␣
↪exchange_constant_J={exchange_constant_J}, k_b={k_b}, u_b={u_b},␣
↪nsteps={nsteps}')

    # Running multiple simulations in parallel
    with Pool(processes=os.cpu_count()) as pool:

        # Calling the imported parallelisable worker function to run multiple␣
↪simulations at the same time using multiprocessing
        # This completes the runs across multiple temperatures much faster than␣
↪using one core only
        per_run_median_quantites = list(tqdm(pool.
↪imap(run_MMC_lattice_side_length_investigation_worker,␣
↪single_lattice_size_run_args), total=nruns, desc='Running parallelised␣
↪metropolis algorithm lattice size sweep:'))

    # Unpacking results from the list generated using multiprocessing
    # This has to be done in the same order as the␣
↪run_single_temp_investigation_simulation(args) returns them in.
    # No need for indexing because the worker function for lattice side length␣
↪only returns one quantity - The heat capacity
    per_run_median_equilibrated_heat_capacities_per_spin_array = np.
↪array([median_results for median_results in per_run_median_quantites])

    # Computing the lattice sizes
    lattice_size_array = np.square(run_lattice_side_lengths_array)

    # Computing the per element logairthm of the lattice sizes
    ln_lattice_size_array = np.log(lattice_size_array)

    # Calculating C_0
    gradient_C_0, error_gradient_C_0, C_0_intercept, error_C_0_intercept =␣
↪calculate_linear_regress_gradient_fit(ln_lattice_size_array,␣
↪per_run_median_equilibrated_heat_capacities_per_spin_array)
```

```
    # Storing Onsager's 1944 value of C_0 - Half of the critical amplitude
    onsager_C_0 = 0.24725

    ## Error Calculation ##
    error_C_0 = np.abs(gradient_C_0 - onsager_C_0)

    # Generating array of x coordinates for plotting straight line
    x_lattice_sizes_straightline = np.linspace(np.min(ln_lattice_size_array),␣
↪np.max(ln_lattice_size_array), num=1001)

    # Printing out the results for the calculated C_0 and its error compared to␣
↪the onsager value.
    print('*** C_0 results ***')
    print(f'Calculated value of C_0 is {gradient_C_0}.')
    print(f'Onsager value of C_0 is {onsager_C_0}.')
    print(f'Error in calculated value of C_0 is {error_C_0}.')

    ## Making plot ##
    fig, ax = plt.subplots(figsize=(6,4), dpi=300)
    ax.plot(ln_lattice_size_array,␣
↪per_run_median_equilibrated_heat_capacities_per_spin_array, '-x')

    # Plotting with the calculated intercept
    ax.plot(x_lattice_sizes_straightline,␣
↪straight_line(x_lattice_sizes_straightline, gradient_C_0, C_0_intercept),␣
↪linestyle='--', color='black', label='Calculated C_0')
    ax.plot(x_lattice_sizes_straightline,␣
↪straight_line(x_lattice_sizes_straightline, onsager_C_0, C_0_intercept),␣
↪linestyle='--', color='red', label='Onsager C_0')
    ax.legend(loc='upper right')
    ax.grid(ls=':', color='0.9')
    ax.set_title('ln(Lattice size D) vs Heat Capacity MMC')
    ax.set_xlabel('ln(Lattice Size (side length^2)')
    ax.set_ylabel('Heat capacity per spin')

    # Saving figure
    fig.savefig(f'ln(Lattice size D) vs heat capacity per spin,␣
↪filename={filename}, MMC.png', dpi=300)

    return fig, ax
```

22 Function to run parallelised lattice side length change investigation (calculating the Onsager critical exponent from lattice size changes) using the wolff algorithm

[21]:
```
def␣
↪run_parallelised_lattice_side_length_change_investigation_simulations_WMC(parameter_file,␣
↪**kwargs):

    """Calls the run_single_temp_investigation_simulation(args) function N␣
↪cores times per instance and packages the results up into lists.
    Once enough instances have been called that the full temperature sweep has␣
↪been completed, the lists of median magnetisations,
    median magnetic susceptibilities and median heat capacities are converted␣
↪into arrays and used for critical temperature and critical
    exponent calculations.

    In order to run the run_single_temp_investigation_simulation(args) function␣
↪in parallel, it uses all available cpu cores and the pool module
    from multiprocessing package.

    Returns plots of magnetisations and magnetic susceptibility vs temperatures.
↪"""

    # Getting the plot mode - condition to show / hide the diagnostic plots
    plot_mode = kwargs.get('plot_mode', None)

    # Setting up ICS and the lattice from the file
    initial_lattice_side_length, magnetic_field_B, temperature,␣
↪lattice_side_length_step, exchange_constant_J, k_b, u_b, nsteps, nruns,␣
↪filename = read_lattice_side_length_investigative_file(parameter_file)

    # Making a list to store temperatures
    run_lattice_side_lenths_list = []

    # Appending to the list using lattice side length step and the amount of␣
↪runs
    for i in range(nruns):

        run_lattice_side_lenths_list.append(initial_lattice_side_length +␣
↪i*lattice_side_length_step)

    # Turning the list into an array as this will be useful later
    run_lattice_side_lengths_array = np.asarray(run_lattice_side_lenths_list)

    ## Setting up parallelised simulation runs ##
```

```
    # Runs for worker function defined in the file
    single_lattice_size_run_args = [(temperature, lattice_side_length,␣
↪magnetic_field_B, exchange_constant_J, k_b, u_b, nsteps) for␣
↪lattice_side_length in run_lattice_side_lengths_array]

    # Printing out initialisation message for parallelised simulations
    print(f'Parallelising {nruns} lattice size runs across {n_cores} CPU cores␣
↪using the wolff algorithm.')
    print(f'Lattice size range: {(run_lattice_side_lengths_array[0])**2} -->␣
↪{(run_lattice_side_lengths_array[-1])**2}.')
    print(f'Per-simulation parameters: temperature={temperature}, lattice side␣
↪length step={lattice_side_length_step}, magnetic_field_B={magnetic_field_B},␣
↪exchange_constant_J={exchange_constant_J}, k_b={k_b}, u_b={u_b},␣
↪nsteps={nsteps}')

    # Running multiple simulations in parallel
    with Pool(processes=os.cpu_count()) as pool:

        # Calling the imported parallelisable worker function to run multiple␣
↪simulations at the same time using multiprocessing
        # This completes the runs across multiple temperatures much faster than␣
↪using one core only
        per_run_median_quantites = list(tqdm(pool.
↪imap(run_WMC_lattice_side_length_investigation_worker,␣
↪single_lattice_size_run_args), total=nruns, desc='Running parallelised wolff␣
↪algorithm lattice size sweep:'))

    # Unpacking results from the list generated using multiprocessing
    # This has to be done in the same order as the␣
↪run_single_temp_investigation_simulation(args) returns them in.
    # No need for indexing because the worker function for lattice side length␣
↪only returns one quantity - The heat capacity
    per_run_median_equilibrated_heat_capacities_per_spin_array = np.
↪array([median_results for median_results in per_run_median_quantites])

    # Computing the lattice sizes
    lattice_size_array = np.square(run_lattice_side_lengths_array)

    # Computing the per element logairthm of the lattice sizes
    ln_lattice_size_array = np.log(lattice_size_array)

    # Calculating C_0
    gradient_C_0, error_gradient_C_0, C_0_intercept, error_C_0_intercept =␣
↪calculate_linear_regress_gradient_fit(ln_lattice_size_array,␣
↪per_run_median_equilibrated_heat_capacities_per_spin_array)
```

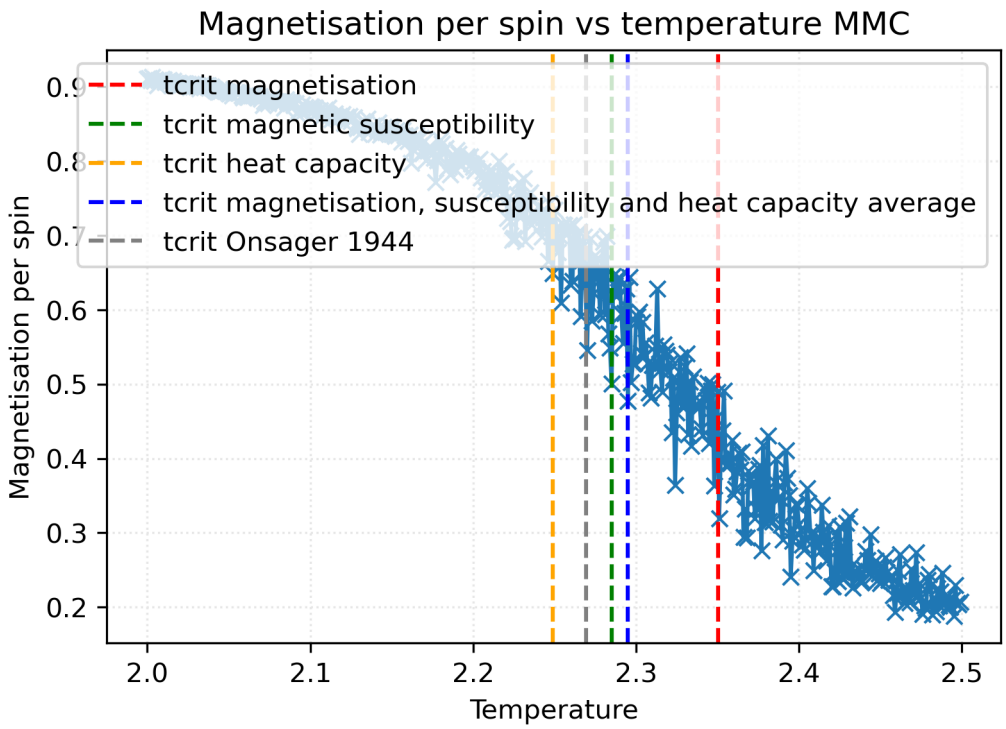
```
    # Storing Onsager's 1944 value of C_0 - Half of the critical amplitude
    onsager_C_0 = 0.24725

    ## Error Calculation ##
    error_C_0 = np.abs(gradient_C_0 - onsager_C_0)

    # Generating array of x coordinates for plotting straight line
    x_lattice_sizes_straightline = np.linspace(np.min(ln_lattice_size_array),␣
↪np.max(ln_lattice_size_array), num=1001)

    # Printing out the results for the calculated C_0 and its error compared to␣
↪the onsager value.
    print('*** C_0 results ***')
    print(f'Calculated value of C_0 is {gradient_C_0}.')
    print(f'Onsager value of C_0 is {onsager_C_0}.')
    print(f'Error in calculated value of C_0 is {error_C_0}.')

    ## Making plot ##
    fig, ax = plt.subplots(figsize=(6,4), dpi=300)
    ax.plot(ln_lattice_size_array,␣
↪per_run_median_equilibrated_heat_capacities_per_spin_array, '-x')

    # Plotting with the calculated intercept
    ax.plot(x_lattice_sizes_straightline,␣
↪straight_line(x_lattice_sizes_straightline, gradient_C_0, C_0_intercept),␣
↪linestyle='--', color='black', label='Calculated C_0')
    ax.plot(x_lattice_sizes_straightline,␣
↪straight_line(x_lattice_sizes_straightline, onsager_C_0, C_0_intercept),␣
↪linestyle='--', color='red', label='Onsager C_0')
    ax.legend(loc='upper right')
    ax.grid(ls=':', color='0.9')
    ax.set_title('ln(Lattice size D) vs Heat Capacity WMC')
    ax.set_xlabel('ln(Lattice Size (side length^2)')
    ax.set_ylabel('Heat capacity per spin')

    # Saving figure
    fig.savefig(f'ln(Lattice size D) vs heat capacity per spin,␣
↪filename={filename}, WMC.png', dpi=300)

    return fig, ax
```

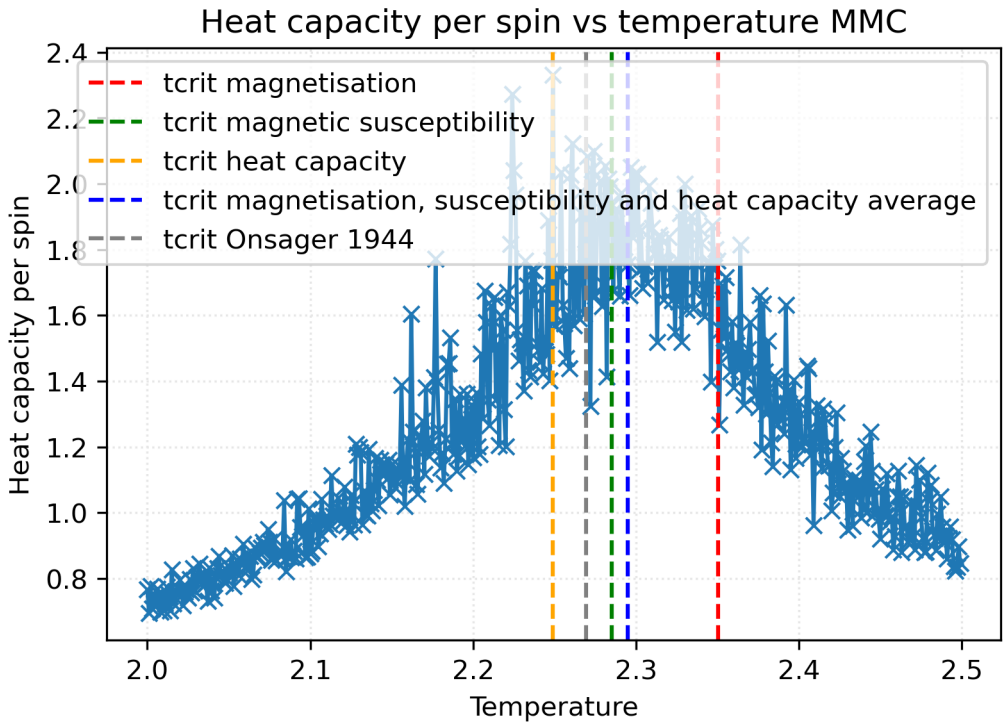
23 Validation run - critical temperature of analytical solution to the 2D ising model using the metropolis algorithm for a 32x32 lattice

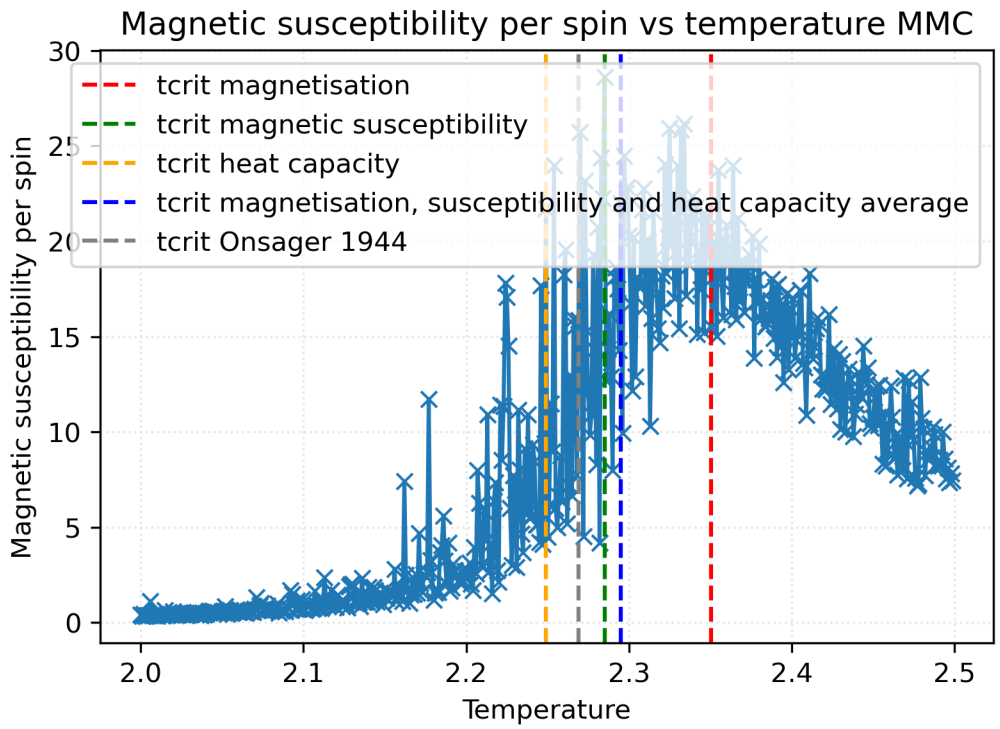
[22]:
```
parameter_file = 'ising_sim_side_len_32_temp_investigation_MMC_ICS.txt'
run_parallelised_temp_change_investigation_simulations_MMC(parameter_file)
```

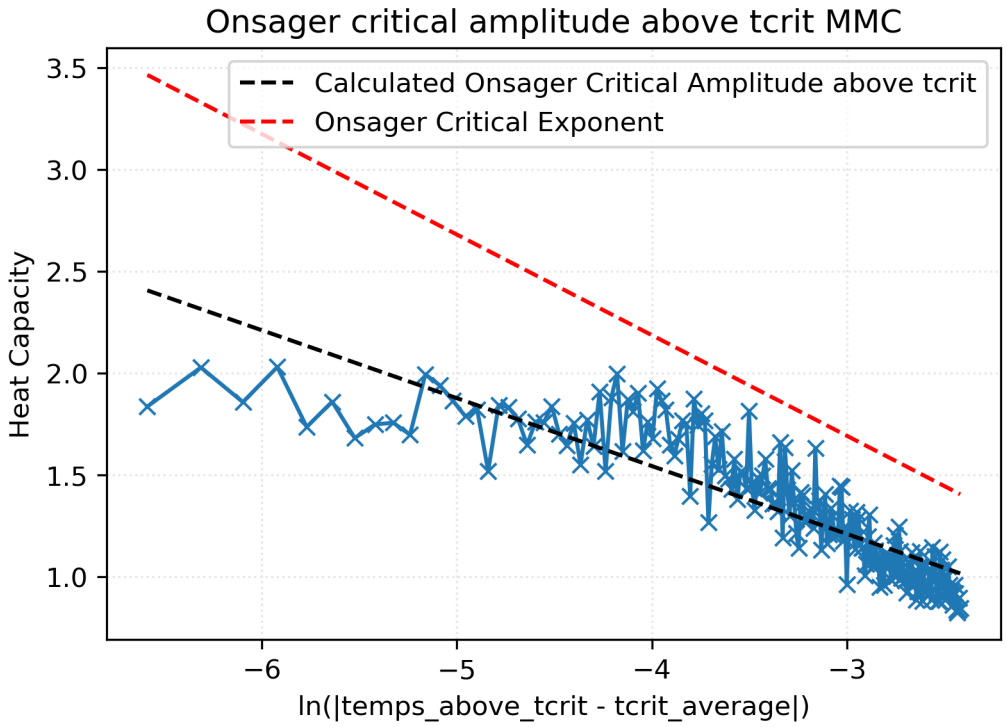
```
Parallelising 500 temperature investigation runs across 24 CPU cores using the
metropolis algorithm.
Temperature range: 2.0 --> 2.499.
Per-simulation parameters: temp_step=0.001K, lattice_side_length=32,
magnetic_field_B=0.0, exchange_constant_J=1.0, k_b=1.0, u_b=0.0,
nsteps=15000000.

Running parallelised metropolis algorithm temperature sweep::   0%|          | 0/
↪500 [00:00<?, ?it/s]

*** Critical Temperature Results ***
The Onsager 1944 result for the critical temperature of an infinite lattice is
2.269185K.
Calculated critical temperature from magnetisation drop using the metropolis
algorithm is 2.3505000000000003K.
Calculated critical temperature from magnetic susceptibility peak using the
metropolis algorithm is 2.285K.
Calculated critical temperature from heat capacity peak using the metropolis
algorithm is 2.249K.
Average from the three methods for calculating critical temperature using the
metropolis algoritm is 2.2948333333333335K.


*** Critical Temperature Errors ***
Error in critical temperature for the magnetisation drop method using the
metropolis algorithm is 0.08131500000000047K.
Error in critical temperature for the magnetic susceptibility peak method using
the metropolis algorithm is 0.015815000000000357K.
Error in critical temperature from heat capacity peak using the metropolis
algorithm is 0.020184999999999675K
Error in critical temperature for the average between the three methods using
the metropolis algorithm is 0.025648333333333717.


*** Onsager Critical Exponents ***
The Onsager 1944 result for the critical amplitude 0.4945.
The Onsager 1944 result for the critical exponent alpha is 0
The Onsager 1944 result for the critical exponent beta is 0.125.
The Onsager 1944 result for the critical exponent gamma is -1.75.
```

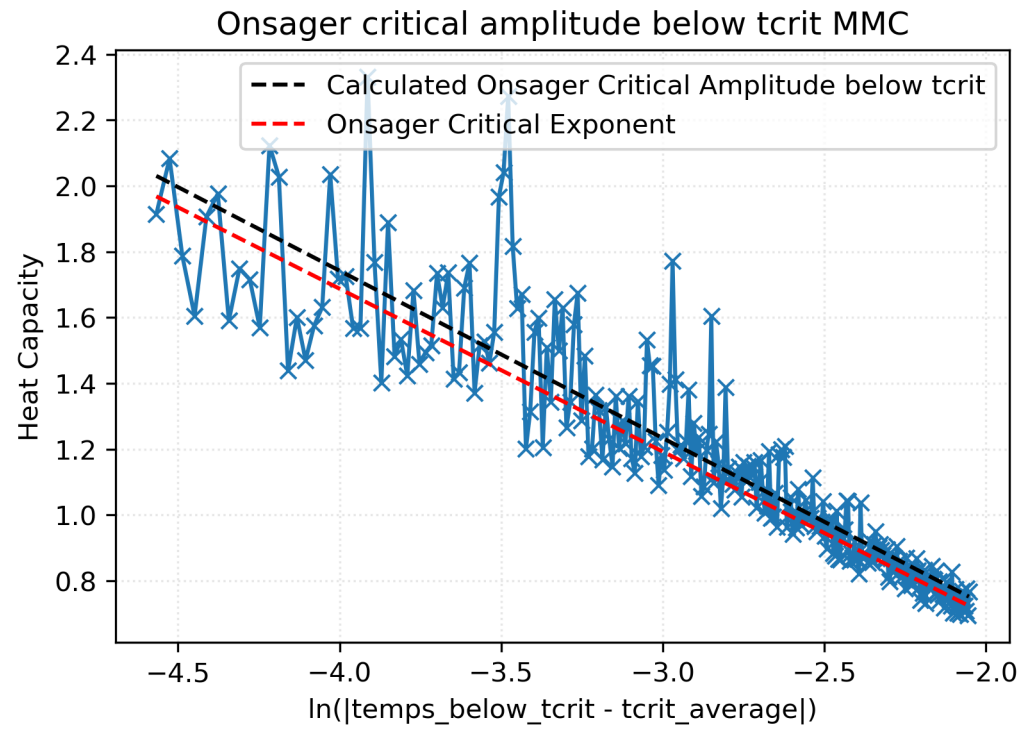
```
Since alpha is 0, the best way to verify it is a linear plot of ln(T) vs the
heat capacity.
If the simulation is working correctly and alpha is behaving as Onsager
predicted, then the plot will be linear and have a gradient of -0.4945, which is
the negative of the Onsager critical amplitude.


*** Results for Onsager Critical Amplitude ***
The calculated value of the negative Onsager critical amplitude above tcrit is
-0.3337481166326708.
The calculated value of the negative Onsager critical amplitude below tcrit is
-0.5081483832198712.
The calculated value of the negative Onsager critical amplitude using the
average of the two temperature regimes is -0.420948249926271.


*** Errors for Onsager critical amplitude ***
Error in Onsager critical amplitude above tcrit is 0.1607518833673292
Error in Onsager critical amplitude below tcrit is 0.013648383219871218
Error in Onsager critical amplitude for the average of both temperature regimes
is 0.073551750073729


*** Results for exponent beta ***
The calculated value of beta below tcrit is 0.14341203242408007


*** Error for exponent beta ***
Error in beta for below tcrit is 0.01841203242408007


*** Results for exponent gamma ***
The calculated value of gamma above tcrit is -0.25041265258481843.
The calculated value of gamma below tcrit is -1.5977198970727455.
The calculated value of gamma using the average of the two temperature regimes
is -0.924066274828782.


*** Errors for exponent gamma ***
Error in gamma above tcrit is 1.4995873474151815
Error in gamma below tcrit is 0.15228010292725447
Error in gamma for the average of both temperature regimes is 0.825933725171218
```

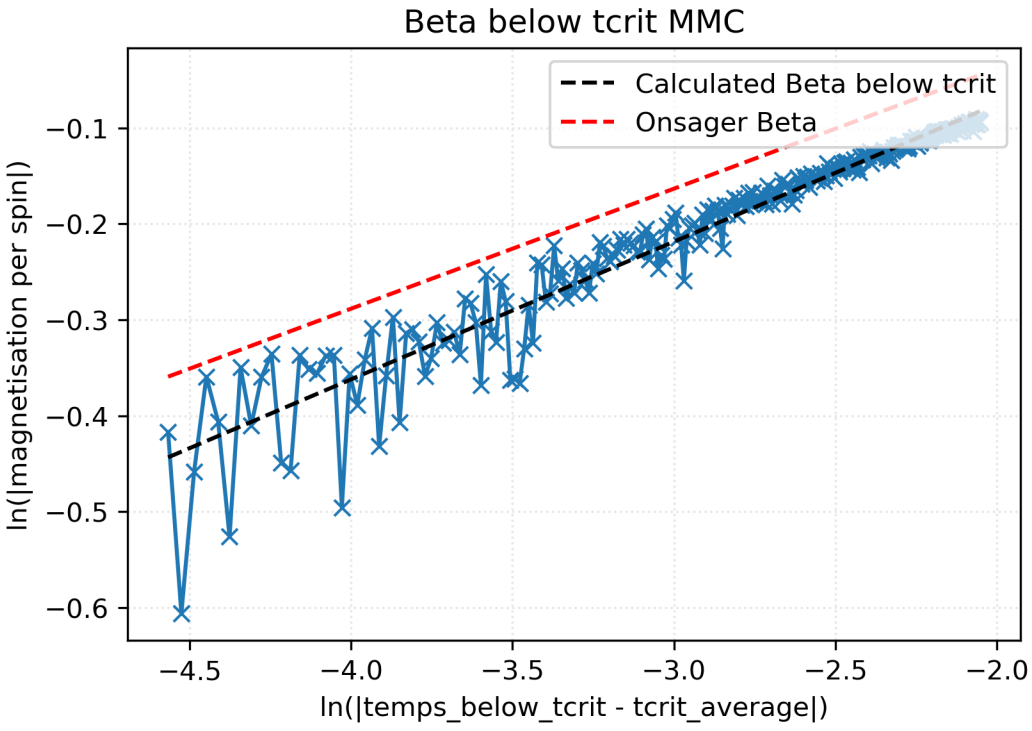

Magnetisation per spin vs temperature MMC
tcrit magnetisation
tcrit magnetic susceptibility
tcrit heat capacity
tcrit magnetisation, susceptibility and heat capacity average
tcrit Onsager 1944
Magnetisation per spin
Temperature
0.9
0.8
0.7
0.6
0.5
0.4
0.3
0.2
2.0
2.1
2.2
2.3
2.4
2.5

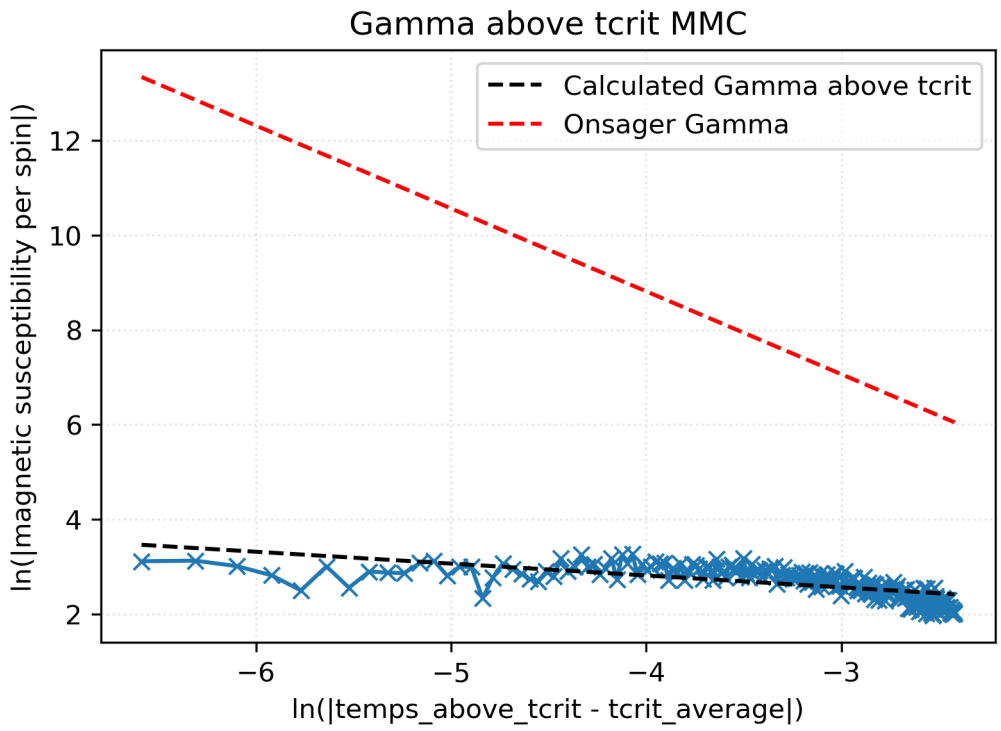

Heat capacity per spin vs temperature MMC
tcrit magnetisation
tcrit magnetic susceptibility
tcrit heat capacity
tcrit magnetisation, susceptibility and heat capacity average
tcrit Onsager 1944
Heat capacity per spin
Temperature
2.4
2.2
2.0
1.8
1.6
1.4
1.2
1.0
0.8
2.0
2.1
2.2
2.3
2.4
2.5

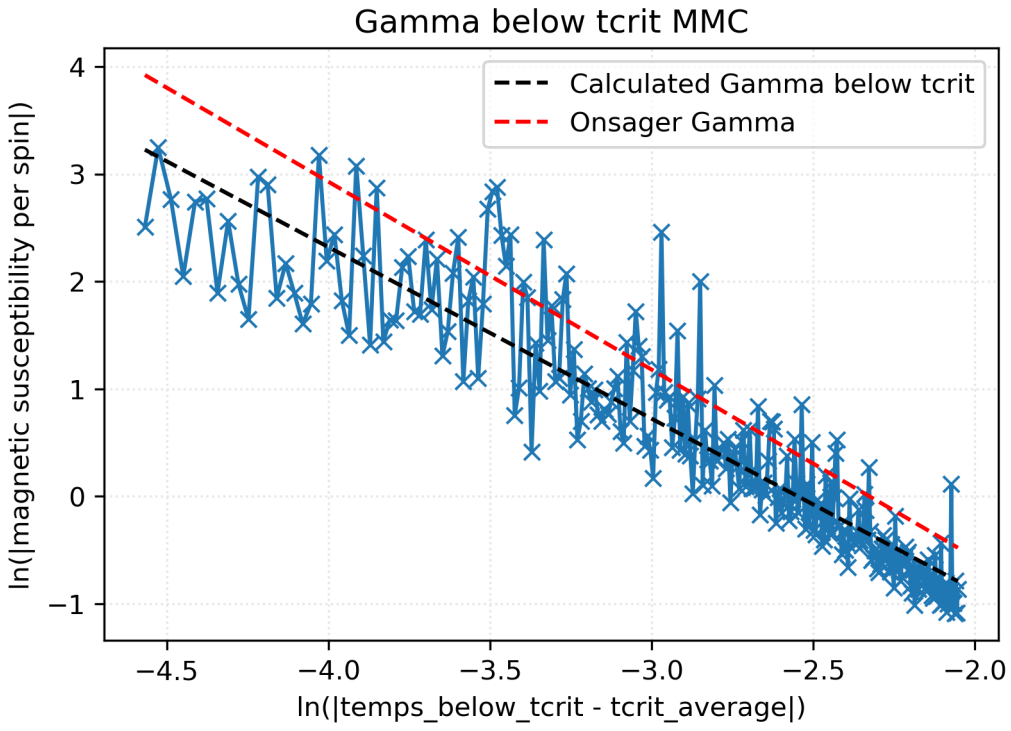

Magnetic susceptibility per spin vs temperature MMC
tcrit magnetisation
tcrit magnetic susceptibility
tcrit heat capacity
tcrit magnetisation, susceptibility and heat capacity average
tcrit Onsager 1944
Magnetic susceptibility per spin
Temperature
0
5
10
15
20
25
30
2.0
2.1
2.2
2.3
2.4
2.5


Onsager critical amplitude above tcrit MMC
Calculated Onsager Critical Amplitude above tcrit
Onsager Critical Exponent
Heat Capacity
ln(|temps_above_tcrit - tcrit_average|)
1.0
1.5
2.0
2.5
3.0
3.5
−6
−5
−4
−3


Onsager critical amplitude below tcrit MMC
Calculated Onsager Critical Amplitude below tcrit
Onsager Critical Exponent
Heat Capacity
2.4
2.2
2.0
1.8
1.6
1.4
1.2
1.0
0.8
−4.5
−4.0
−3.5
−3.0
−2.5
−2.0
ln(|temps_below_tcrit - tcrit_average|)


Beta below tcrit MMC
Calculated Beta below tcrit
Onsager Beta
ln(|magnetisation per spin|)
−0.1
−0.2
−0.3
−0.4
−0.5
−0.6
−4.5
−4.0
−3.5
−3.0
−2.5
−2.0
ln(|temps_below_tcrit - tcrit_average|)


Gamma above tcrit MMC
Calculated Gamma above tcrit
Onsager Gamma
ln(|magnetic susceptibility per spin|)
ln(|temps_above_tcrit - tcrit_average|)


Gamma below tcrit MMC
Calculated Gamma below tcrit
Onsager Gamma
ln(|magnetic susceptibility per spin|)
ln(|temps_below_tcrit - tcrit_average|)

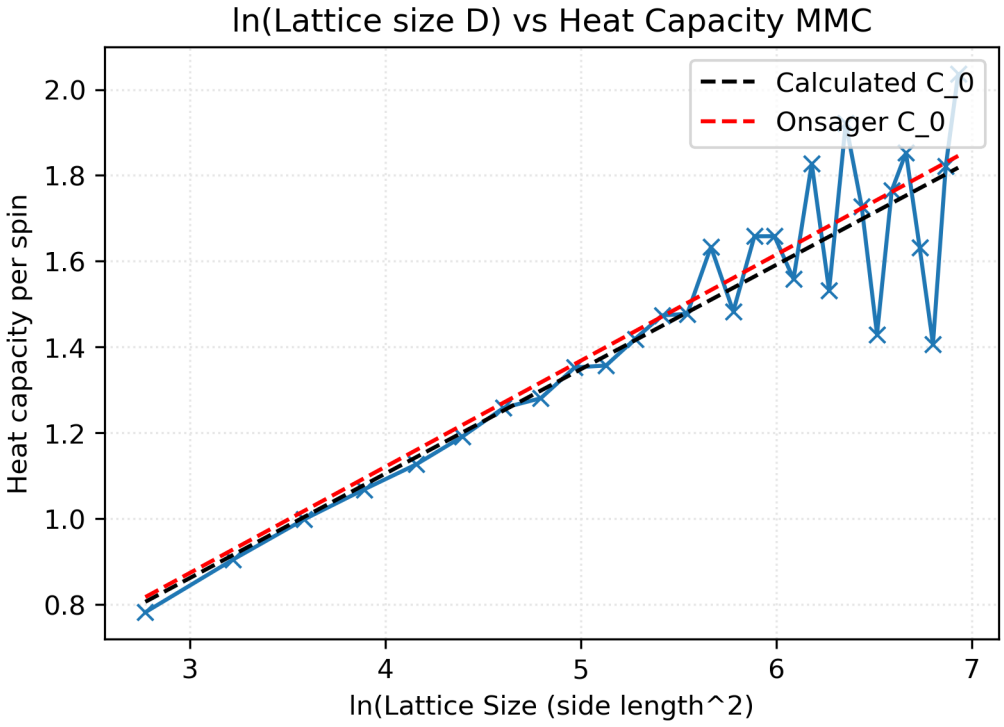
[22]:
```
(<Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Magnetisation per spin vs temperature MMC'},
xlabel='Temperature', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Heat capacity per spin vs temperature MMC'},
xlabel='Temperature', ylabel='Heat capacity per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Magnetic susceptibility per spin vs temperature MMC'},
xlabel='Temperature', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Onsager critical amplitude above tcrit MMC'},
xlabel='ln(|temps_above_tcrit - tcrit_average|)', ylabel='Heat Capacity'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Onsager critical amplitude below tcrit MMC'},
xlabel='ln(|temps_below_tcrit - tcrit_average|)', ylabel='Heat Capacity'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Beta below tcrit MMC'}, xlabel='ln(|temps_below_tcrit
- tcrit_average|)', ylabel='ln(|magnetisation per spin|)'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Gamma above tcrit MMC'}, xlabel='ln(|temps_above_tcrit
- tcrit_average|)', ylabel='ln(|magnetic susceptibility per spin|)'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Gamma below tcrit MMC'}, xlabel='ln(|temps_below_tcrit
- tcrit_average|)', ylabel='ln(|magnetic susceptibility per spin|)'>)
```

24 Validation run - investigating the effect of changing lattice size on the value of C_0 and the heat capacity using the metropolis algorithm

[23]:
```
parameter_file = 'ising_sim_C_0_side_length_investigation_MMC_ICS.txt'
run_parallelised_lattice_side_length_change_investigation_simulations_MMC(parameter_file)
```

```
Parallelising 29 lattice size runs across 24 CPU cores using the metropolis
algorithm.
Lattice size range: 16 --> 1024.
Per-simulation parameters: temperature=2.269185, lattice side length step=1,
magnetic_field_B=0.0, exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=15000000

Running parallelised metropolis algorithm lattice size sweep::   0%|          |␣
 ↪0/29 [00:00<?, ?it/s]

*** C_0 results ***
Calculated value of C_0 is 0.24327429268517567.
Onsager value of C_0 is 0.24725.
Error in calculated value of C_0 is 0.003975707314824328.
```

[23]:
```
(<Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'ln(Lattice size D) vs Heat Capacity MMC'},
xlabel='ln(Lattice Size (side length^2)', ylabel='Heat capacity per spin'>)
```



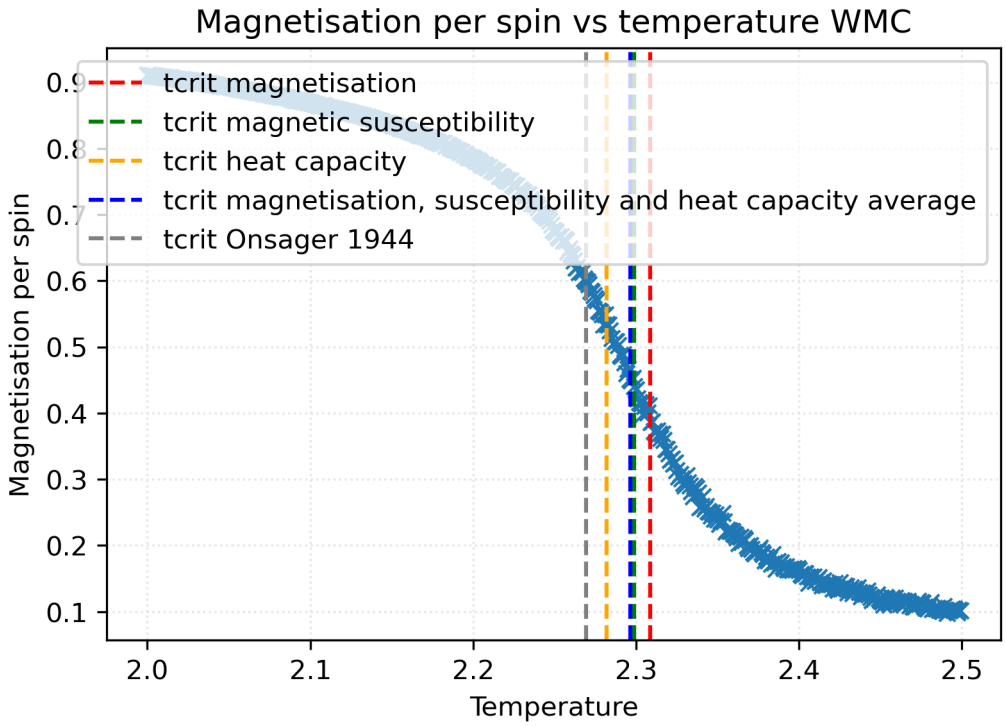
25 Validation run - critical temperature of analytical solution to the 2D ising model using the wolff algorithm

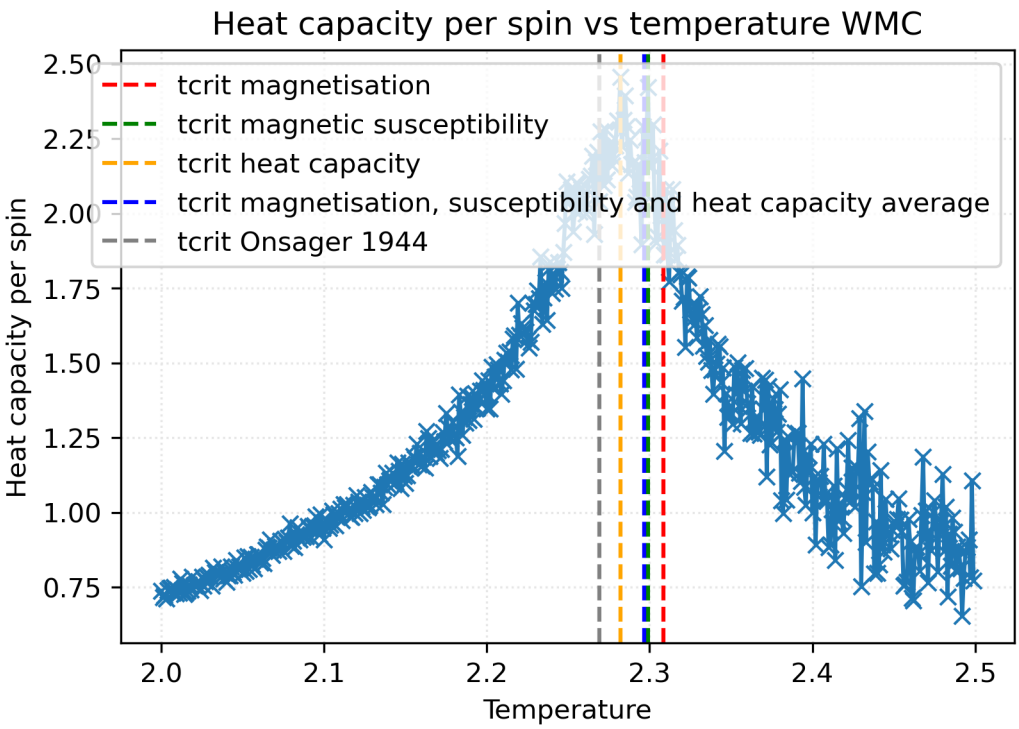
[24]:
```
parameter_file = 'ising_sim_side_len_64_temp_investigation_WMC_ICS.txt'
run_parallelised_temp_change_investigation_simulations_WMC(parameter_file)
```

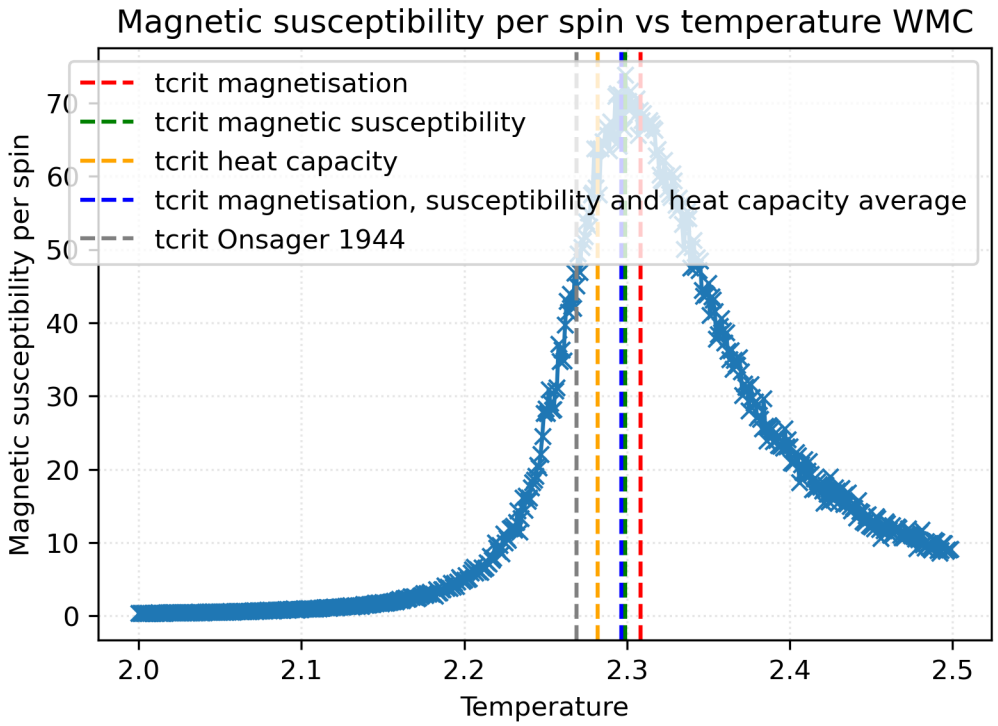
```
Parallelising 500 temperature investigation runs across 24 CPU cores using the
wolff algorithm.
Temperature range: 2.0 --> 2.499.
Per-simulation parameters: temp_step=0.001K, lattice_side_length=64,
magnetic_field_B=0.0, exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=15000.

Running parallelised wolff algorithm temperature sweep::   0%|          | 0/500␣
↪[00:00<?, ?it/s]

*** Critical Temperature Results ***
The Onsager 1944 result for the critical temperature of an infinite lattice is
2.269185K.
Calculated critical temperature from magnetisation drop using the wolff
```

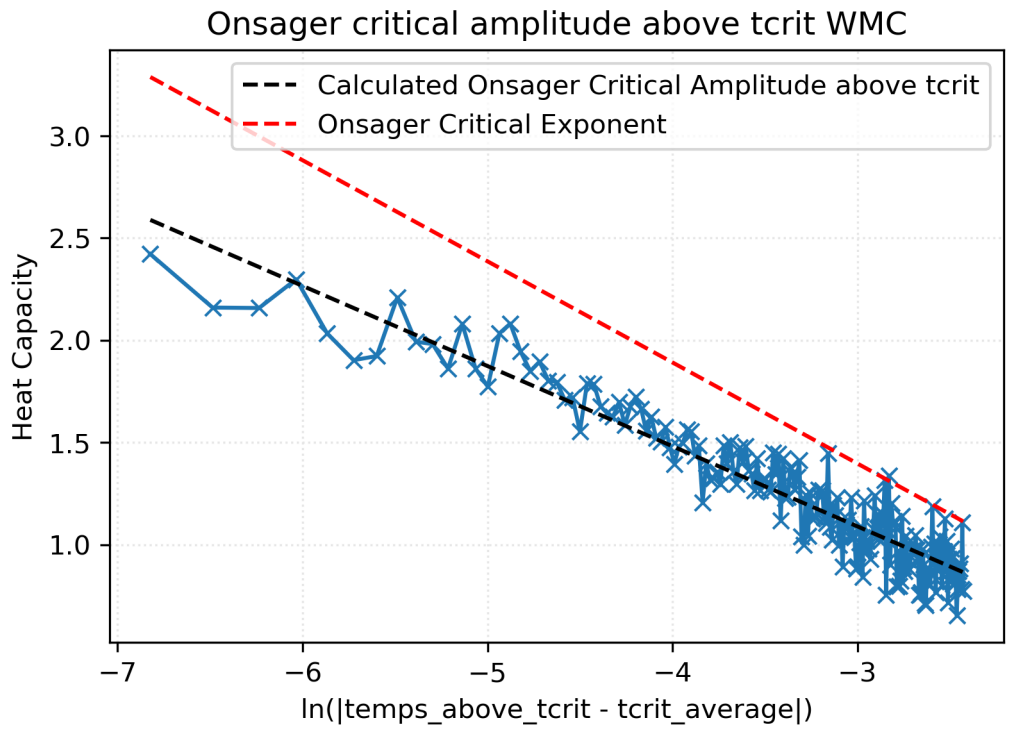
```
algorithm is 2.3085K.
Calculated critical temperature from magnetic susceptibility peak using the
wolff algorithm is 2.299K.
Calculated critical temperature from heat capacity peak using the wolff
algorithm is 2.282K.
Average from the three methods for calculating critical temperature using the
wolff algoritm is 2.2965K.


*** Critical Temperature Errors ***
Error in critical temperature for the magnetisation drop method using the wolff
algorithm is 0.03931500000000021K.
Error in critical temperature for the magnetic susceptibility peak method using
the wolff algorithm is 0.029815000000000147K.
Error in critical temperature from heat capacity peak using the wolff algorithm
is 0.012815000000000243K
Error in critical temperature for the average between the three methods using
the wolff algorithm is 0.0273150000000002.


*** Onsager Critical Exponents ***
The Onsager 1944 result for the critical amplitude 0.4945.
The Onsager 1944 result for the critical exponent alpha is 0
The Onsager 1944 result for the critical exponent beta is 0.125.
The Onsager 1944 result for the critical exponent gamma is -1.75.


Since alpha is 0, the best way to verify it is a linear plot of ln(T) vs the
heat capacity.
If the simulation is working correctly and alpha is behaving as Onsager
predicted, then the plot will be linear and have a gradient of -0.4945, which is
the negative of the Onsager critical amplitude.


*** Results for Onsager Critical Amplitude ***
The calculated value of the negative Onsager critical amplitude above tcrit is
-0.3920842096789374.
The calculated value of the negative Onsager critical amplitude below tcrit is
-0.6401309988238675.
The calculated value of the negative Onsager critical amplitude using the
average of the two temperature regimes is -0.5161076042514025.


*** Errors for Onsager critical amplitude ***
Error in Onsager critical amplitude above tcrit is 0.10241579032106257
Error in Onsager critical amplitude below tcrit is 0.14563099882386749
Error in Onsager critical amplitude for the average of both temperature regimes
is 0.021607604251402457
```

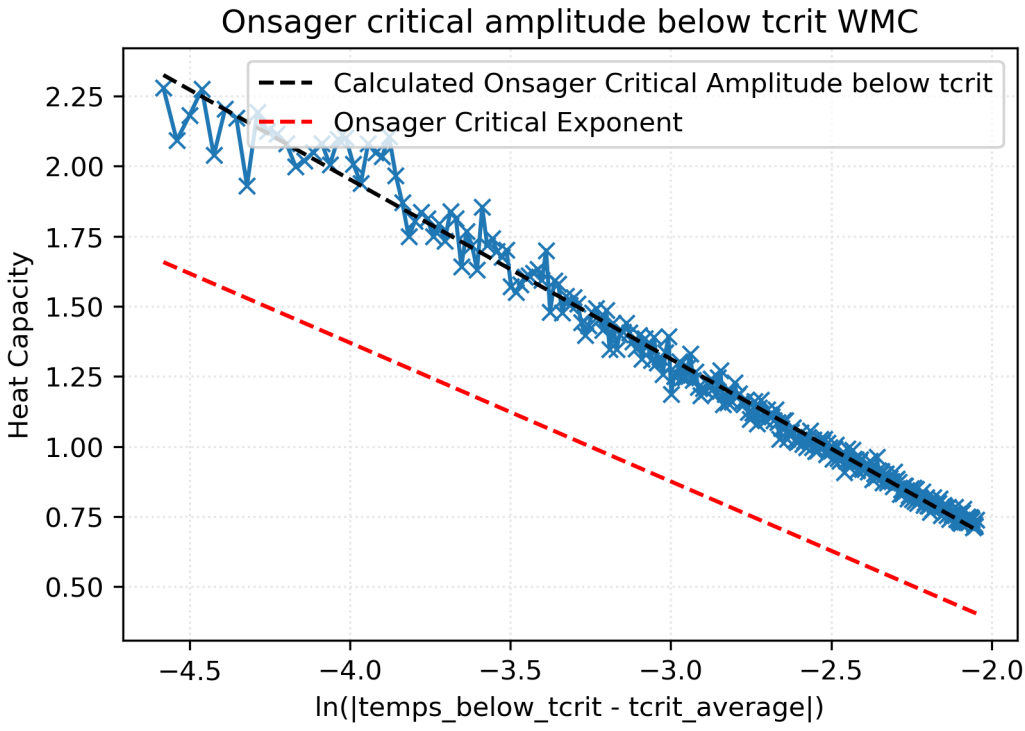
```
*** Results for exponent beta ***
The calculated value of beta below tcrit is 0.1659094812067389


*** Error for exponent beta ***
Error in beta for below tcrit is 0.0409094812067389


*** Results for exponent gamma ***
The calculated value of gamma above tcrit is -0.6792495385301973.
The calculated value of gamma below tcrit is -2.184368863727049.
The calculated value of gamma using the average of the two temperature regimes
is -1.4318092011286232.


*** Errors for exponent gamma ***
Error in gamma above tcrit is 1.0707504614698027
Error in gamma below tcrit is 0.4343688637270491
Error in gamma for the average of both temperature regimes is 0.3181907988713768
```

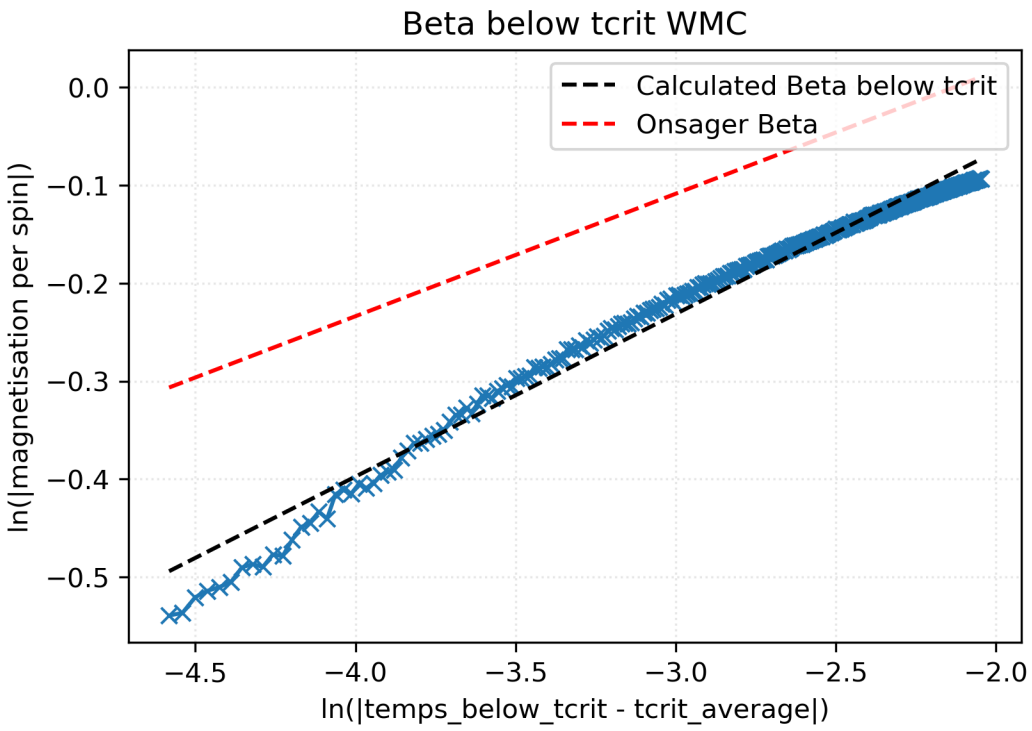


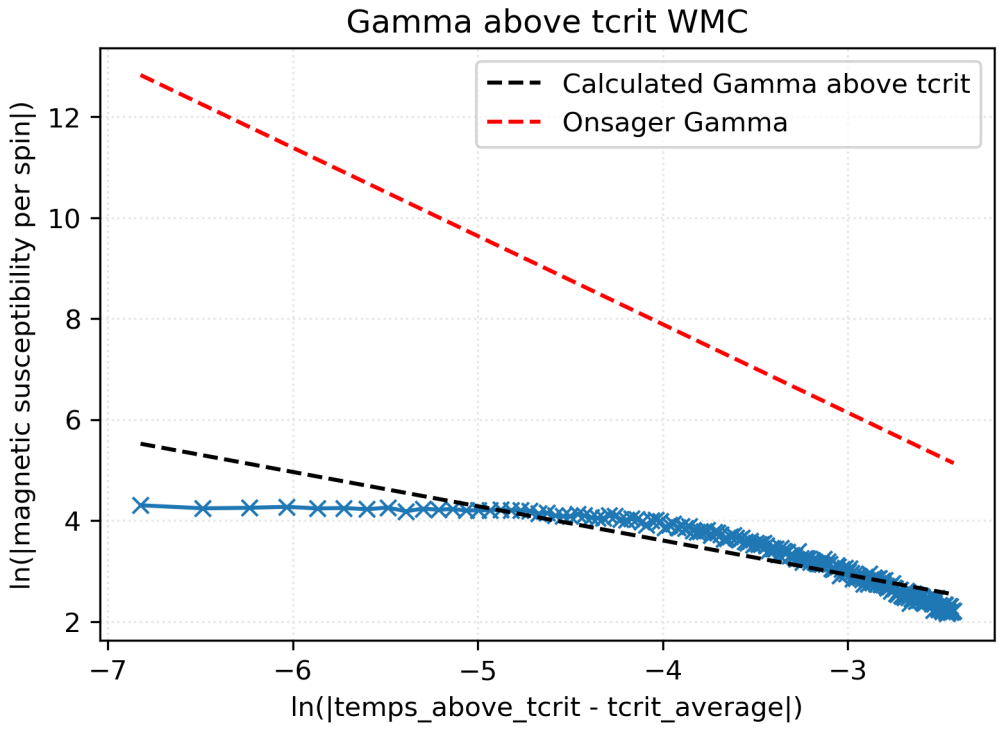

Heat capacity per spin vs temperature WMC
tcrit magnetisation
tcrit magnetic susceptibility
tcrit heat capacity
tcrit magnetisation, susceptibility and heat capacity average
tcrit Onsager 1944
Heat capacity per spin
Temperature
2.50
2.25
2.00
1.75
1.50
1.25
1.00
0.75
2.0
2.1
2.2
2.3
2.4
2.5

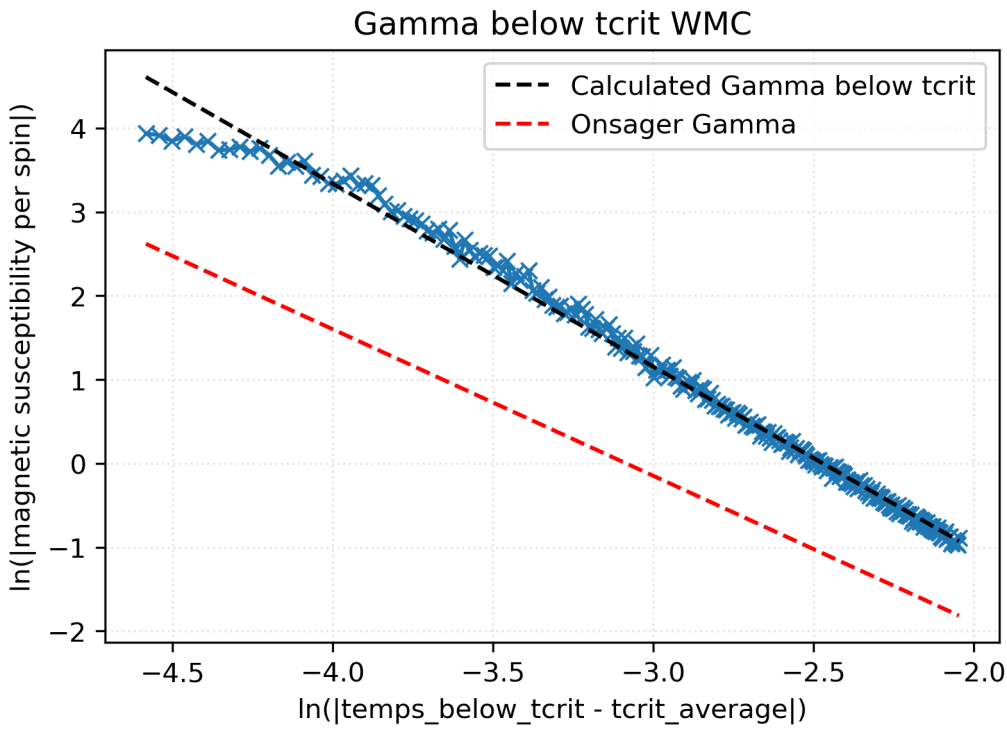

Magnetic susceptibility per spin vs temperature WMC
tcrit magnetisation
tcrit magnetic susceptibility
tcrit heat capacity
tcrit magnetisation, susceptibility and heat capacity average
tcrit Onsager 1944
Magnetic susceptibility per spin
Temperature
70
60
50
40
30
20
10
0
2.0
2.1
2.2
2.3
2.4
2.5


Onsager critical amplitude above tcrit WMC
Calculated Onsager Critical Amplitude above tcrit
Onsager Critical Exponent
Heat Capacity
3.0
2.5
2.0
1.5
1.0
−7
−6
−5
−4
−3
ln(|temps_above_tcrit - tcrit_average|)


Onsager critical amplitude below tcrit WMC
Calculated Onsager Critical Amplitude below tcrit
Onsager Critical Exponent
Heat Capacity
2.25
2.00
1.75
1.50
1.25
1.00
0.75
0.50
−4.5
−4.0
−3.5
−3.0
−2.5
−2.0
ln(|temps_below_tcrit - tcrit_average|)


Beta below tcrit WMC
Calculated Beta below tcrit
Onsager Beta
ln(|magnetisation per spin|)
ln(|temps_below_tcrit - tcrit_average|)
0.0
−0.1
−0.2
−0.3
−0.4
−0.5
−4.5
−4.0
−3.5
−3.0
−2.5
−2.0


Gamma above tcrit WMC
Calculated Gamma above tcrit
Onsager Gamma
ln(|magnetic susceptibility per spin|)
ln(|temps_above_tcrit - tcrit_average|)
12
10
8
6
4
2
−7
−6
−5
−4
−3



[24]: (<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Magnetisation per spin vs temperature WMC'}, xlabel='Temperature', ylabel='Magnetisation per spin'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Heat capacity per spin vs temperature WMC'}, xlabel='Temperature', ylabel='Heat capacity per spin'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Magnetic susceptibility per spin vs temperature WMC'}, xlabel='Temperature', ylabel='Magnetic susceptibility per spin'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Onsager critical amplitude above tcrit WMC'}, xlabel='ln(|temps_above_tcrit - tcrit_average|)', ylabel='Heat Capacity'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Onsager critical amplitude below tcrit WMC'}, xlabel='ln(|temps_below_tcrit - tcrit_average|)', ylabel='Heat Capacity'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Beta below tcrit WMC'}, xlabel='ln(|temps_below_tcrit - tcrit_average|)', ylabel='ln(|magnetisation per spin|)'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Gamma above tcrit WMC'}, xlabel='ln(|temps_above_tcrit - tcrit_average|)', ylabel='ln(|magnetic susceptibility per spin|)'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Gamma below tcrit WMC'}, xlabel='ln(|temps_below_tcrit

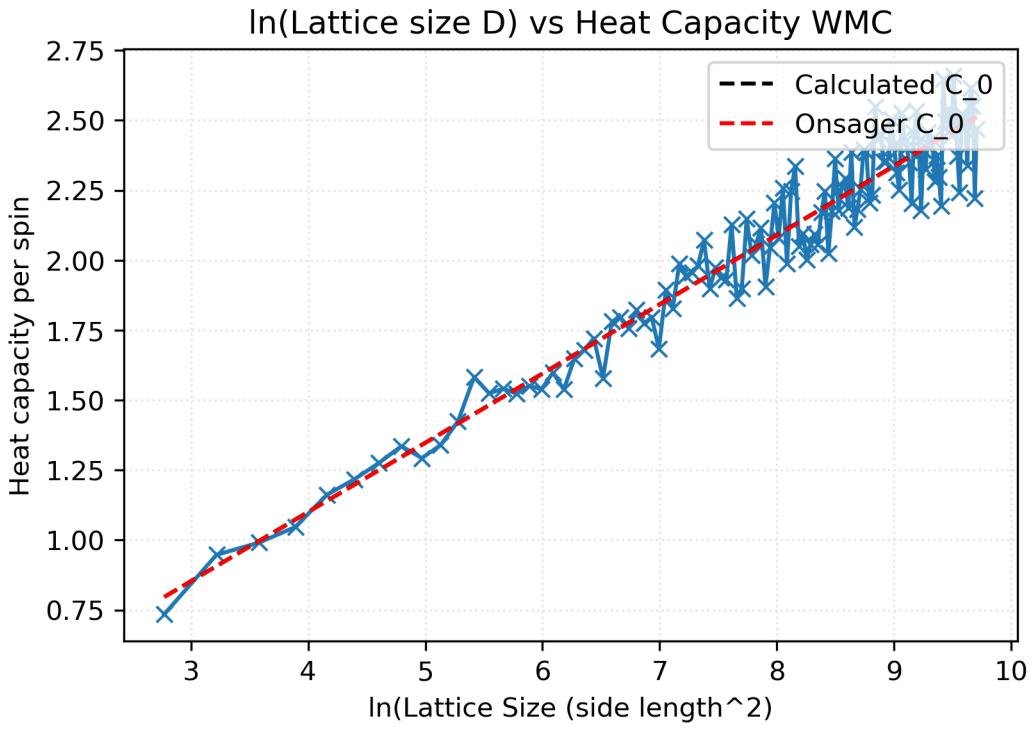
- tcrit_average|)', ylabel='ln(|magnetic susceptibility per spin|)'>)

26 Validation run - investigating the effect of changing lattice size on the value of C_0 and the heat capacity using the wolff algorithm

[25]:
```
parameter_file = 'ising_sim_C_0_side_length_investigation_WMC_ICS.txt'
run_parallelised_lattice_side_length_change_investigation_simulations_WMC(parameter_file)
```

```
Parallelising 125 lattice size runs across 24 CPU cores using the wolff
algorithm.
Lattice size range: 16 --> 16384.
Per-simulation parameters: temperature=2.269185, lattice side length step=1,
magnetic_field_B=0.0, exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=15000

Running parallelised wolff algorithm lattice size sweep::   0%|          | 0/125␣
↪[00:00<?, ?it/s]

*** C_0 results ***
Calculated value of C_0 is 0.24742337288323016.
Onsager value of C_0 is 0.24725.
Error in calculated value of C_0 is 0.00017337288323016065.
```

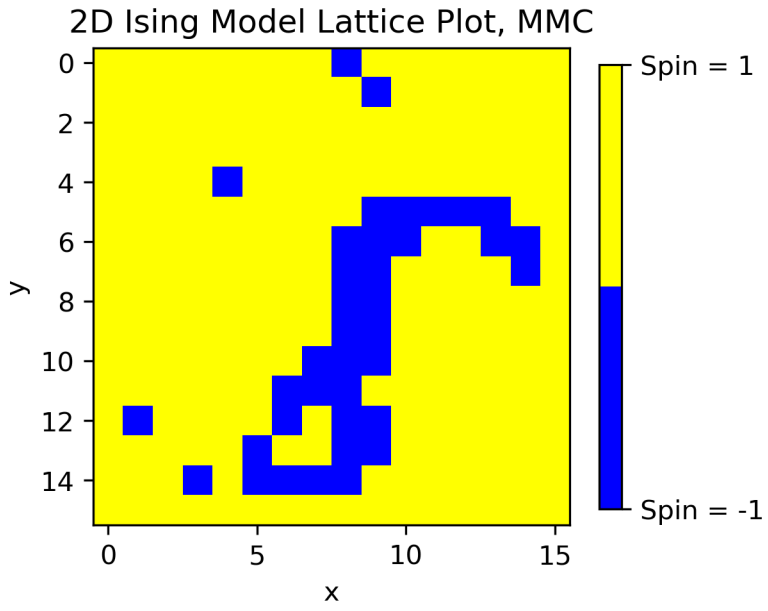
[25]:
```
(<Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'ln(Lattice size D) vs Heat Capacity WMC'},
xlabel='ln(Lattice Size (side length^2)', ylabel='Heat capacity per spin'>)
```

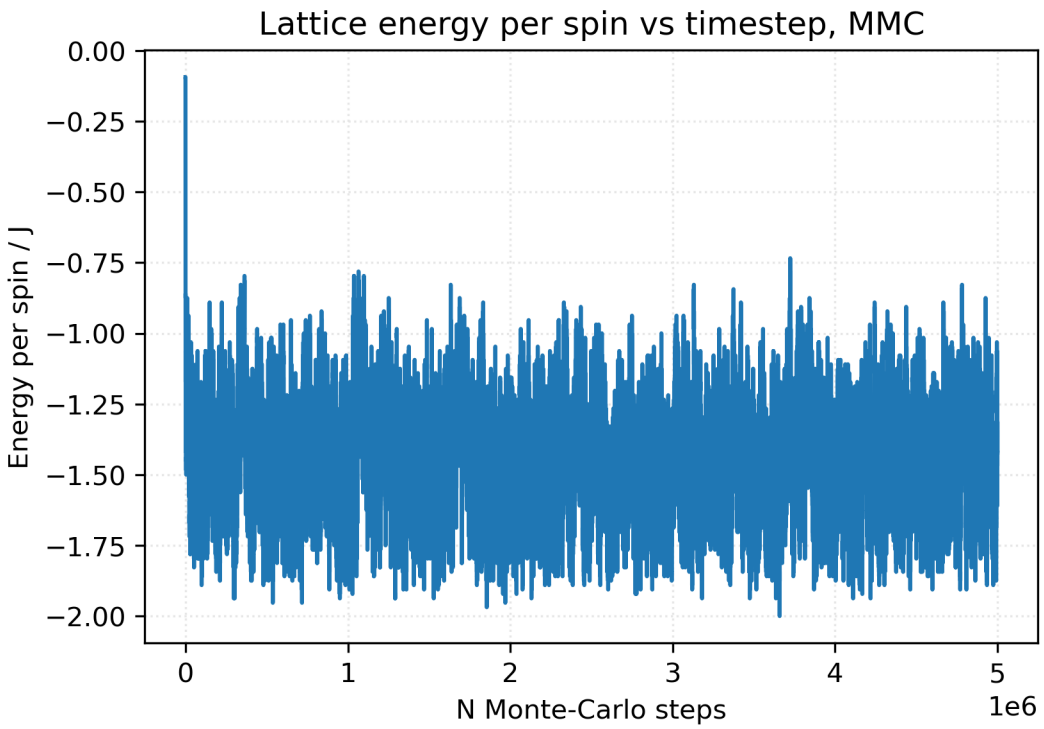


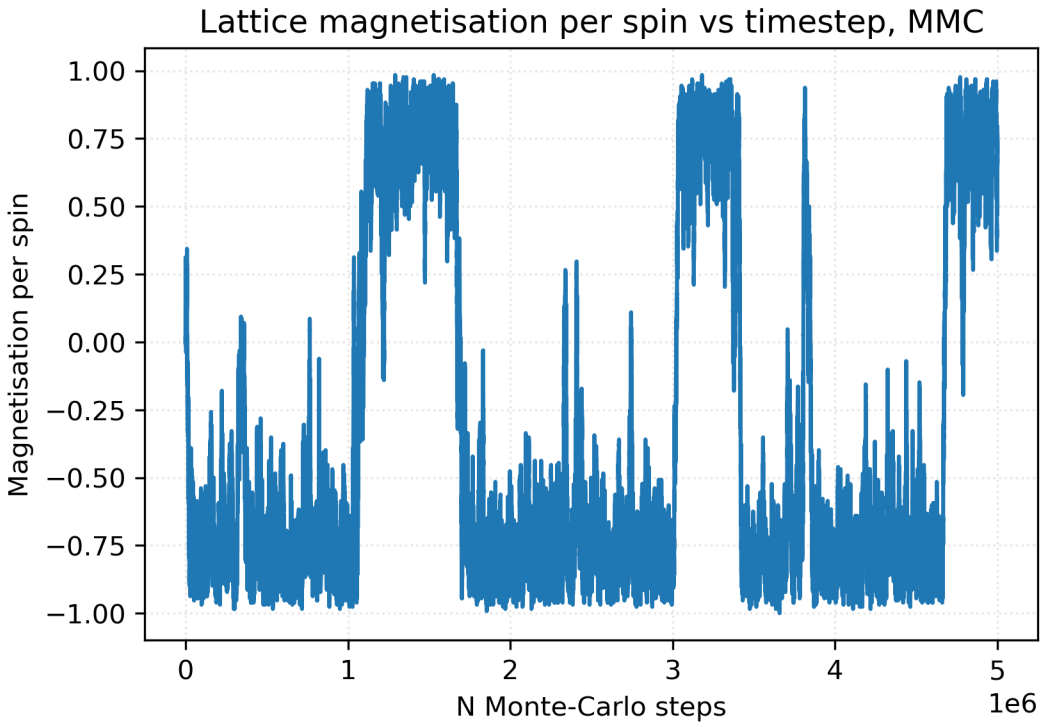
27 Visualising how changing lattice size impacts the simulations using the metropolis algorithm, whilst remaining at the Onsager critical temperature

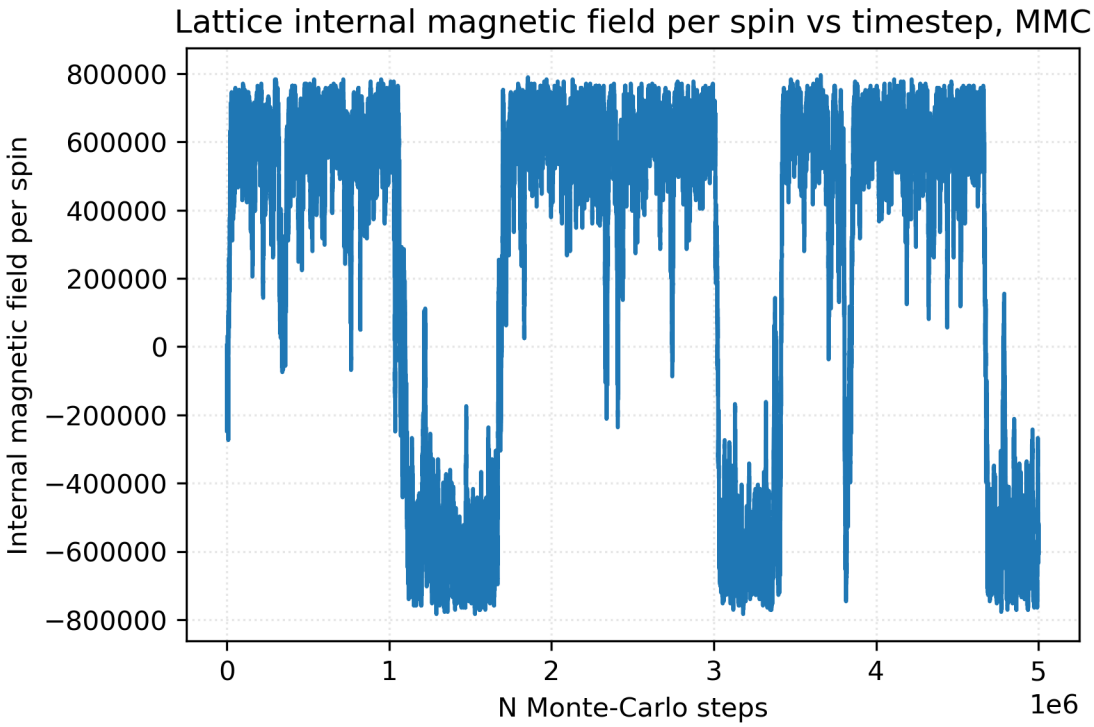
[26]:
```
# 16x16 lattice
parameter_file = 'ising_single_sim_side_len_16_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

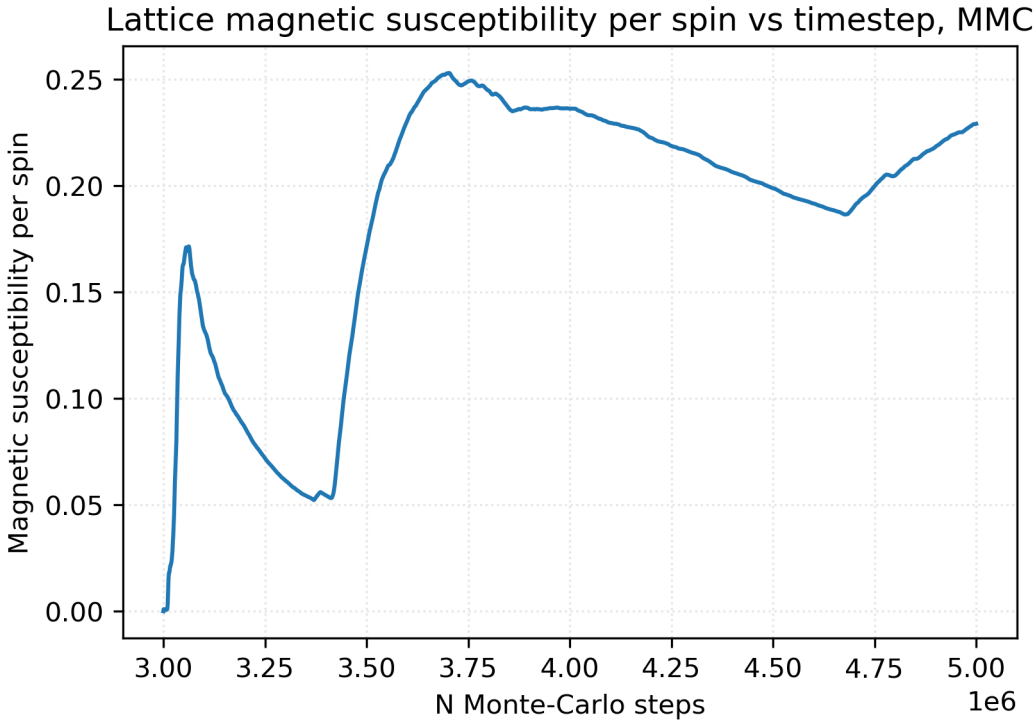
```
Running single Metropolis Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=16, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=5000000

Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

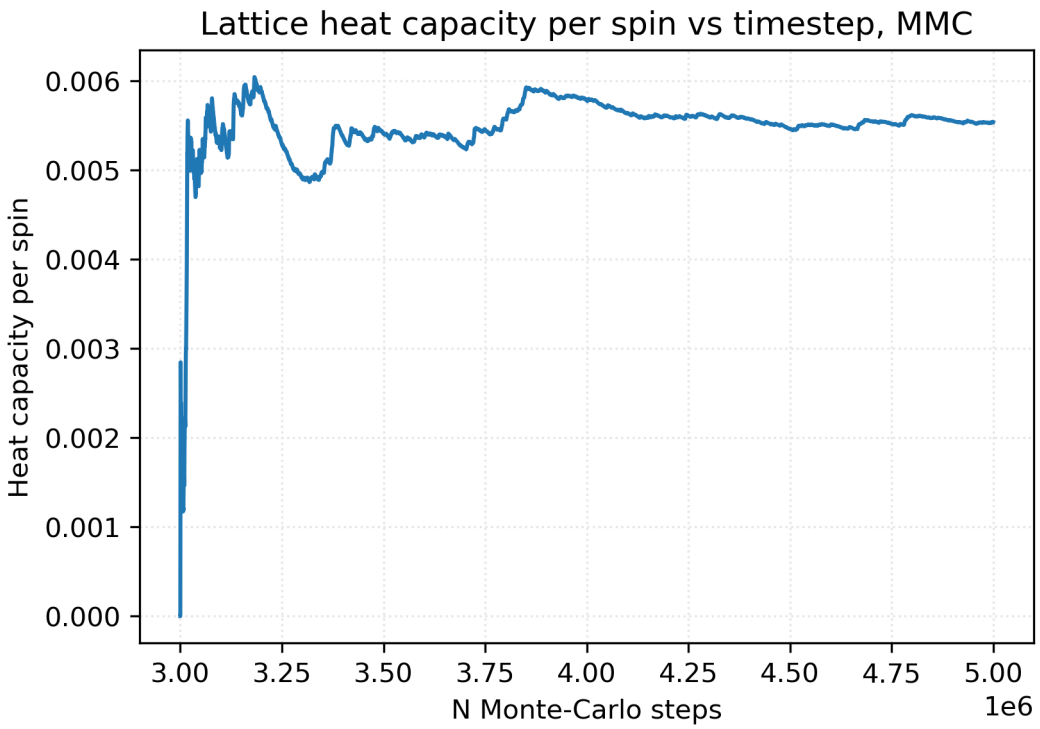



Lattice energy per spin vs timestep, MMC
Energy per spin / J
0.00
−0.25
−0.50
−0.75
−1.00
−1.25
−1.50
−1.75
−2.00
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice magnetisation per spin vs timestep, MMC
Magnetisation per spin
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
800000
600000
400000
200000
0
−200000
−400000
−600000
−800000
0
1
2
3
4
5
N Monte-Carlo steps
1e6


Lattice magnetic susceptibility per spin vs timestep, MMC
Magnetic susceptibility per spin
0.25
0.20
0.15
0.10
0.05
0.00
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
N Monte-Carlo steps
1e6

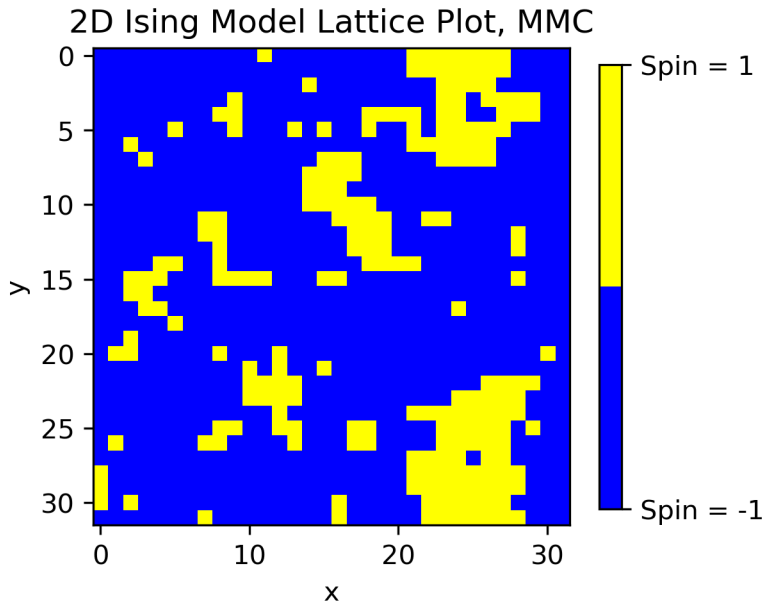


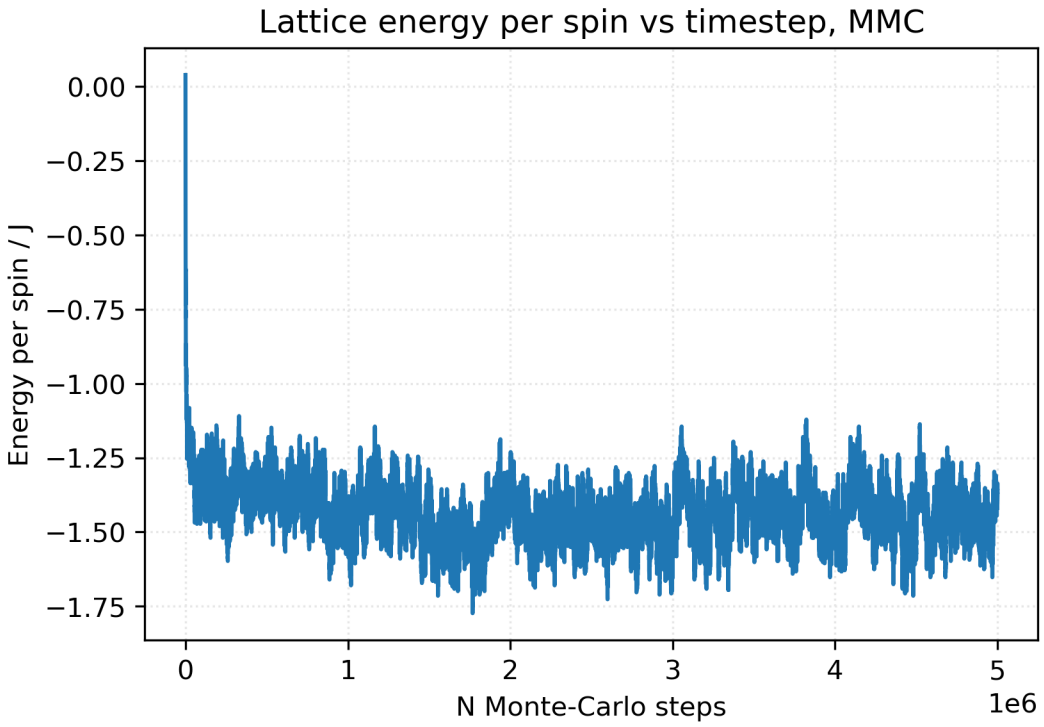
[26]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

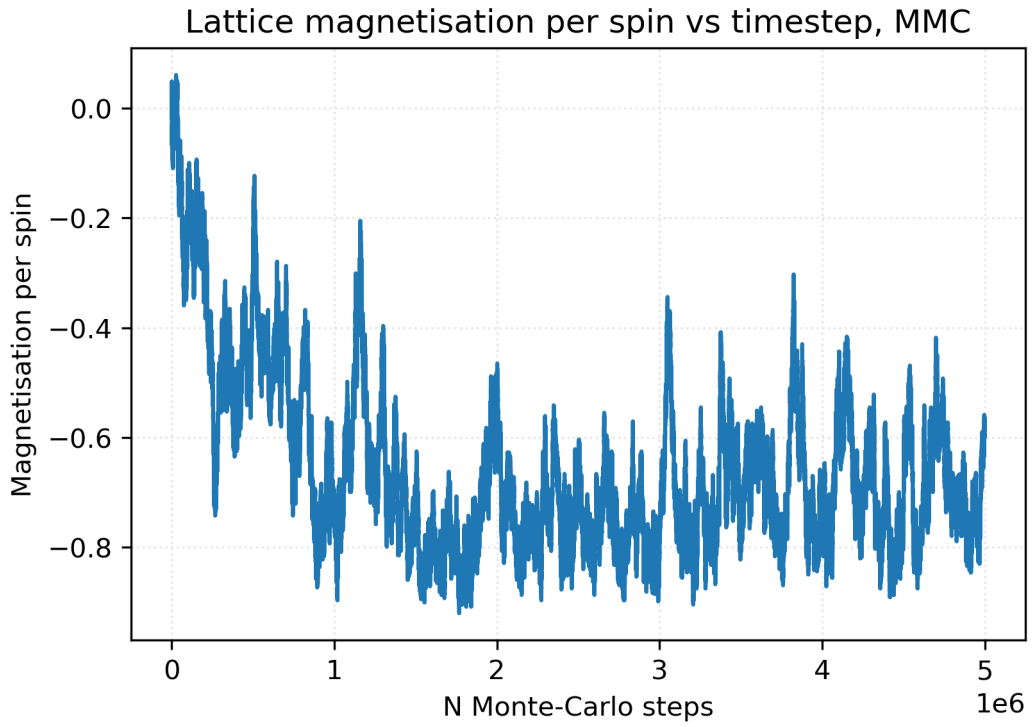
[27]:
```
# 32x32 lattice
parameter_file = 'ising_single_sim_side_len_32_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

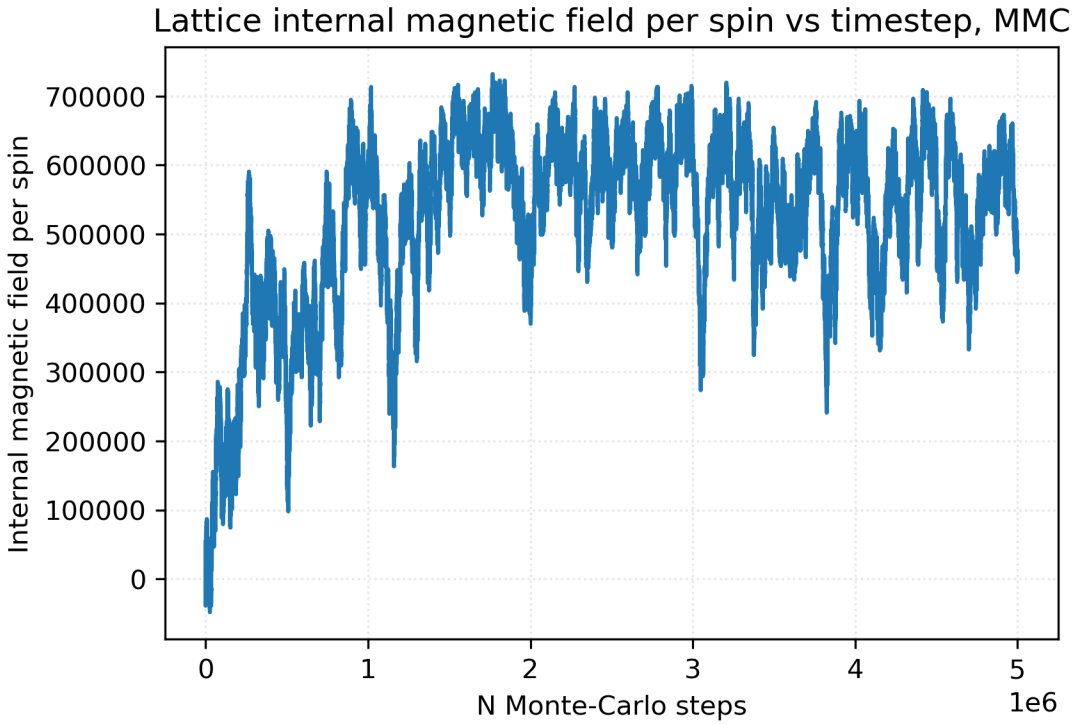
```
Running single Metropolis Algorithm simulation with parameters:
```

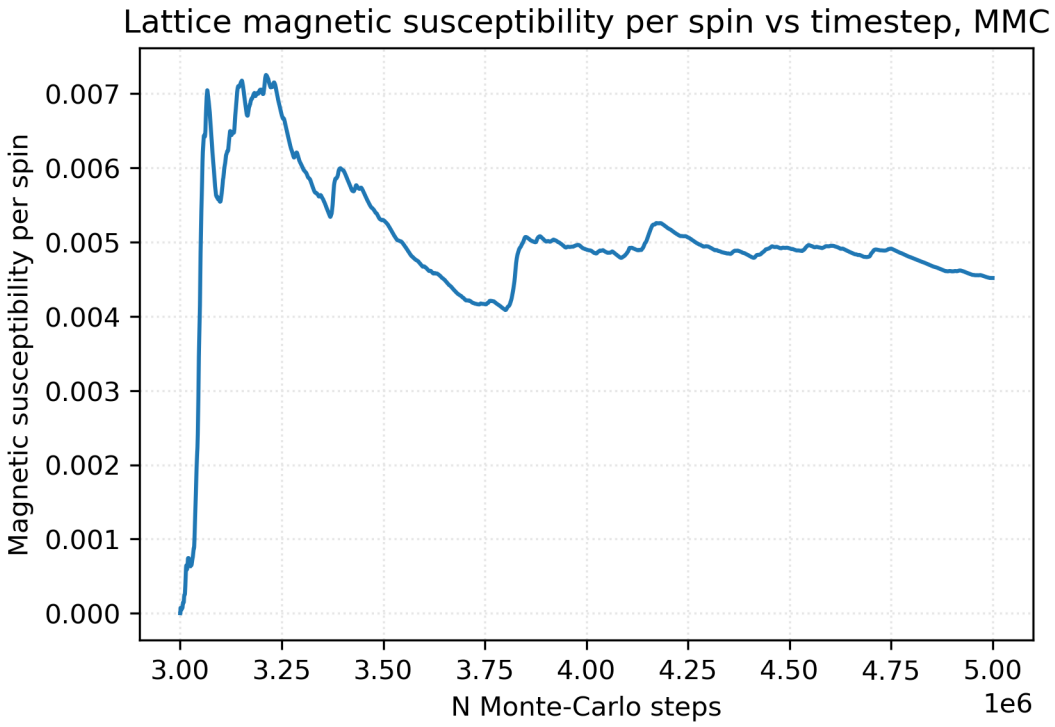
```
temperature=2.269185K, lattice_side_length=32, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=5000000

Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

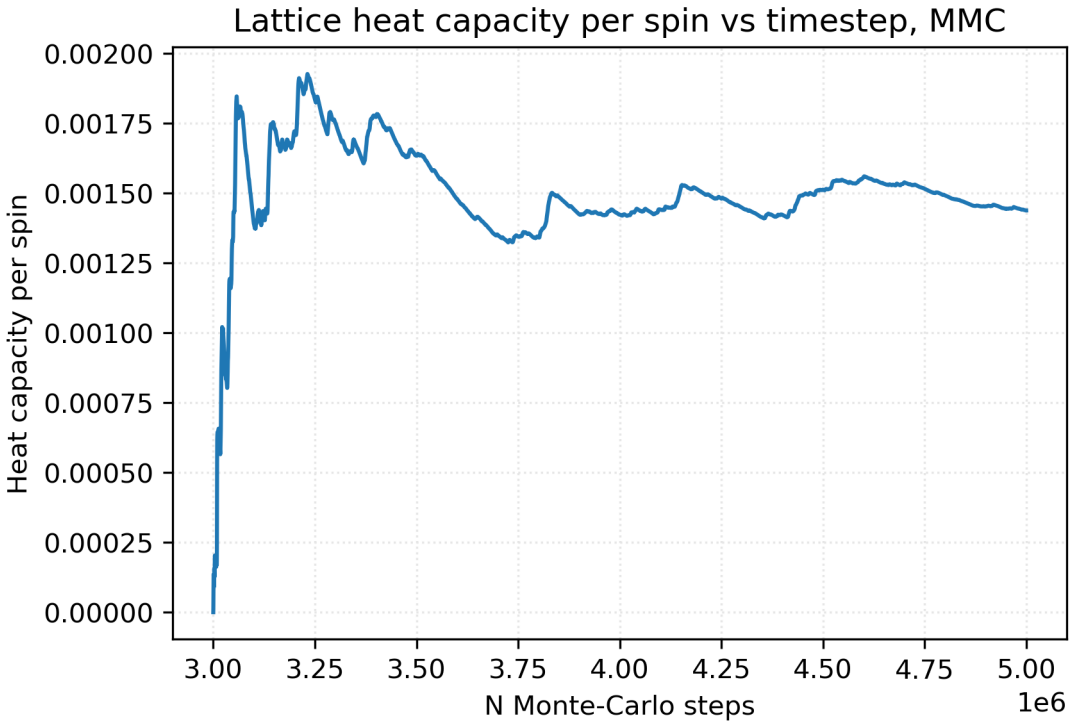





Lattice magnetisation per spin vs timestep, MMC
Magnetisation per spin
0.0
−0.2
−0.4
−0.6
−0.8
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
700000
600000
500000
400000
300000
200000
100000
0
0
1
2
3
4
5
1e6
N Monte-Carlo steps

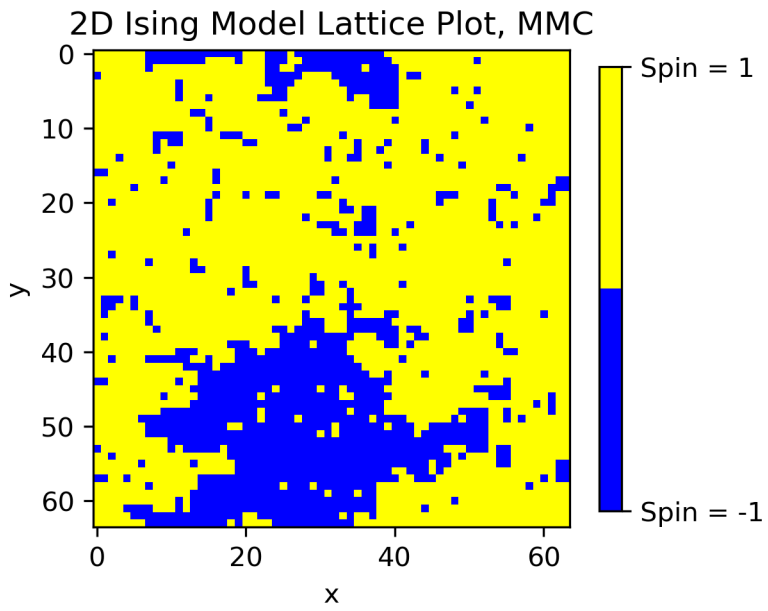

Lattice magnetic susceptibility per spin vs timestep, MMC
Magnetic susceptibility per spin
0.007
0.006
0.005
0.004
0.003
0.002
0.001
0.000
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
N Monte-Carlo steps
1e6

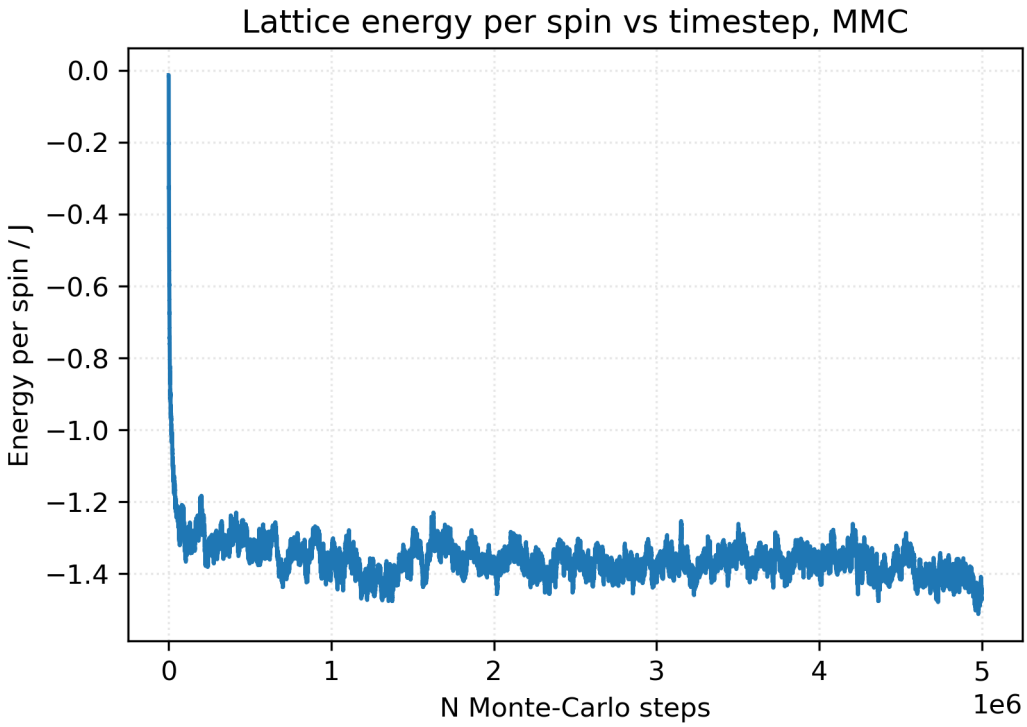

Lattice heat capacity per spin vs timestep, MMC
Heat capacity per spin
0.00200
0.00175
0.00150
0.00125
0.00100
0.00075
0.00050
0.00025
0.00000
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
N Monte-Carlo steps
1e6

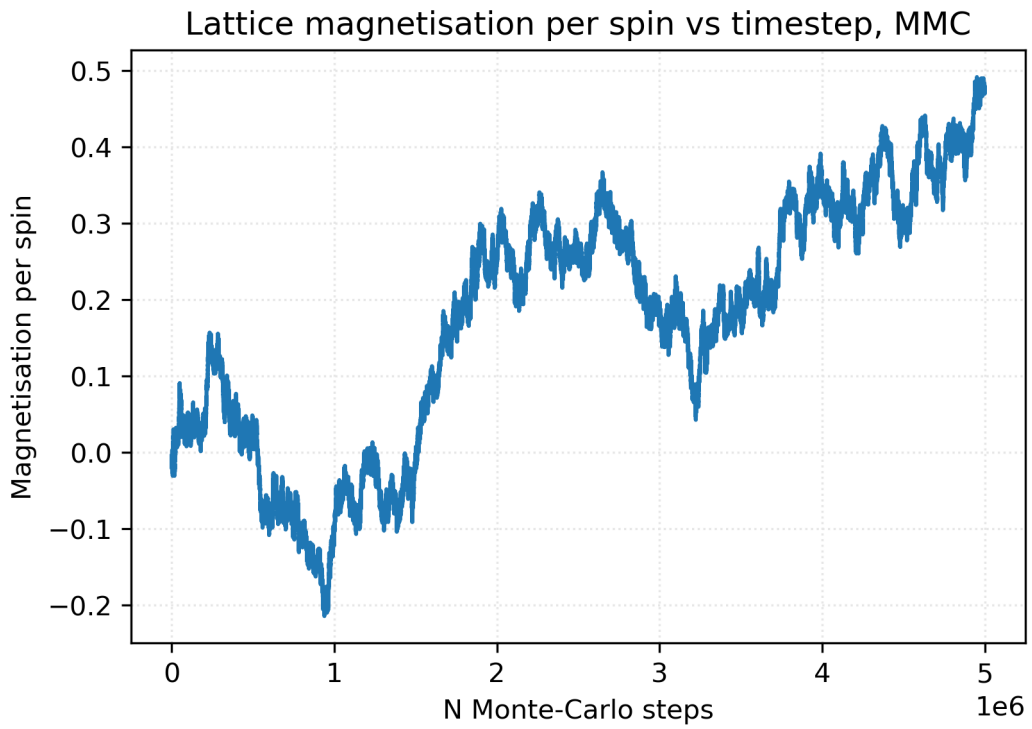
[27]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

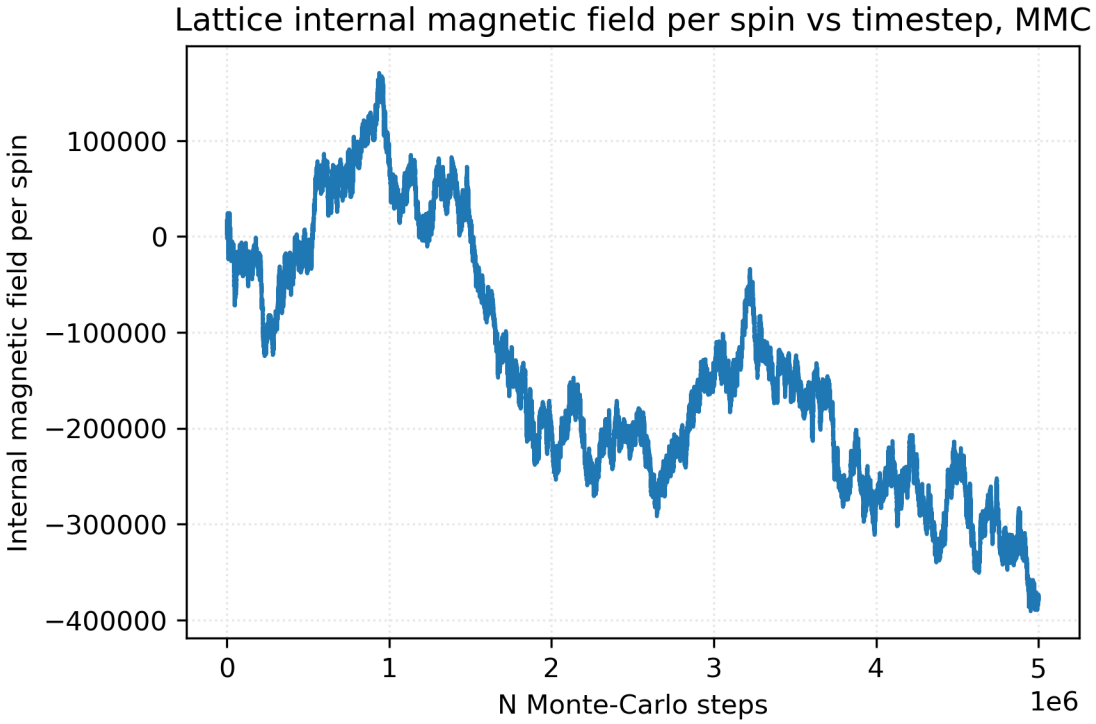
[28]:
```
# 64x64 lattice
parameter_file = 'ising_single_sim_side_len_64_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

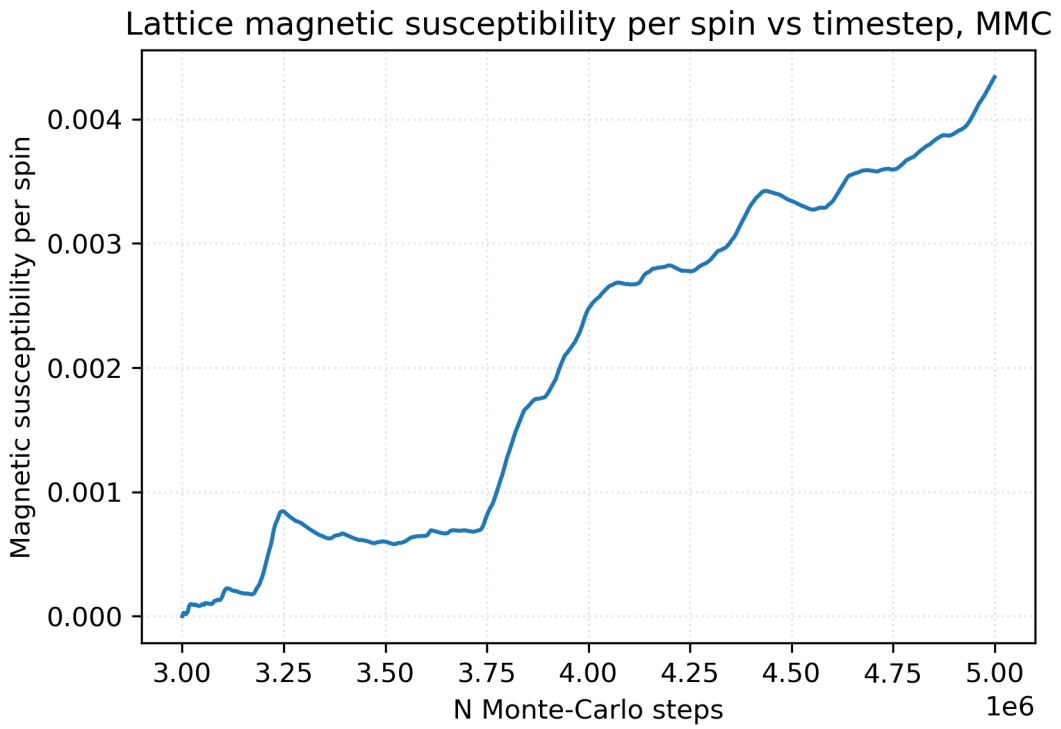
```
Running single Metropolis Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=5000000


Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

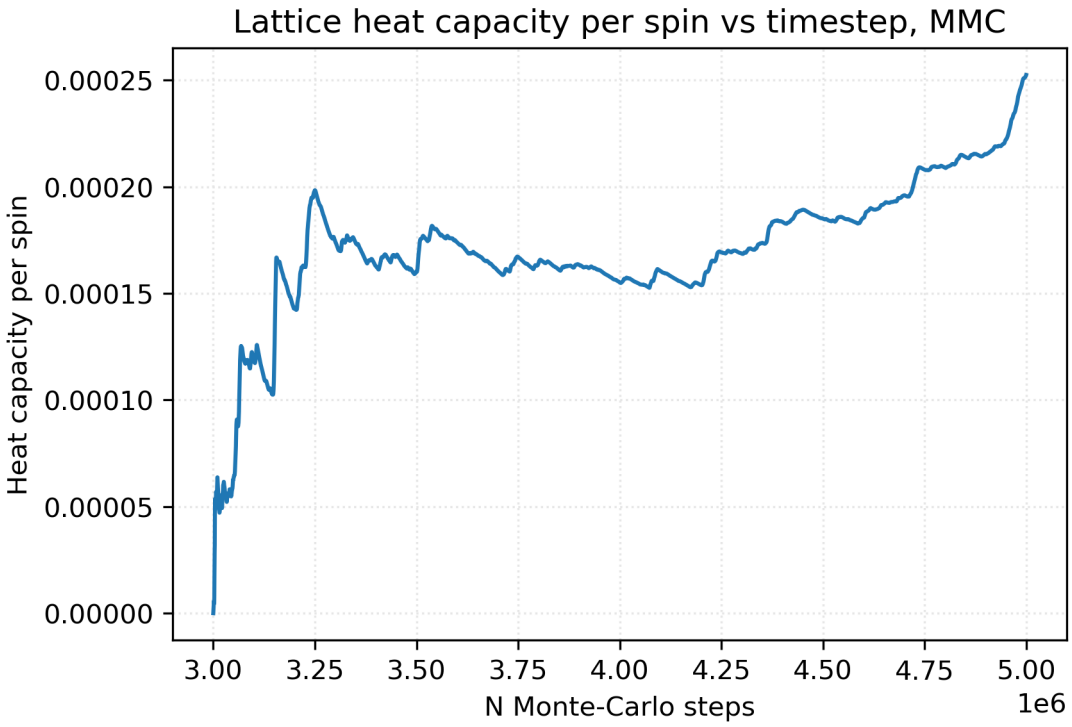

2D Ising Model Lattice Plot, MMC
0
10
20
30
40
50
60
y
0
20
40
60
x
Spin = 1
Spin = -1


Lattice energy per spin vs timestep, MMC
0.0
−0.2
−0.4
−0.6
−0.8
−1.0
−1.2
−1.4
Energy per spin / J
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice magnetisation per spin vs timestep, MMC
Magnetisation per spin
0.5
0.4
0.3
0.2
0.1
0.0
−0.1
−0.2
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
100000
0
−100000
−200000
−300000
−400000
0
1
2
3
4
5
1e6
N Monte-Carlo steps

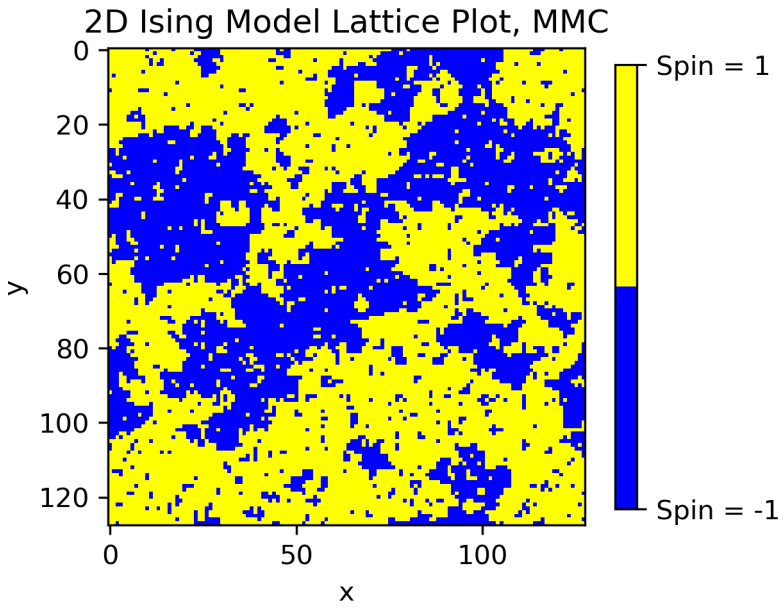

Lattice magnetic susceptibility per spin vs timestep, MMC
Magnetic susceptibility per spin
0.004
0.003
0.002
0.001
0.000
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
1e6
N Monte-Carlo steps

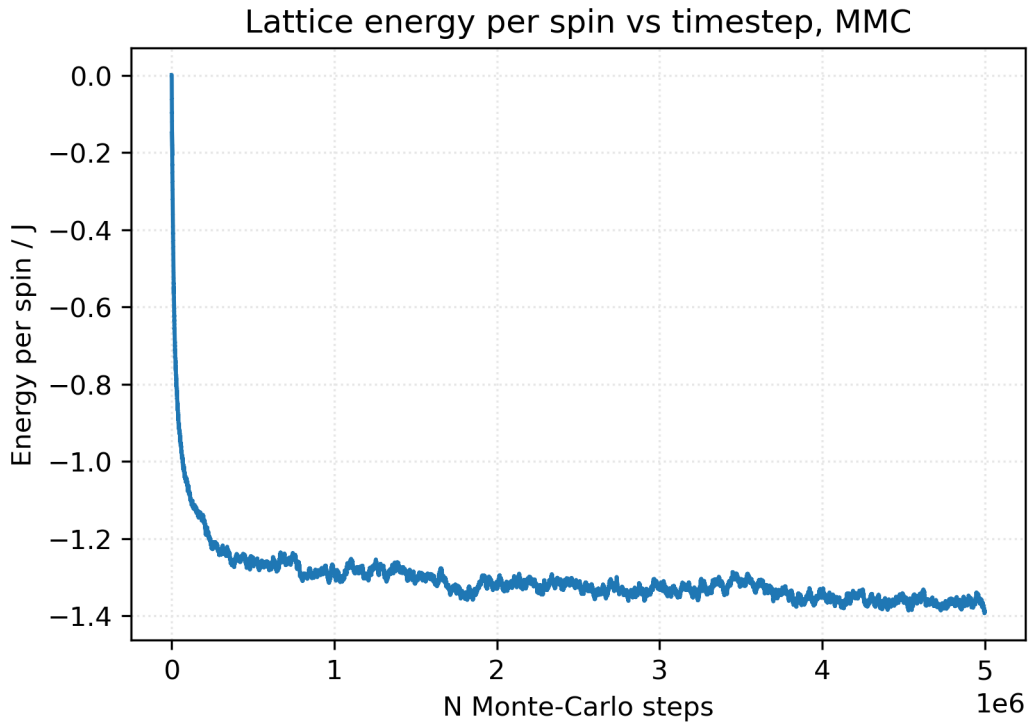

Lattice heat capacity per spin vs timestep, MMC
Heat capacity per spin
0.00025
0.00020
0.00015
0.00010
0.00005
0.00000
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
1e6
N Monte-Carlo steps

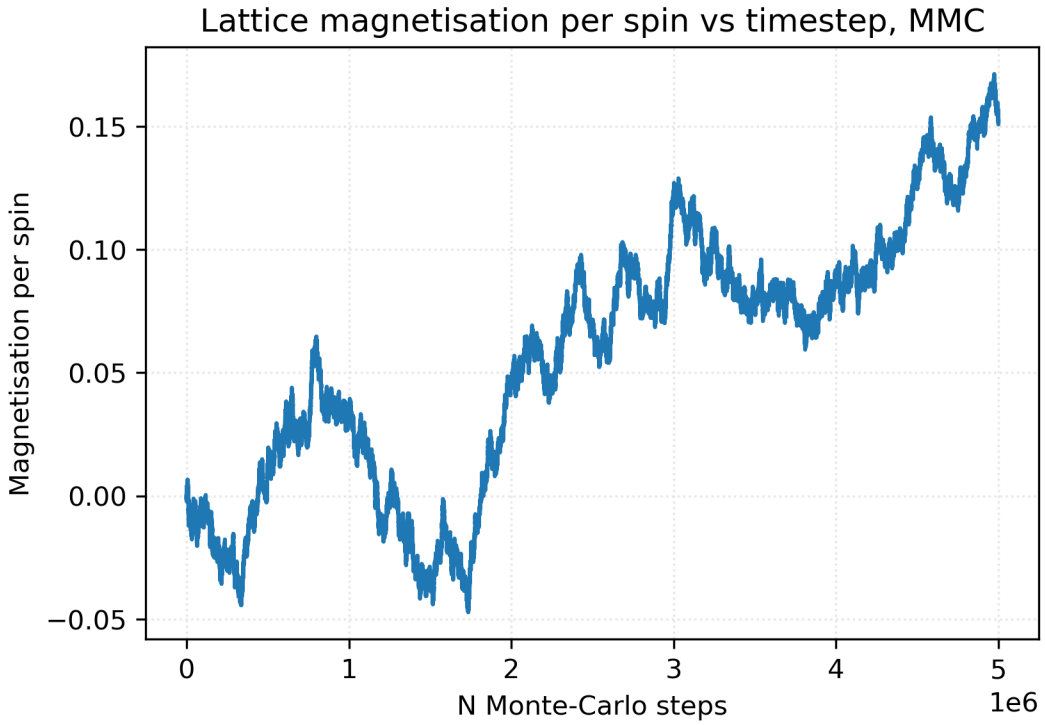
[28]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

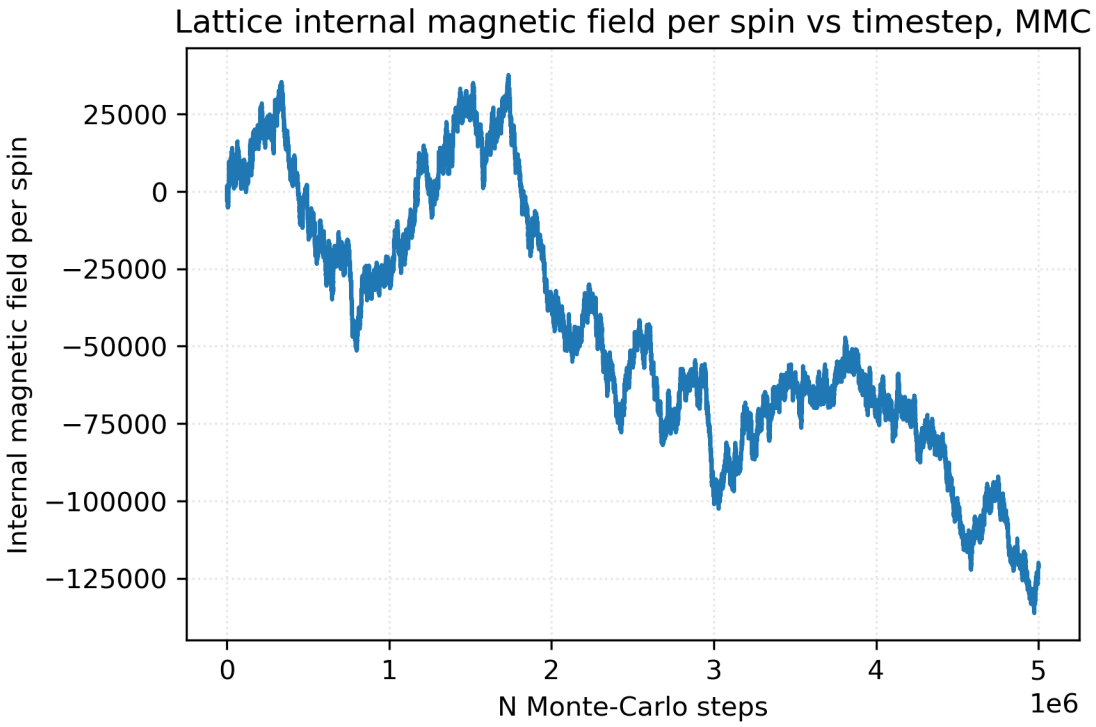
[29]:
```
# 128x128 lattice
parameter_file = 'ising_single_sim_side_len_128_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

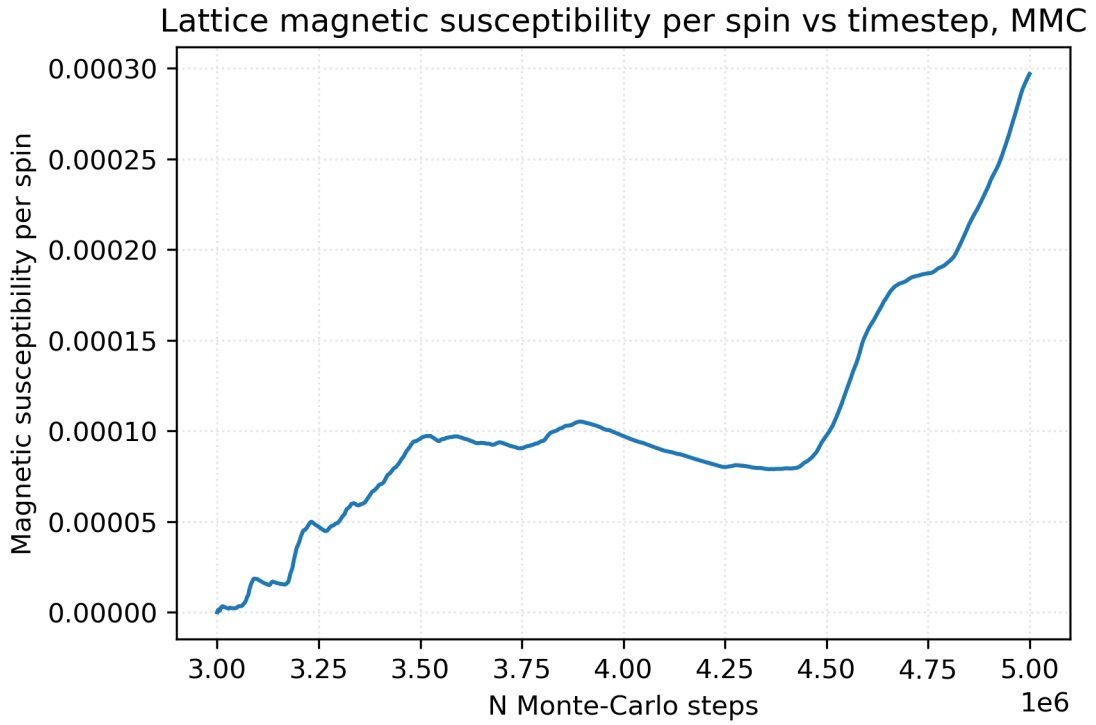
```
Running single Metropolis Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=128, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=5000000


Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

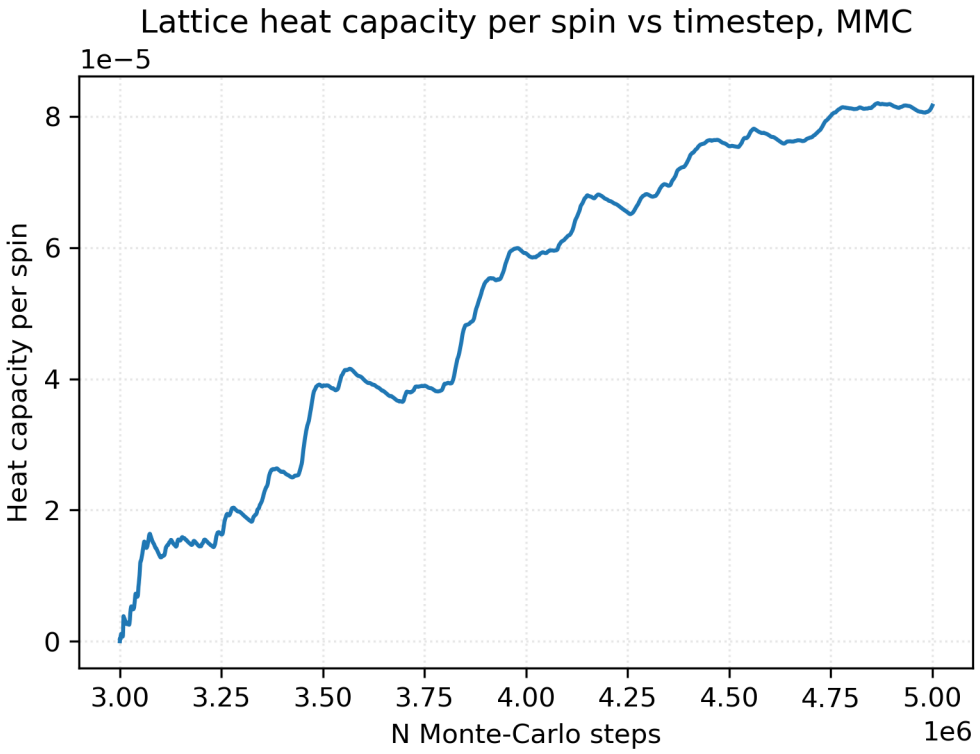



Lattice energy per spin vs timestep, MMC
Energy per spin / J
0.0
−0.2
−0.4
−0.6
−0.8
−1.0
−1.2
−1.4
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice magnetisation per spin vs timestep, MMC
Magnetisation per spin
0.15
0.10
0.05
0.00
−0.05
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
25000
0
−25000
−50000
−75000
−100000
−125000
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice magnetic susceptibility per spin vs timestep, MMC
Magnetic susceptibility per spin
0.00030
0.00025
0.00020
0.00015
0.00010
0.00005
0.00000
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
1e6
N Monte-Carlo steps

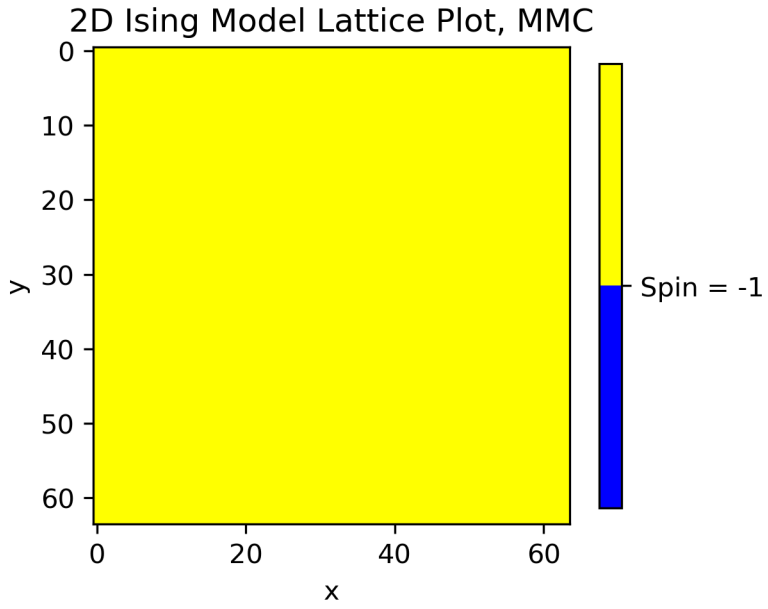


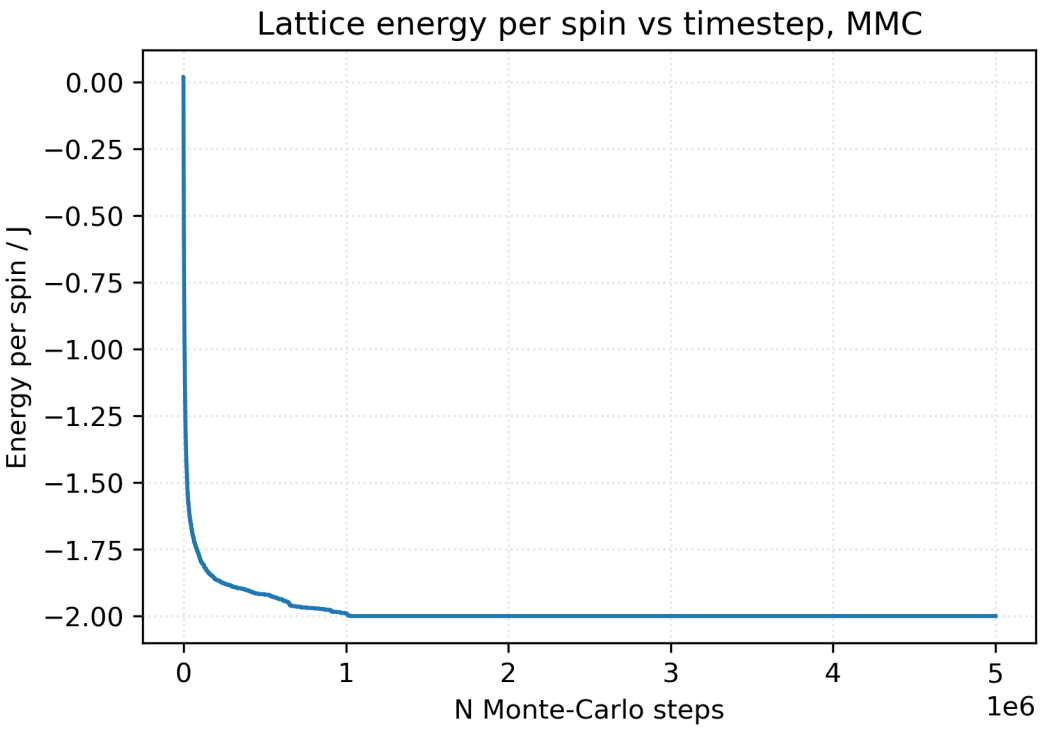
[29]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

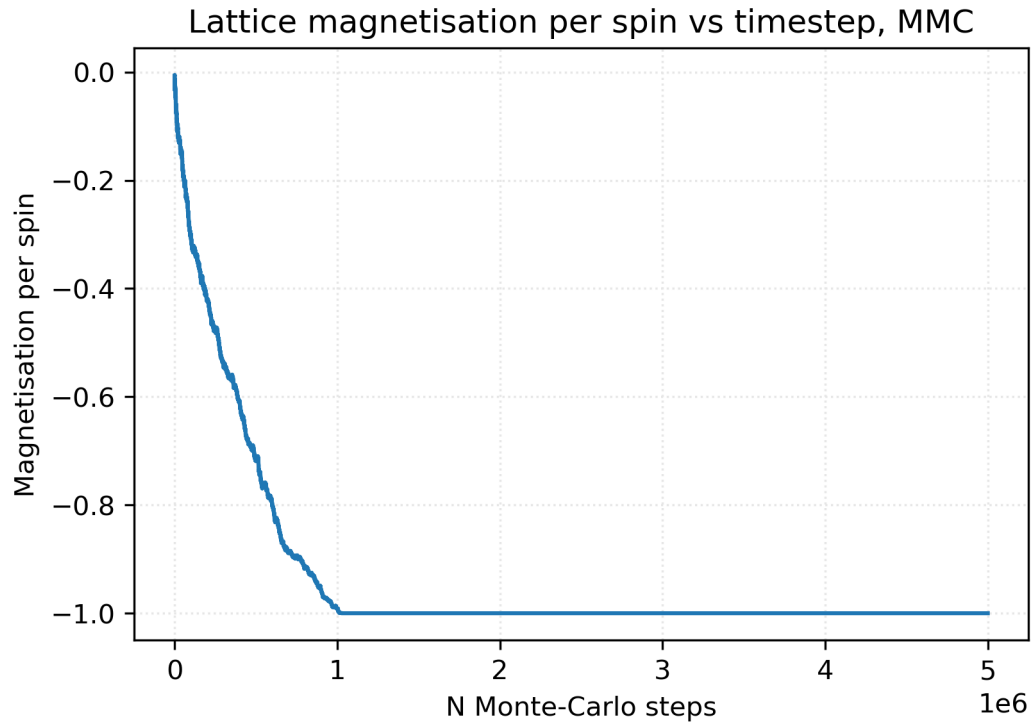
28 Visualising the effect of changing the temperature for a standard lattice size of 64x64 spins using the metropolis algorithm

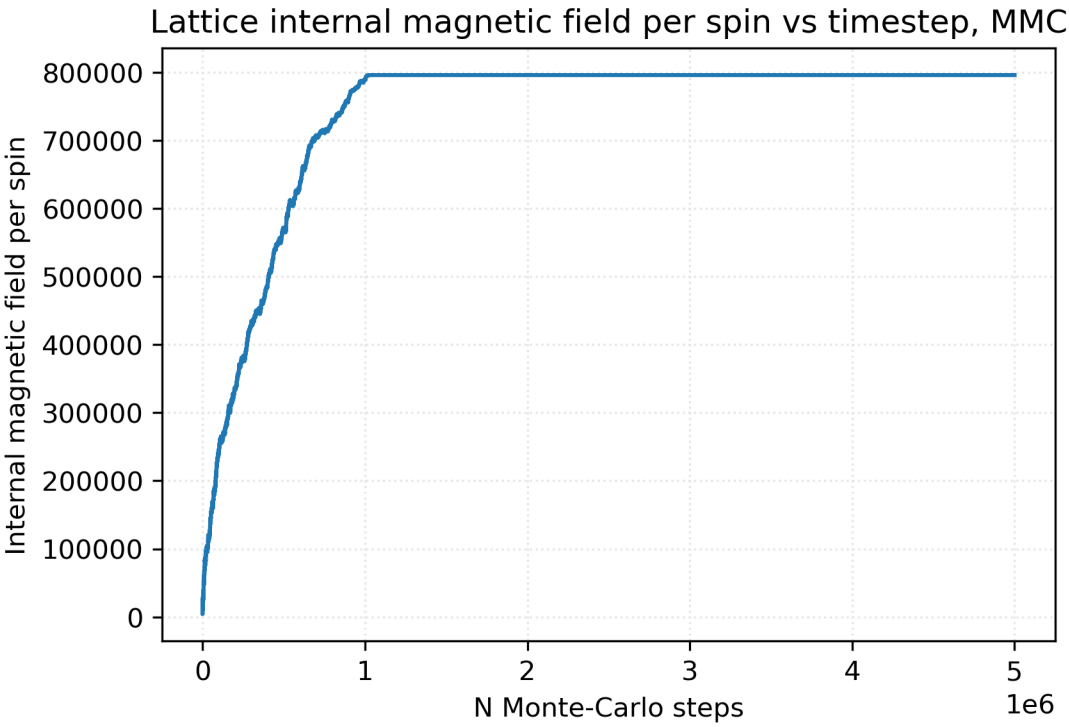
[30]:
```
# 0.001K (close to absolute 0)
parameter_file = 'ising_single_sim_T_0001_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

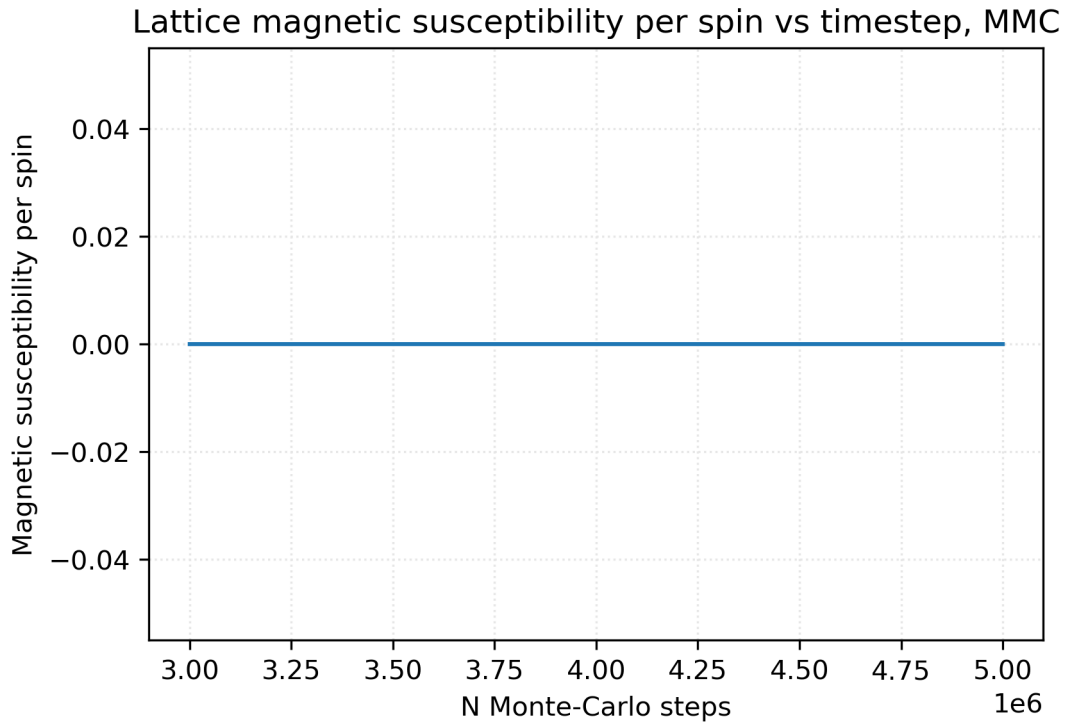
```
Running single Metropolis Algorithm simulation with parameters:
temperature=0.001K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=5000000

Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

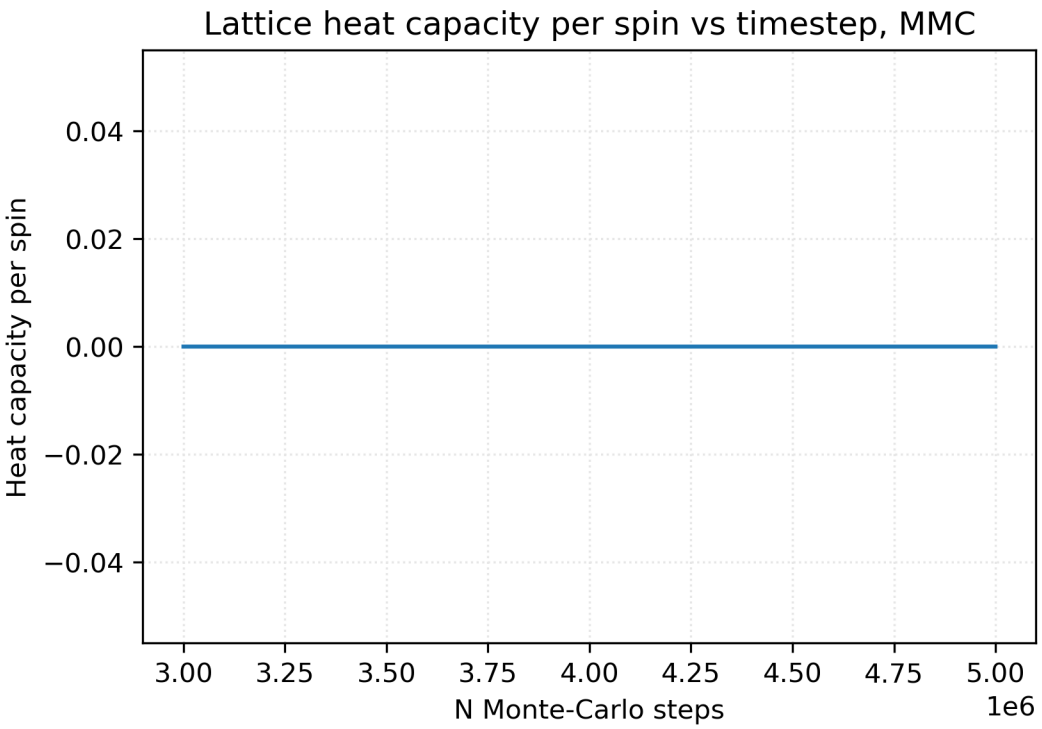



Lattice energy per spin vs timestep, MMC
0.00
−0.25
−0.50
−0.75
−1.00
−1.25
−1.50
−1.75
−2.00
Energy per spin / J
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice magnetisation per spin vs timestep, MMC
0.0
−0.2
−0.4
−0.6
−0.8
−1.0
Magnetisation per spin
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
800000
700000
600000
500000
400000
300000
200000
100000
0
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice magnetic susceptibility per spin vs timestep, MMC
Magnetic susceptibility per spin
0.04
0.02
0.00
−0.02
−0.04
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
1e6
N Monte-Carlo steps

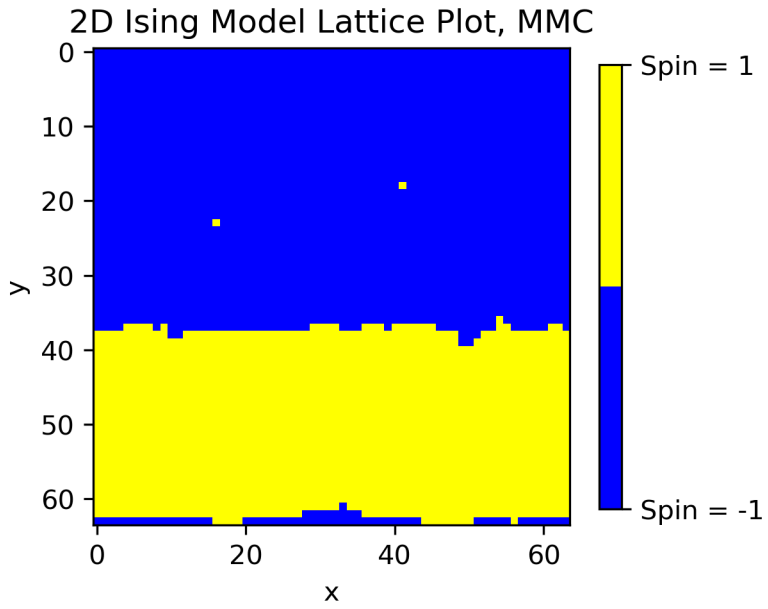


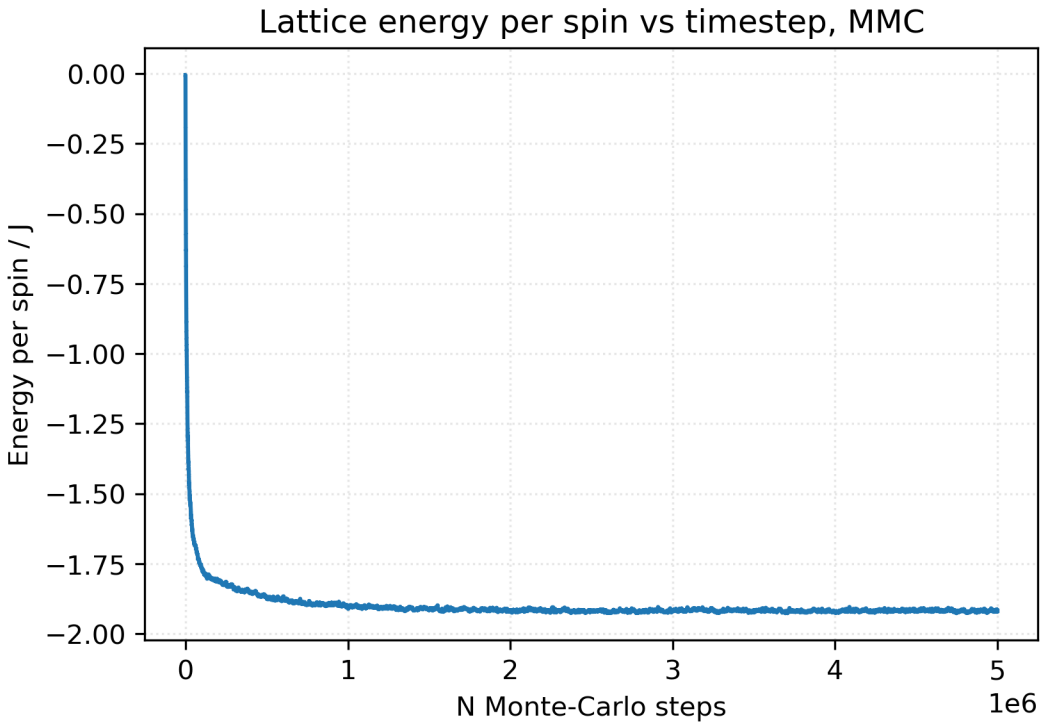
[30]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

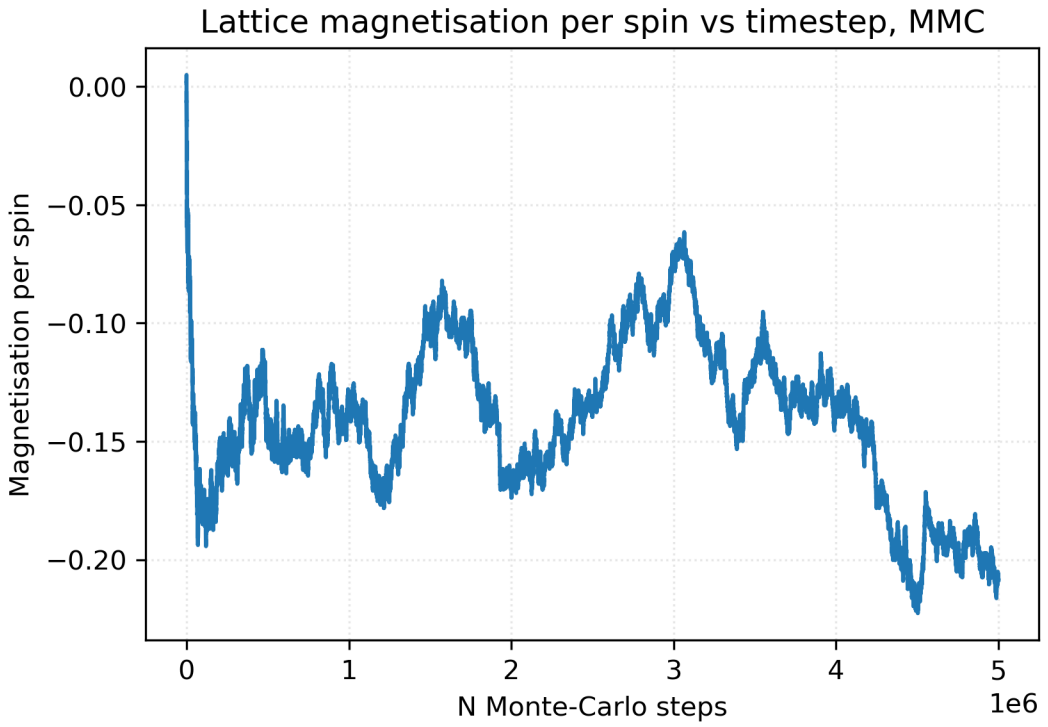
[31]:
```
# 1K (below tcrit)
parameter_file = 'ising_single_sim_T_1_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

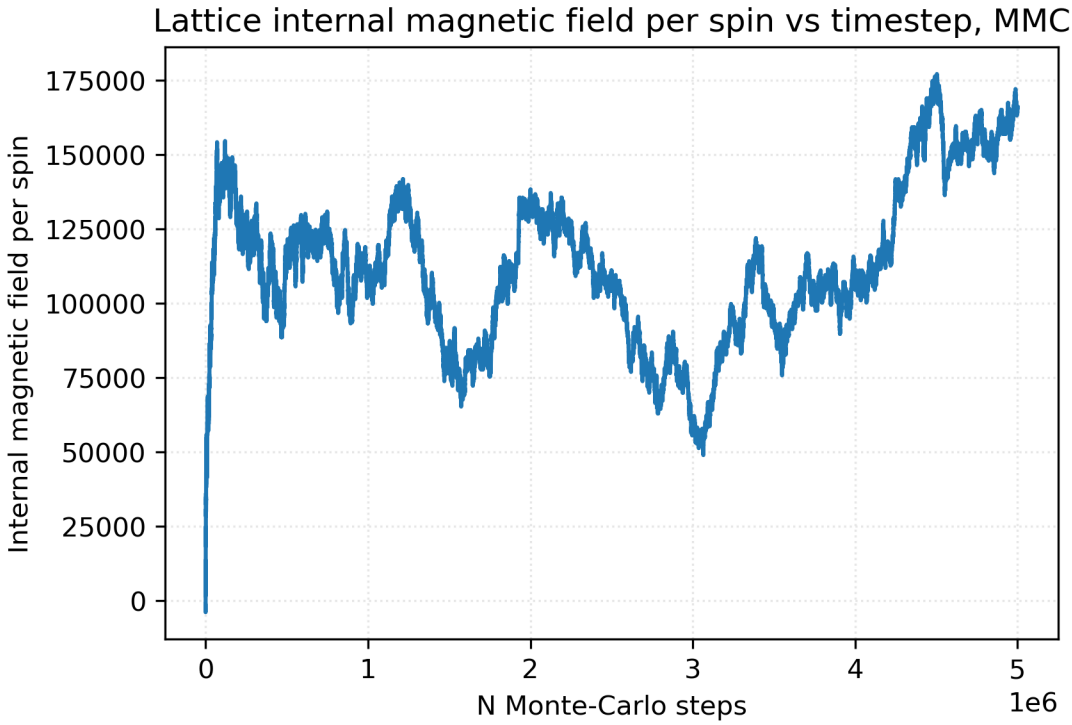
```
Running single Metropolis Algorithm simulation with parameters:
```

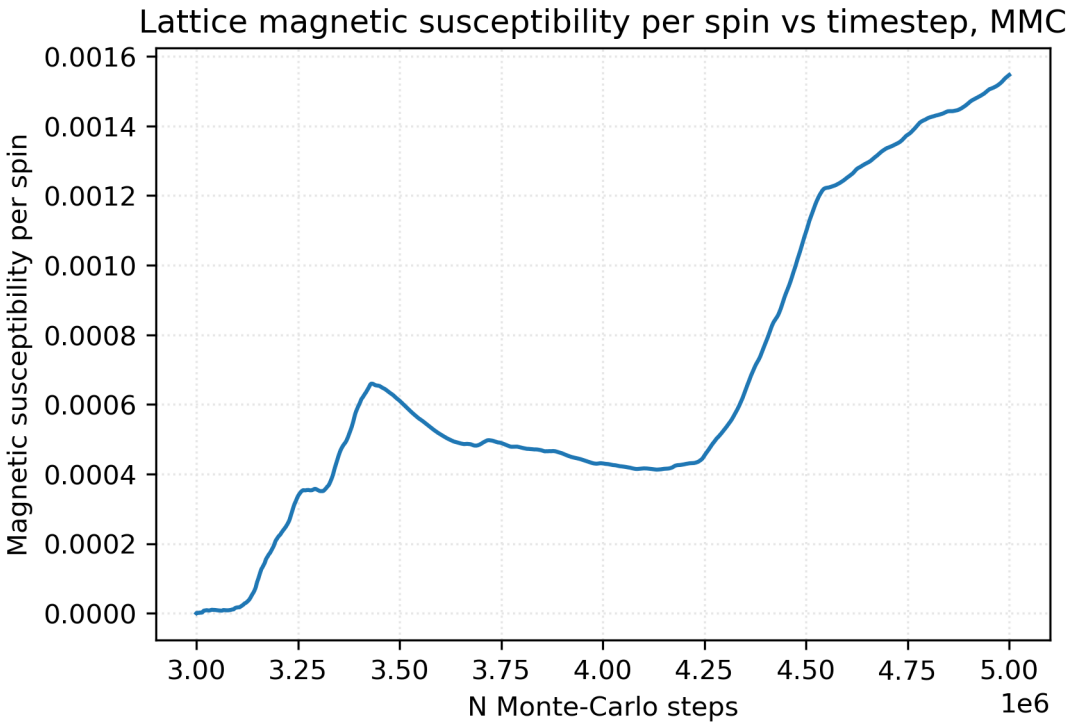
```
temperature=1.0K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=5000000

Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

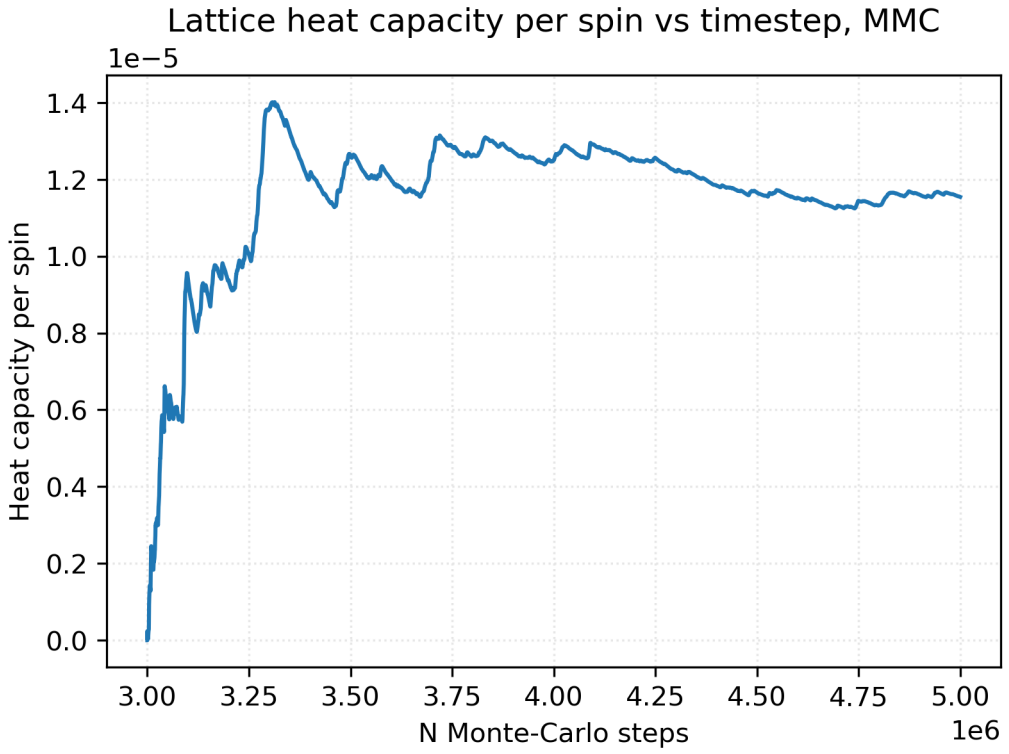





Lattice magnetisation per spin vs timestep, MMC
Magnetisation per spin
0.00
−0.05
−0.10
−0.15
−0.20
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
175000
150000
125000
100000
75000
50000
25000
0
0
1
2
3
4
5
1e6
N Monte-Carlo steps

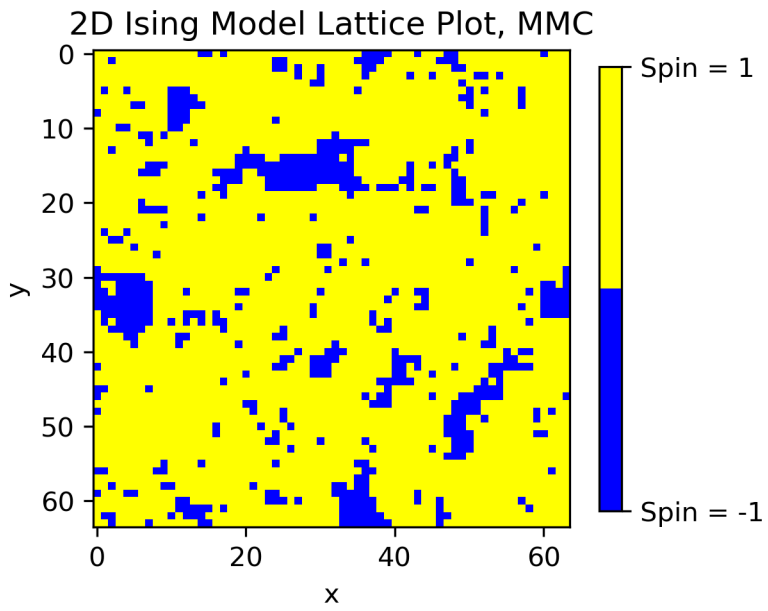

Lattice magnetic susceptibility per spin vs timestep, MMC
Magnetic susceptibility per spin
0.0016
0.0014
0.0012
0.0010
0.0008
0.0006
0.0004
0.0002
0.0000
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
1e6
N Monte-Carlo steps

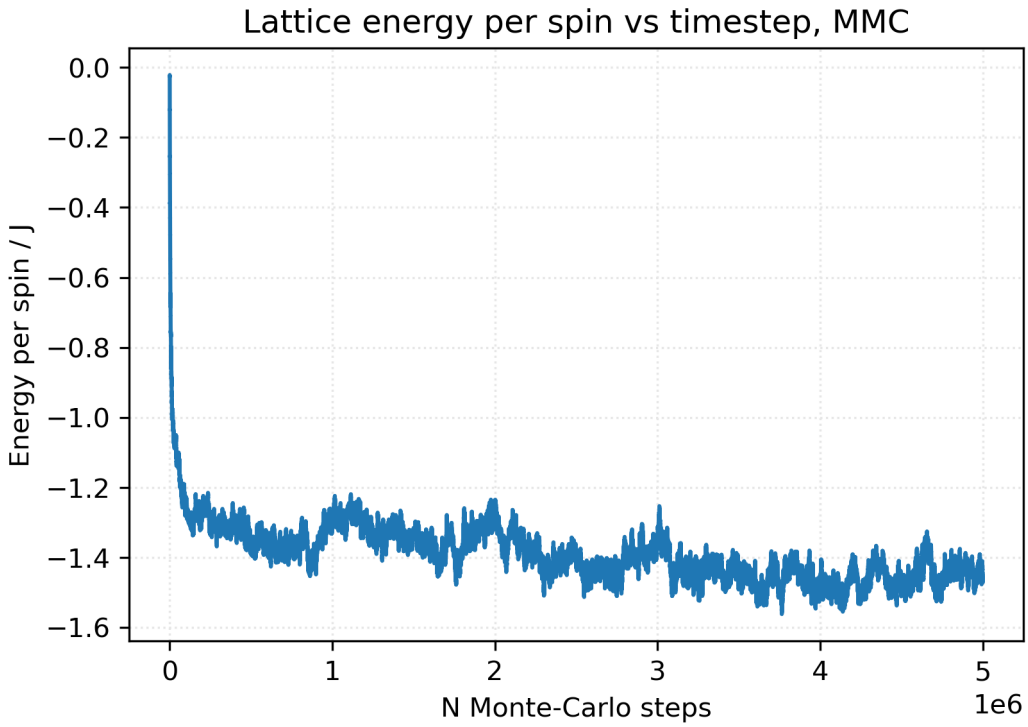

Lattice heat capacity per spin vs timestep, MMC
1e−5
Heat capacity per spin
1.4
1.2
1.0
0.8
0.6
0.4
0.2
0.0
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
1e6
N Monte-Carlo steps

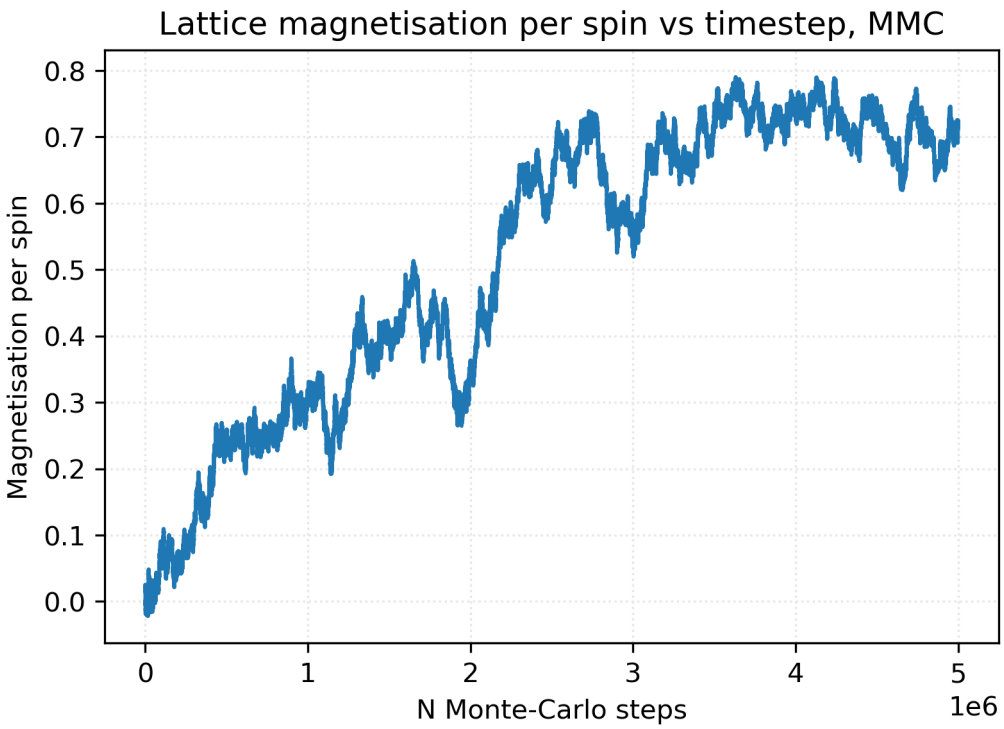
[31]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

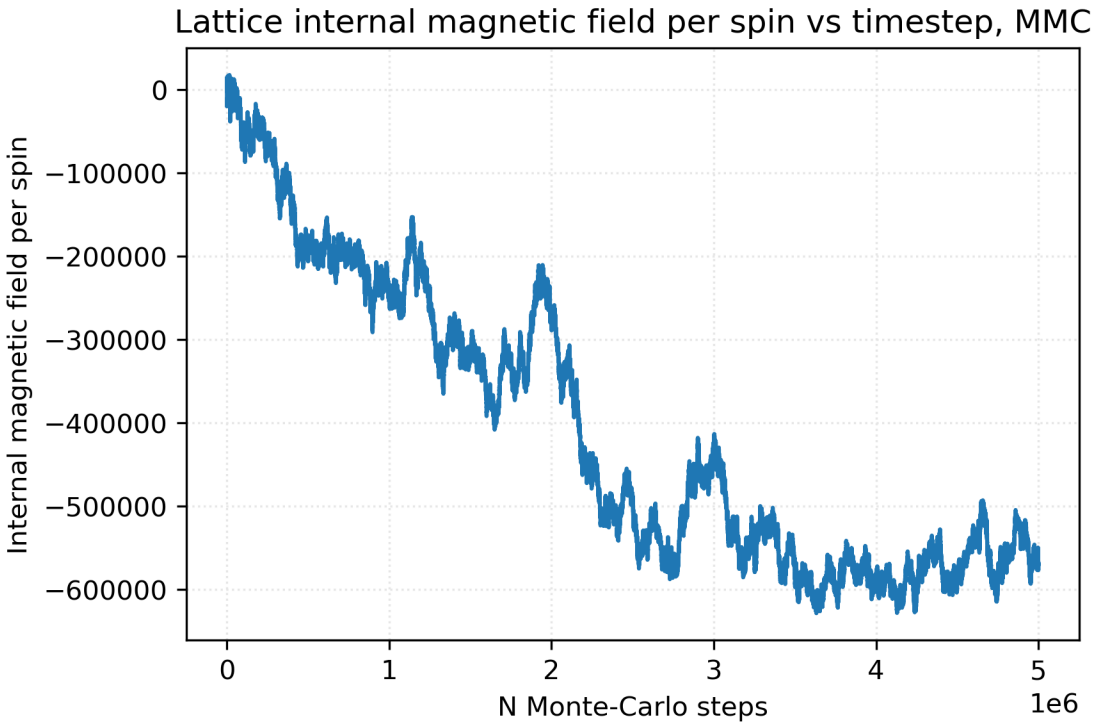
[32]:
```
# 2.269185K (tcrit)
parameter_file = 'ising_single_sim_T_crit_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

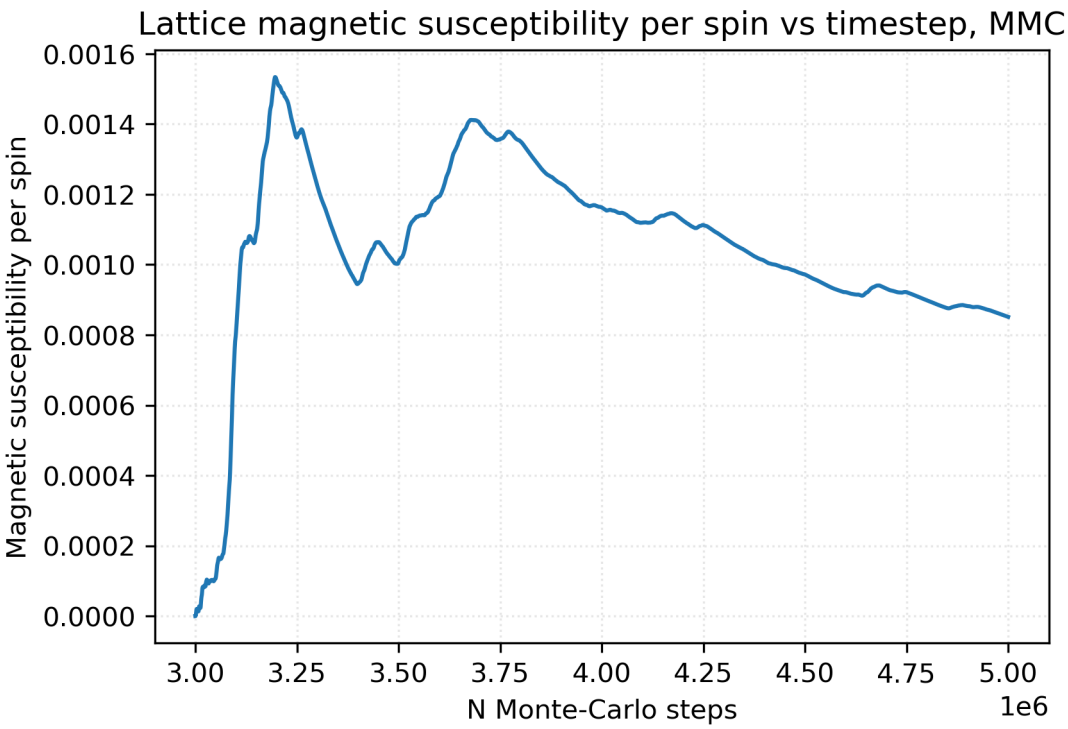
```
Running single Metropolis Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=5000000


Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

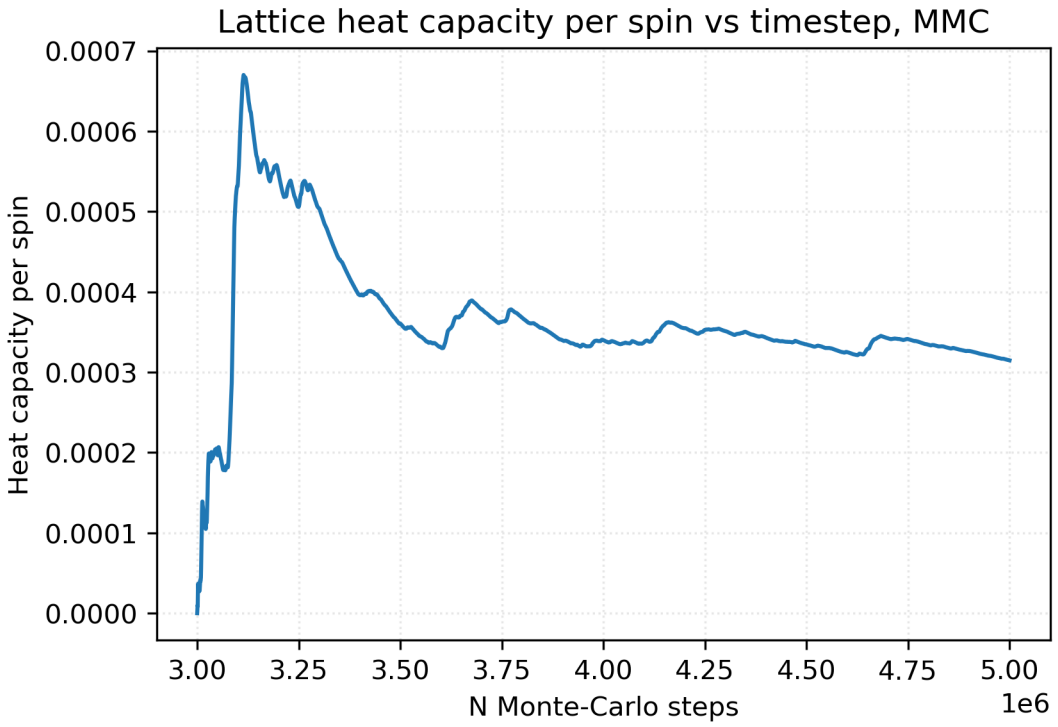

2D Ising Model Lattice Plot, MMC
0
10
20
30
40
50
60
y
0
20
40
60
x
Spin = 1
Spin = -1


Lattice energy per spin vs timestep, MMC
0.0
−0.2
−0.4
−0.6
−0.8
−1.0
−1.2
−1.4
−1.6
Energy per spin / J
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice magnetisation per spin vs timestep, MMC
Magnetisation per spin
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
0
−100000
−200000
−300000
−400000
−500000
−600000
0
1
2
3
4
5
1e6
N Monte-Carlo steps

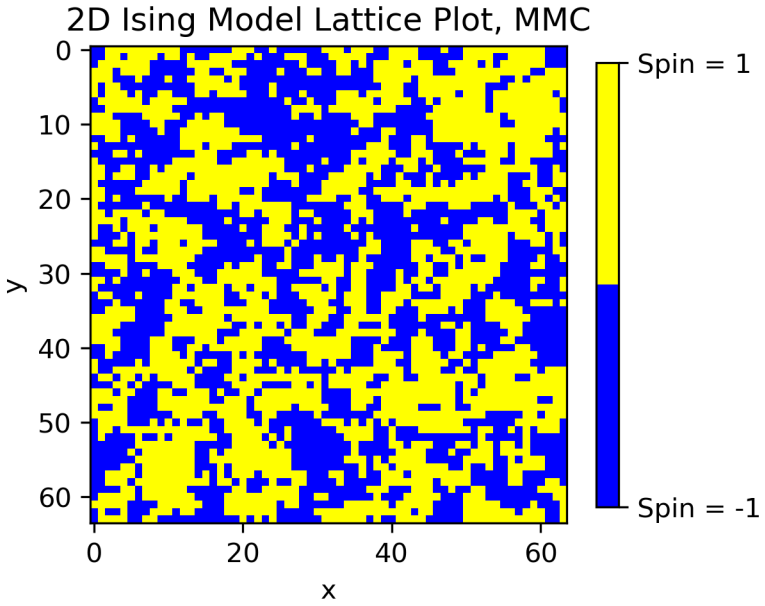

Lattice magnetic susceptibility per spin vs timestep, MMC
Magnetic susceptibility per spin
0.0016
0.0014
0.0012
0.0010
0.0008
0.0006
0.0004
0.0002
0.0000
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
N Monte-Carlo steps
1e6

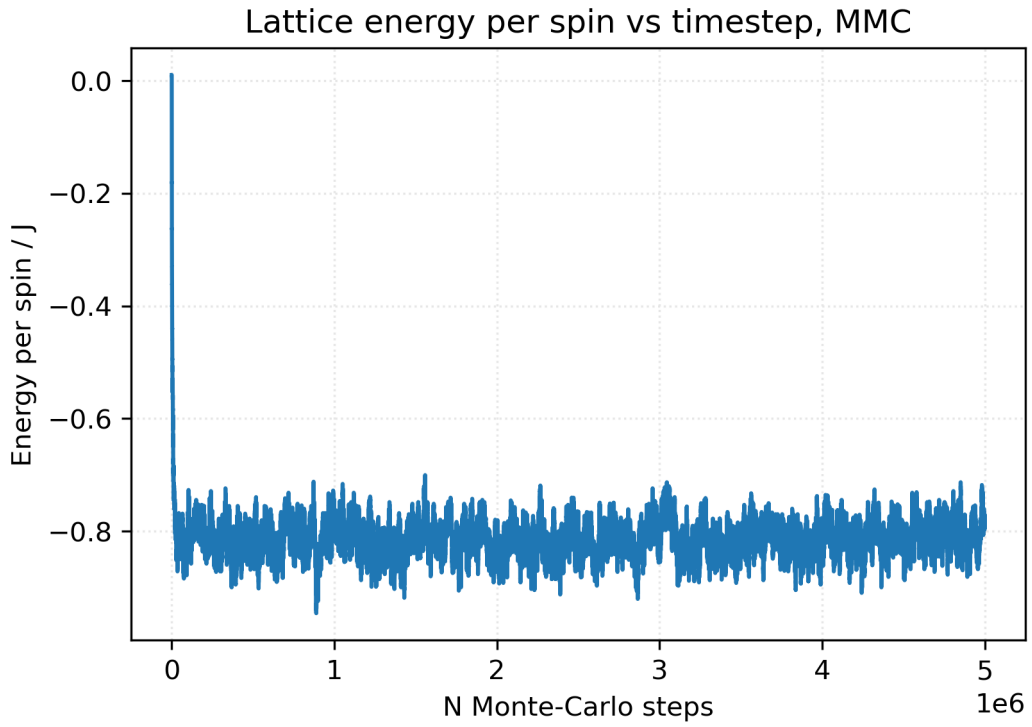

Lattice heat capacity per spin vs timestep, MMC
Heat capacity per spin
0.0007
0.0006
0.0005
0.0004
0.0003
0.0002
0.0001
0.0000
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
N Monte-Carlo steps
1e6

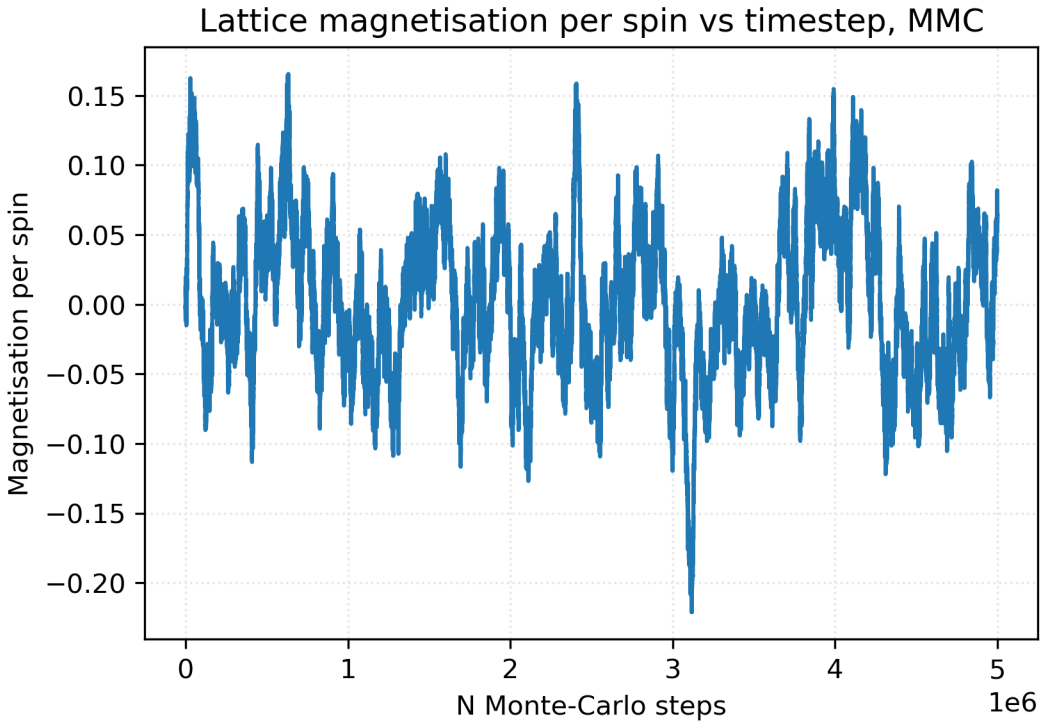
[32]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

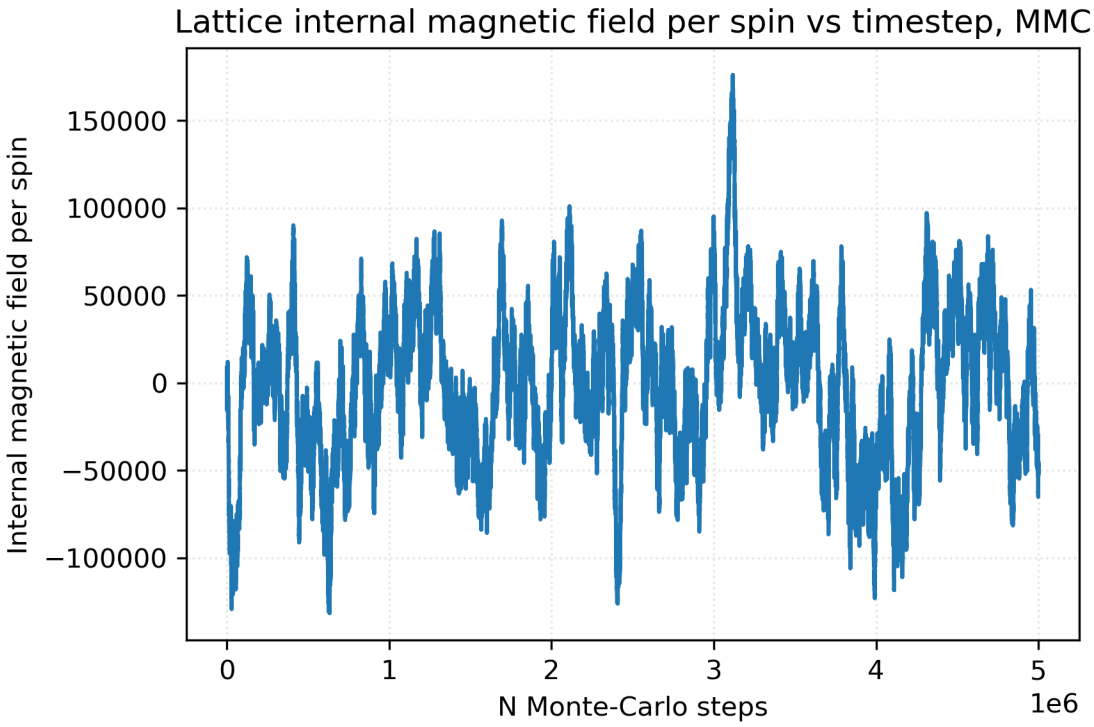
[33]:
```
# 3K (above tcrit)
parameter_file = 'ising_single_sim_T_3_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

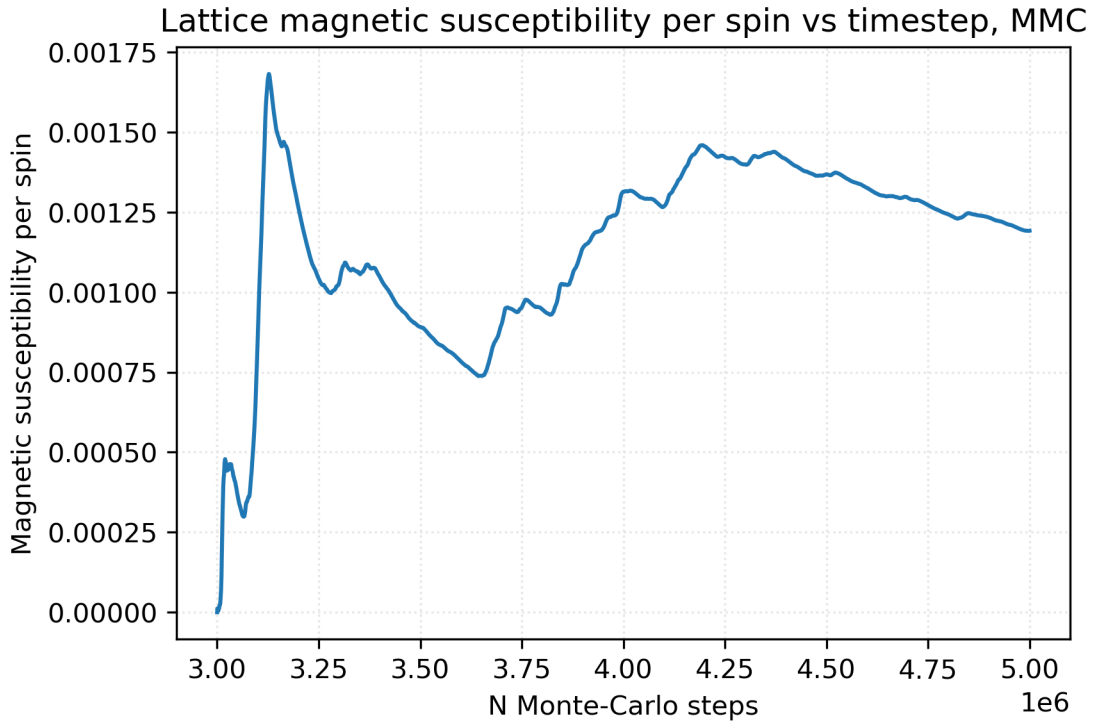
```
Running single Metropolis Algorithm simulation with parameters:
temperature=3.0K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=5000000


Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

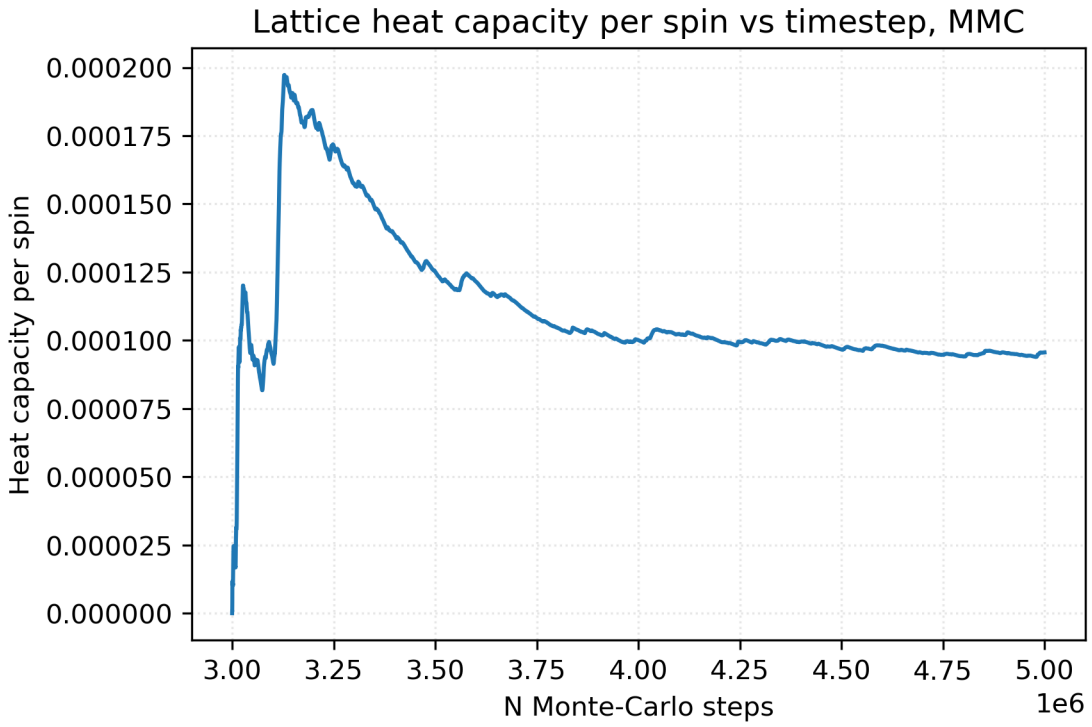



Lattice energy per spin vs timestep, MMC
0.0
−0.2
−0.4
−0.6
−0.8
Energy per spin / J
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice magnetisation per spin vs timestep, MMC
0.15
0.10
0.05
0.00
−0.05
−0.10
−0.15
−0.20
Magnetisation per spin
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
150000
100000
50000
0
−50000
−100000
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice magnetic susceptibility per spin vs timestep, MMC
Magnetic susceptibility per spin
0.00175
0.00150
0.00125
0.00100
0.00075
0.00050
0.00025
0.00000
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
1e6
N Monte-Carlo steps

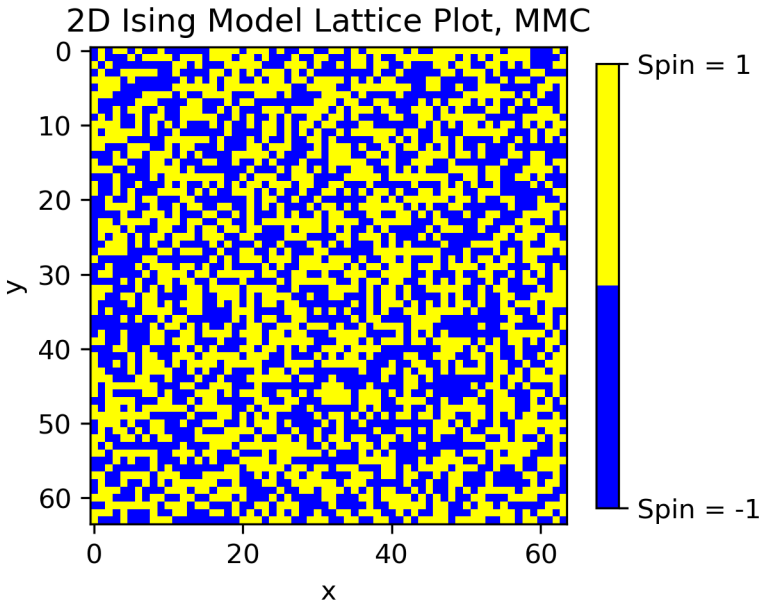


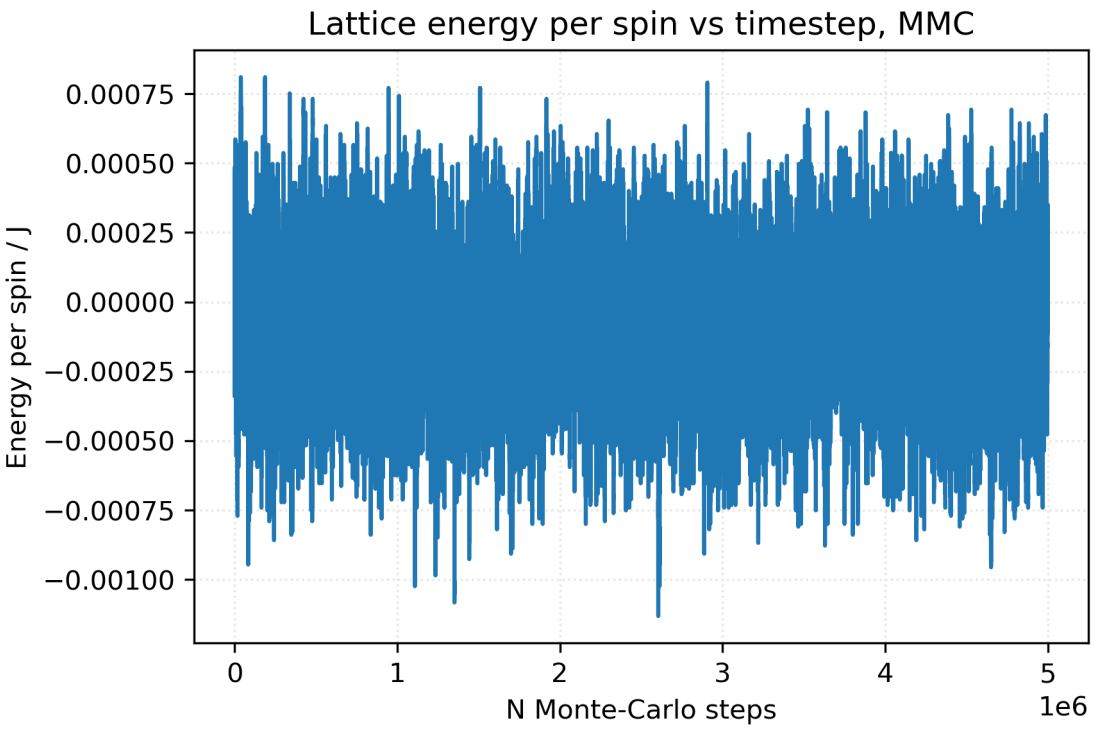
[33]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

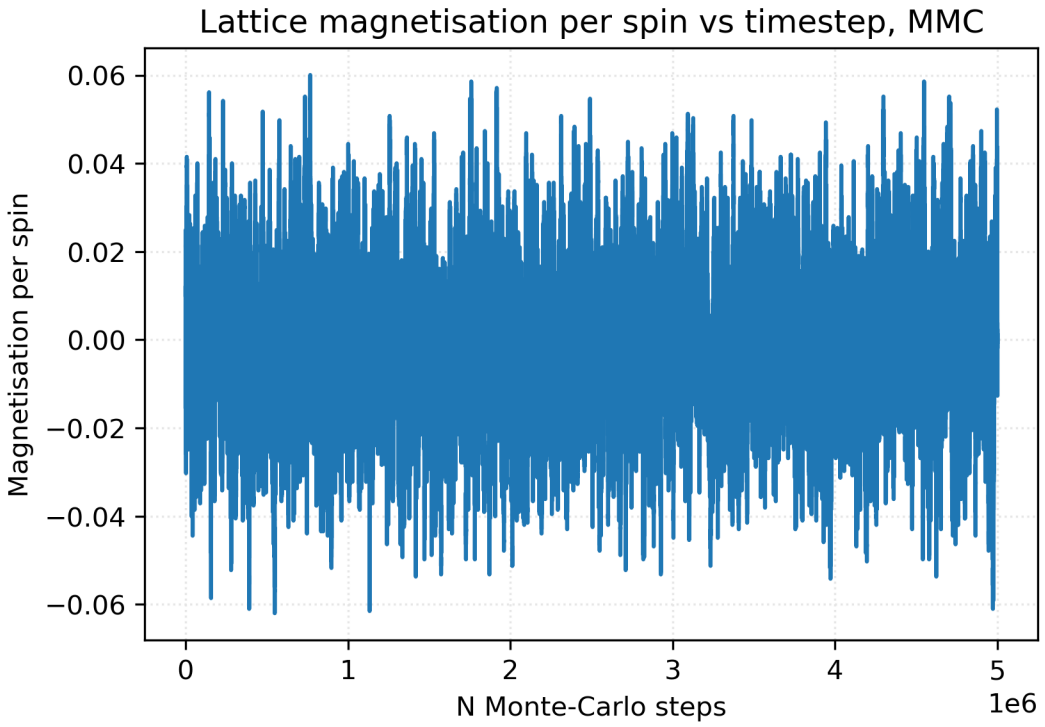
29 Visualising the effect of changing exchange interaction constant J for a standard lattice size of 64x64 spins using the metropolis algorithm, whilst remaining at the onsager critical temperature

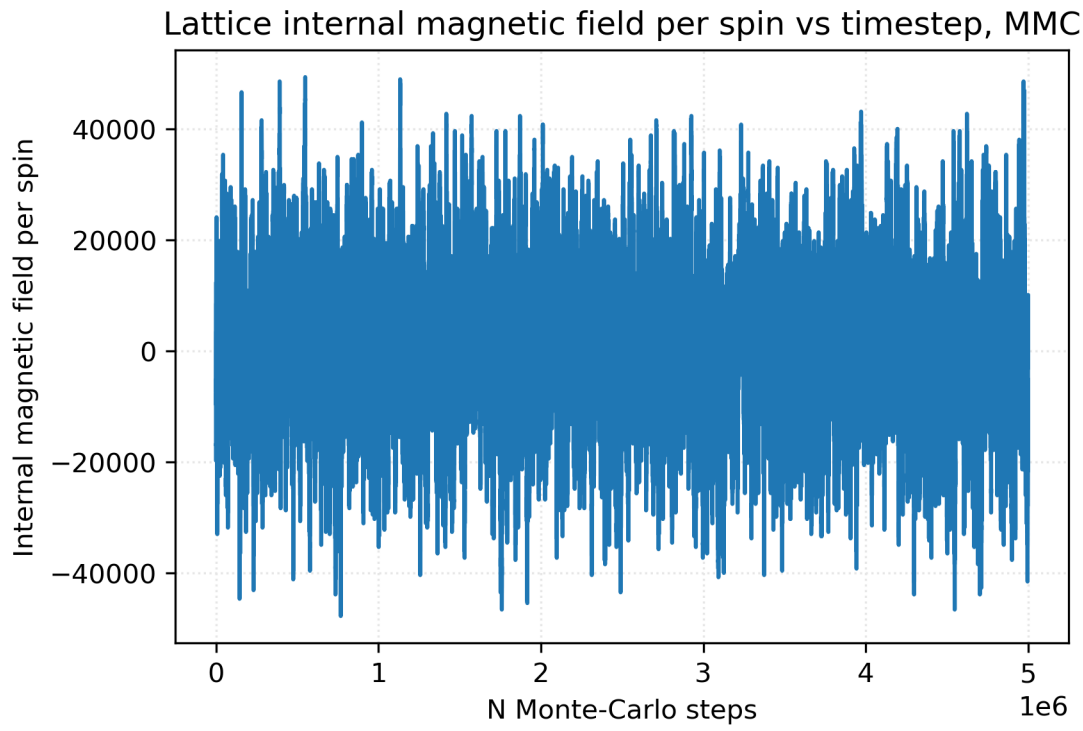
[34]:
```
# J = 0.01
parameter_file = 'ising_single_sim_J_001_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

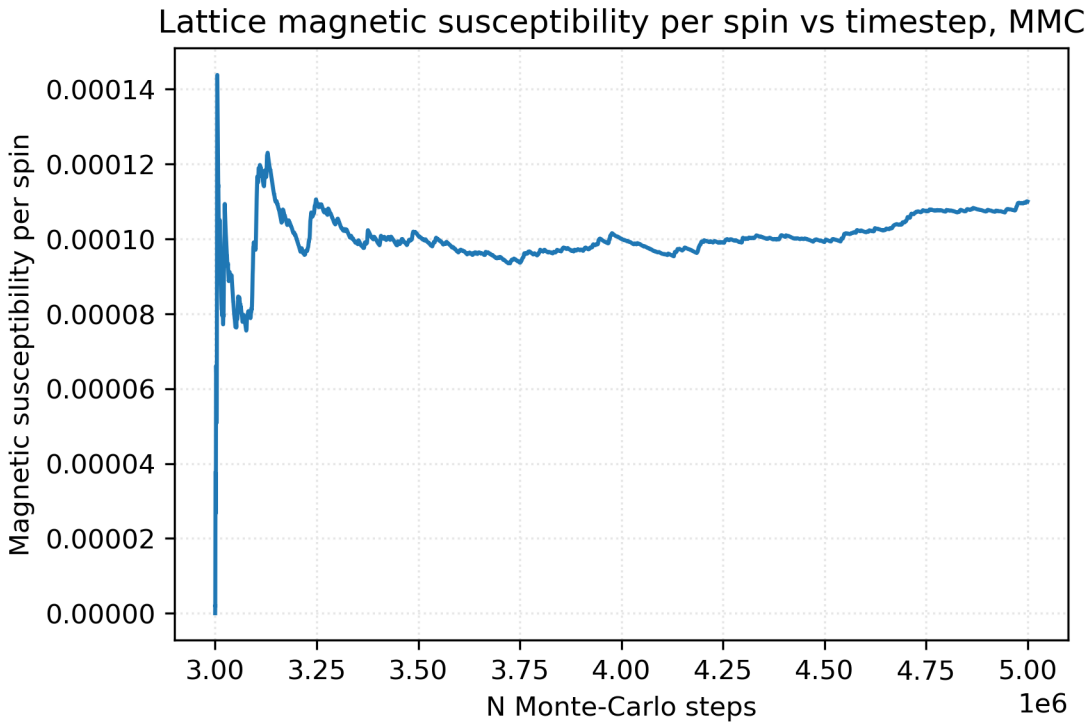
```
Running single Metropolis Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=0.01, k_b=1.0, u_b=0.0, nsteps=5000000


Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

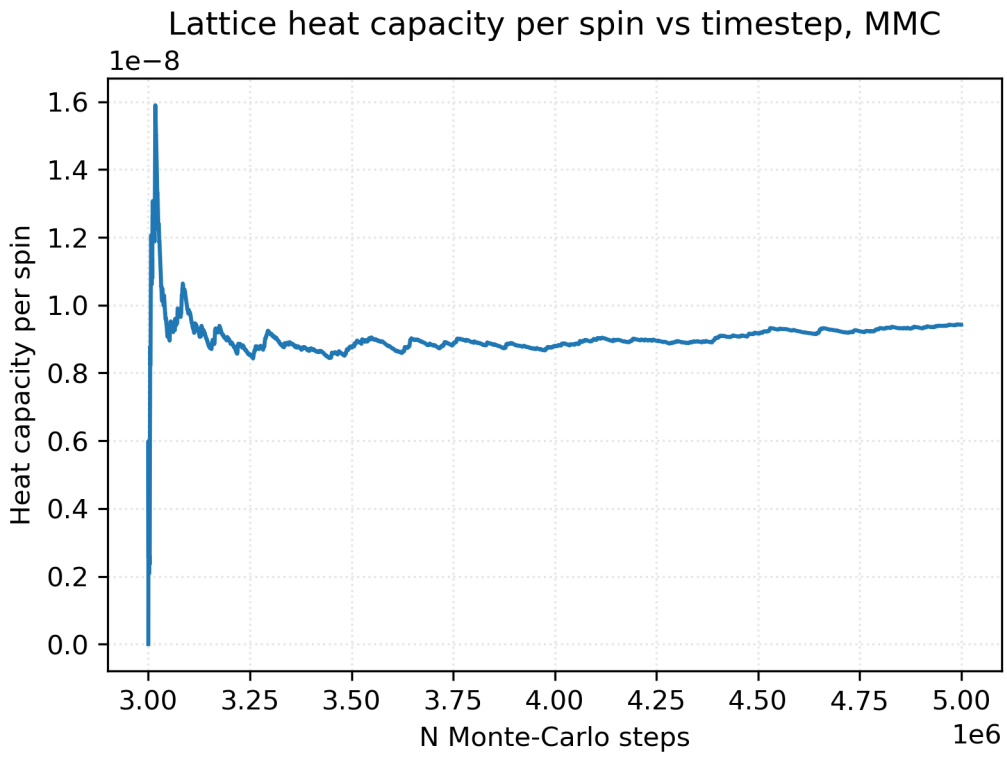



Lattice energy per spin vs timestep, MMC
Energy per spin / J
0.00075
0.00050
0.00025
0.00000
−0.00025
−0.00050
−0.00075
−0.00100
0
1
2
3
4
5
N Monte-Carlo steps
1e6


Lattice magnetisation per spin vs timestep, MMC
Magnetisation per spin
0.06
0.04
0.02
0.00
−0.02
−0.04
−0.06
0
1
2
3
4
5
N Monte-Carlo steps
1e6


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
40000
20000
0
−20000
−40000
0
1
2
3
4
5
1e6
N Monte-Carlo steps

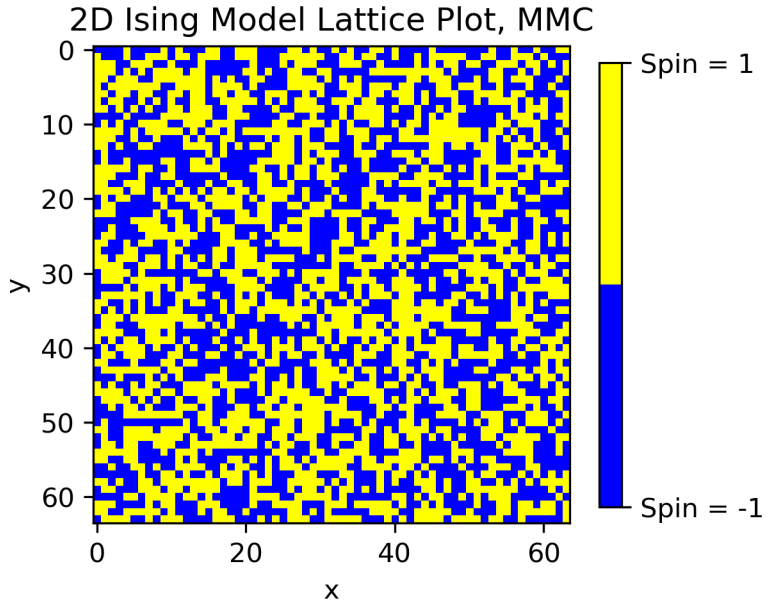

Lattice magnetic susceptibility per spin vs timestep, MMC
Magnetic susceptibility per spin
0.00014
0.00012
0.00010
0.00008
0.00006
0.00004
0.00002
0.00000
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
1e6
N Monte-Carlo steps

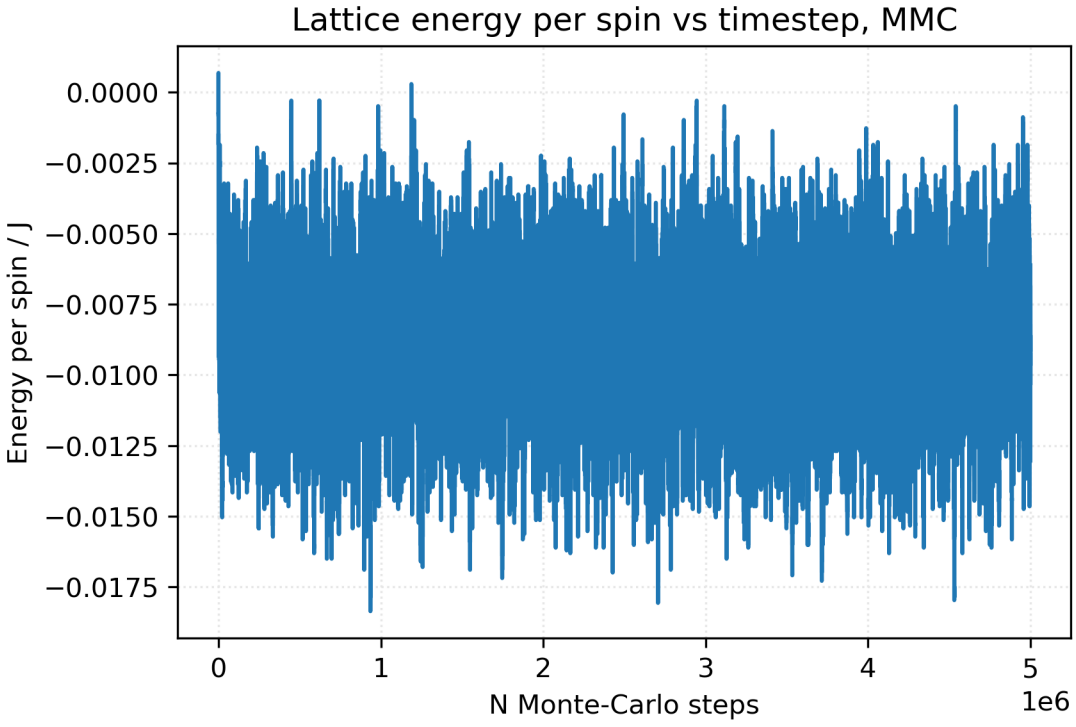


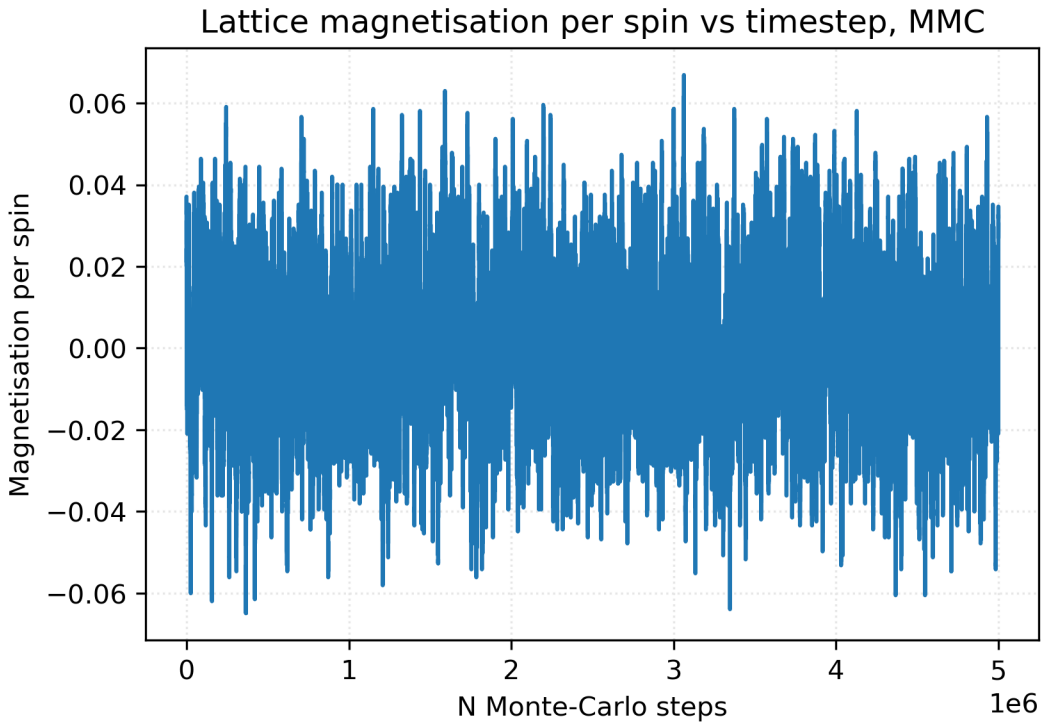
[34]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

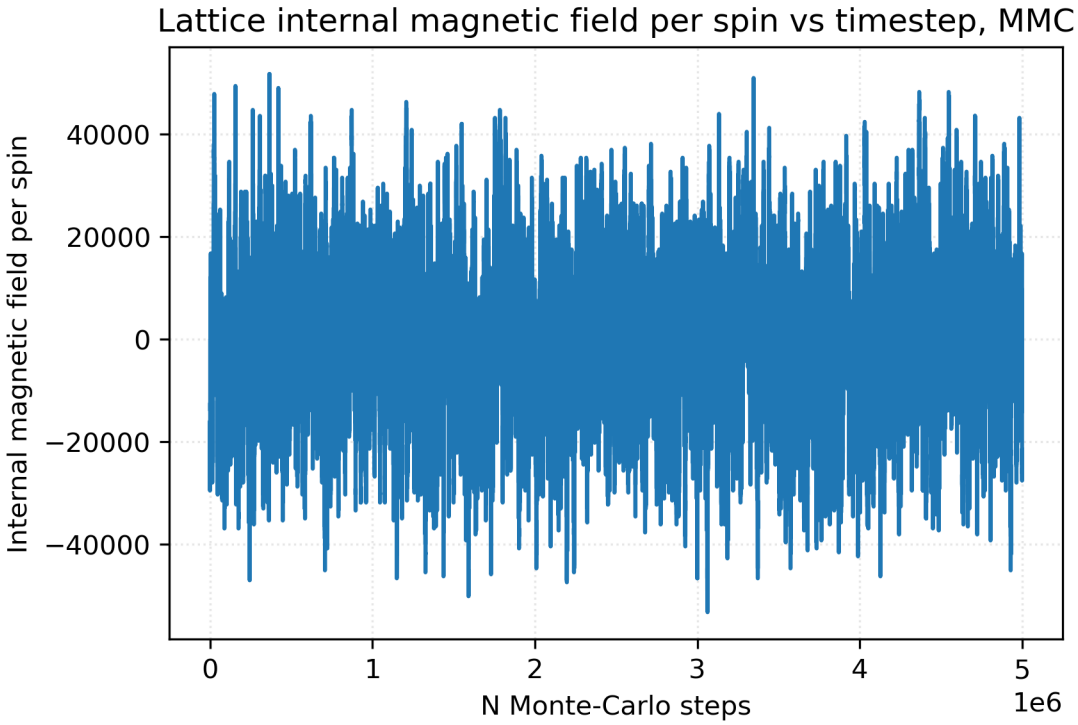
[35]:
```
# J = 0.1
parameter_file = 'ising_single_sim_J_01_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

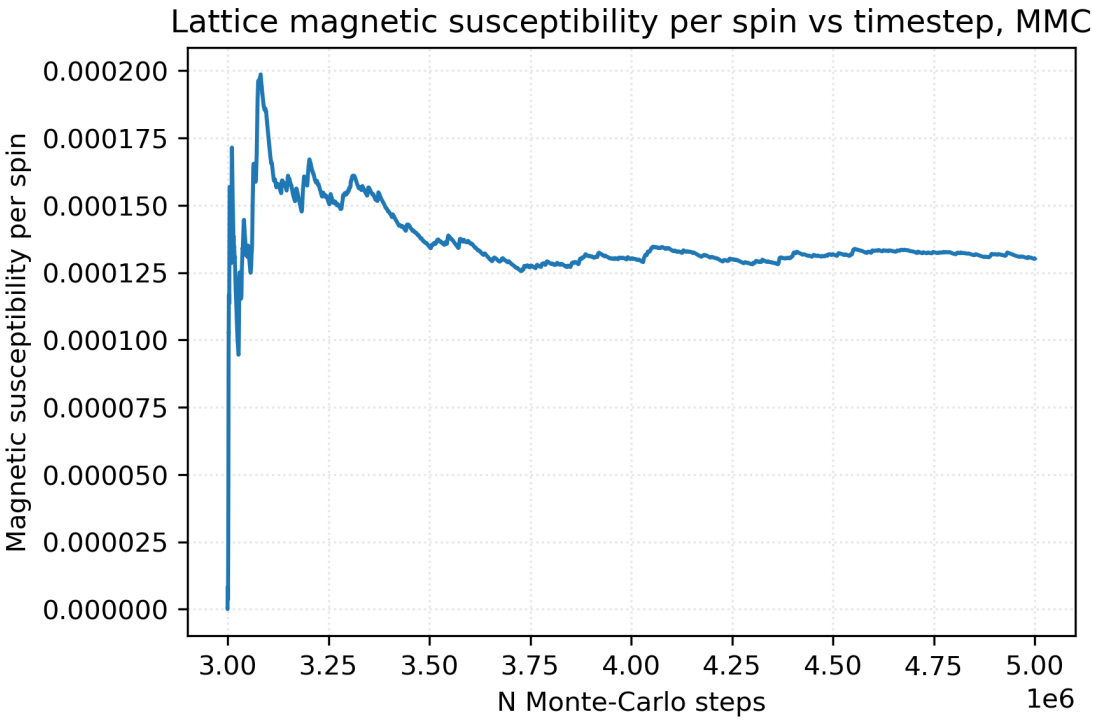
```
Running single Metropolis Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=0.1, k_b=1.0, u_b=0.0, nsteps=5000000

Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

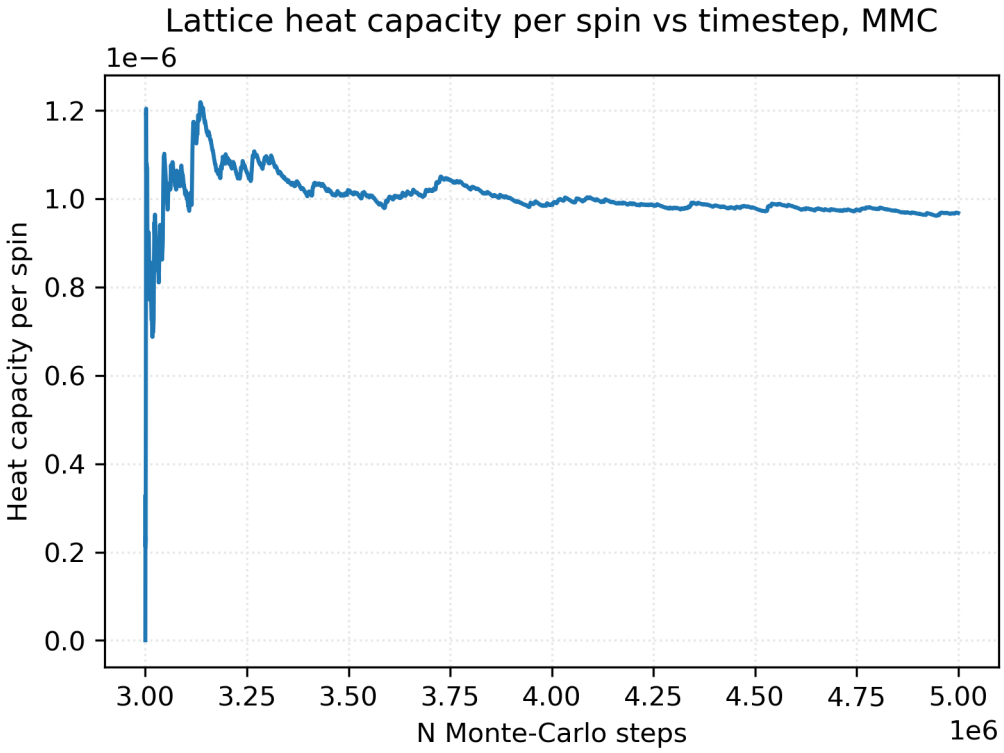





Lattice magnetisation per spin vs timestep, MMC
Magnetisation per spin
0.06
0.04
0.02
0.00
−0.02
−0.04
−0.06
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
40000
20000
0
−20000
−40000
0
1
2
3
4
5
1e6
N Monte-Carlo steps

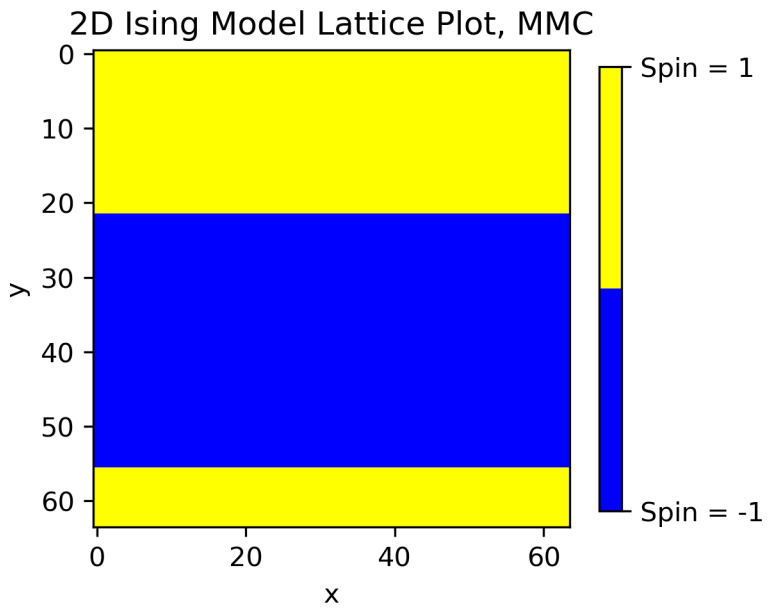

Lattice magnetic susceptibility per spin vs timestep, MMC
Magnetic susceptibility per spin
0.000200
0.000175
0.000150
0.000125
0.000100
0.000075
0.000050
0.000025
0.000000
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
1e6
N Monte-Carlo steps

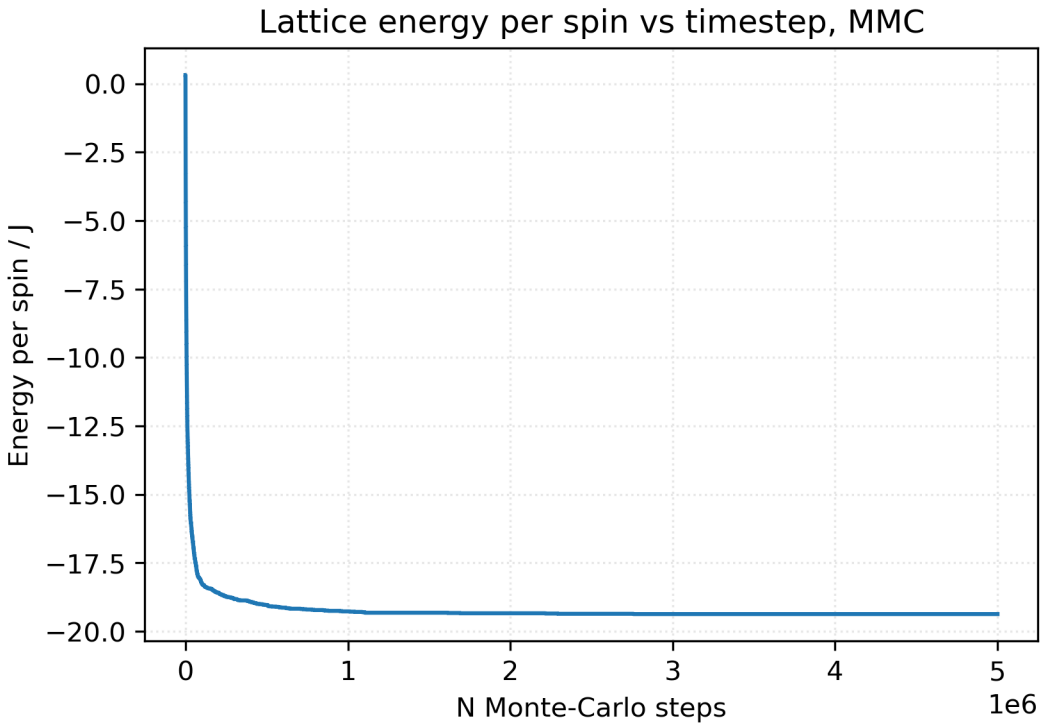

Lattice heat capacity per spin vs timestep, MMC
1e−6
Heat capacity per spin
1.2
1.0
0.8
0.6
0.4
0.2
0.0
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
1e6
N Monte-Carlo steps

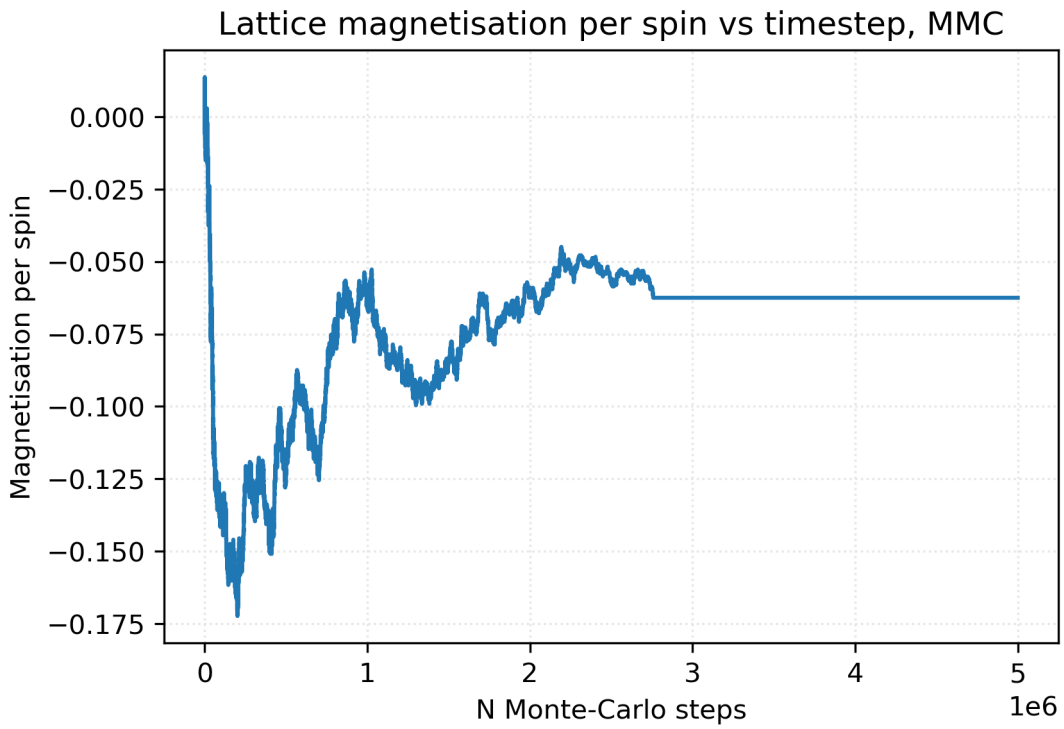
[35]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

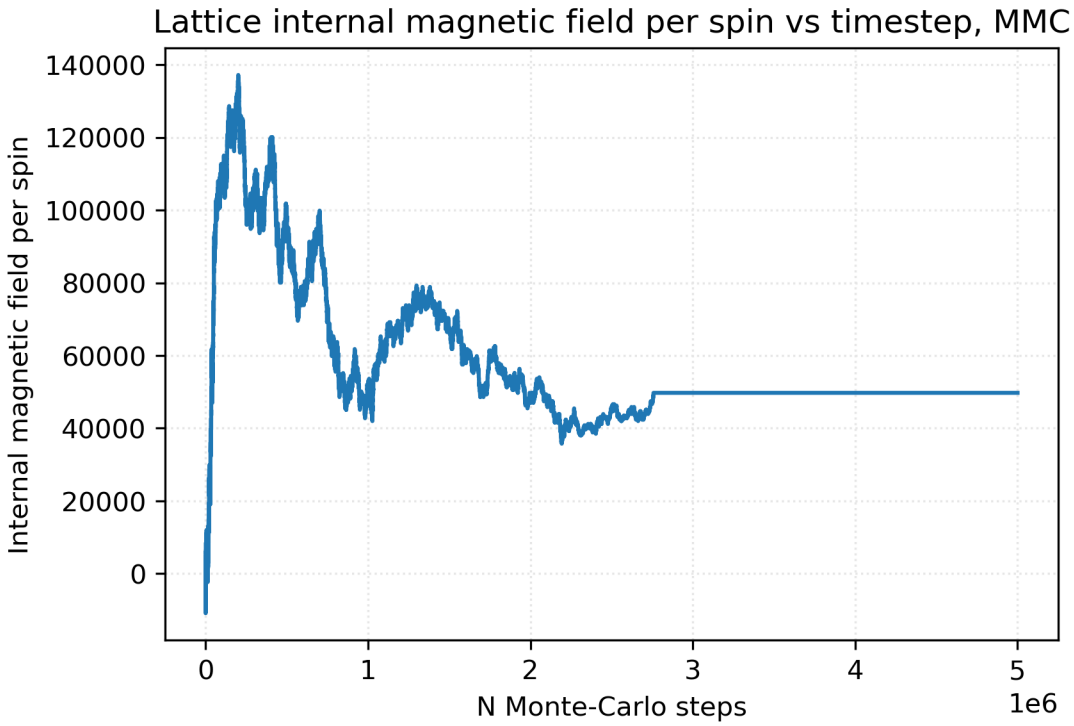
[36]:
```
# J = 10
parameter_file = 'ising_single_sim_J_10_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

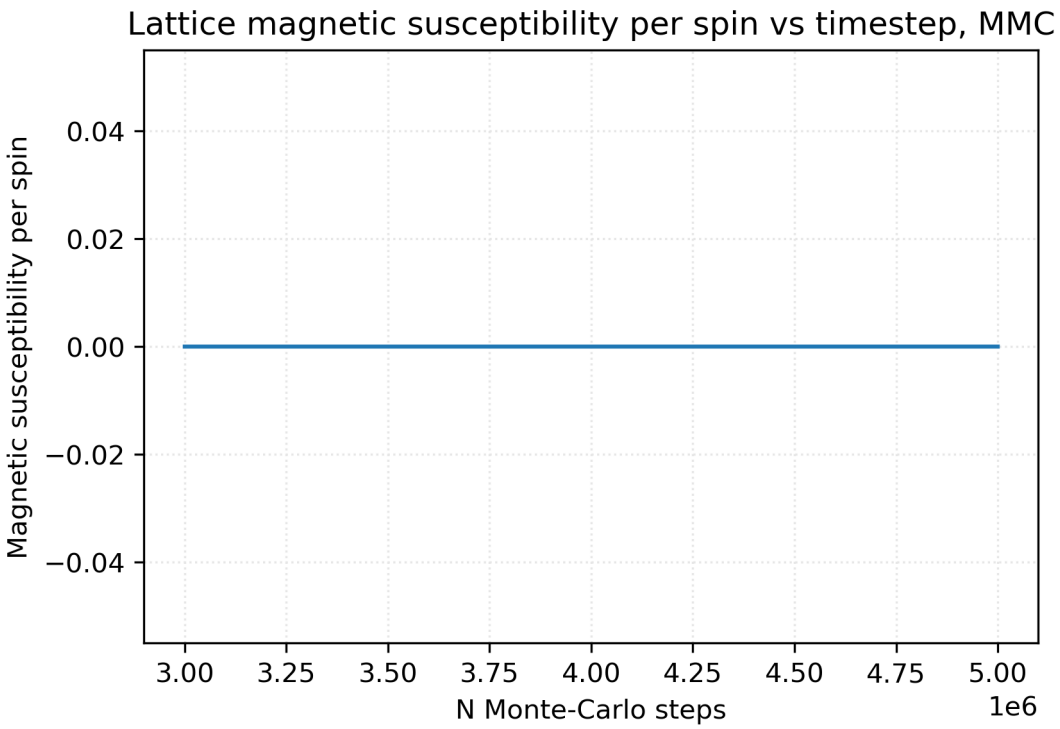
```
Running single Metropolis Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=10.0, k_b=1.0, u_b=0.0, nsteps=5000000


Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

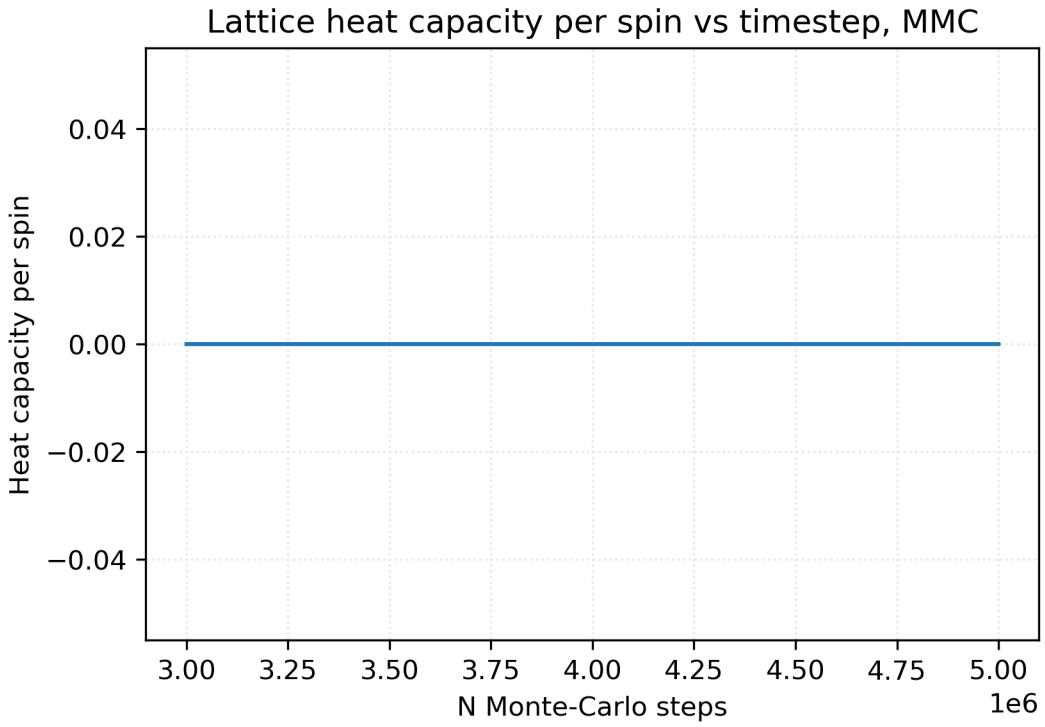

2D Ising Model Lattice Plot, MMC
0
10
20
30
40
50
60
y
0
20
40
60
x
Spin = 1
Spin = -1


Lattice energy per spin vs timestep, MMC
0.0
−2.5
−5.0
−7.5
−10.0
−12.5
−15.0
−17.5
−20.0
Energy per spin / J
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice magnetisation per spin vs timestep, MMC
Magnetisation per spin
0.000
−0.025
−0.050
−0.075
−0.100
−0.125
−0.150
−0.175
0
1
2
3
4
5
N Monte-Carlo steps
1e6


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
140000
120000
100000
80000
60000
40000
20000
0
0
1
2
3
4
5
N Monte-Carlo steps
1e6

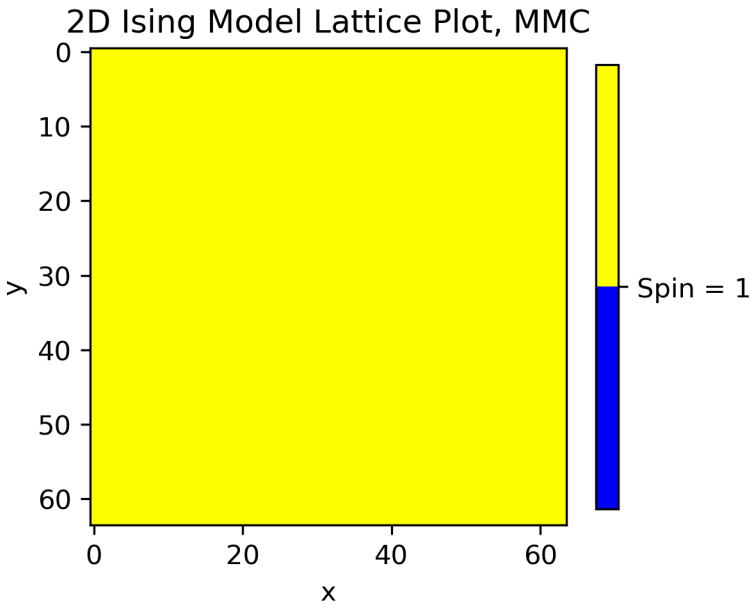

Lattice magnetic susceptibility per spin vs timestep, MMC
Magnetic susceptibility per spin
0.04
0.02
0.00
−0.02
−0.04
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
1e6
N Monte-Carlo steps

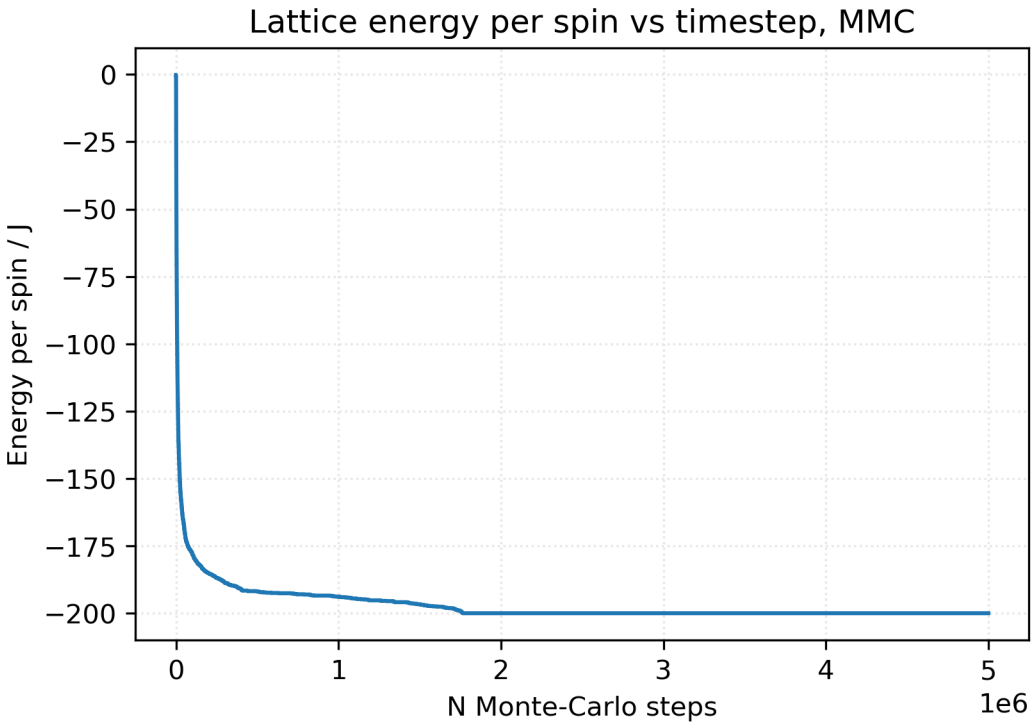

Lattice heat capacity per spin vs timestep, MMC
Heat capacity per spin
0.04
0.02
0.00
−0.02
−0.04
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
1e6
N Monte-Carlo steps

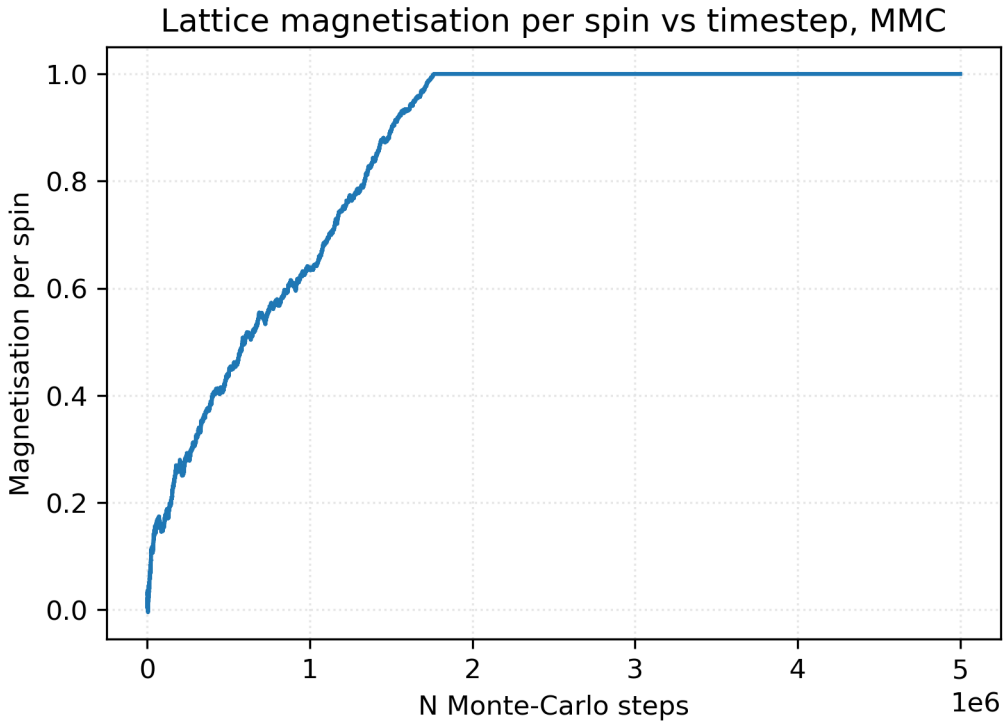
[36]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

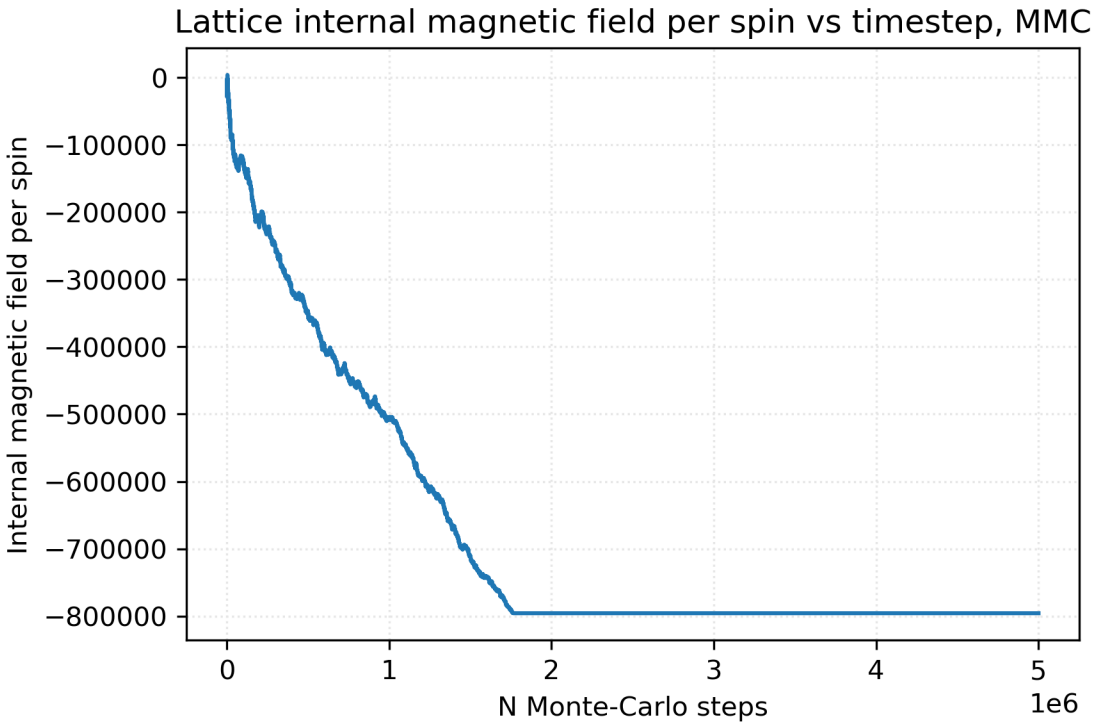
[37]:
```
# J = 100
parameter_file = 'ising_single_sim_J_100_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

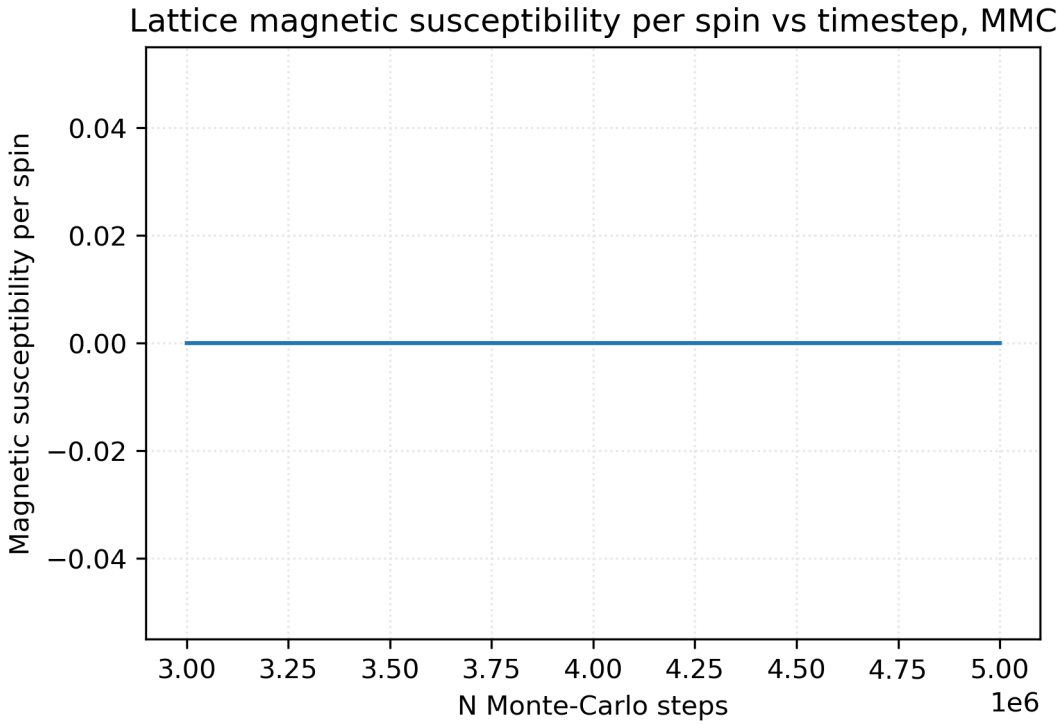
```
Running single Metropolis Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=100.0, k_b=1.0, u_b=0.0, nsteps=5000000


Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

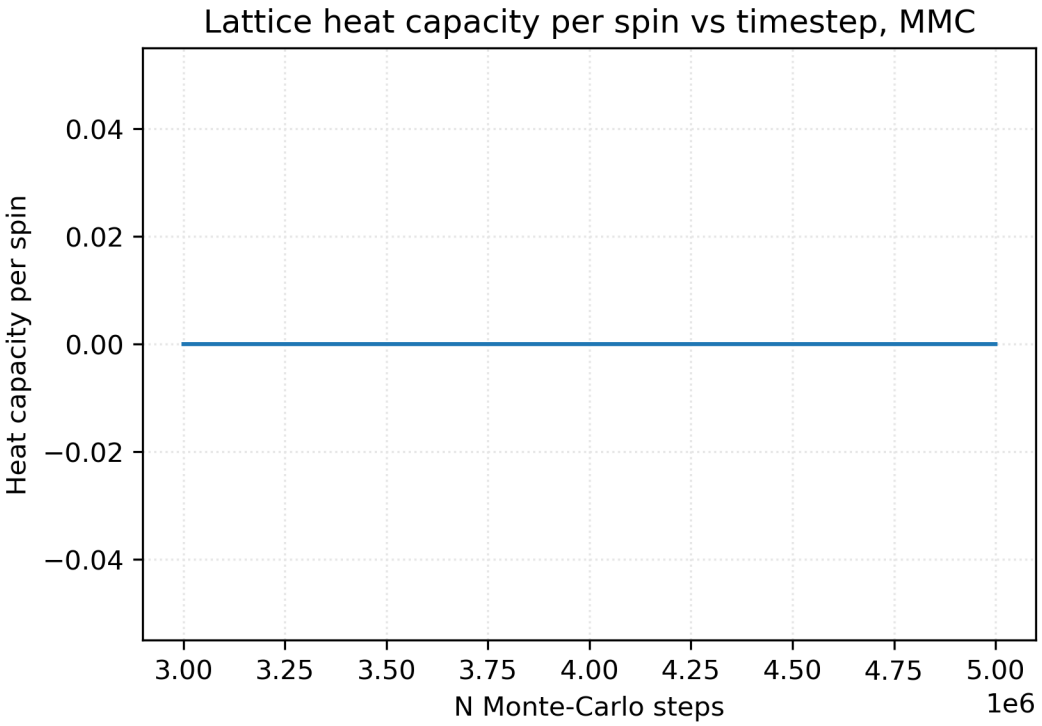



Lattice energy per spin vs timestep, MMC
Energy per spin / J
0
−25
−50
−75
−100
−125
−150
−175
−200
0
1
2
3
4
5
N Monte-Carlo steps
1e6


Lattice magnetisation per spin vs timestep, MMC
Magnetisation per spin
1.0
0.8
0.6
0.4
0.2
0.0
0
1
2
3
4
5
N Monte-Carlo steps
1e6


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
0
−100000
−200000
−300000
−400000
−500000
−600000
−700000
−800000
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice magnetic susceptibility per spin vs timestep, MMC
Magnetic susceptibility per spin
0.04
0.02
0.00
−0.02
−0.04
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
1e6
N Monte-Carlo steps

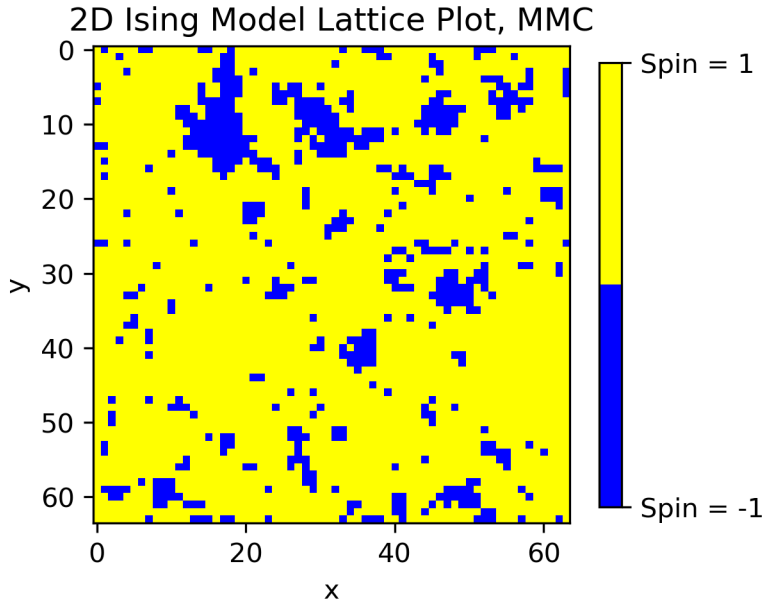


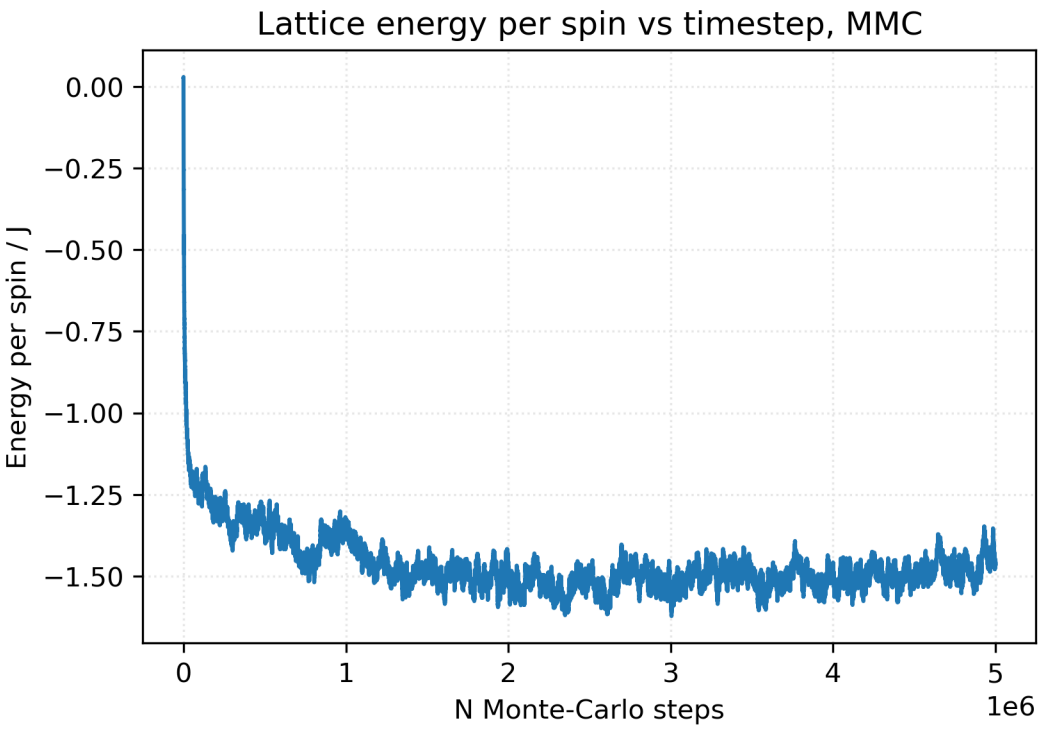
[37]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

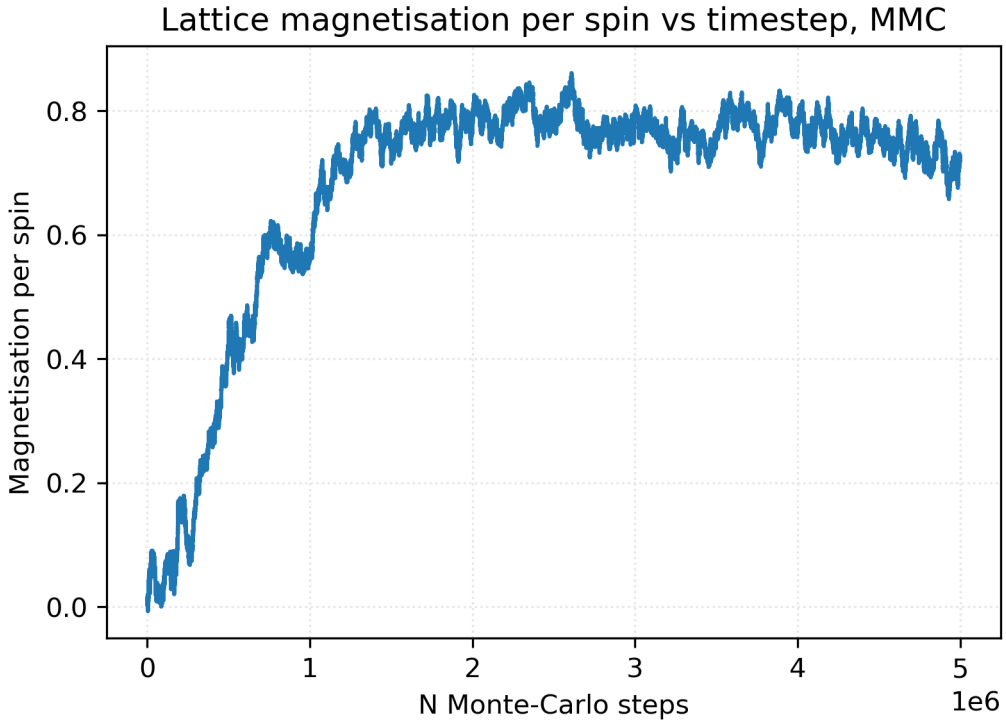
30 Visualising the effect of changing the magnetic induction field strength B for a standard lattice size of 64x64 spins using the metropolis algorithm, whilst remaining at the onsager critical temperature

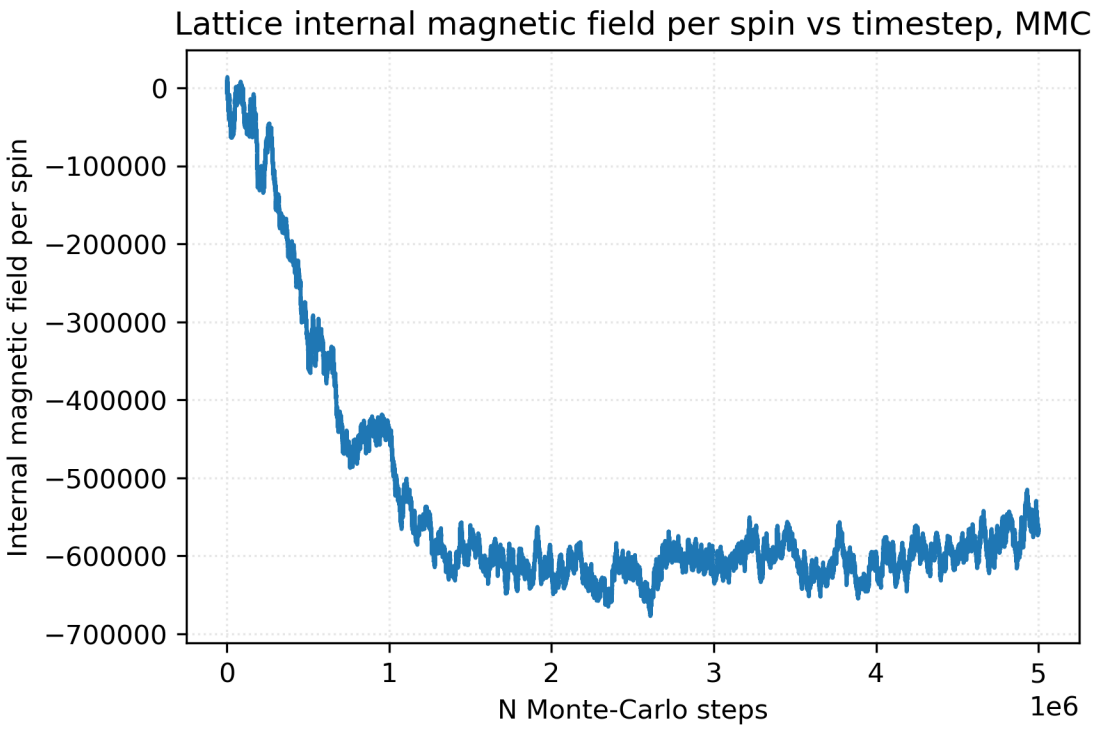
[38]:
```
# B = 0.01 (comparable to a fridge magnet)
parameter_file = 'ising_single_sim_B_001_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

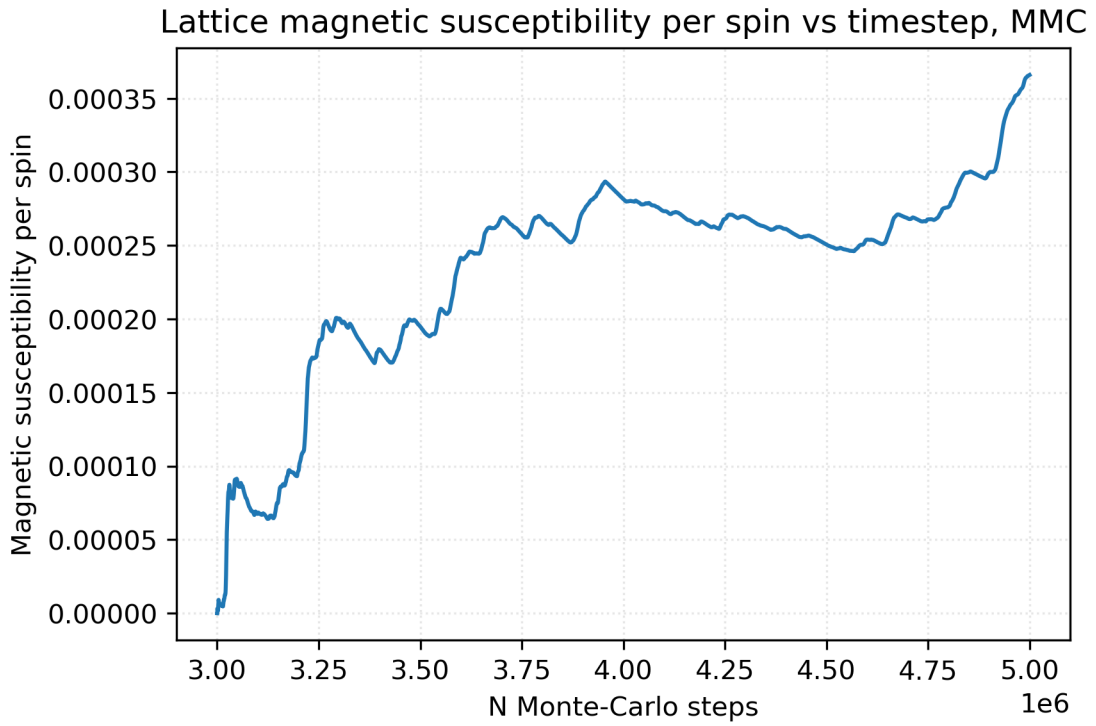
```
Running single Metropolis Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=64, magnetic_field=0.01,
exchange_constant_J=1.0, k_b=1.0, u_b=1.0, nsteps=5000000

Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

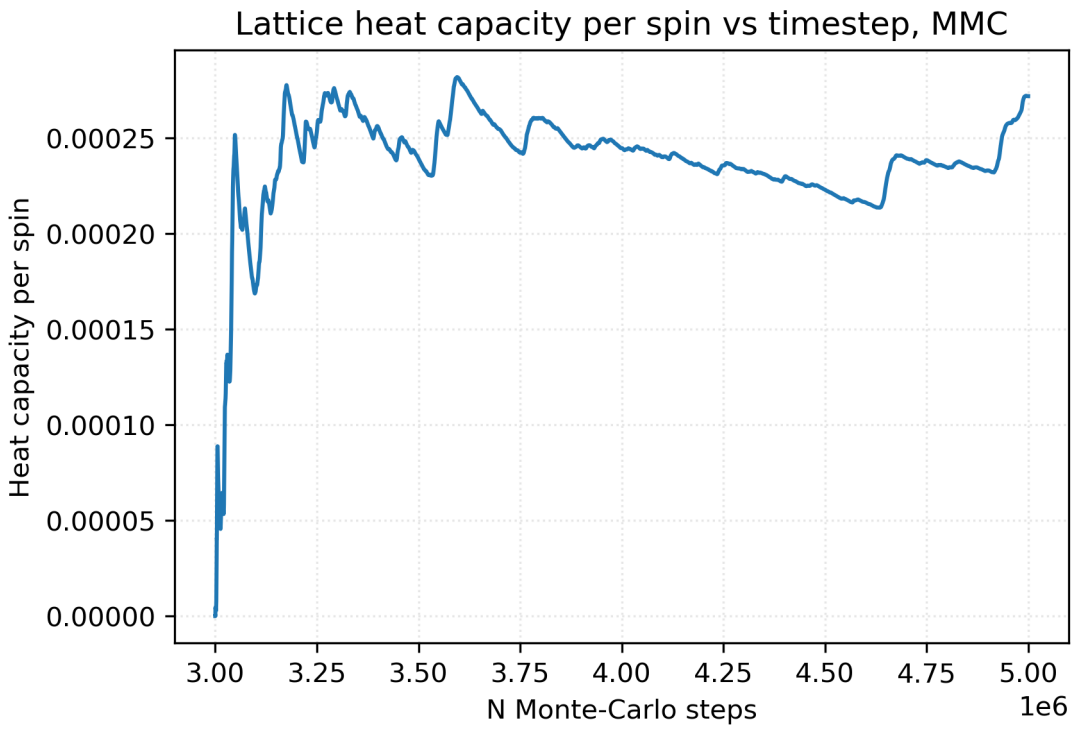



Lattice energy per spin vs timestep, MMC
0.00
−0.25
−0.50
−0.75
−1.00
−1.25
−1.50
Energy per spin / J
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice magnetisation per spin vs timestep, MMC
0.8
0.6
0.4
0.2
0.0
Magnetisation per spin
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
0
−100000
−200000
−300000
−400000
−500000
−600000
−700000
0
1
2
3
4
5
N Monte-Carlo steps
1e6


Lattice magnetic susceptibility per spin vs timestep, MMC
Magnetic susceptibility per spin
0.00035
0.00030
0.00025
0.00020
0.00015
0.00010
0.00005
0.00000
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
N Monte-Carlo steps
1e6

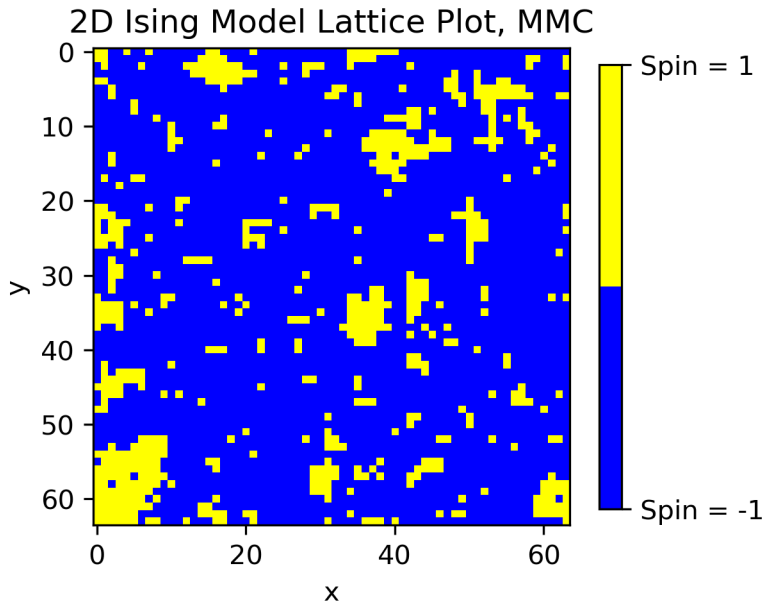


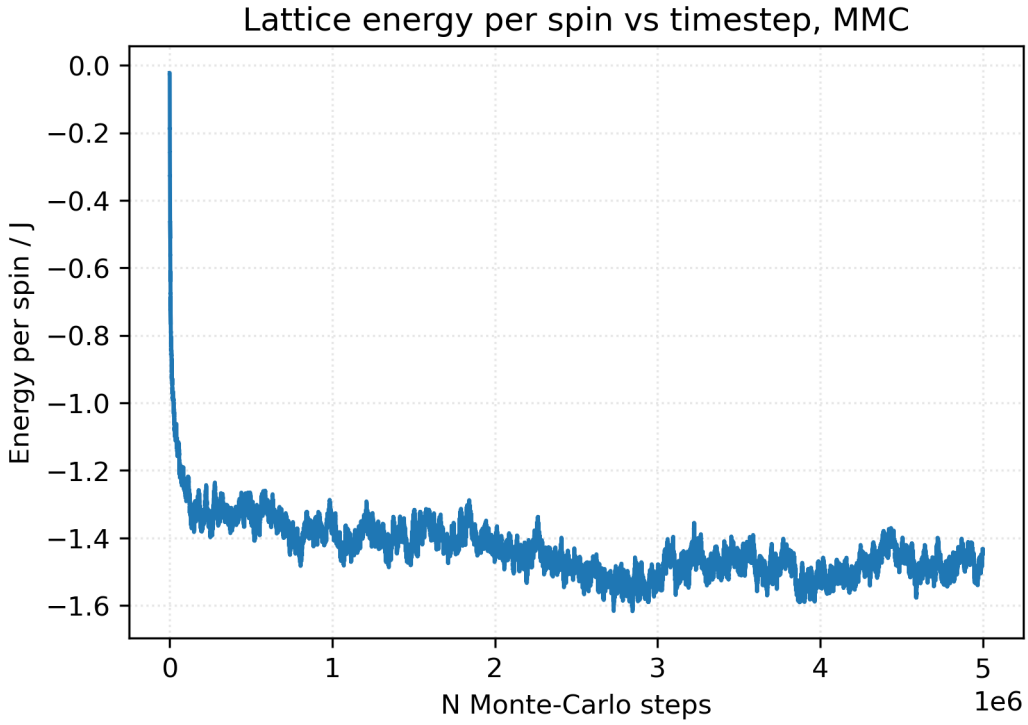
[38]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

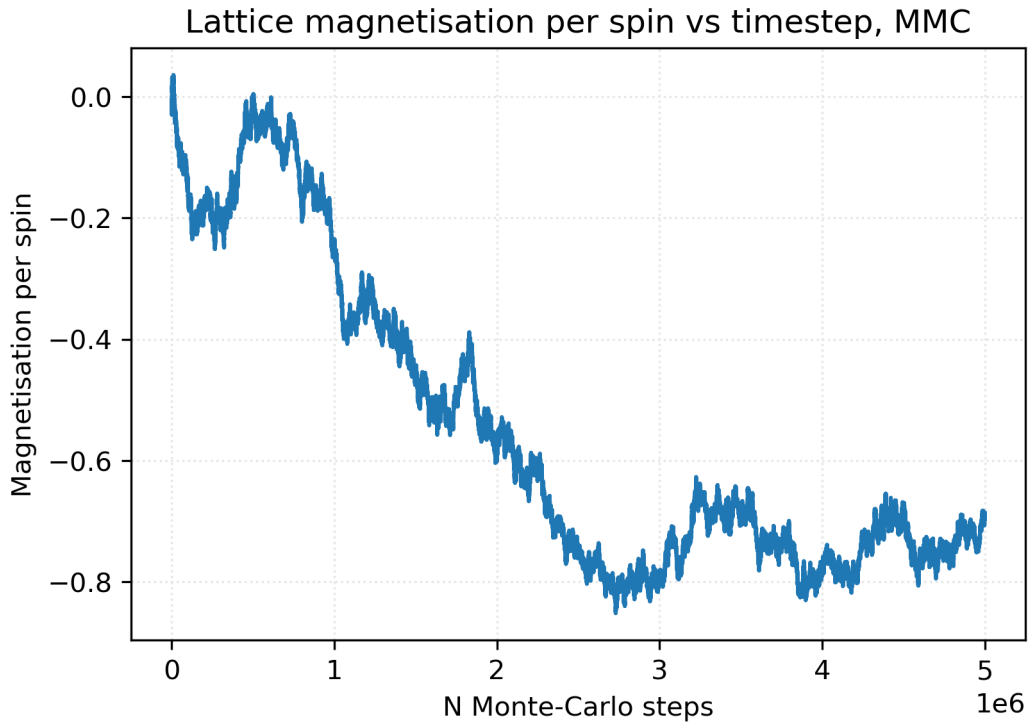
[39]:
```
# B = -0.01 (comparable to a fridge magnet)
parameter_file = 'ising_single_sim_B_-001_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

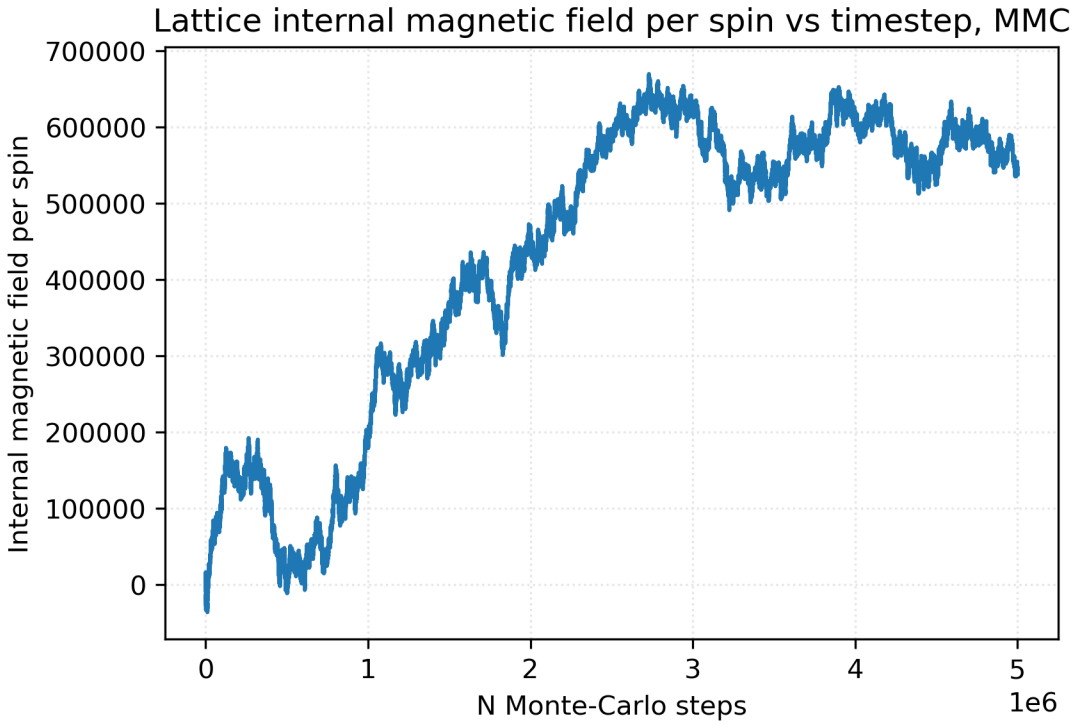
```
Running single Metropolis Algorithm simulation with parameters:
```

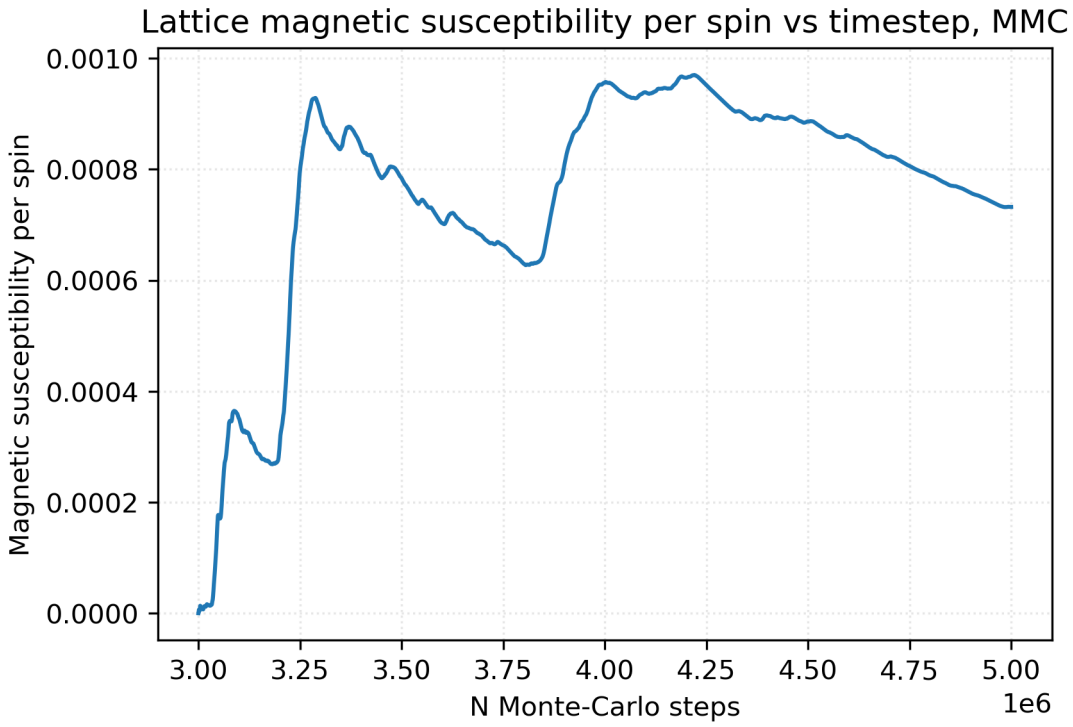
```
temperature=2.269185K, lattice_side_length=64, magnetic_field=-0.01,
exchange_constant_J=1.0, k_b=1.0, u_b=1.0, nsteps=5000000

Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

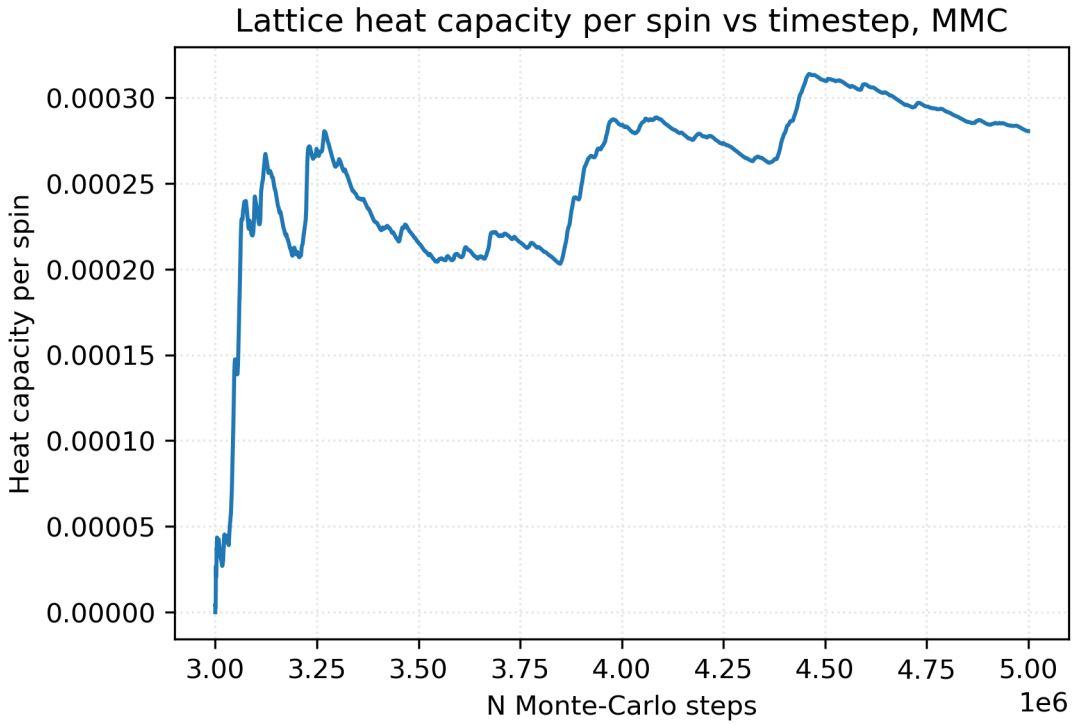





Lattice magnetisation per spin vs timestep, MMC
Magnetisation per spin
0.0
−0.2
−0.4
−0.6
−0.8
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
700000
600000
500000
400000
300000
200000
100000
0
0
1
2
3
4
5
1e6
N Monte-Carlo steps

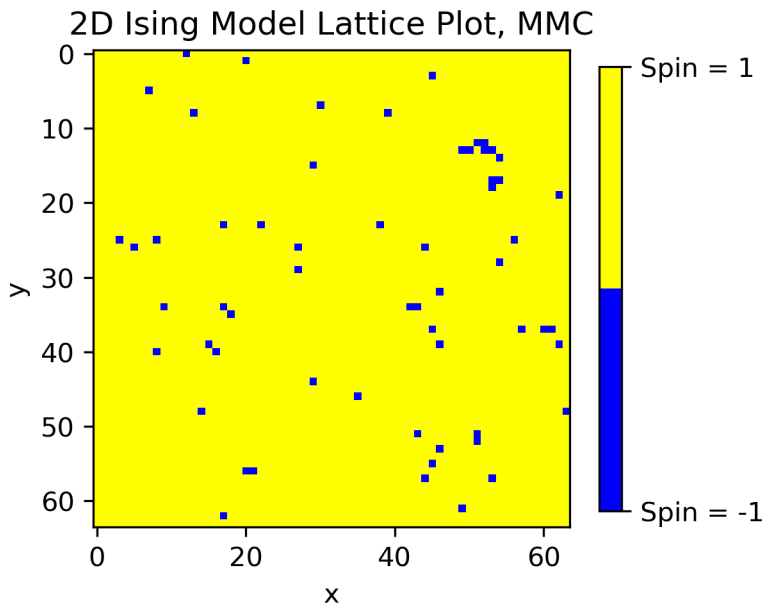

Lattice magnetic susceptibility per spin vs timestep, MMC
Magnetic susceptibility per spin
0.0010
0.0008
0.0006
0.0004
0.0002
0.0000
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
N Monte-Carlo steps
1e6

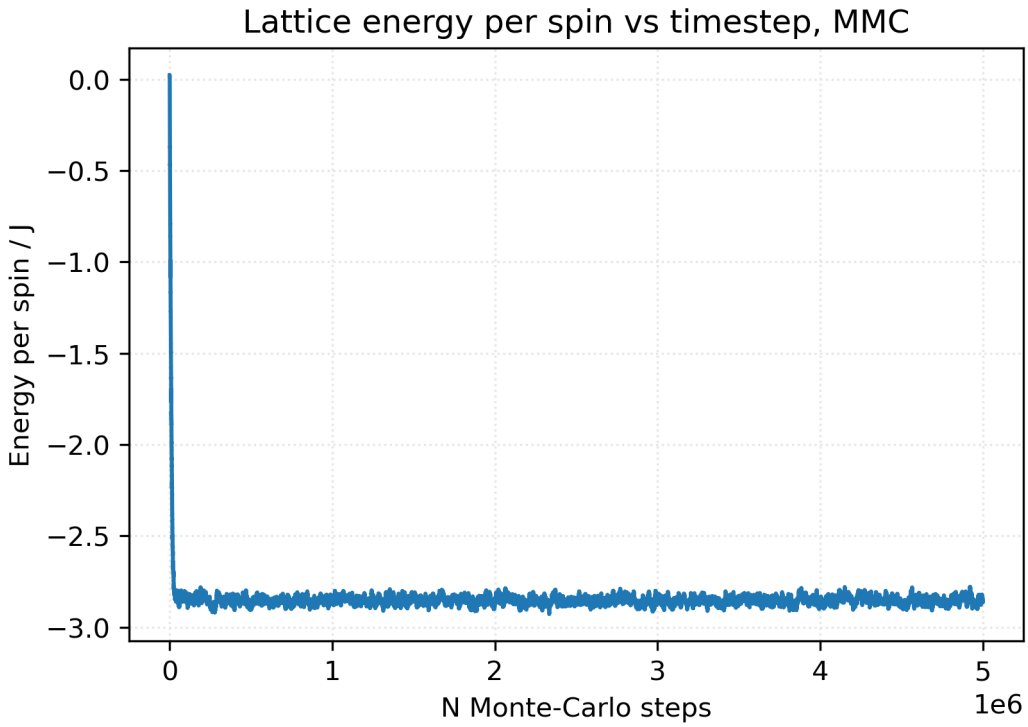

Lattice heat capacity per spin vs timestep, MMC
Heat capacity per spin
0.00030
0.00025
0.00020
0.00015
0.00010
0.00005
0.00000
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
N Monte-Carlo steps
1e6

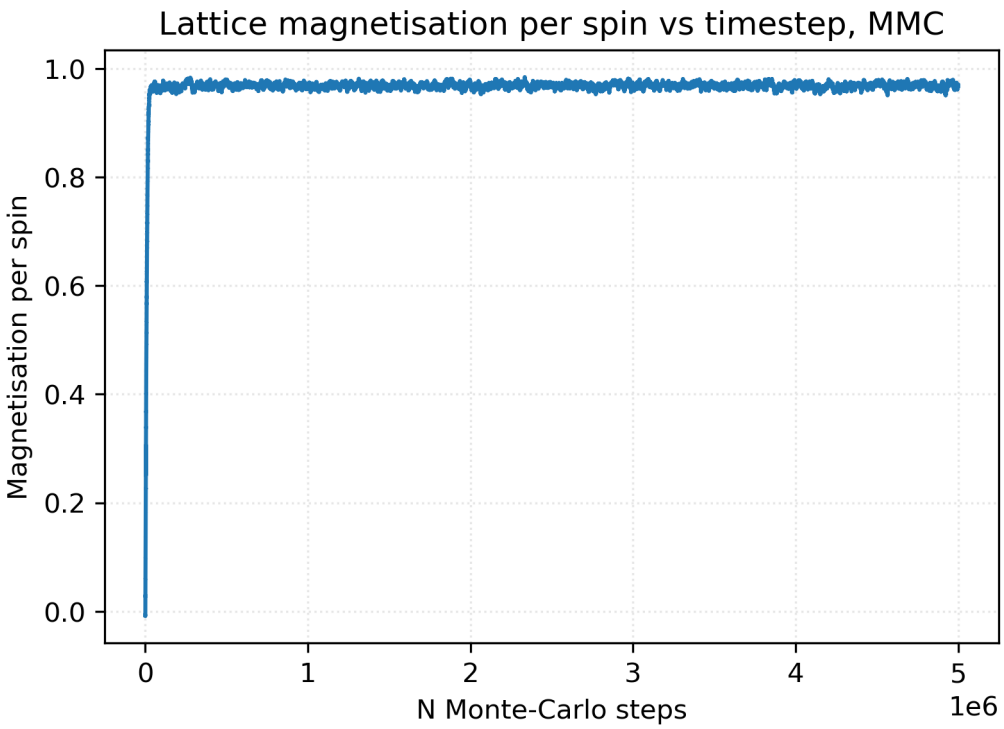
[39]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

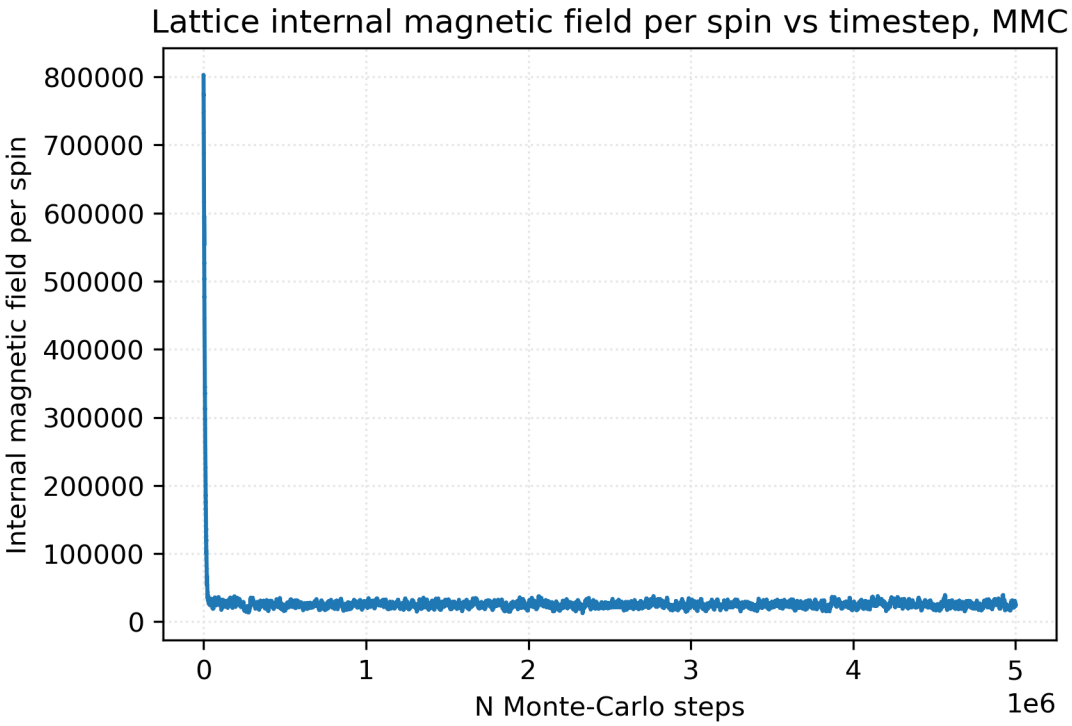
[40]:
```
# B = 1 (comparable to an MRI machine)
parameter_file = 'ising_single_sim_B_1_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

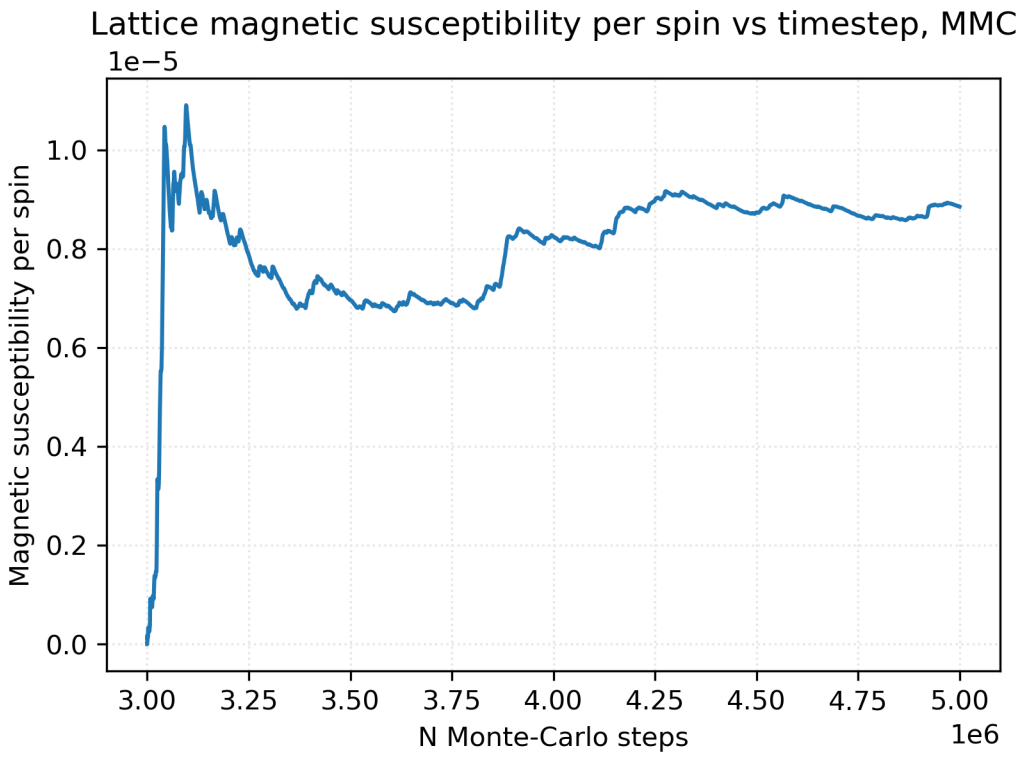
```
Running single Metropolis Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=64, magnetic_field=1.0,
exchange_constant_J=1.0, k_b=1.0, u_b=1.0, nsteps=5000000


Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

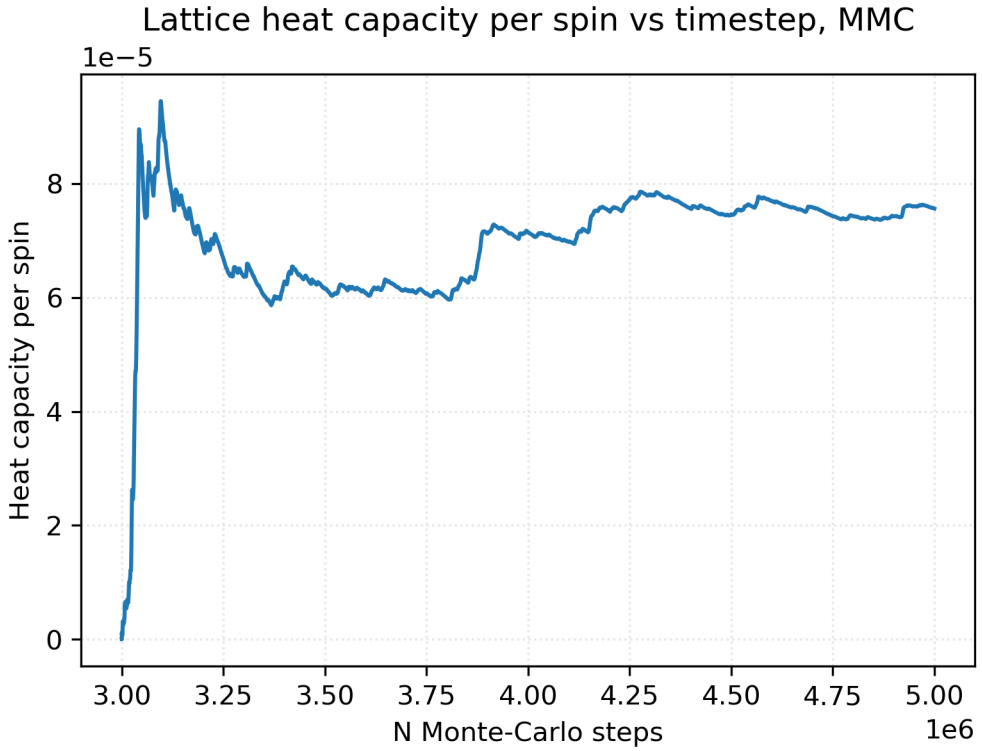

2D Ising Model Lattice Plot, MMC
Spin = 1
Spin = -1
y
x
0
10
20
30
40
50
60
0
20
40
60


Lattice energy per spin vs timestep, MMC
Energy per spin / J
N Monte-Carlo steps
1e6
0.0
−0.5
−1.0
−1.5
−2.0
−2.5
−3.0
0
1
2
3
4
5


Lattice magnetisation per spin vs timestep, MMC
Magnetisation per spin
1.0
0.8
0.6
0.4
0.2
0.0
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
800000
700000
600000
500000
400000
300000
200000
100000
0
0
1
2
3
4
5
1e6
N Monte-Carlo steps

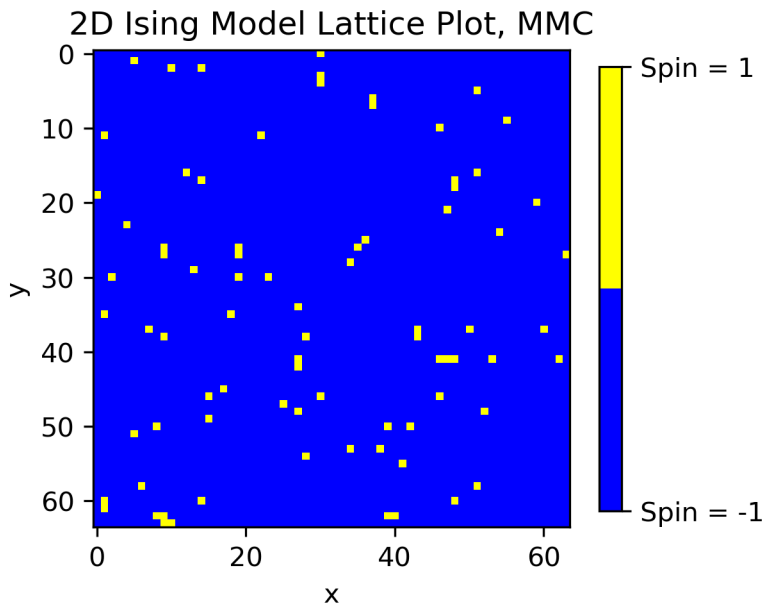

Lattice magnetic susceptibility per spin vs timestep, MMC
1e−5
Magnetic susceptibility per spin
1.0
0.8
0.6
0.4
0.2
0.0
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
1e6
N Monte-Carlo steps

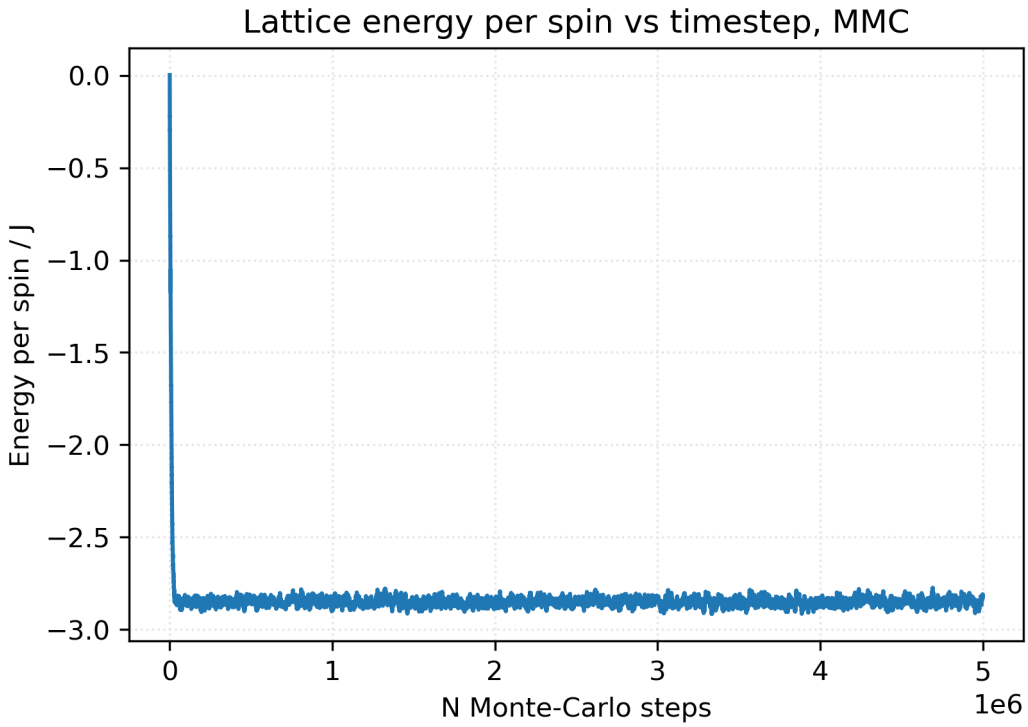

Lattice heat capacity per spin vs timestep, MMC
1e−5
Heat capacity per spin
8
6
4
2
0
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
1e6
N Monte-Carlo steps

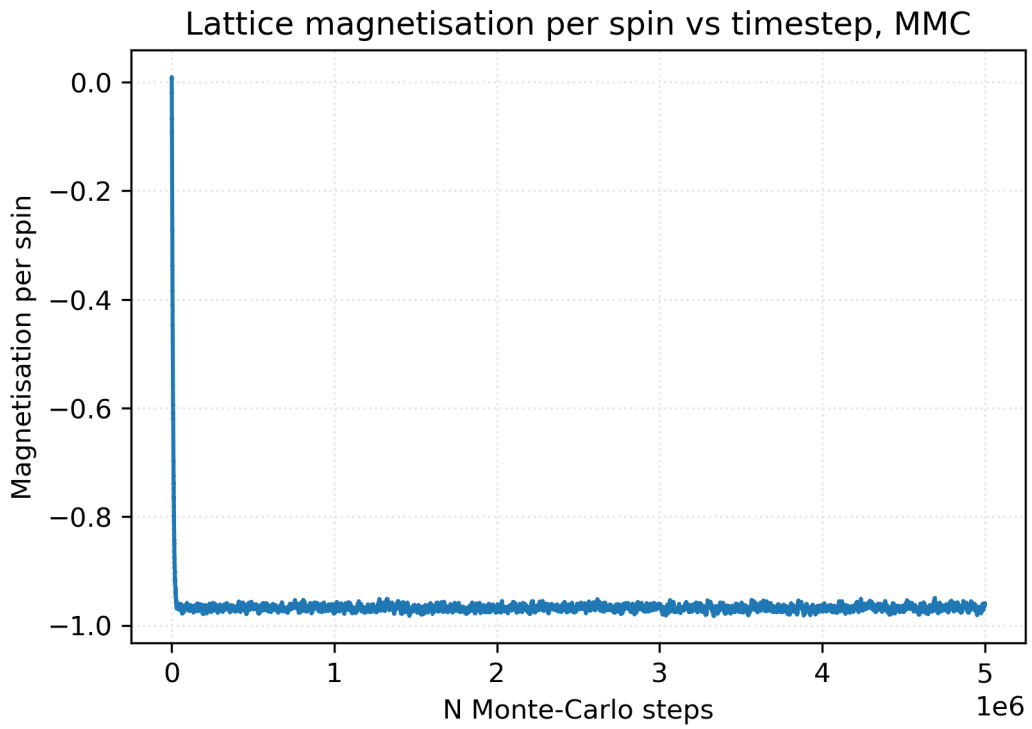
[40]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

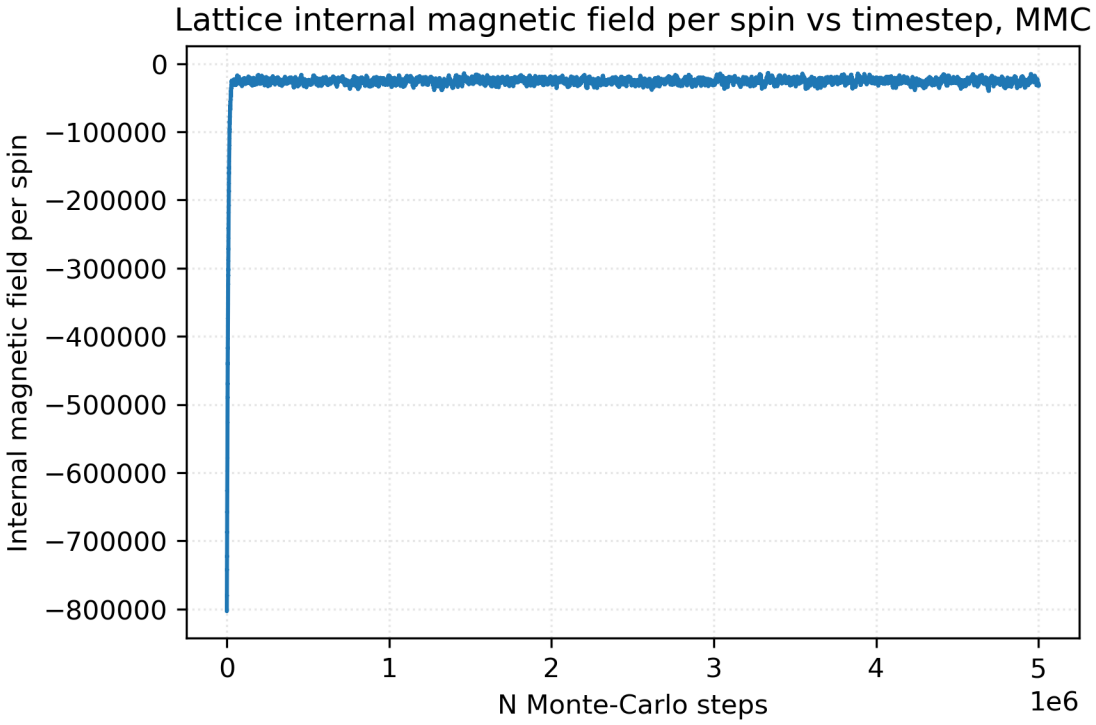
[41]:
```
# B = -1 (comparable to an MRI machine)
parameter_file = 'ising_single_sim_B_-1_MMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_MMC(parameter_file)
```

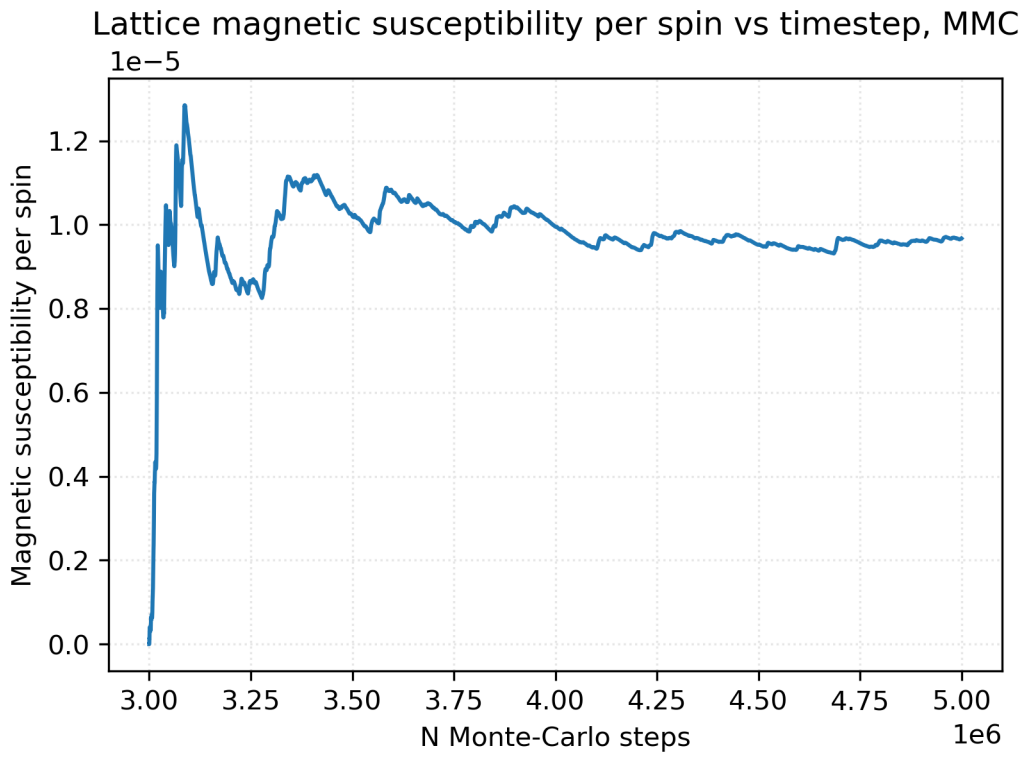
```
Running single Metropolis Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=64, magnetic_field=-1.0,
exchange_constant_J=1.0, k_b=1.0, u_b=1.0, nsteps=5000000


Running MMC Ising Model::   0%|          | 0/5000000 [00:00<?, ?it/s]
```

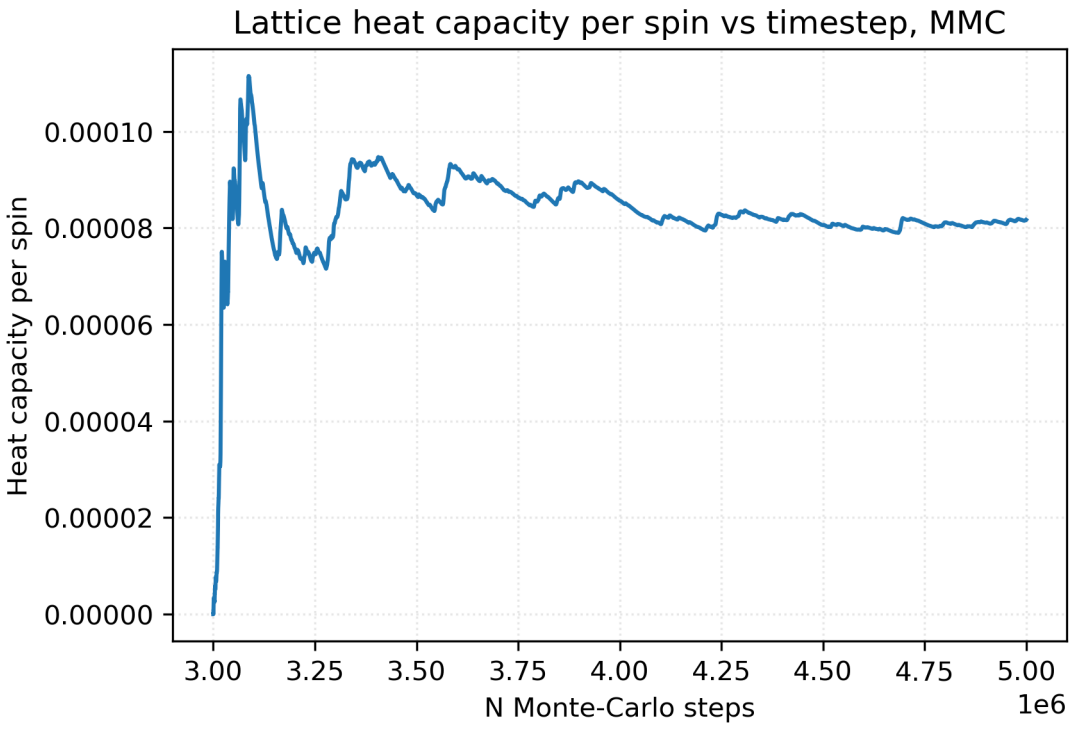

2D Ising Model Lattice Plot, MMC
0
10
20
30
40
50
60
y
0
20
40
60
x
Spin = 1
Spin = -1


Lattice energy per spin vs timestep, MMC
0.0
−0.5
−1.0
−1.5
−2.0
−2.5
−3.0
Energy per spin / J
0
1
2
3
4
5
1e6
N Monte-Carlo steps


Lattice magnetisation per spin vs timestep, MMC
Magnetisation per spin
0.0
−0.2
−0.4
−0.6
−0.8
−1.0
0
1
2
3
4
5
N Monte-Carlo steps
1e6


Lattice internal magnetic field per spin vs timestep, MMC
Internal magnetic field per spin
0
−100000
−200000
−300000
−400000
−500000
−600000
−700000
−800000
0
1
2
3
4
5
N Monte-Carlo steps
1e6


Lattice magnetic susceptibility per spin vs timestep, MMC
1e−5
Magnetic susceptibility per spin
1.2
1.0
0.8
0.6
0.4
0.2
0.0
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
N Monte-Carlo steps
1e6

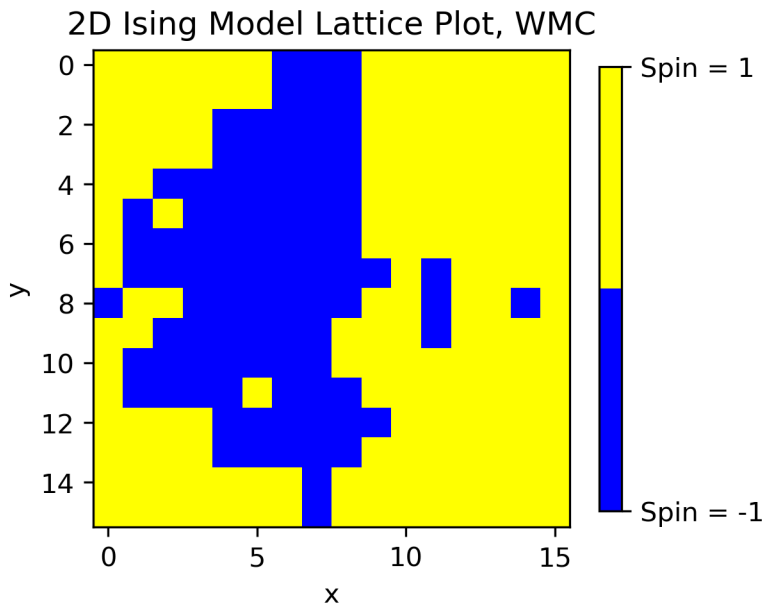

Lattice heat capacity per spin vs timestep, MMC
Heat capacity per spin
0.00010
0.00008
0.00006
0.00004
0.00002
0.00000
3.00
3.25
3.50
3.75
4.00
4.25
4.50
4.75
5.00
N Monte-Carlo steps
1e6

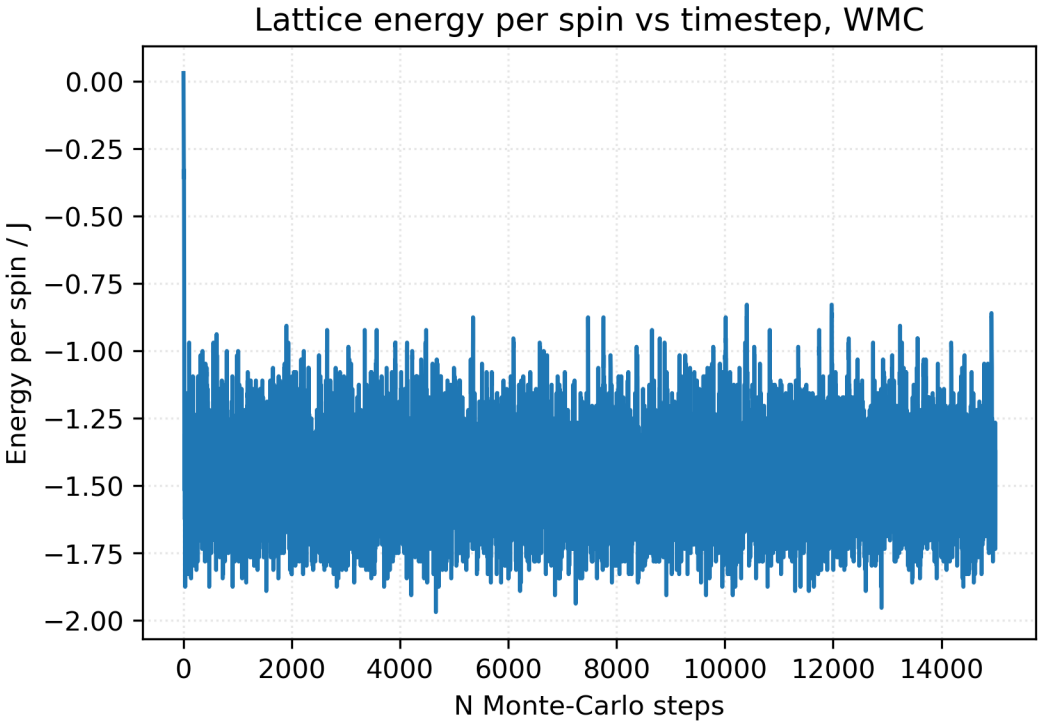
[41]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, MMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, MMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetisation per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice internal magnetic field per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Internal magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
MMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, MMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

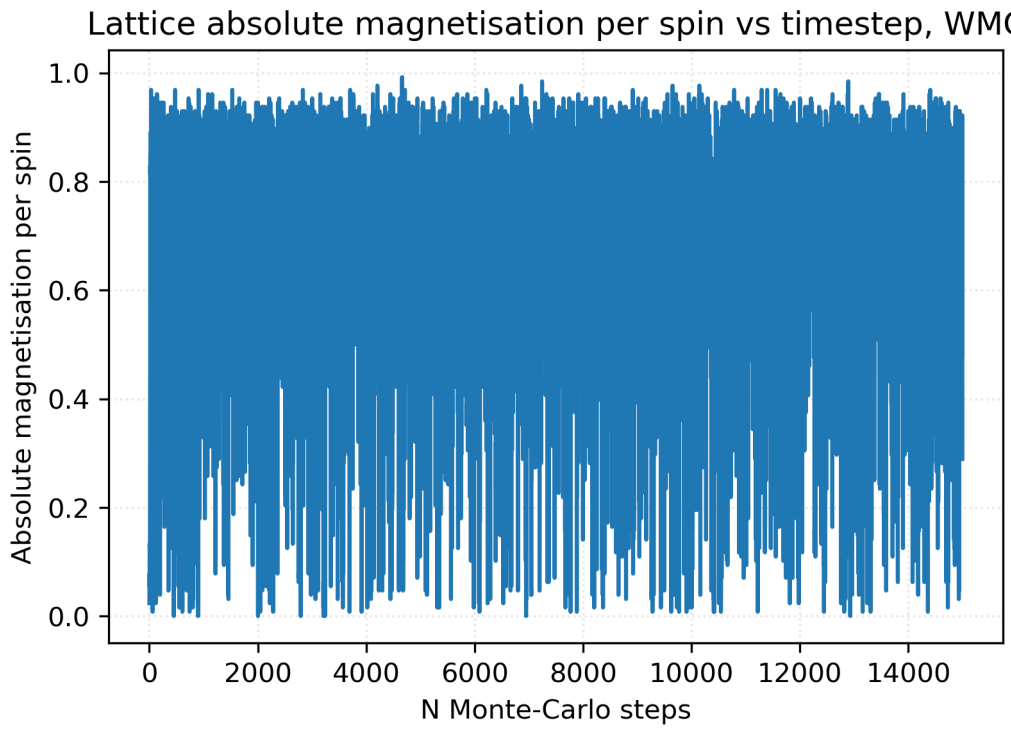
31 Visualising how changing lattice size impacts the simulations using the wolff algorithm, whilst remaining at the onsager critical temperature

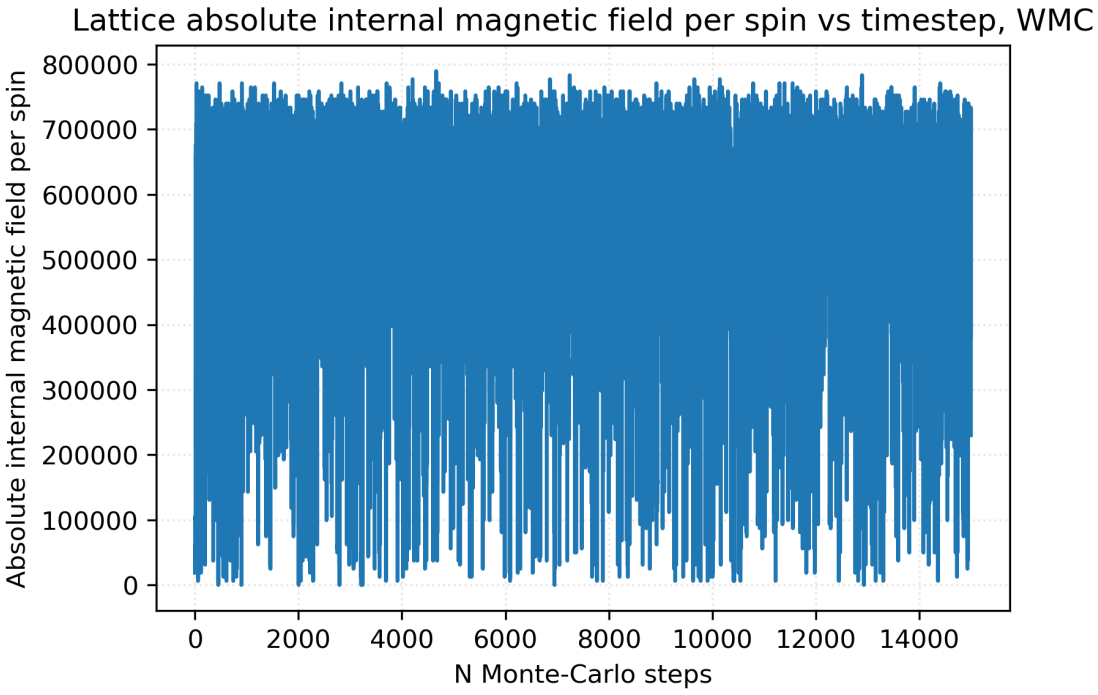
[42]:
```
# 16x16 lattice
parameter_file = 'ising_single_sim_side_len_16_WMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_WMC(parameter_file)
```

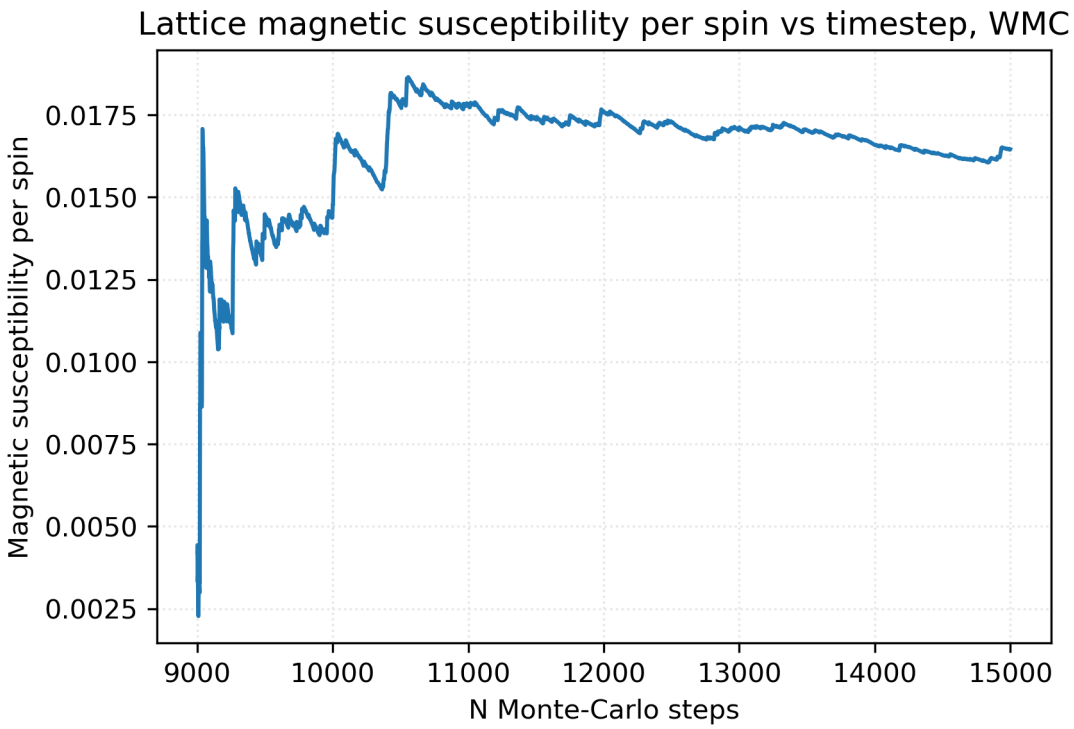
```
Running single Wolff Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=16, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=15000


Running WMC Ising Model::   0%|          | 0/15000 [00:00<?, ?it/s]
```

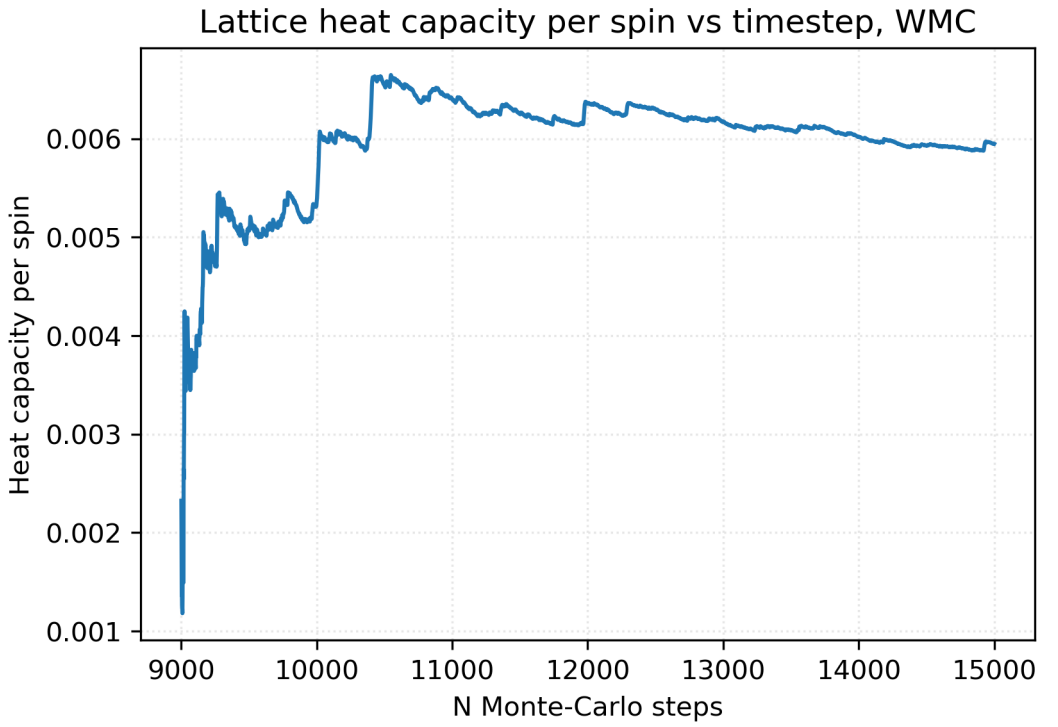

2D Ising Model Lattice Plot, WMC
0
2
4
6
8
10
12
14
y
0
5
10
15
x
Spin = 1
Spin = -1


Lattice energy per spin vs timestep, WMC
0.00
−0.25
−0.50
−0.75
−1.00
−1.25
−1.50
−1.75
−2.00
Energy per spin / J
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute magnetisation per spin vs timestep, WMC
Absolute magnetisation per spin
1.0
0.8
0.6
0.4
0.2
0.0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute internal magnetic field per spin vs timestep, WMC
Absolute internal magnetic field per spin
800000
700000
600000
500000
400000
300000
200000
100000
0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps

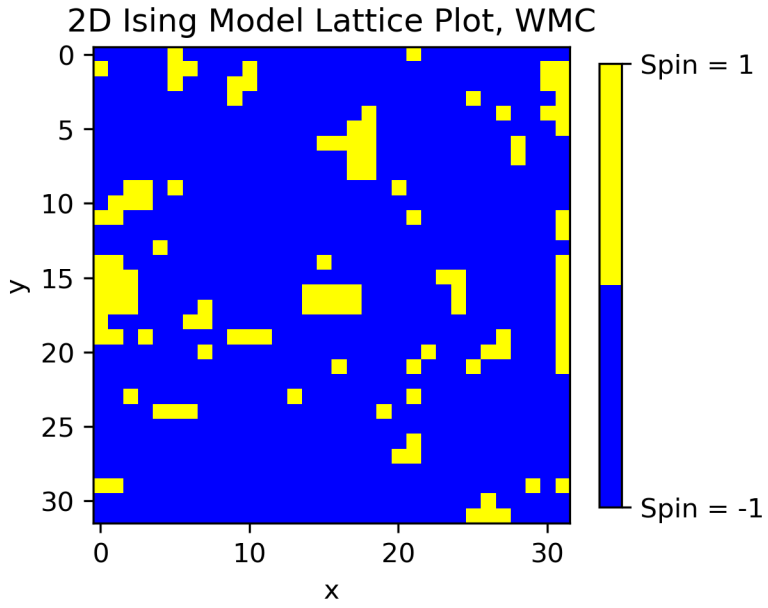

Lattice magnetic susceptibility per spin vs timestep, WMC
Magnetic susceptibility per spin
0.0175
0.0150
0.0125
0.0100
0.0075
0.0050
0.0025
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

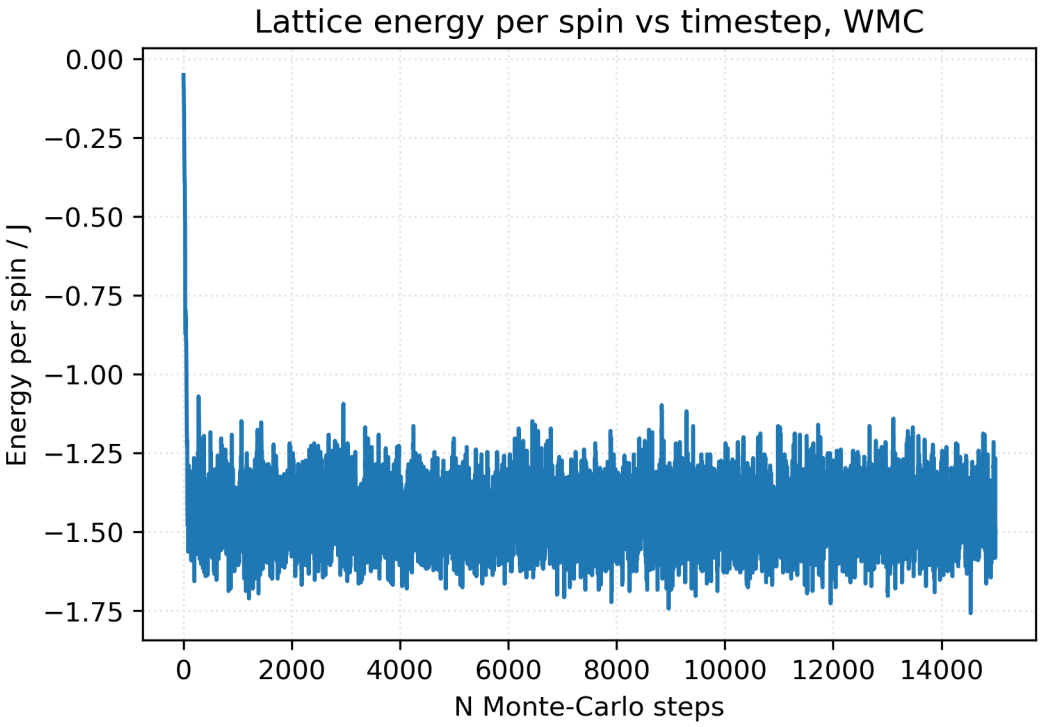

Lattice heat capacity per spin vs timestep, WMC
Heat capacity per spin
0.006
0.005
0.004
0.003
0.002
0.001
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

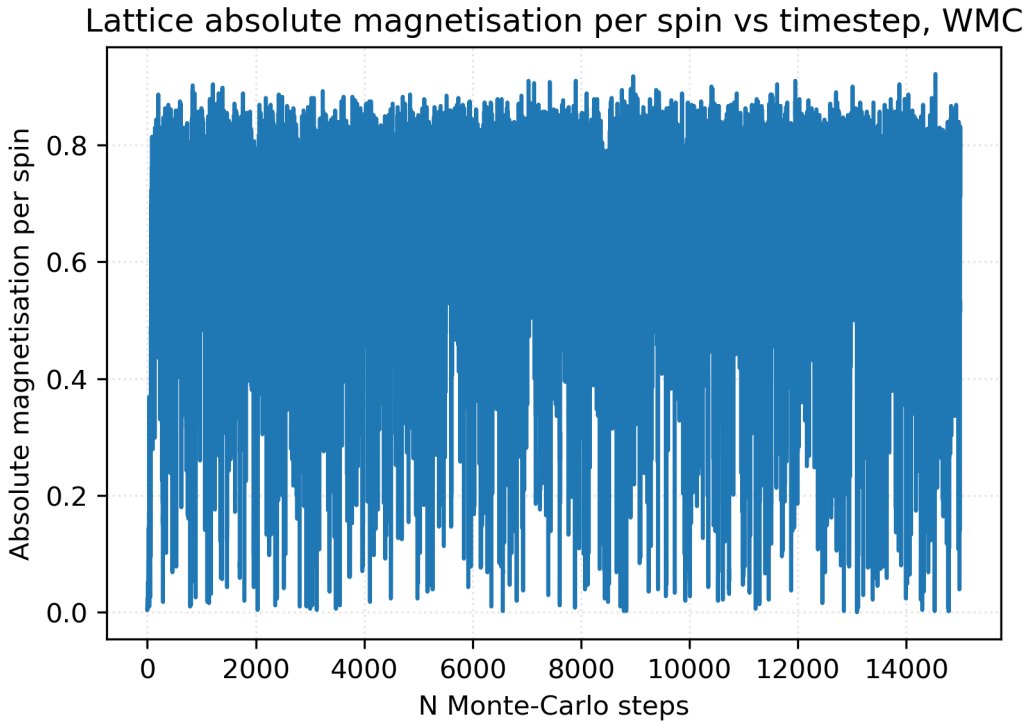
[42]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, WMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, WMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute magnetisation per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute internal magnetic field per spin vs
timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute internal
magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, WMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

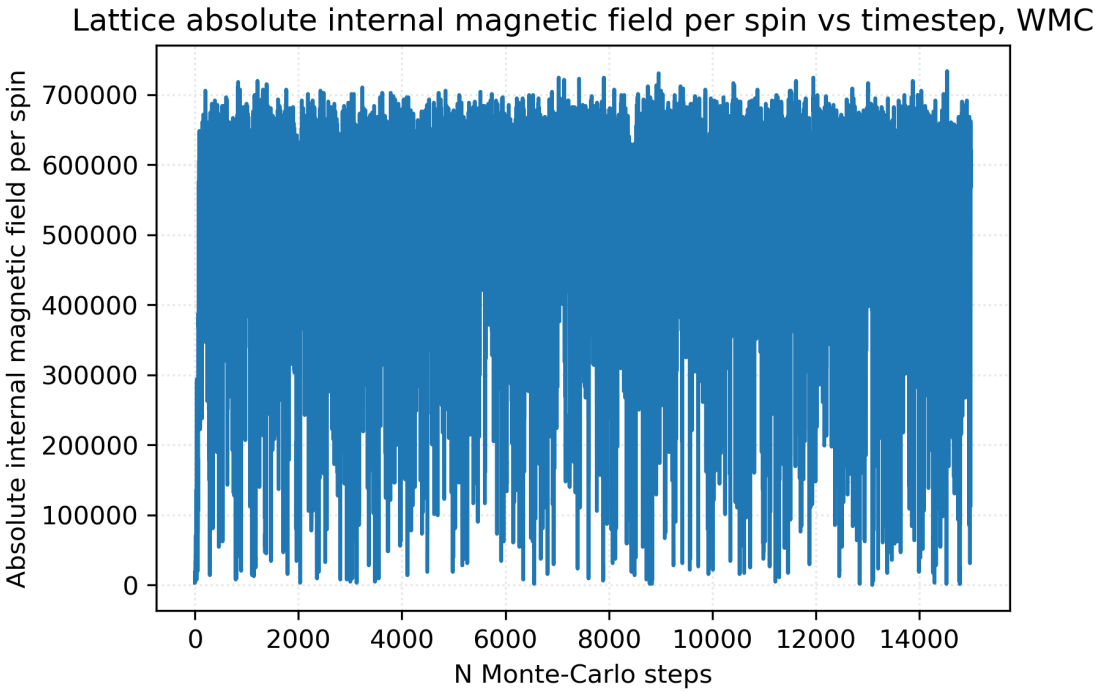
[43]:
```
# 32x32 lattice
parameter_file = 'ising_single_sim_side_len_32_WMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_WMC(parameter_file)
```

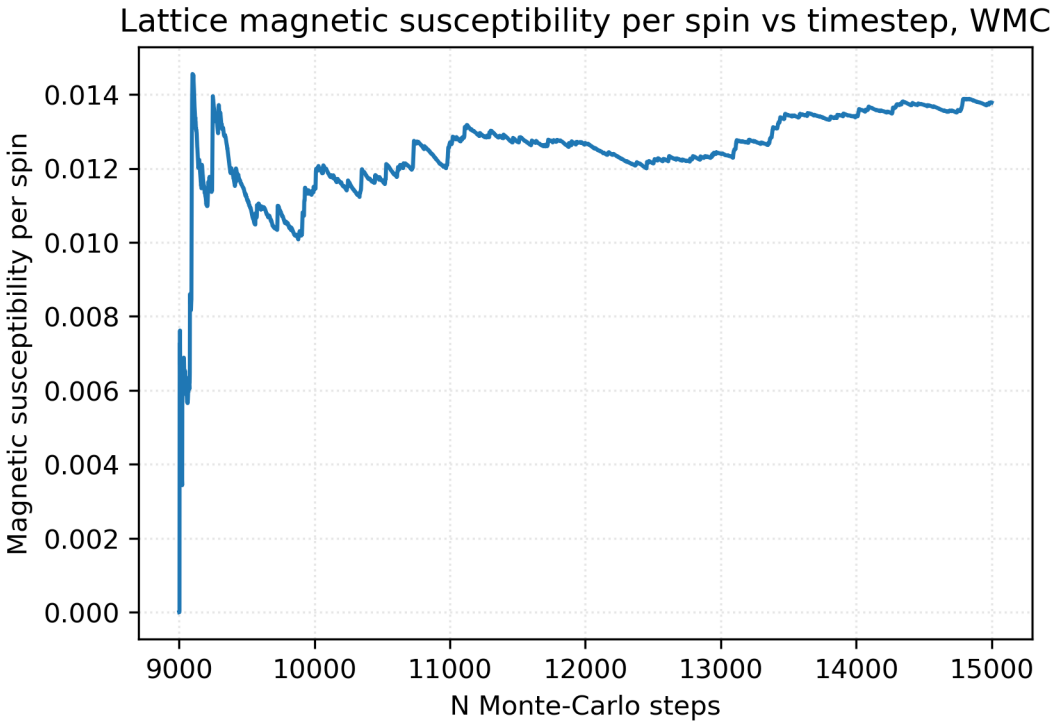
```
Running single Wolff Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=32, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=15000


Running WMC Ising Model::   0%|          | 0/15000 [00:00<?, ?it/s]
```

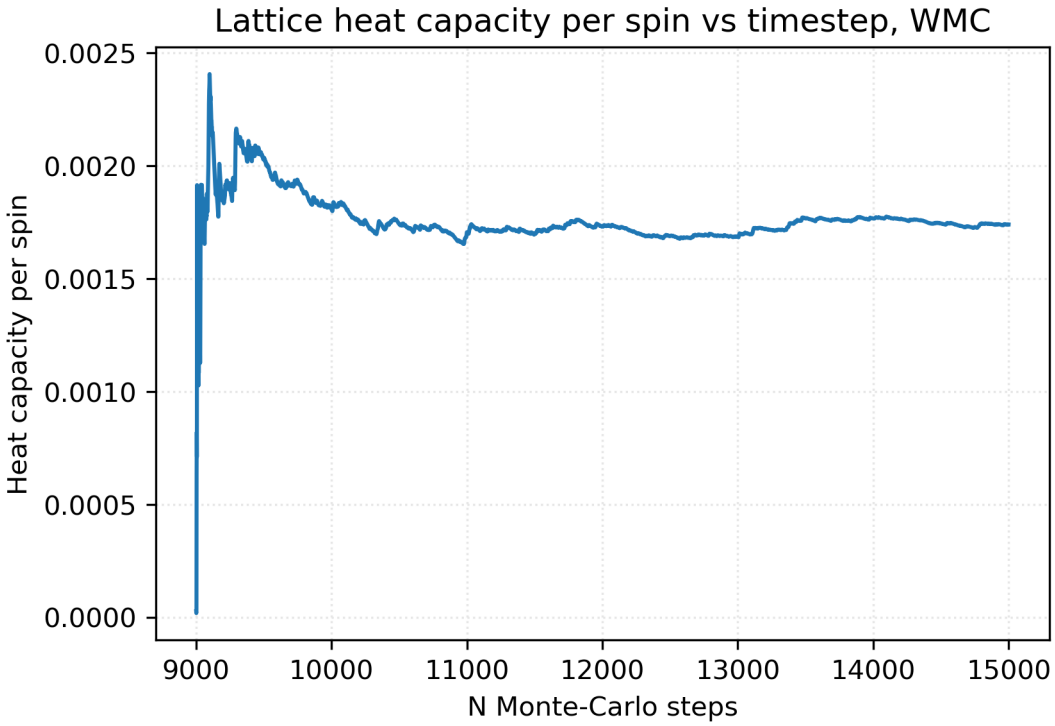



Lattice energy per spin vs timestep, WMC
Energy per spin / J
0.00
−0.25
−0.50
−0.75
−1.00
−1.25
−1.50
−1.75
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute magnetisation per spin vs timestep, WMC
Absolute magnetisation per spin
0.8
0.6
0.4
0.2
0.0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute internal magnetic field per spin vs timestep, WMC
Absolute internal magnetic field per spin
700000
600000
500000
400000
300000
200000
100000
0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps

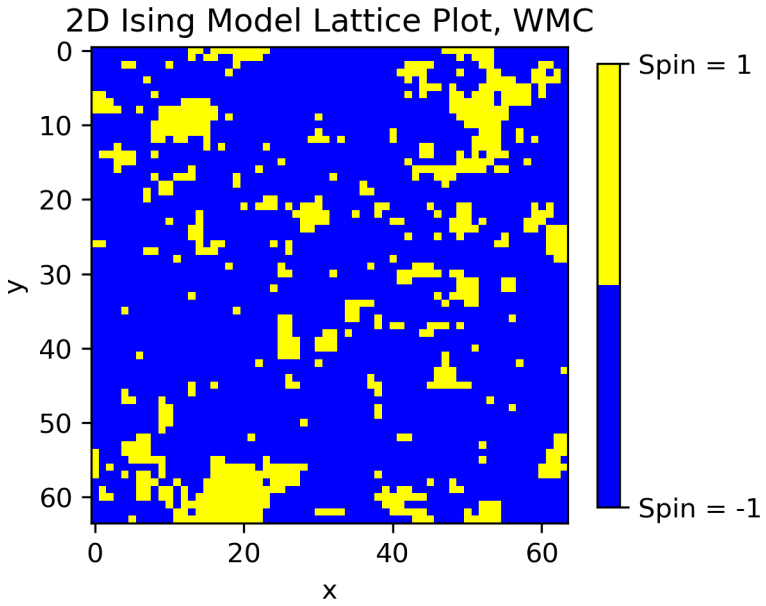

Lattice magnetic susceptibility per spin vs timestep, WMC
Magnetic susceptibility per spin
0.014
0.012
0.010
0.008
0.006
0.004
0.002
0.000
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

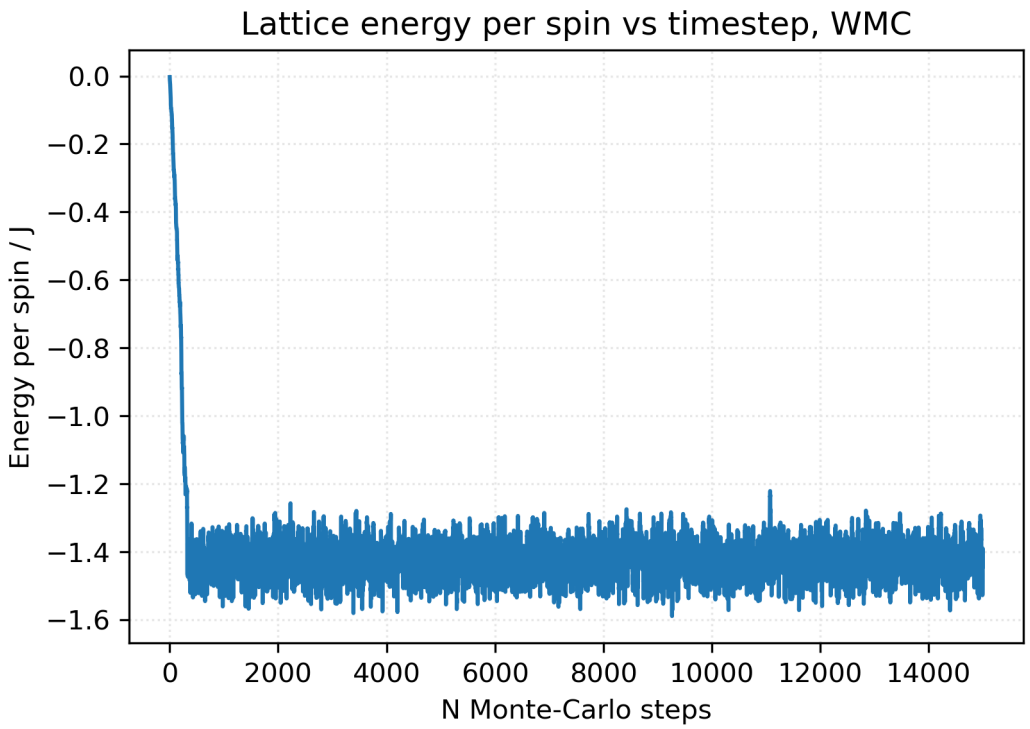


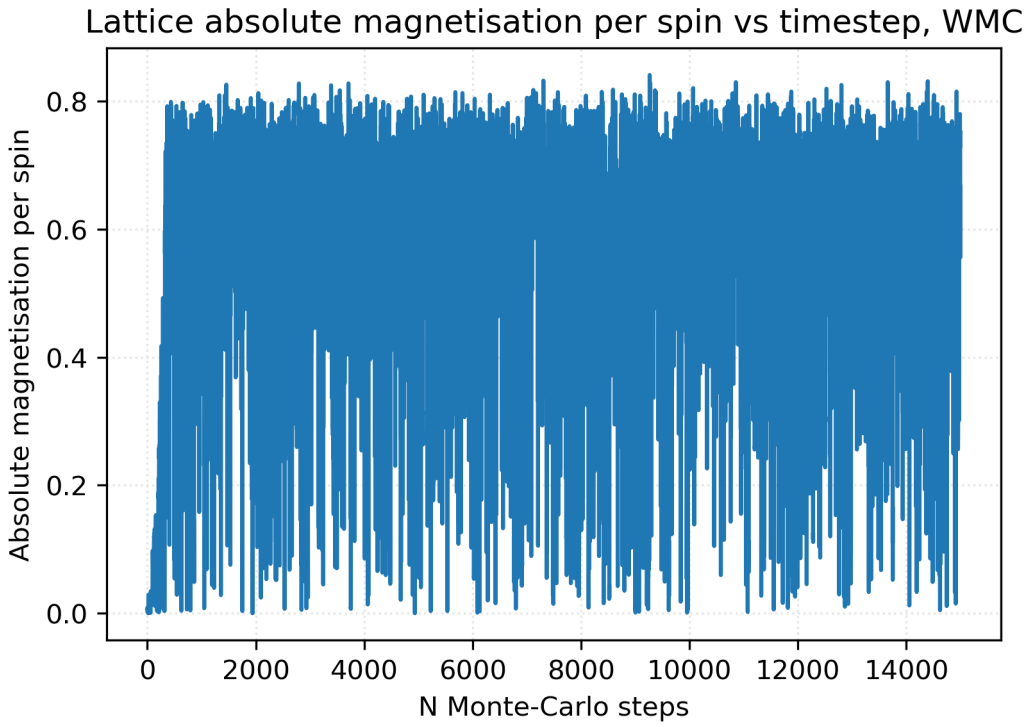
[43]: (<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, WMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, WMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute magnetisation per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute internal magnetic field per spin vs
timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute internal
magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, WMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)

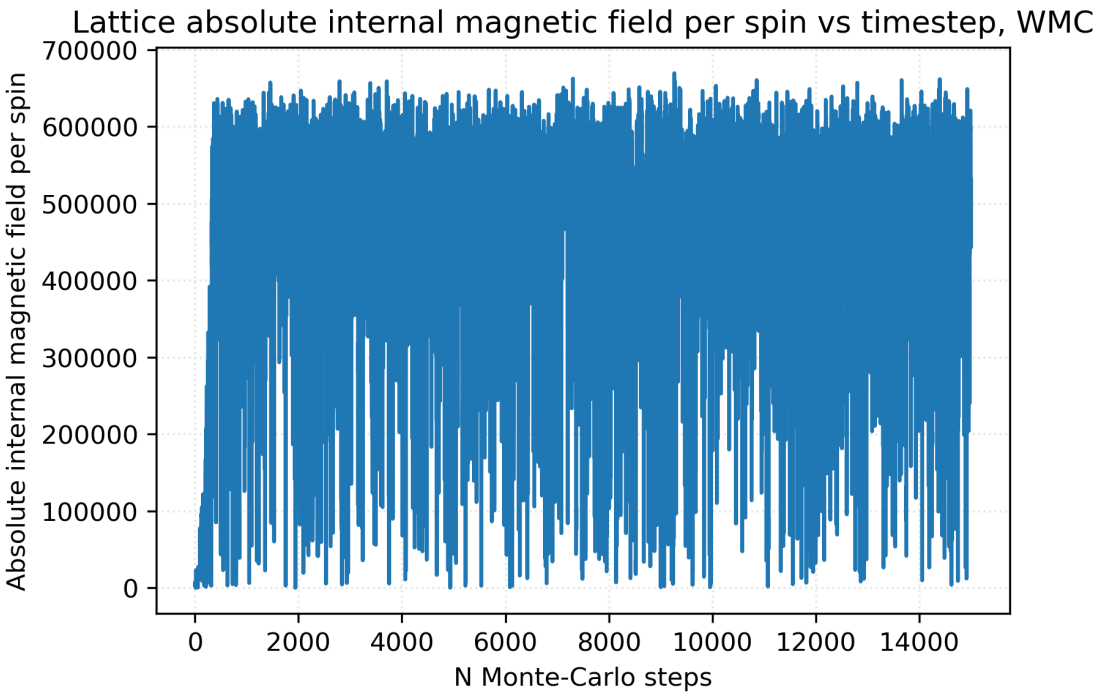
[44]:
```
# 64x64 lattice
parameter_file = 'ising_single_sim_side_len_64_WMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_WMC(parameter_file)
```

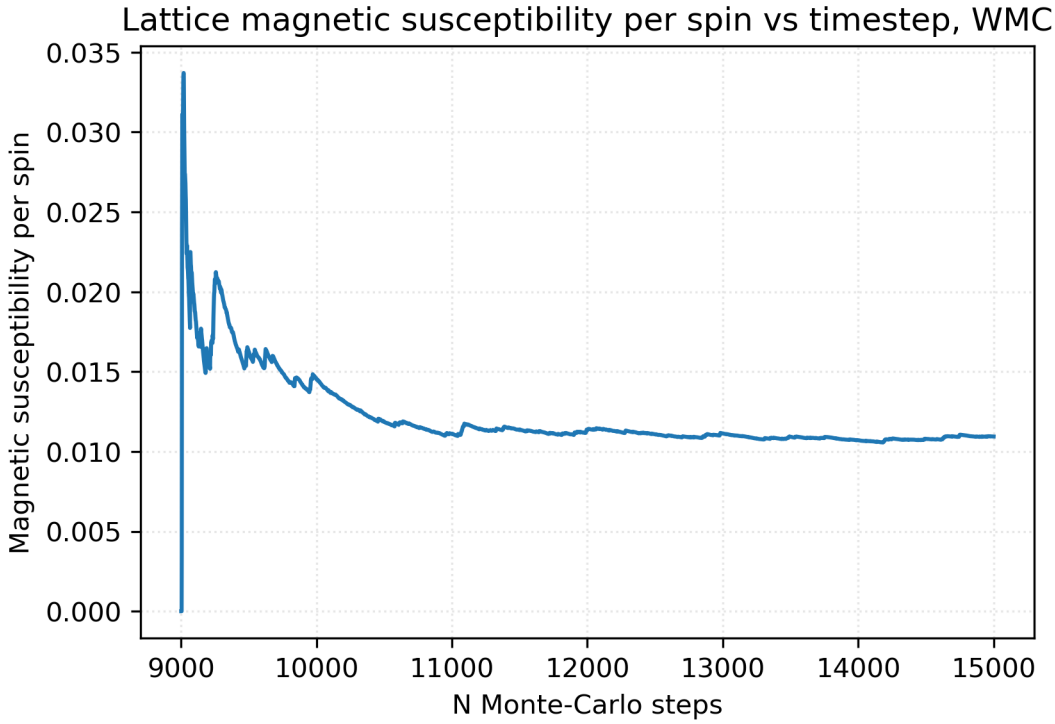
```
Running single Wolff Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=15000


Running WMC Ising Model::   0%|          | 0/15000 [00:00<?, ?it/s]
```

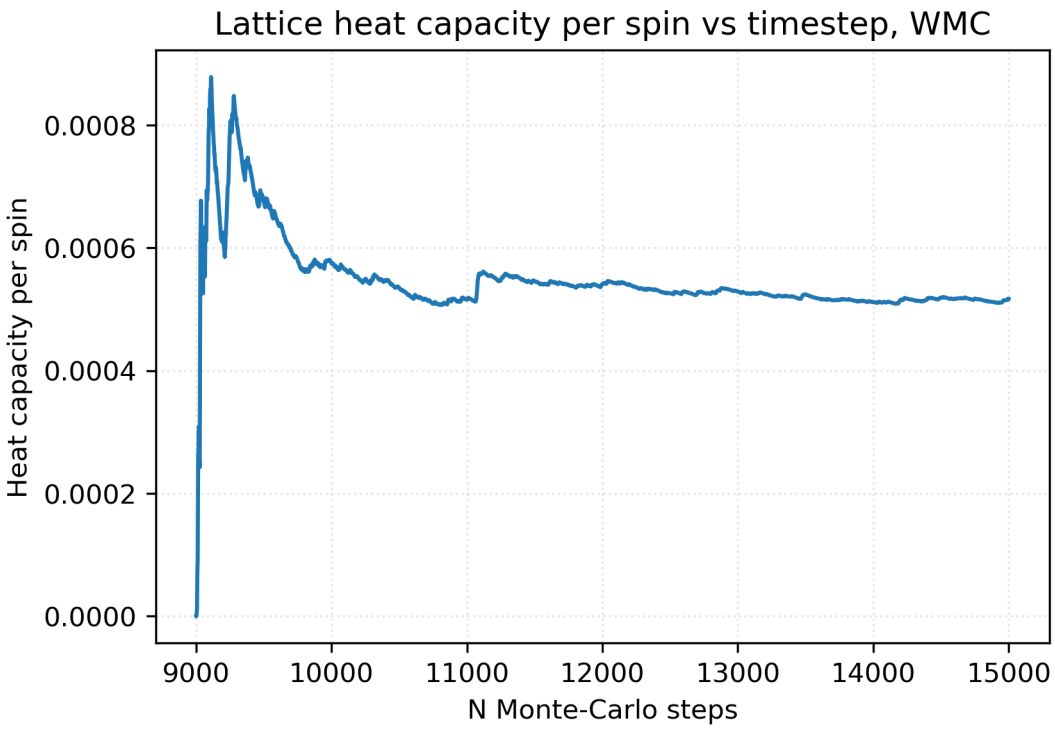



Lattice energy per spin vs timestep, WMC
Energy per spin / J
0.0
−0.2
−0.4
−0.6
−0.8
−1.0
−1.2
−1.4
−1.6
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute magnetisation per spin vs timestep, WMC
Absolute magnetisation per spin
0.8
0.6
0.4
0.2
0.0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute internal magnetic field per spin vs timestep, WMC
Absolute internal magnetic field per spin
700000
600000
500000
400000
300000
200000
100000
0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps

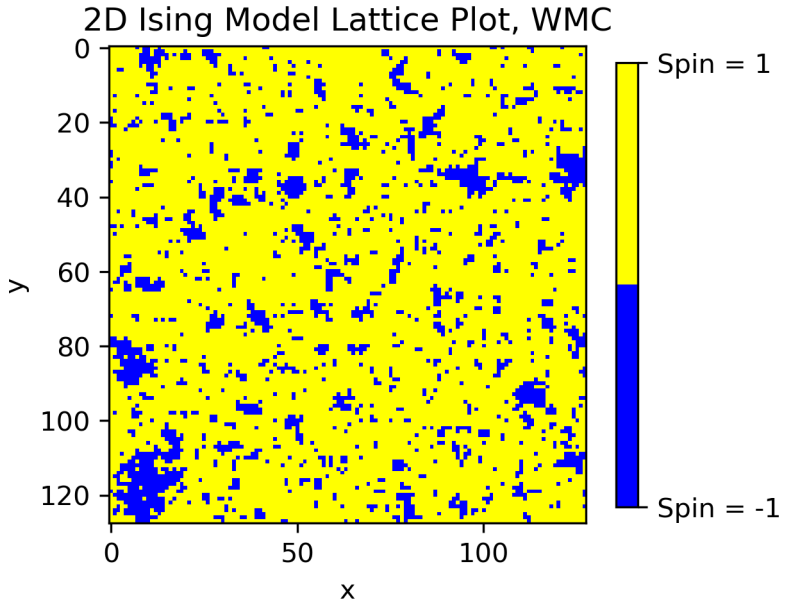

Lattice magnetic susceptibility per spin vs timestep, WMC
Magnetic susceptibility per spin
0.035
0.030
0.025
0.020
0.015
0.010
0.005
0.000
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

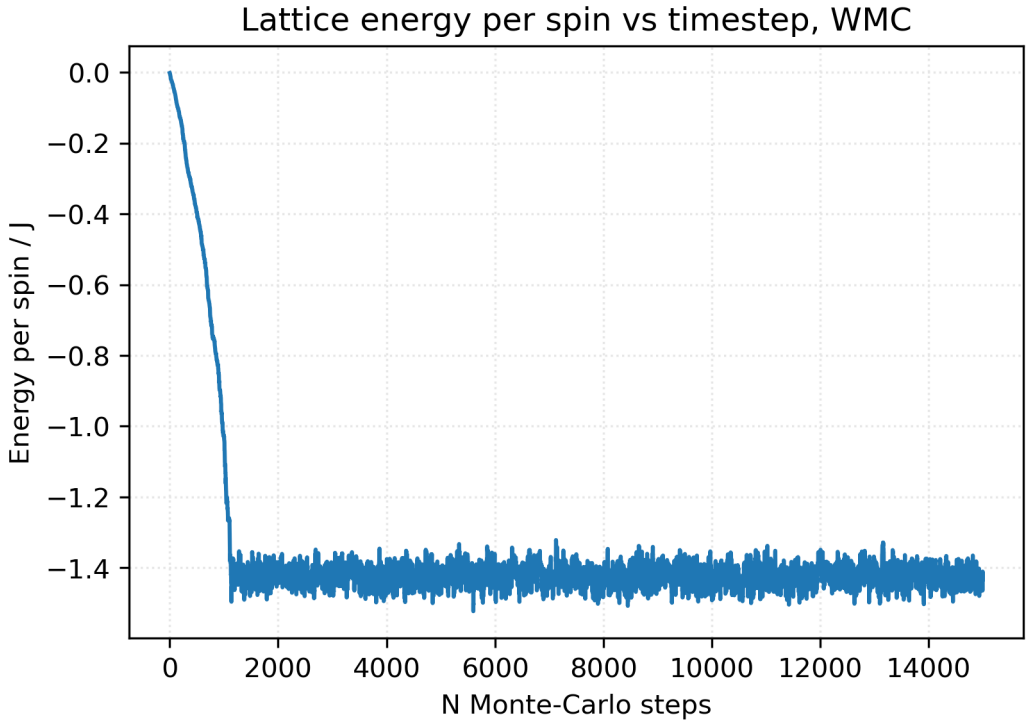


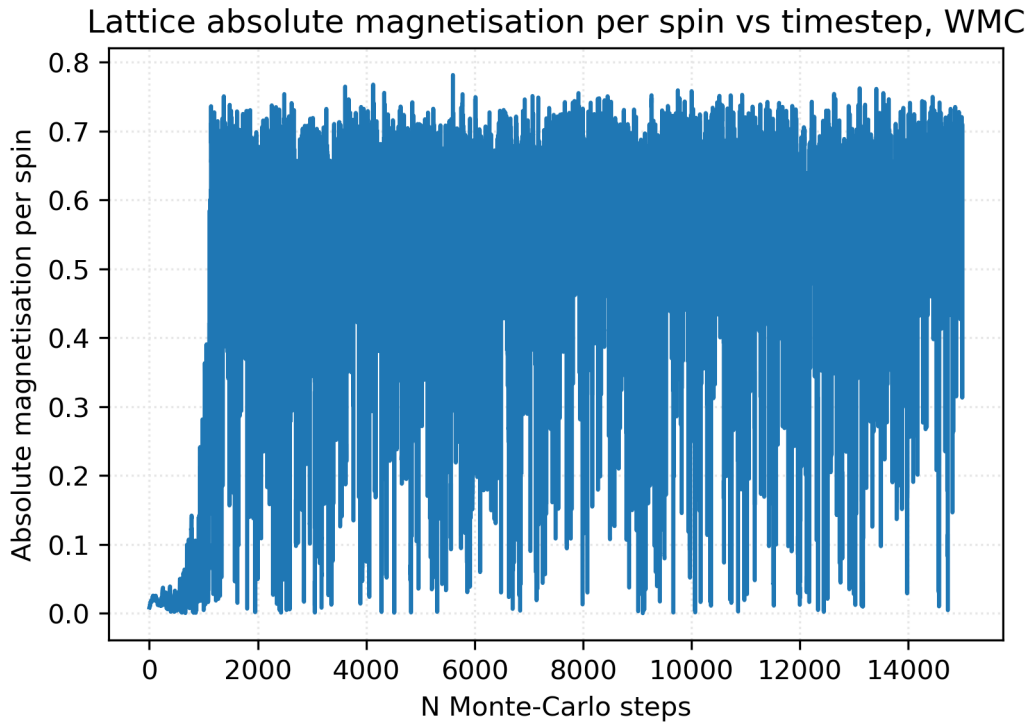
[44]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, WMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, WMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute magnetisation per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute internal magnetic field per spin vs
timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute internal
magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, WMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

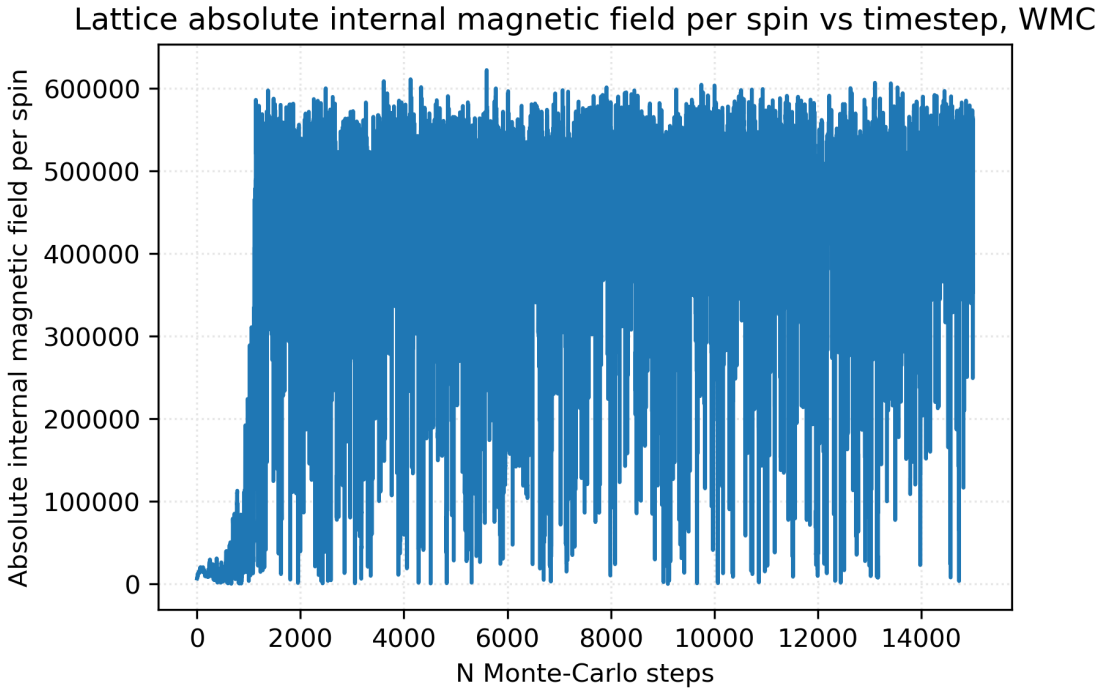
[45]:
```
# 128x128 lattice
parameter_file = 'ising_single_sim_side_len_128_WMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_WMC(parameter_file)
```

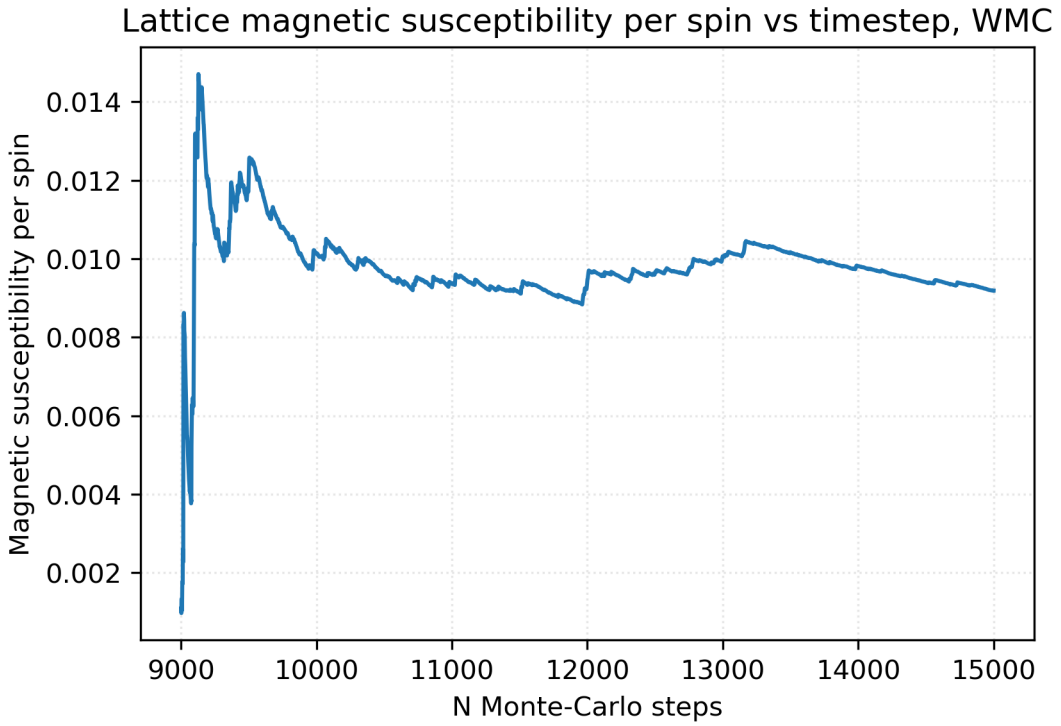
```
Running single Wolff Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=128, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=15000

Running WMC Ising Model::   0%|          | 0/15000 [00:00<?, ?it/s]
```

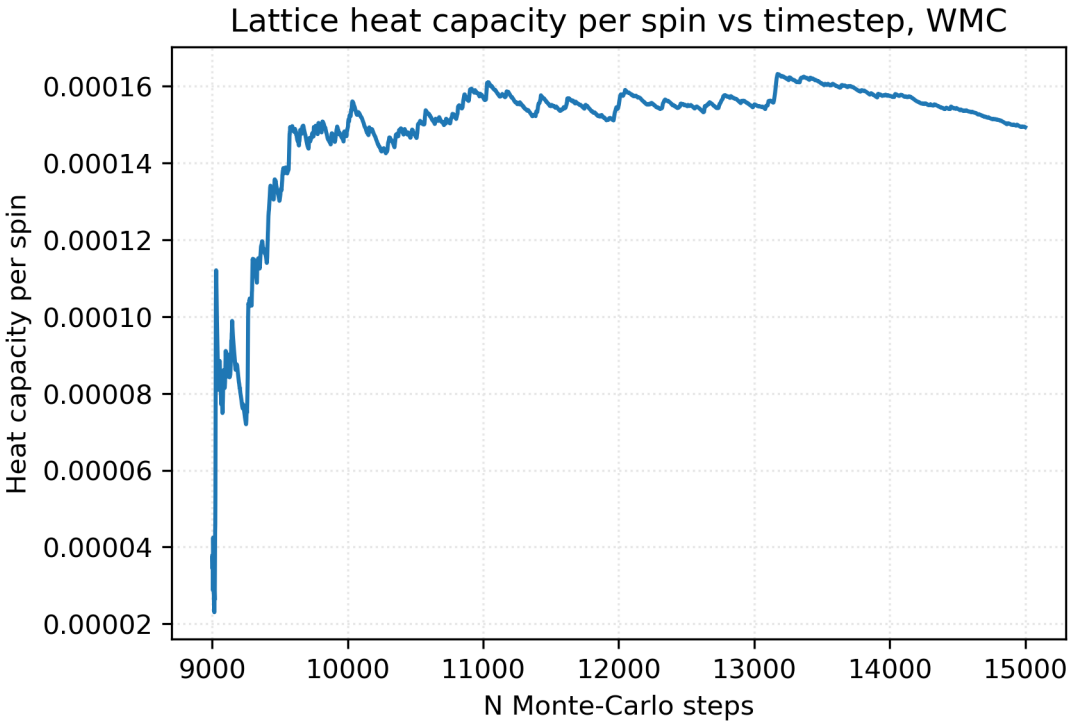





Lattice absolute magnetisation per spin vs timestep, WMC
Absolute magnetisation per spin
0.0
0.1
0.2
0.3
0.4
0.5
0.6
0.7
0.8
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute internal magnetic field per spin vs timestep, WMC
Absolute internal magnetic field per spin
0
100000
200000
300000
400000
500000
600000
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice magnetic susceptibility per spin vs timestep, WMC
Magnetic susceptibility per spin
0.014
0.012
0.010
0.008
0.006
0.004
0.002
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

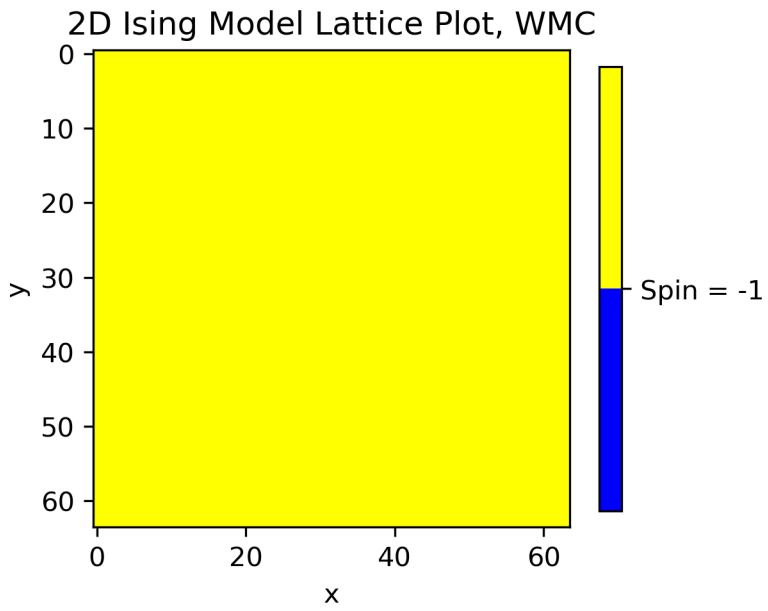

Lattice heat capacity per spin vs timestep, WMC
Heat capacity per spin
0.00016
0.00014
0.00012
0.00010
0.00008
0.00006
0.00004
0.00002
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

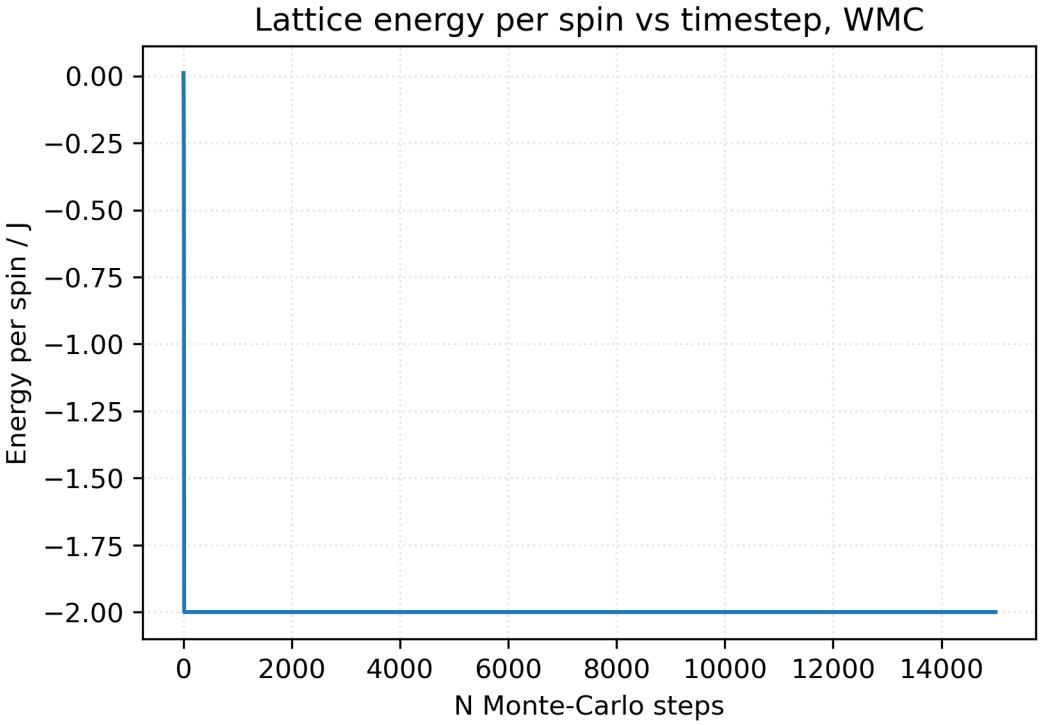
[45]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, WMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, WMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute magnetisation per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute internal magnetic field per spin vs
timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute internal
magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, WMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

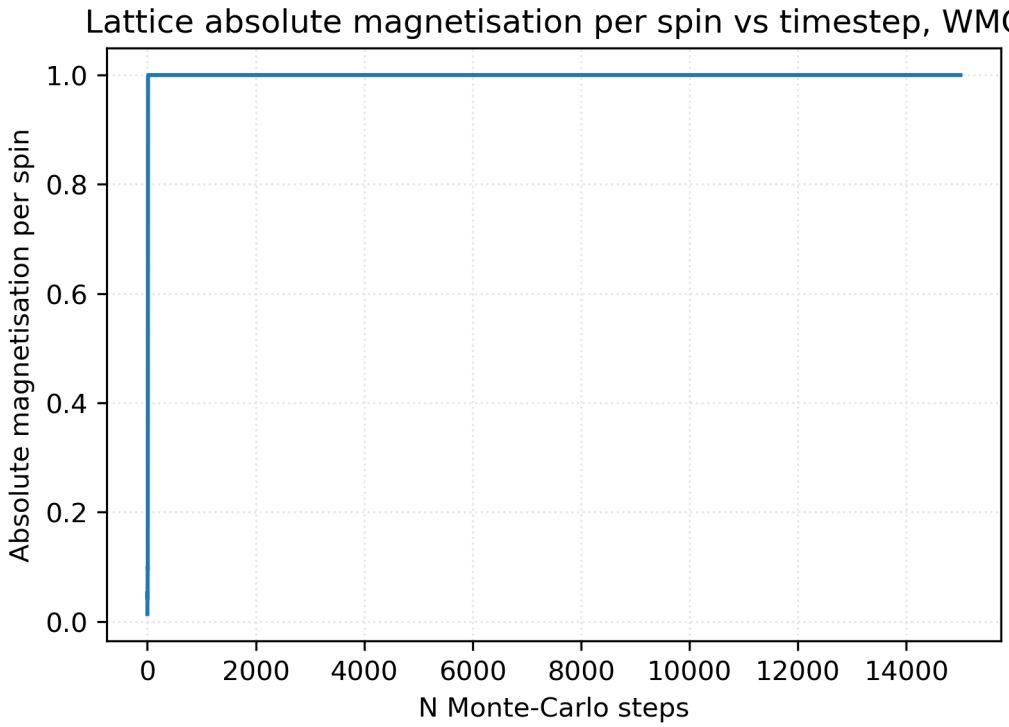
32 Visualising the effect of changing the temperature for a standard lattice size of 64x64 spins using the wolff algorithm

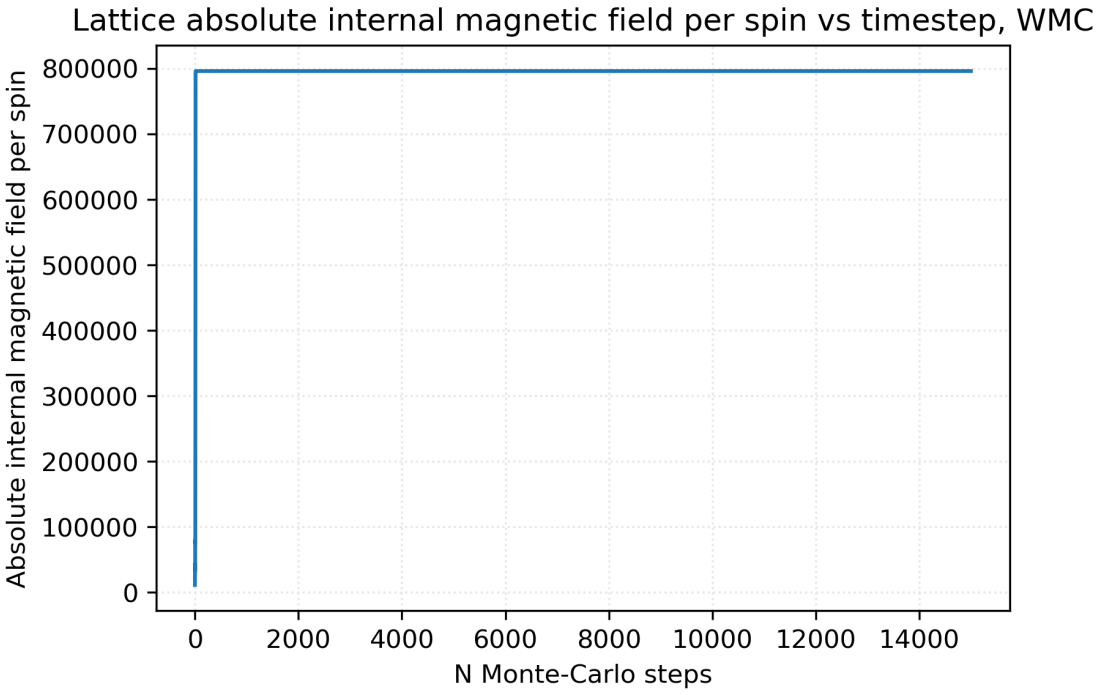
[46]:
```
# 0.001K (close to absolute 0)
parameter_file = 'ising_single_sim_T_0001_WMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_WMC(parameter_file)
```

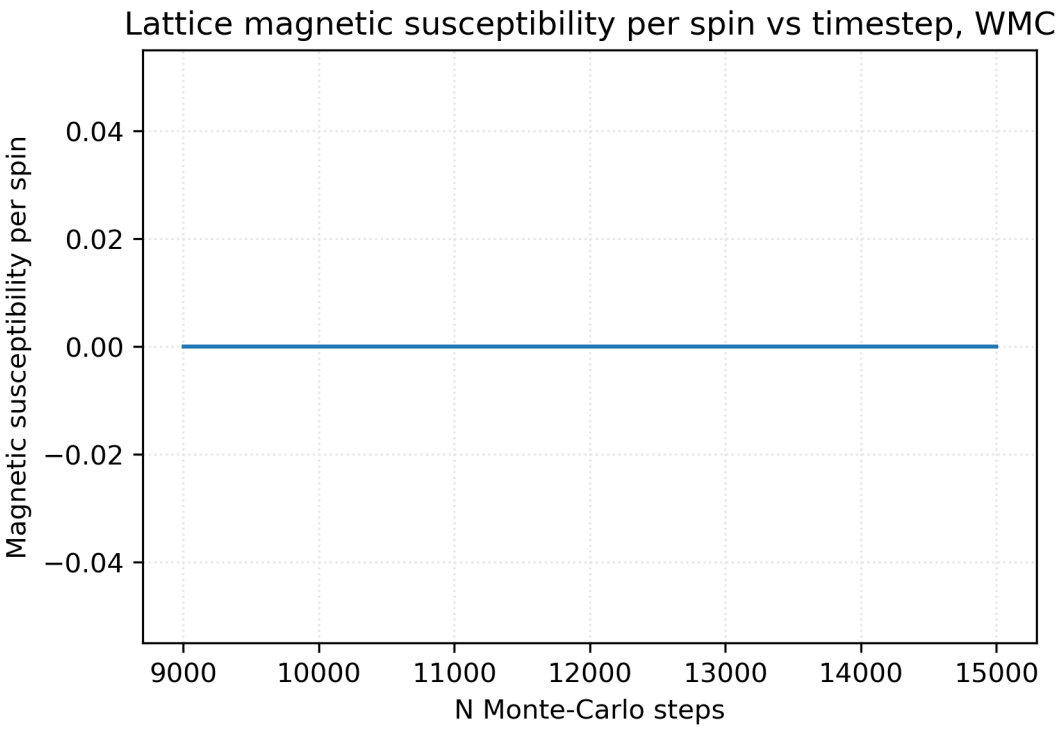
```
Running single Wolff Algorithm simulation with parameters:
temperature=0.001K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=15000


Running WMC Ising Model::   0%|          | 0/15000 [00:00<?, ?it/s]
```

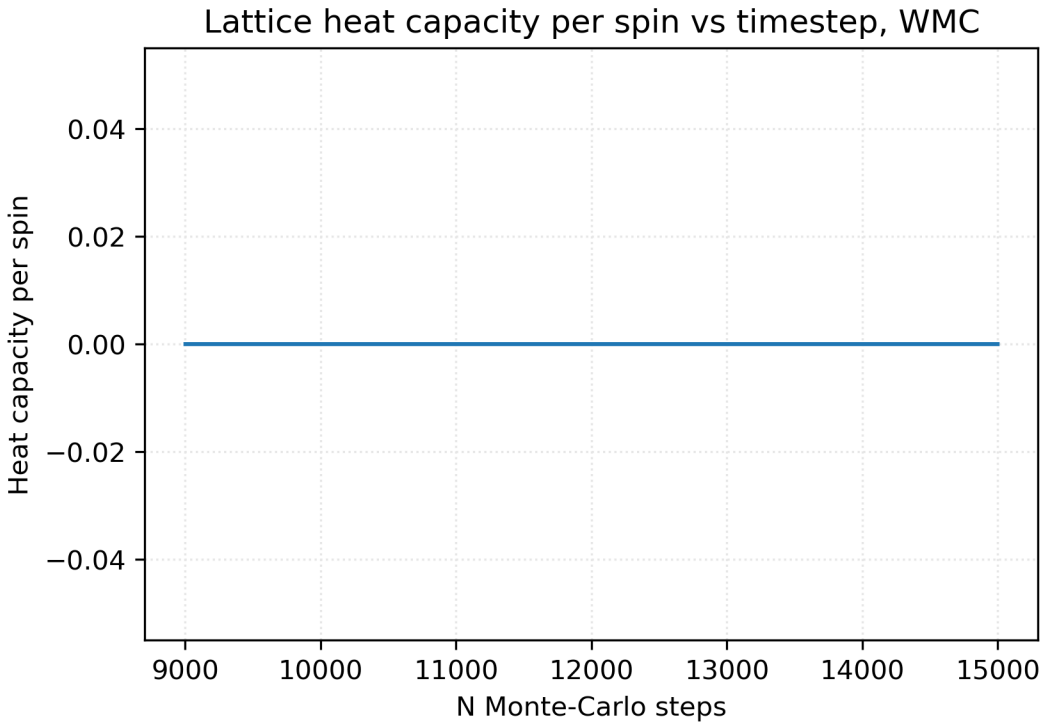

2D Ising Model Lattice Plot, WMC
0
10
20
30
40
50
60
y
0
20
40
60
x
Spin = -1


Lattice energy per spin vs timestep, WMC
0.00
−0.25
−0.50
−0.75
−1.00
−1.25
−1.50
−1.75
−2.00
Energy per spin / J
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute magnetisation per spin vs timestep, WMC
Absolute magnetisation per spin
1.0
0.8
0.6
0.4
0.2
0.0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute internal magnetic field per spin vs timestep, WMC
Absolute internal magnetic field per spin
800000
700000
600000
500000
400000
300000
200000
100000
0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps

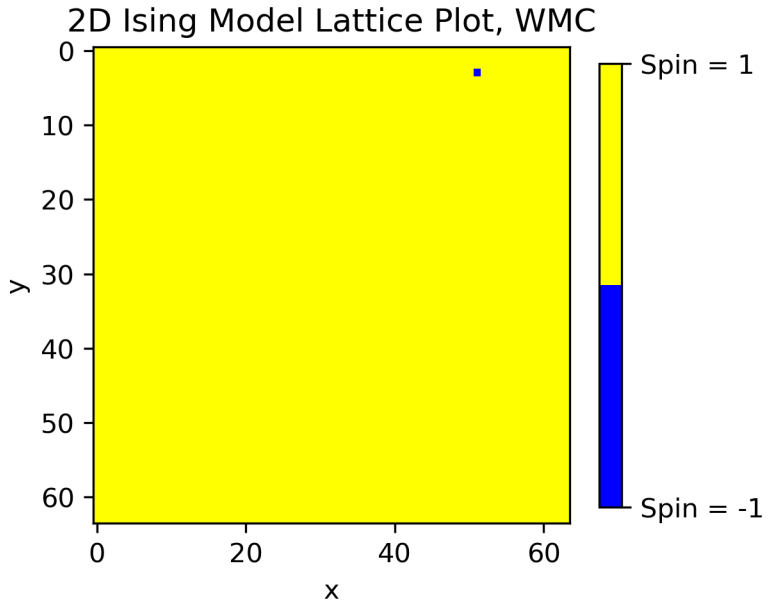

Lattice magnetic susceptibility per spin vs timestep, WMC
Magnetic susceptibility per spin
0.04
0.02
0.00
−0.02
−0.04
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

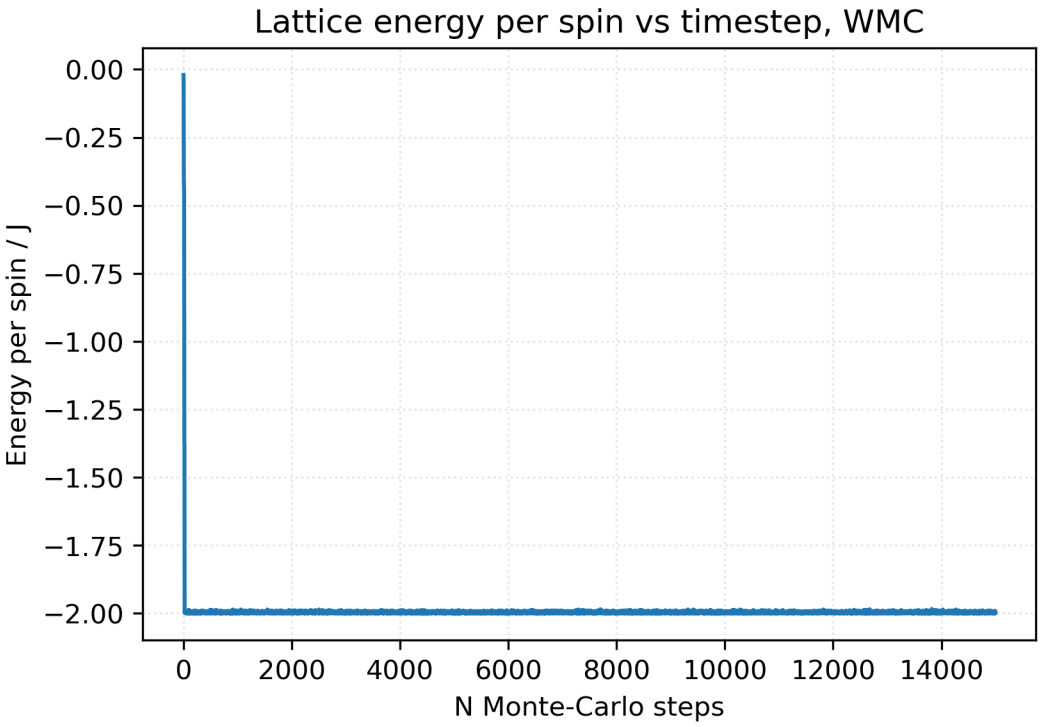

Lattice heat capacity per spin vs timestep, WMC
Heat capacity per spin
0.04
0.02
0.00
−0.02
−0.04
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

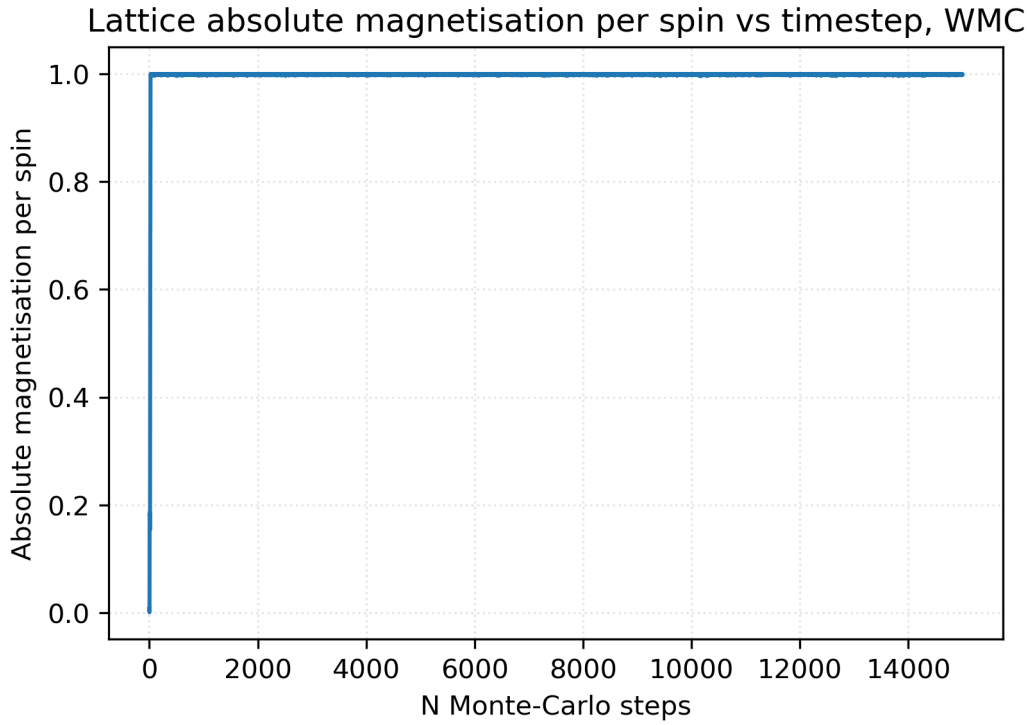
[46]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, WMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, WMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute magnetisation per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute internal magnetic field per spin vs
timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute internal
magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, WMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

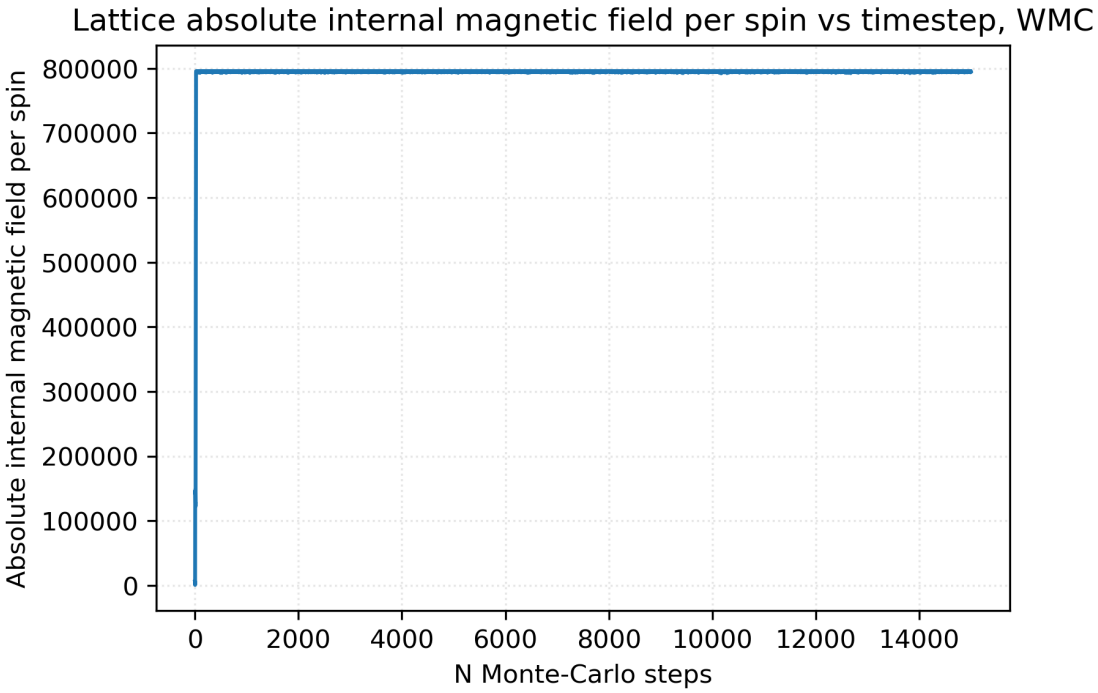
[47]:
```
# 1K
parameter_file = 'ising_single_sim_T_1_WMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_WMC(parameter_file)
```

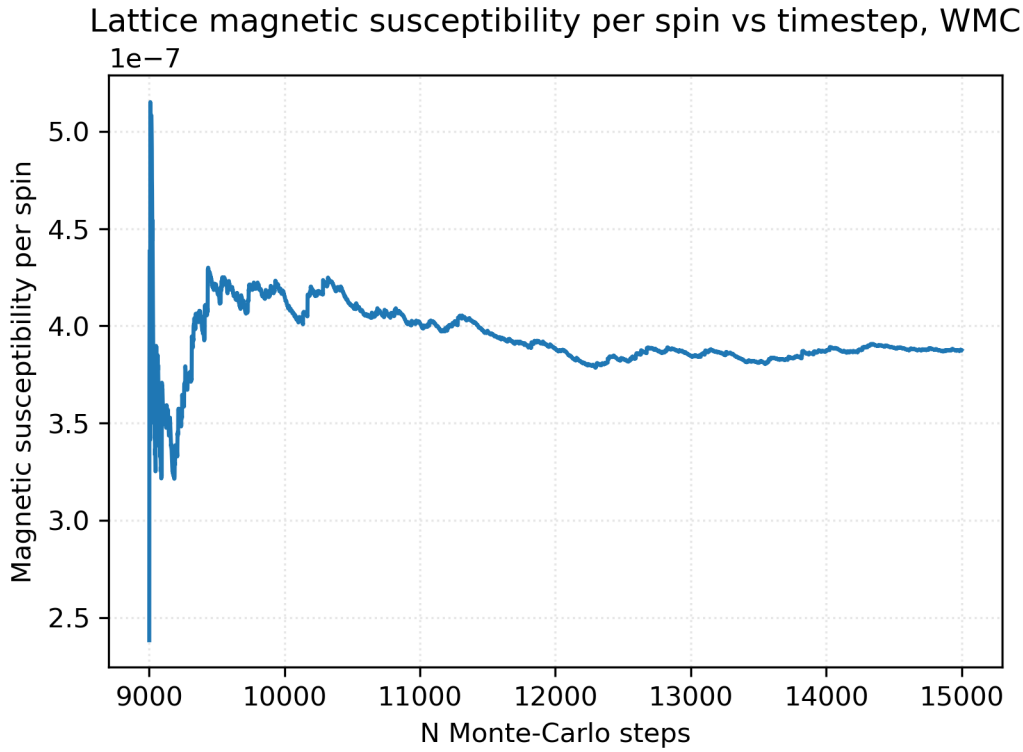
```
Running single Wolff Algorithm simulation with parameters:
temperature=1.0K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=15000


Running WMC Ising Model::   0%|          | 0/15000 [00:00<?, ?it/s]
```

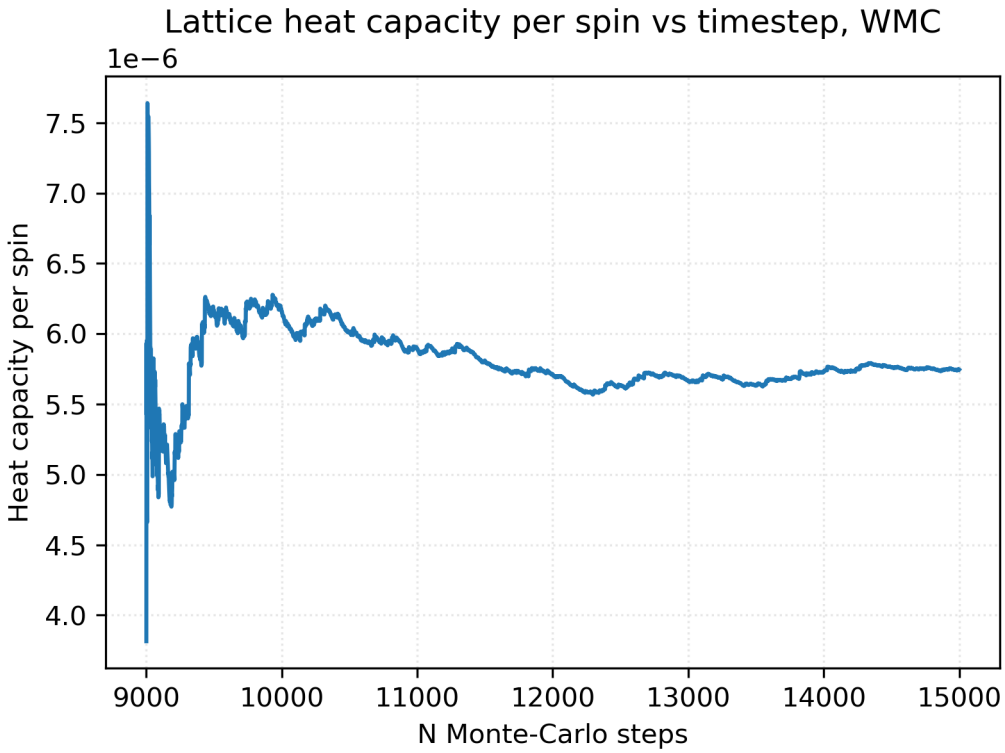



Lattice energy per spin vs timestep, WMC
Energy per spin / J
0.00
−0.25
−0.50
−0.75
−1.00
−1.25
−1.50
−1.75
−2.00
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute magnetisation per spin vs timestep, WMC
Absolute magnetisation per spin
1.0
0.8
0.6
0.4
0.2
0.0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute internal magnetic field per spin vs timestep, WMC
Absolute internal magnetic field per spin
800000
700000
600000
500000
400000
300000
200000
100000
0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps

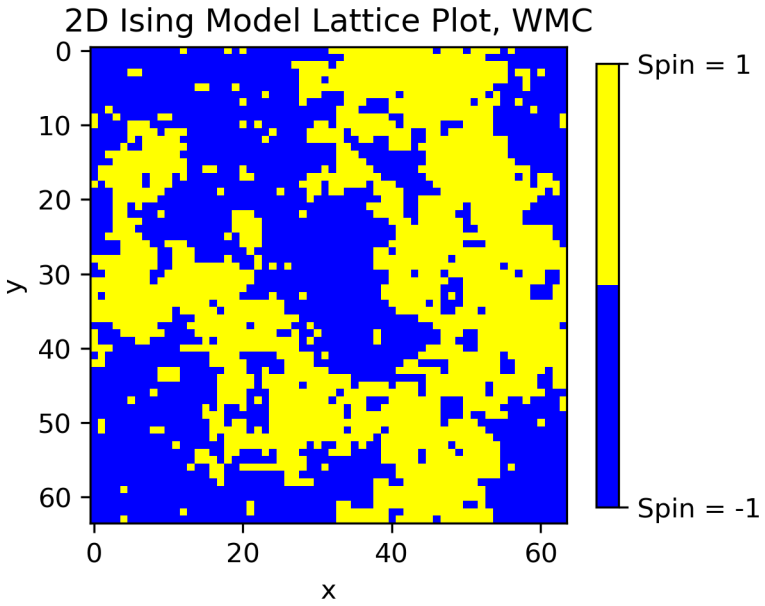

Lattice magnetic susceptibility per spin vs timestep, WMC
1e−7
Magnetic susceptibility per spin
5.0
4.5
4.0
3.5
3.0
2.5
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

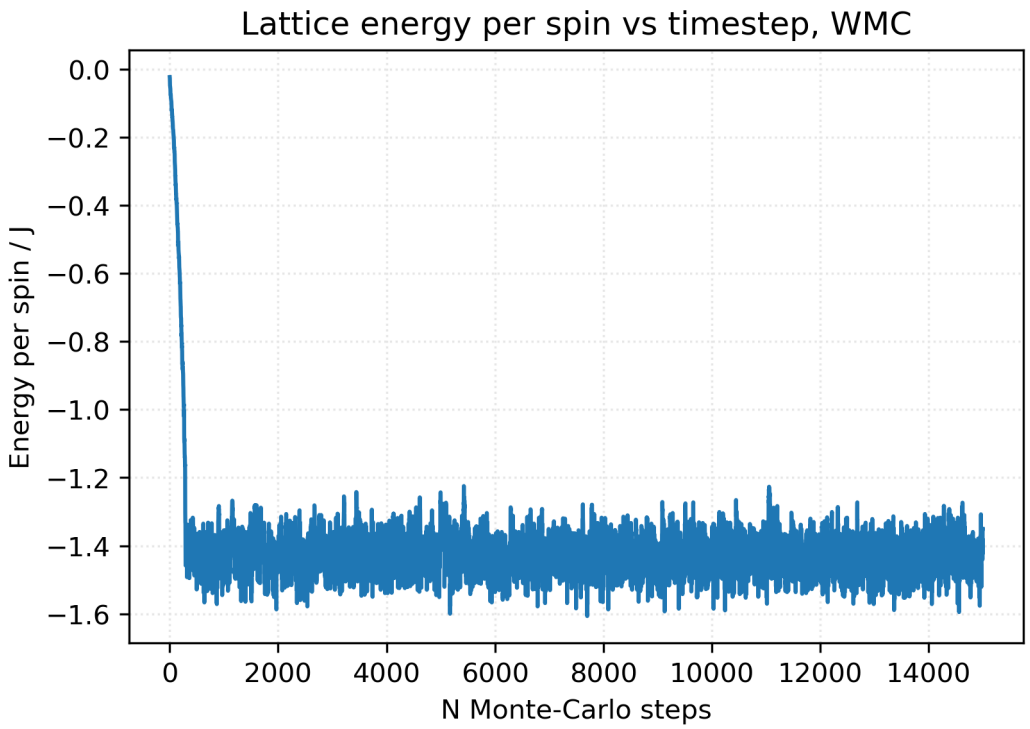


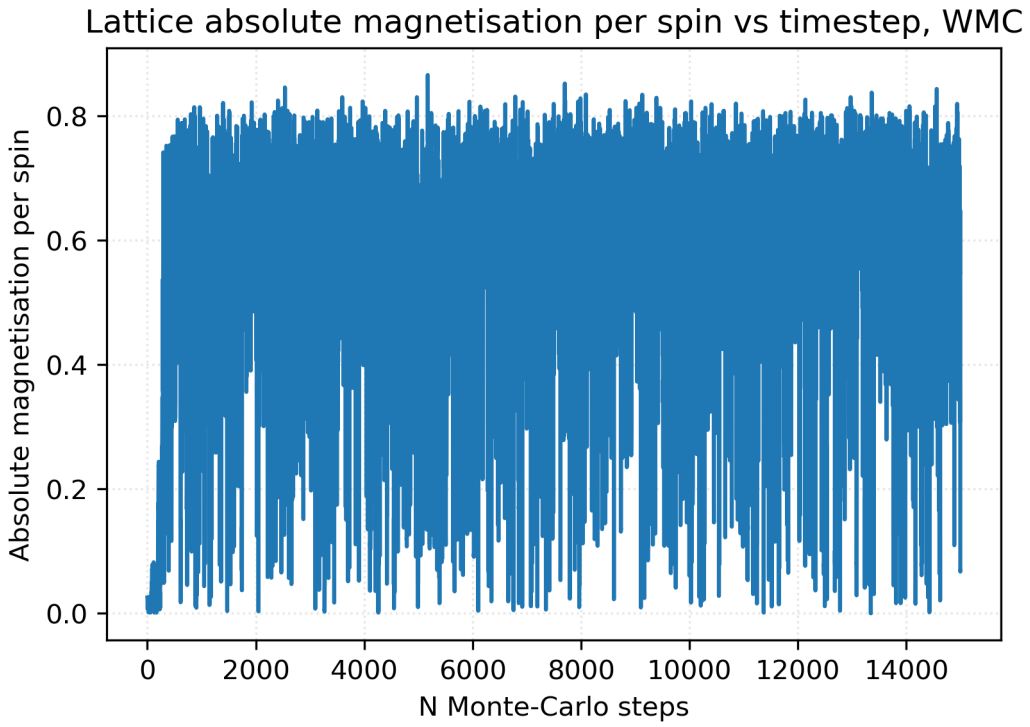
[47]: (<Figure size 1200x1200 with 2 Axes>,
<Axes: title={'center': '2D Ising Model Lattice Plot, WMC'}, xlabel='x', ylabel='y'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Lattice energy per spin vs timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Energy per spin / J'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Lattice absolute magnetisation per spin vs timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute magnetisation per spin'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Lattice absolute internal magnetic field per spin vs timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute internal magnetic field per spin'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Lattice heat capacity per spin vs timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)

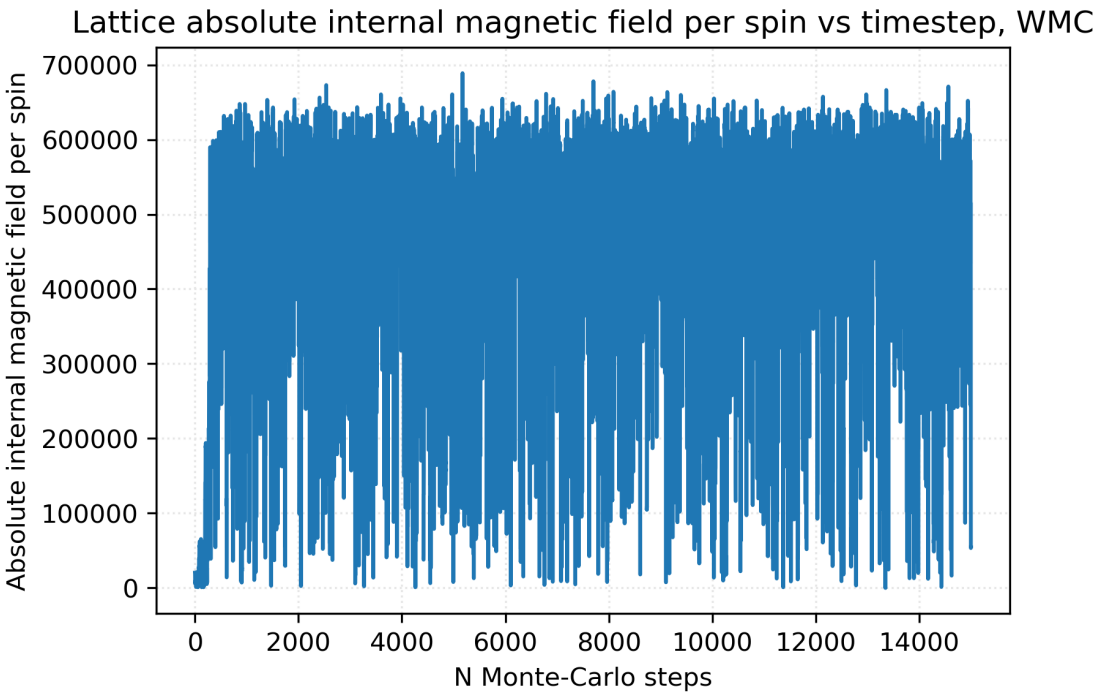
[48]:
```
# 2.269185K (close to tcrit)
parameter_file = 'ising_single_sim_T_crit_WMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_WMC(parameter_file)
```

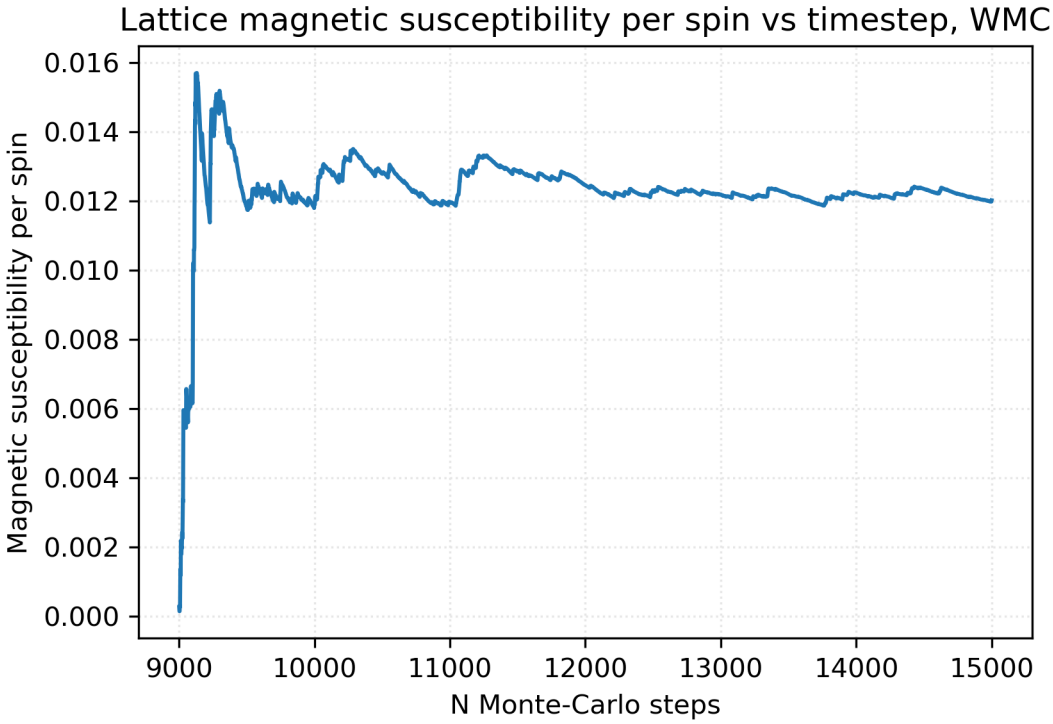
```
Running single Wolff Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=15000


Running WMC Ising Model::   0%|          | 0/15000 [00:00<?, ?it/s]
```

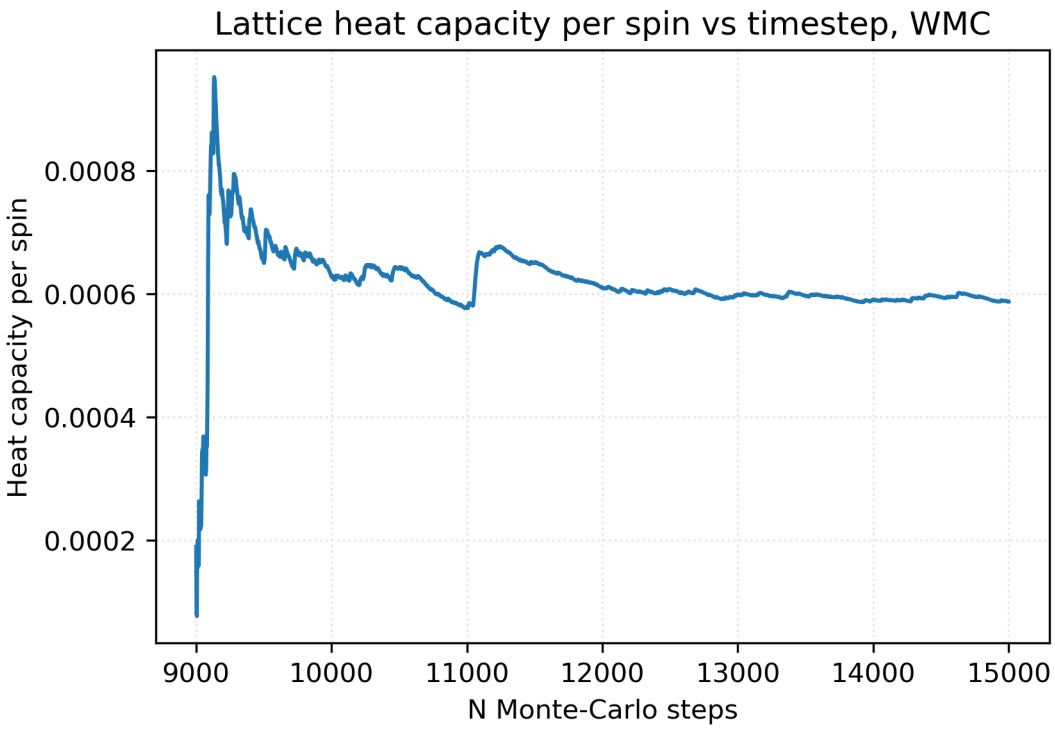



Lattice energy per spin vs timestep, WMC
Energy per spin / J
0.0
−0.2
−0.4
−0.6
−0.8
−1.0
−1.2
−1.4
−1.6
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute magnetisation per spin vs timestep, WMC
Absolute magnetisation per spin
0.8
0.6
0.4
0.2
0.0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute internal magnetic field per spin vs timestep, WMC
Absolute internal magnetic field per spin
700000
600000
500000
400000
300000
200000
100000
0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps

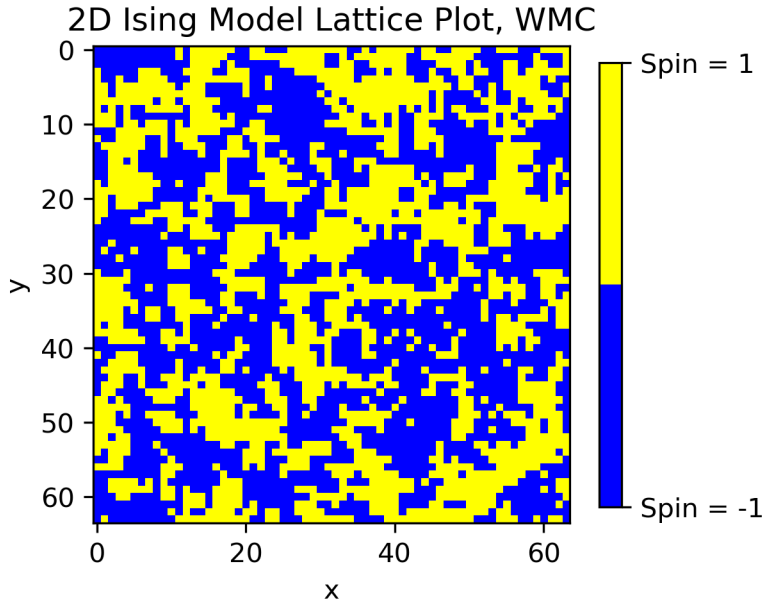

Lattice magnetic susceptibility per spin vs timestep, WMC
Magnetic susceptibility per spin
0.016
0.014
0.012
0.010
0.008
0.006
0.004
0.002
0.000
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

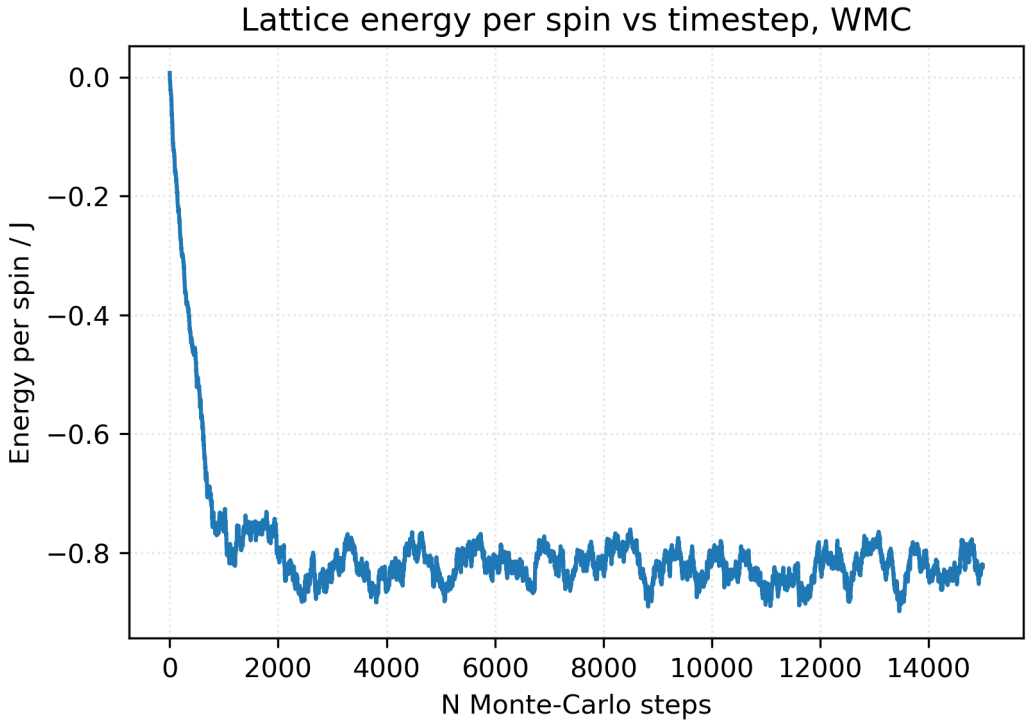


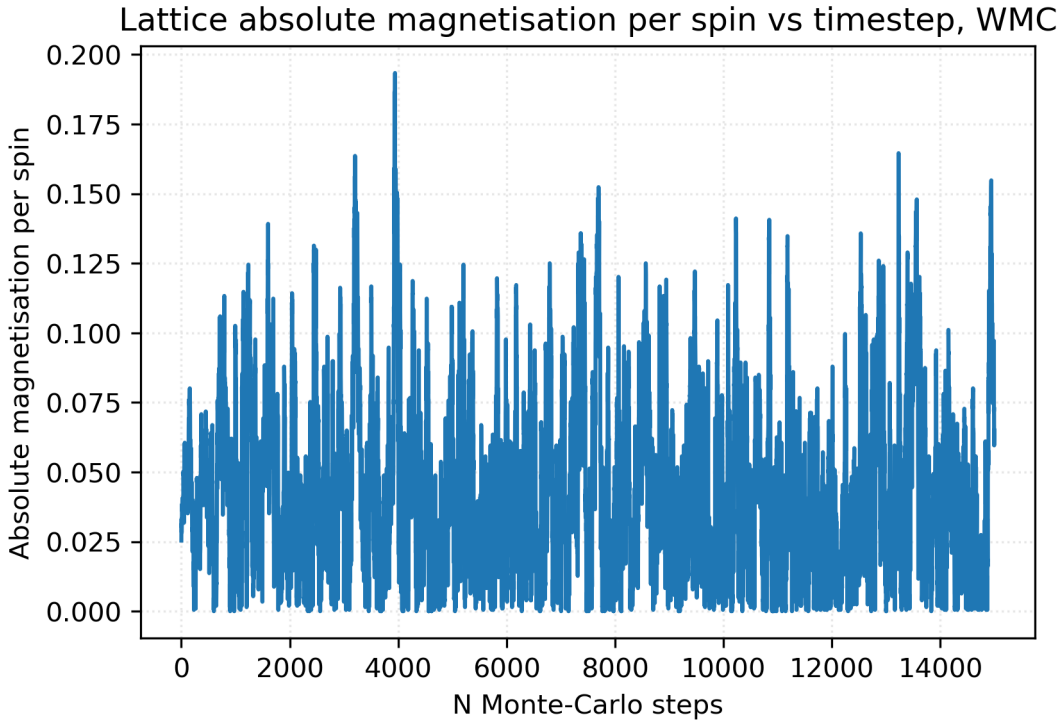
[48]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, WMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, WMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute magnetisation per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute internal magnetic field per spin vs
timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute internal
magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, WMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

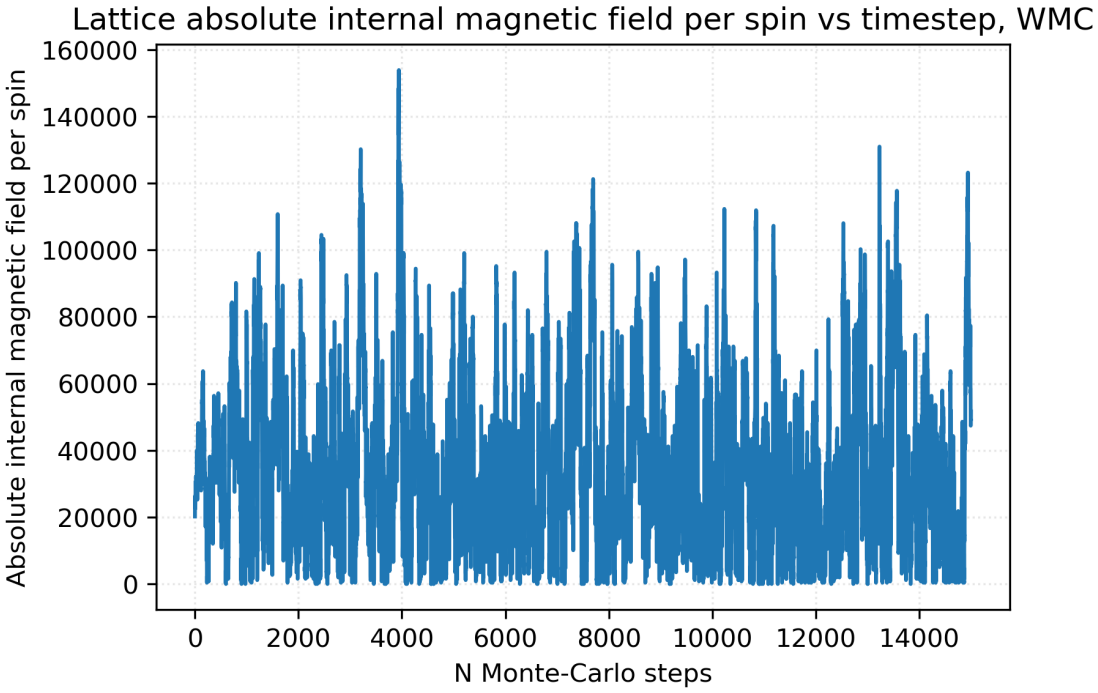
[49]:
```
# 3K
parameter_file = 'ising_single_sim_T_3_WMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_WMC(parameter_file)
```

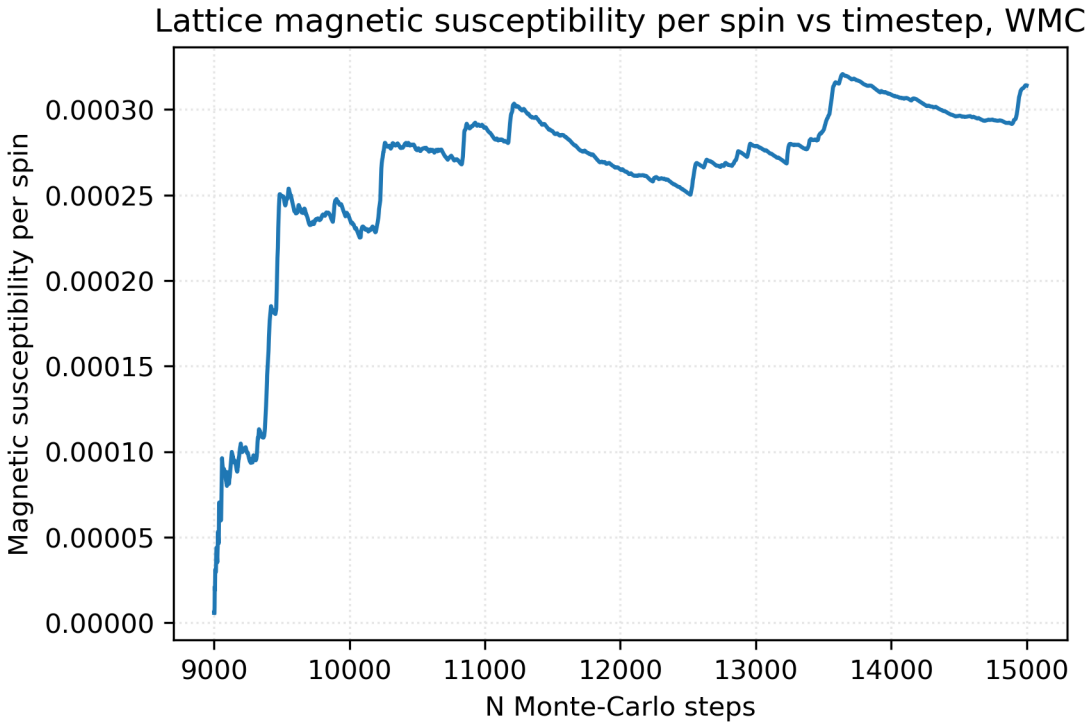
```
Running single Wolff Algorithm simulation with parameters:
temperature=3.0K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=1.0, k_b=1.0, u_b=0.0, nsteps=15000

Running WMC Ising Model::   0%|          | 0/15000 [00:00<?, ?it/s]
```

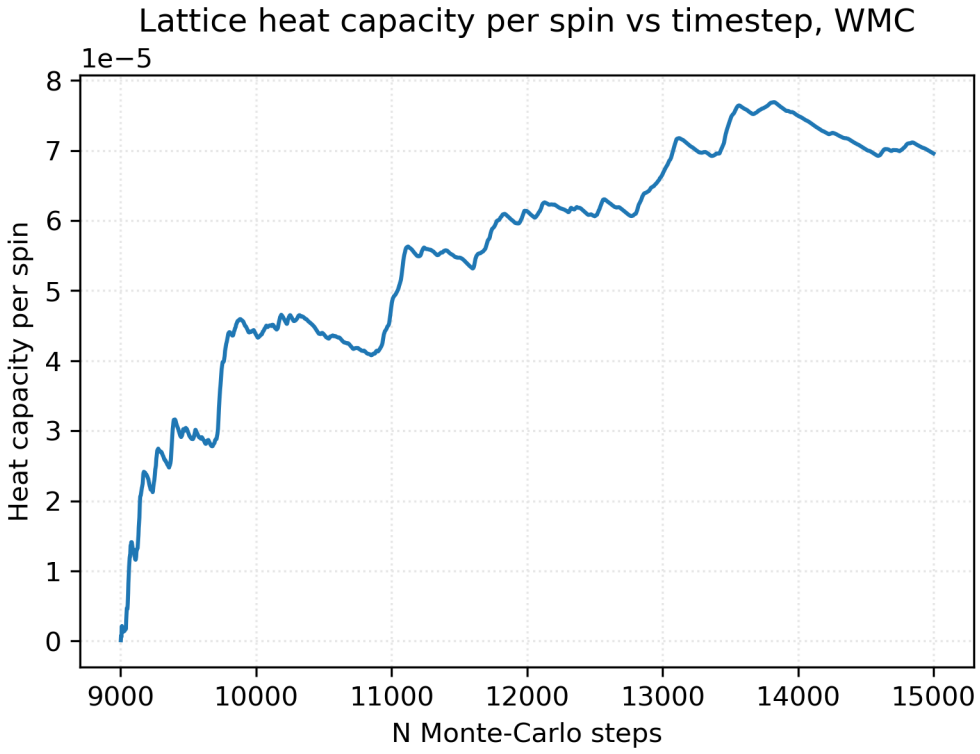





Lattice absolute magnetisation per spin vs timestep, WMC
Absolute magnetisation per spin
0.200
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute internal magnetic field per spin vs timestep, WMC
Absolute internal magnetic field per spin
160000
140000
120000
100000
80000
60000
40000
20000
0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice magnetic susceptibility per spin vs timestep, WMC
Magnetic susceptibility per spin
0.00030
0.00025
0.00020
0.00015
0.00010
0.00005
0.00000
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

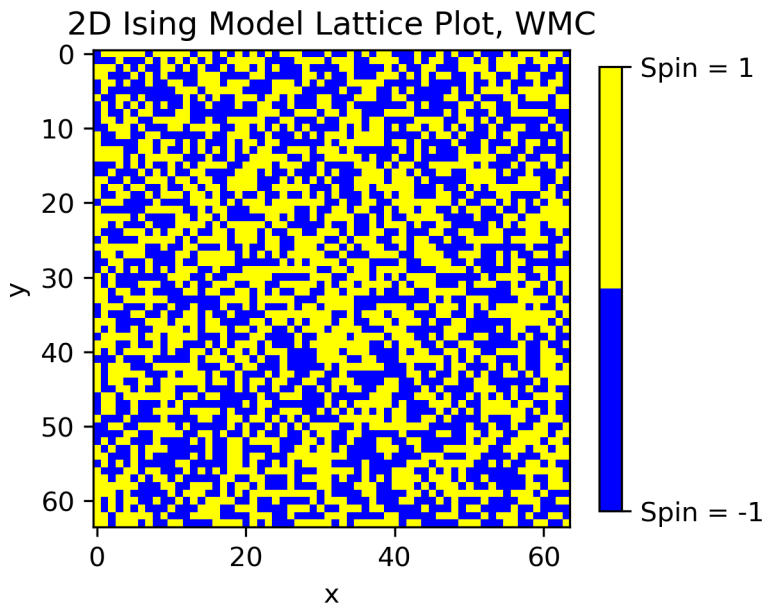

Lattice heat capacity per spin vs timestep, WMC
1e−5
Heat capacity per spin
8
7
6
5
4
3
2
1
0
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

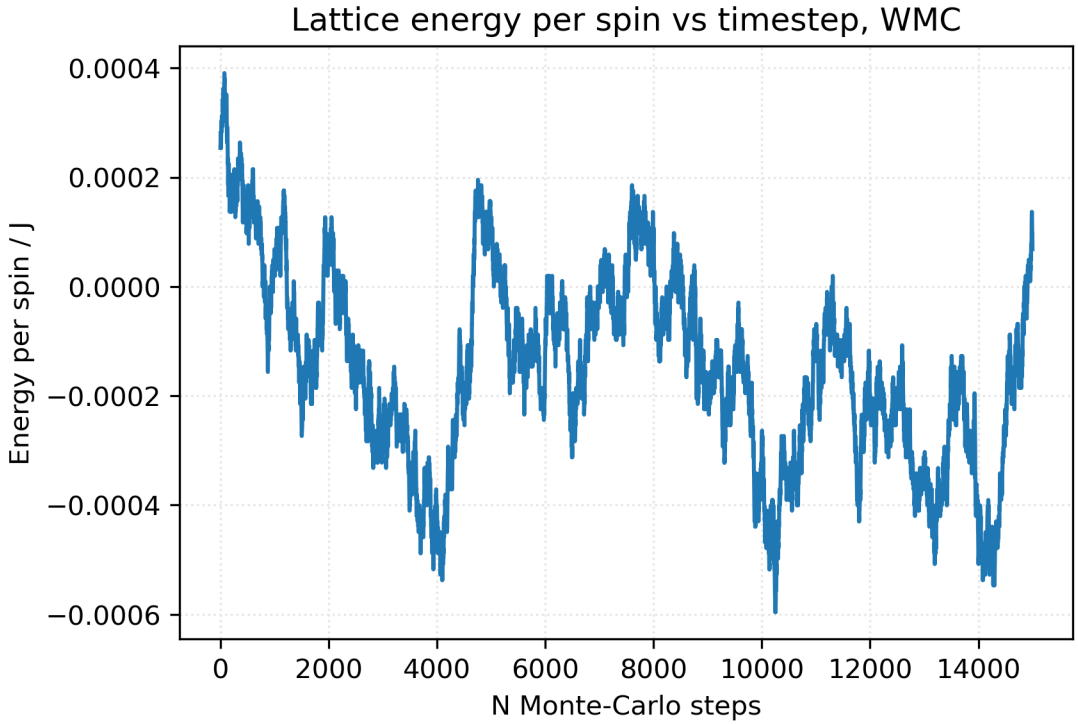
[49]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, WMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, WMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute magnetisation per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute internal magnetic field per spin vs
timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute internal
magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, WMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

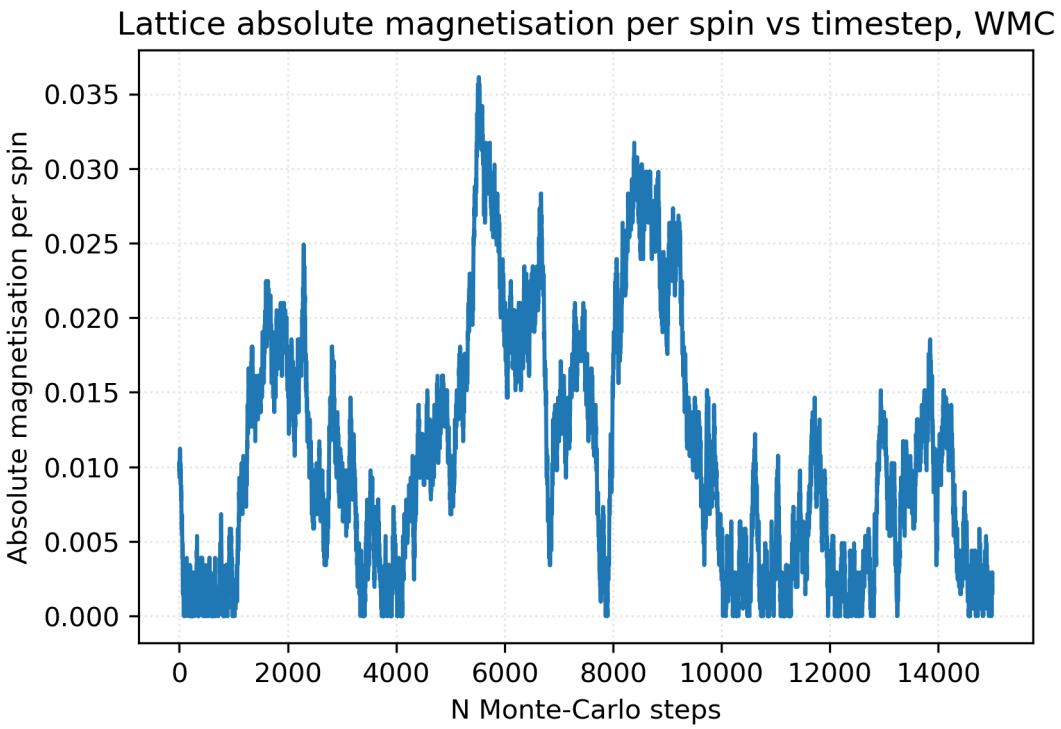
33 Visualising the effect of changing exchange interaction constant J for a standard lattice size of 64x64 spins using the wolff algorithm whilst remaining at the onsager critical temperature

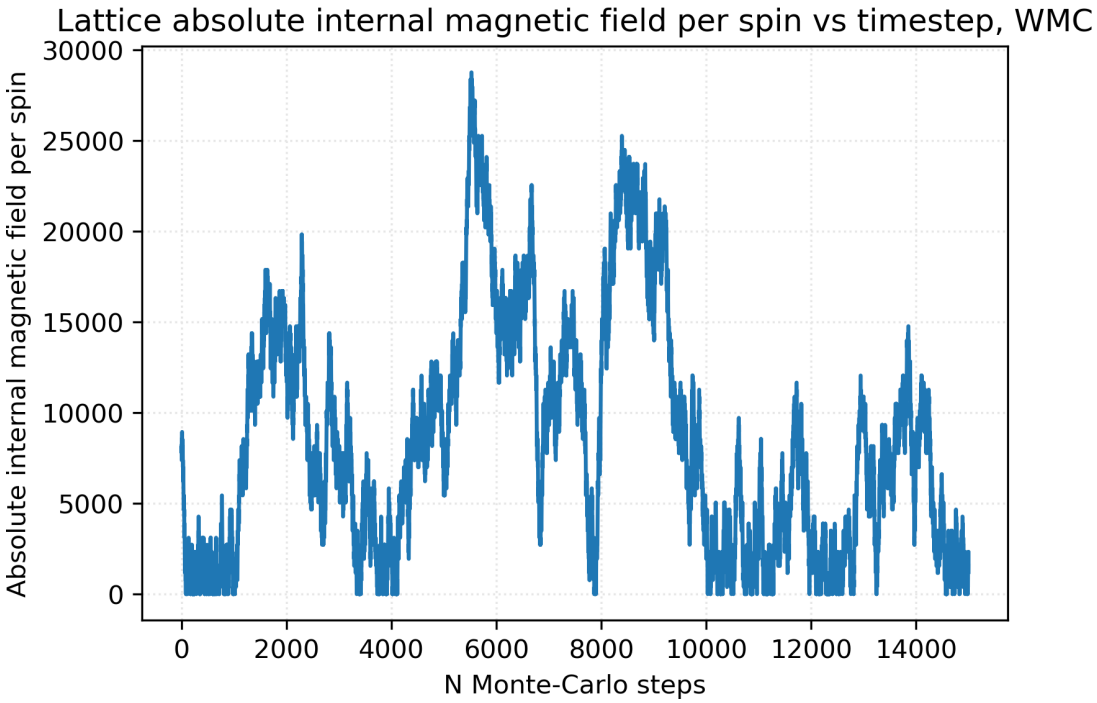
[50]:
```
# J = 0.01
parameter_file = 'ising_single_sim_J_001_WMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_WMC(parameter_file)
```

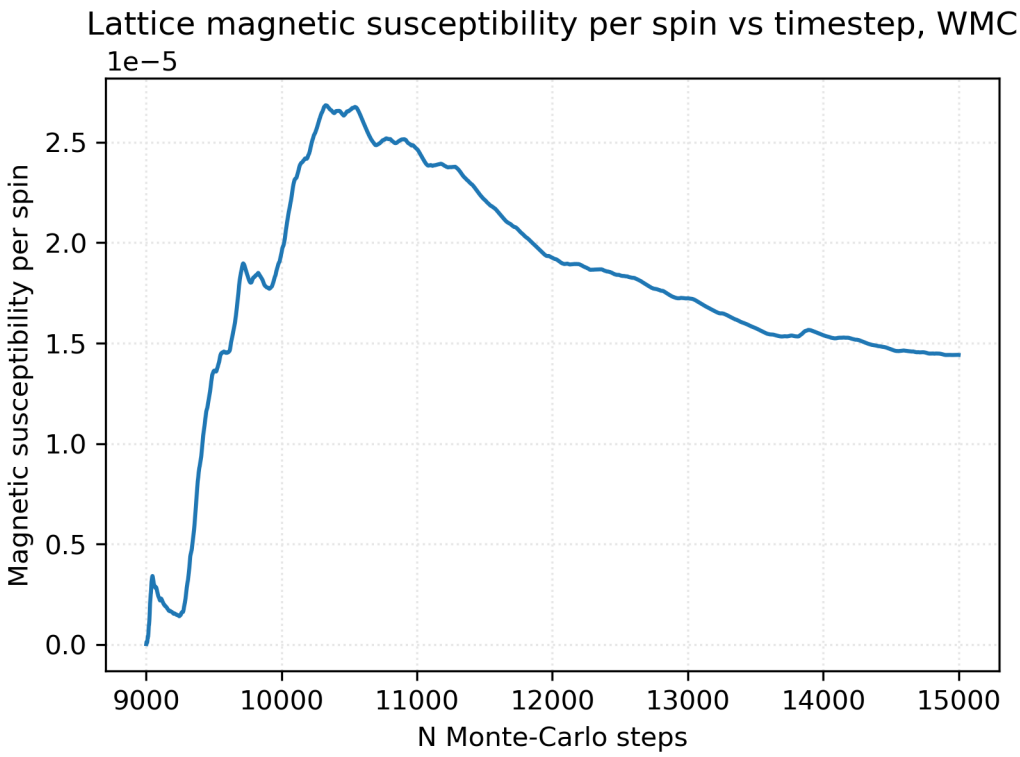
```
Running single Wolff Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=0.01, k_b=1.0, u_b=0.0, nsteps=15000


Running WMC Ising Model::   0%|          | 0/15000 [00:00<?, ?it/s]
```

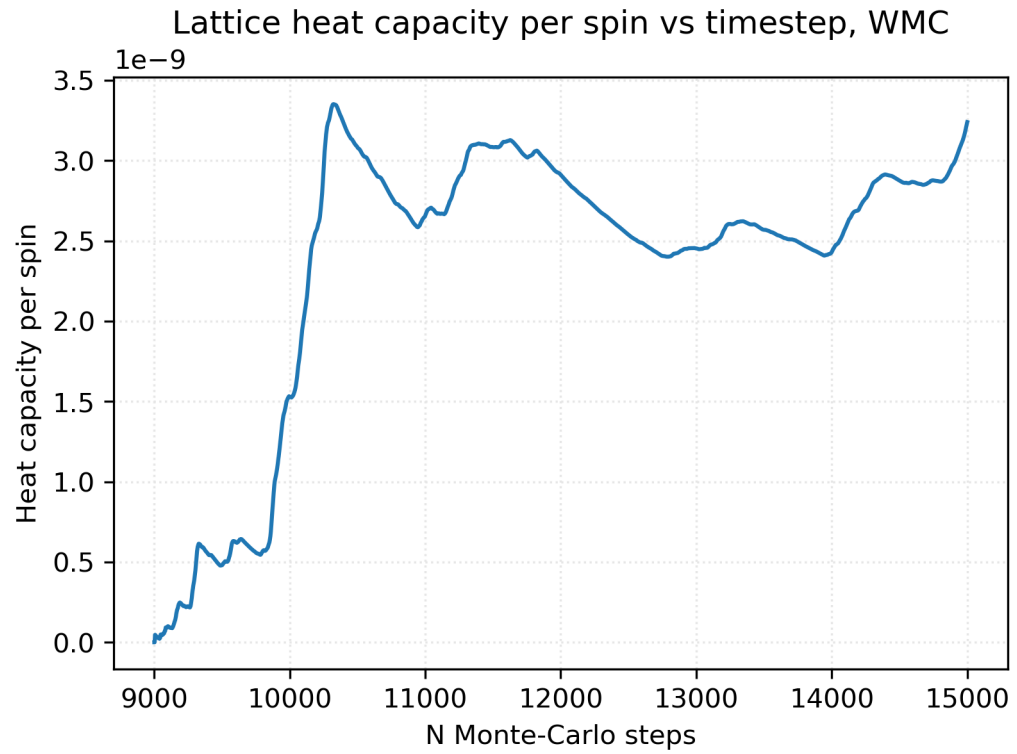

2D Ising Model Lattice Plot, WMC
0
10
20
30
40
50
60
y
0
20
40
60
x
Spin = 1
Spin = -1


Lattice energy per spin vs timestep, WMC
0.0004
0.0002
0.0000
−0.0002
−0.0004
−0.0006
Energy per spin / J
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute magnetisation per spin vs timestep, WMC
Absolute magnetisation per spin
0.035
0.030
0.025
0.020
0.015
0.010
0.005
0.000
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute internal magnetic field per spin vs timestep, WMC
Absolute internal magnetic field per spin
30000
25000
20000
15000
10000
5000
0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps

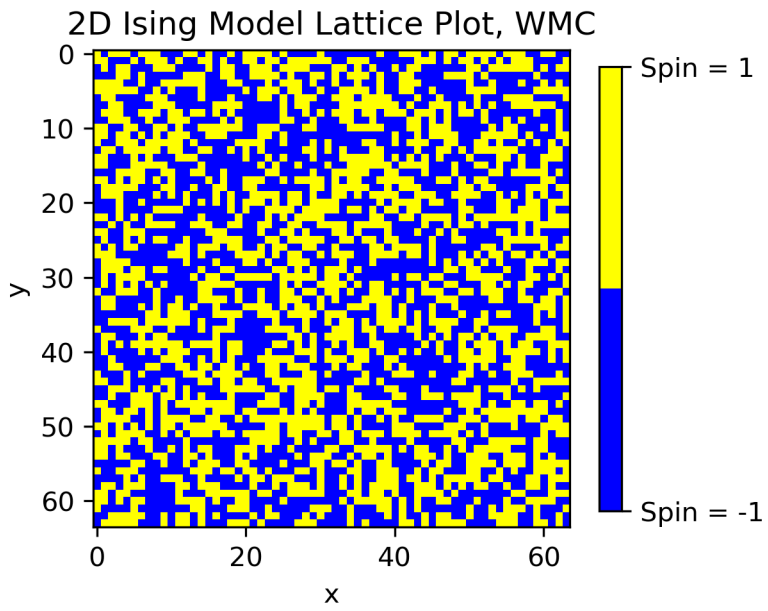

Lattice magnetic susceptibility per spin vs timestep, WMC
1e−5
Magnetic susceptibility per spin
2.5
2.0
1.5
1.0
0.5
0.0
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

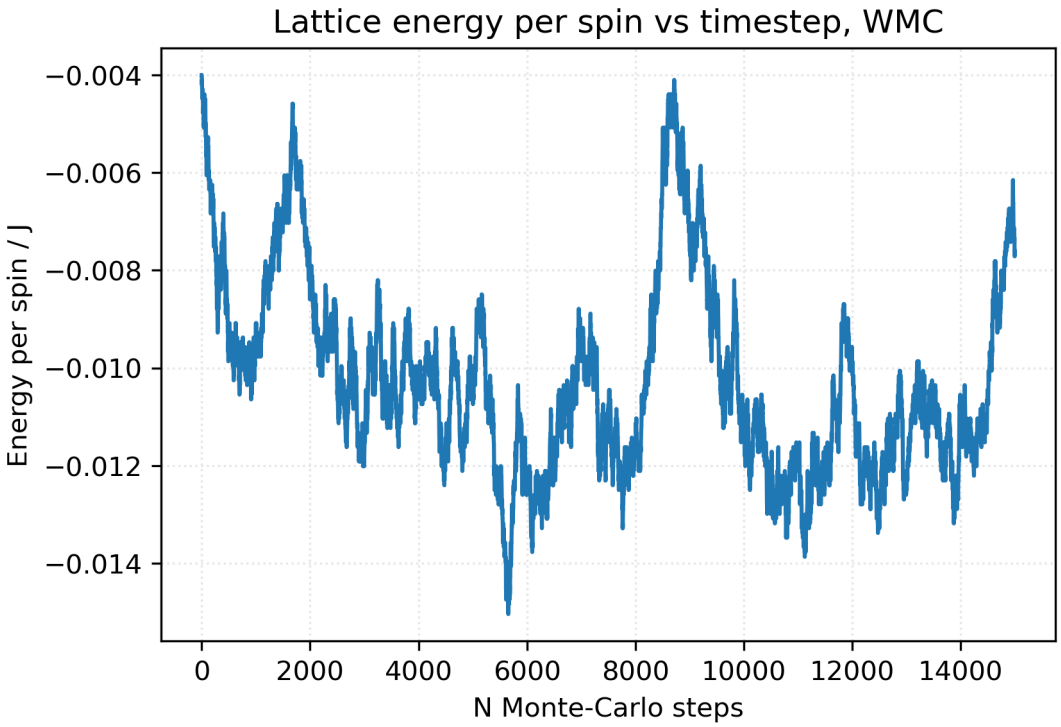

Lattice heat capacity per spin vs timestep, WMC
1e−9
Heat capacity per spin
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

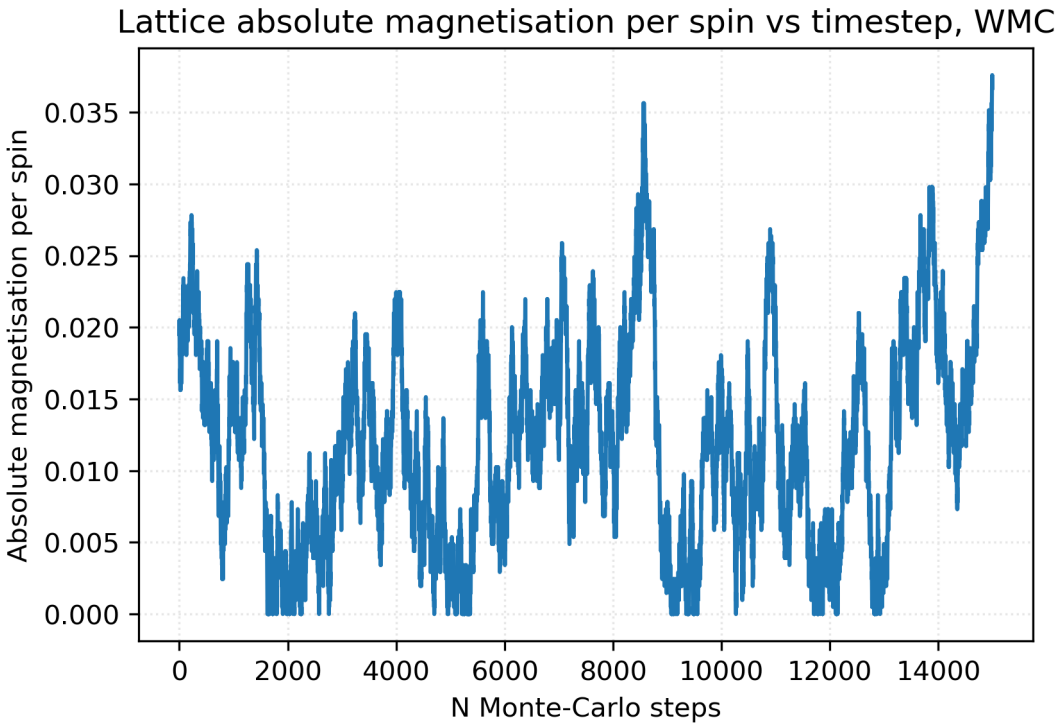
[50]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, WMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, WMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute magnetisation per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute internal magnetic field per spin vs
timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute internal
magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, WMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

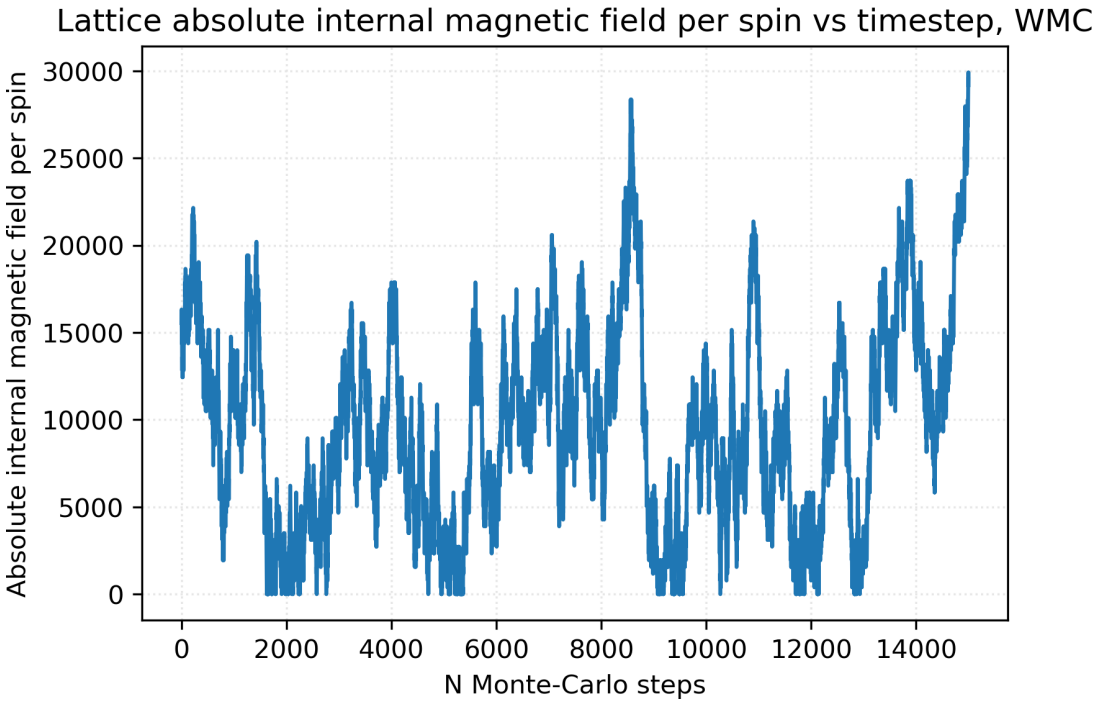
[51]:
```
# J = 0.1
parameter_file = 'ising_single_sim_J_01_WMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_WMC(parameter_file)
```

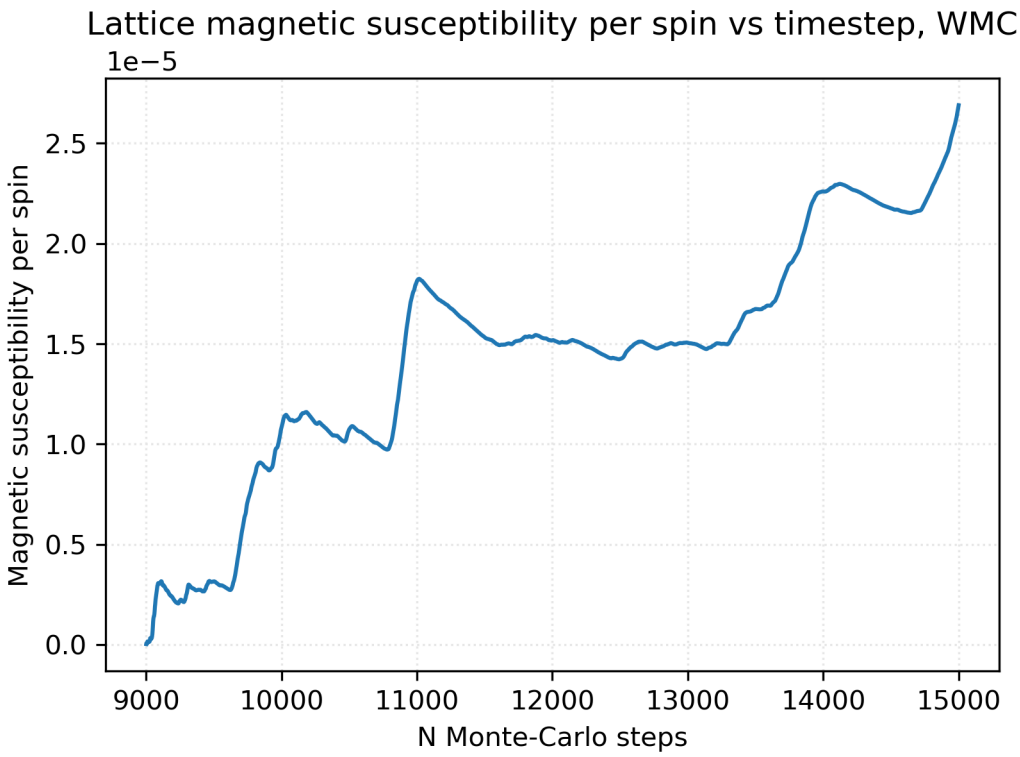
```
Running single Wolff Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=0.1, k_b=1.0, u_b=0.0, nsteps=15000


Running WMC Ising Model::   0%|          | 0/15000 [00:00<?, ?it/s]
```

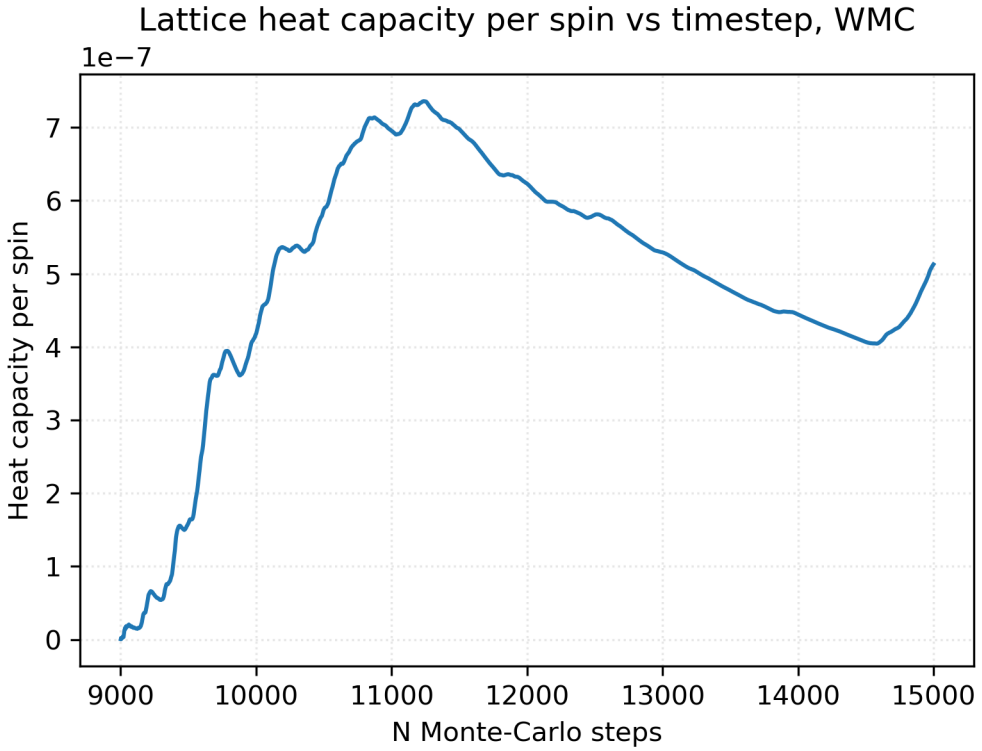

2D Ising Model Lattice Plot, WMC
0
10
20
30
40
50
60
y
0
20
40
60
x
Spin = 1
Spin = -1


Lattice energy per spin vs timestep, WMC
−0.004
−0.006
−0.008
−0.010
−0.012
−0.014
Energy per spin / J
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute magnetisation per spin vs timestep, WMC
Absolute magnetisation per spin
0.035
0.030
0.025
0.020
0.015
0.010
0.005
0.000
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute internal magnetic field per spin vs timestep, WMC
Absolute internal magnetic field per spin
30000
25000
20000
15000
10000
5000
0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps

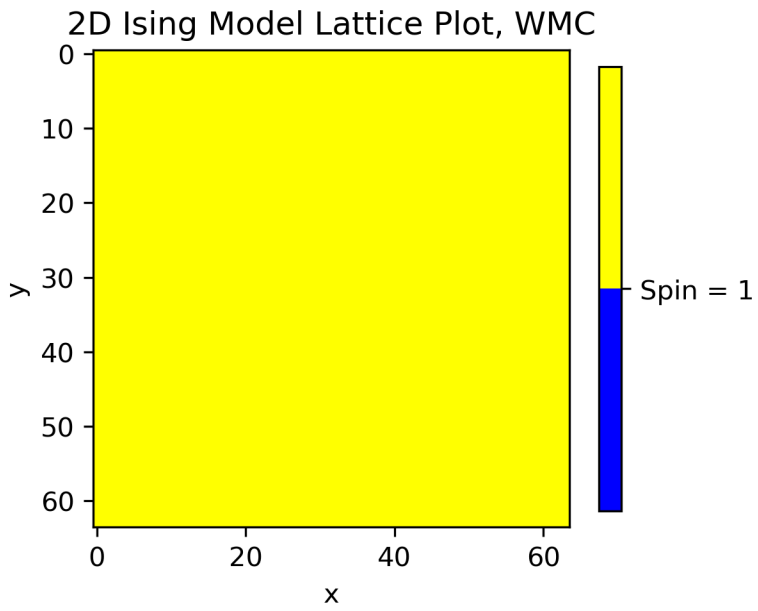

Lattice magnetic susceptibility per spin vs timestep, WMC
1e−5
Magnetic susceptibility per spin
2.5
2.0
1.5
1.0
0.5
0.0
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

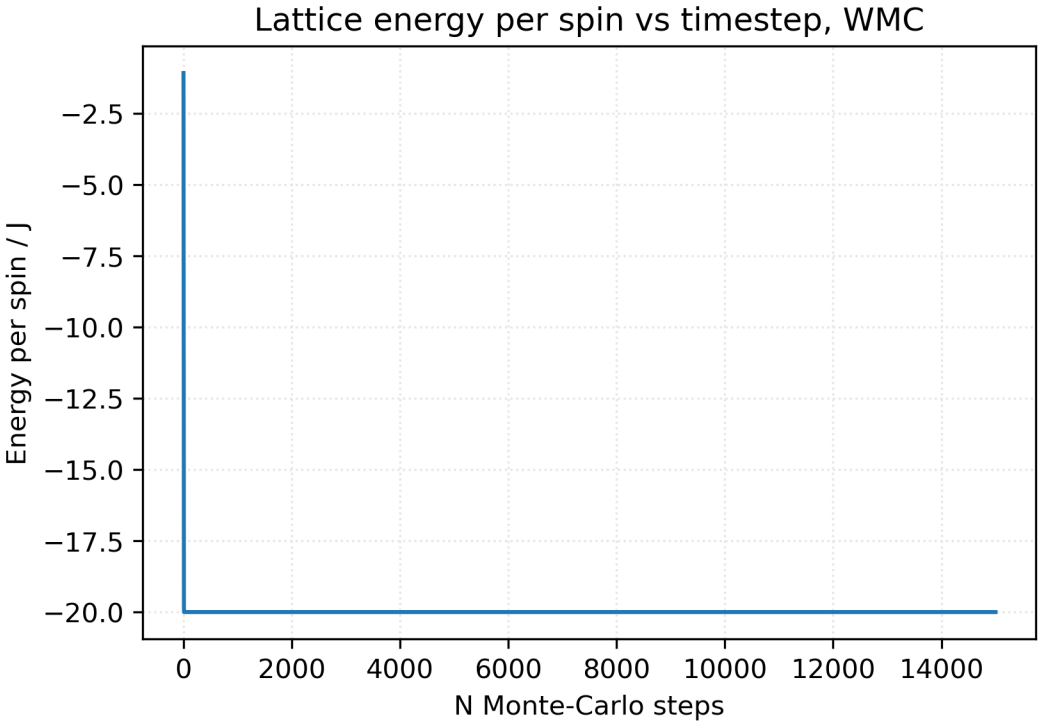

Lattice heat capacity per spin vs timestep, WMC
1e−7
Heat capacity per spin
7
6
5
4
3
2
1
0
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

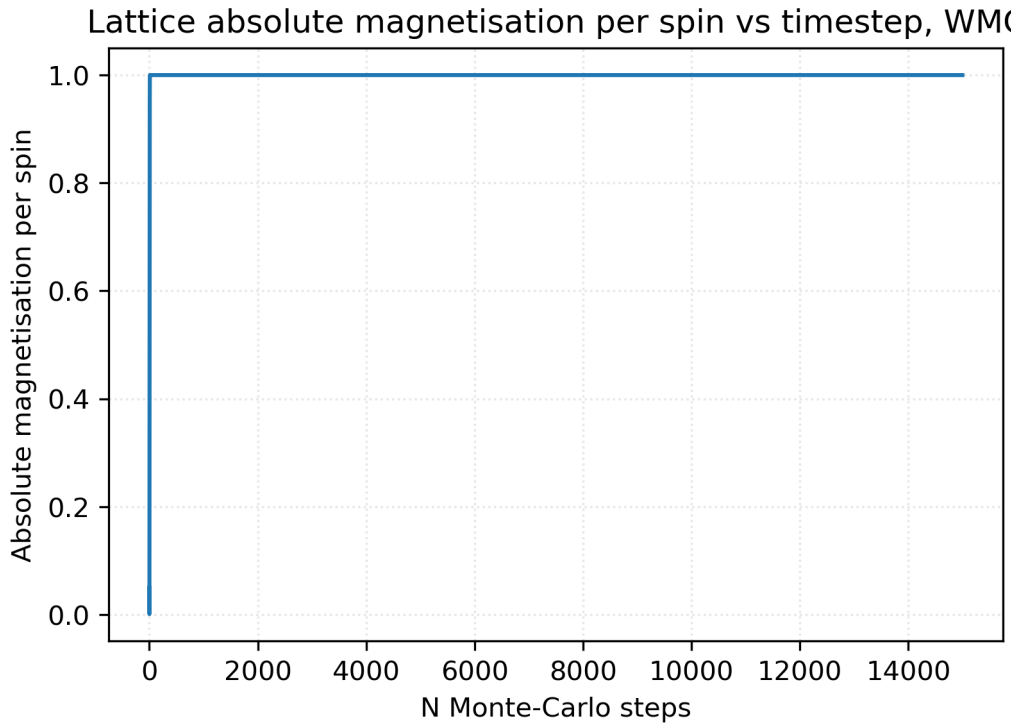
[51]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, WMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, WMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute magnetisation per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute internal magnetic field per spin vs
timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute internal
magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, WMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

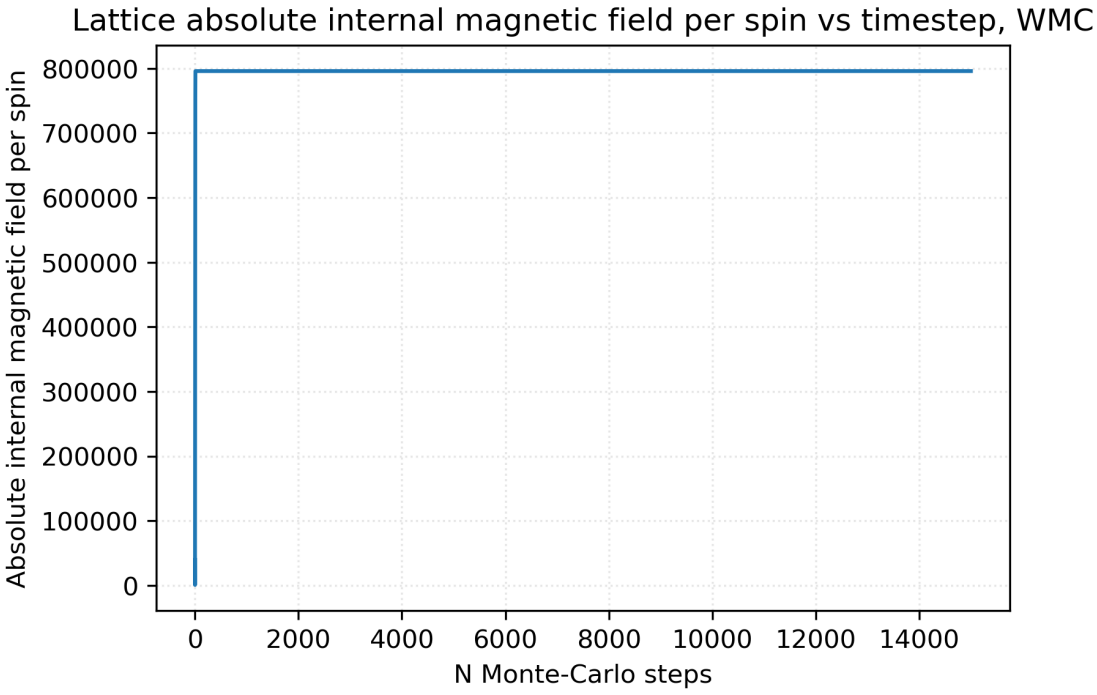
[52]:
```
# J = 10
parameter_file = 'ising_single_sim_J_10_WMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_WMC(parameter_file)
```

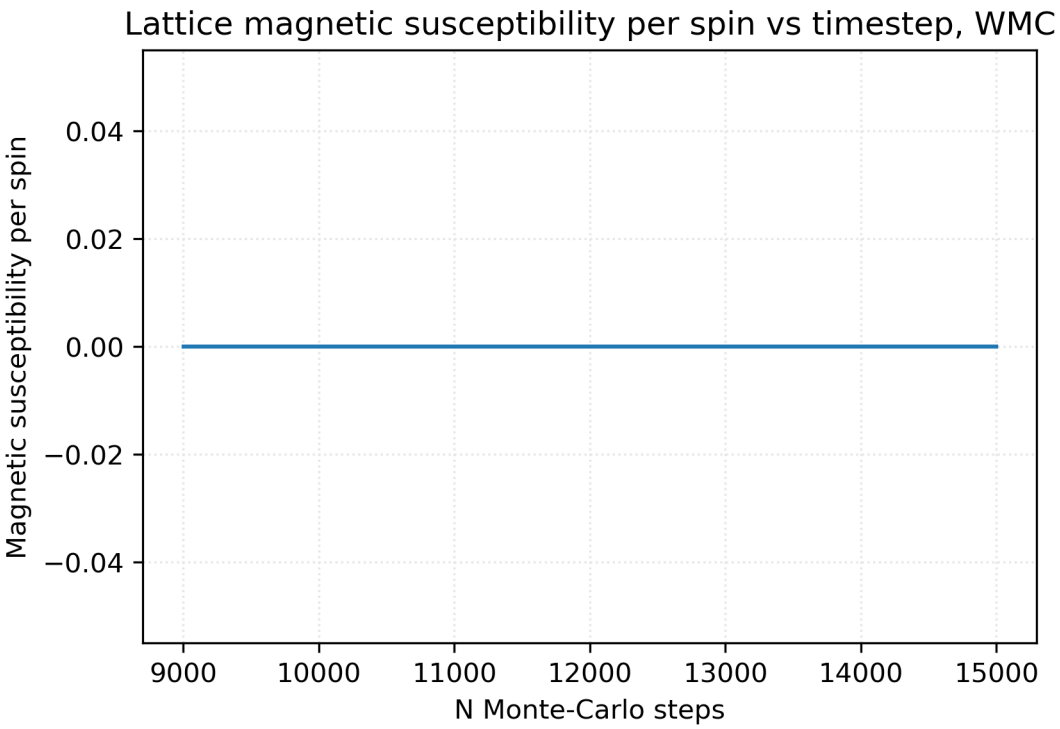
```
Running single Wolff Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=10.0, k_b=1.0, u_b=0.0, nsteps=15000


Running WMC Ising Model::   0%|          | 0/15000 [00:00<?, ?it/s]
```

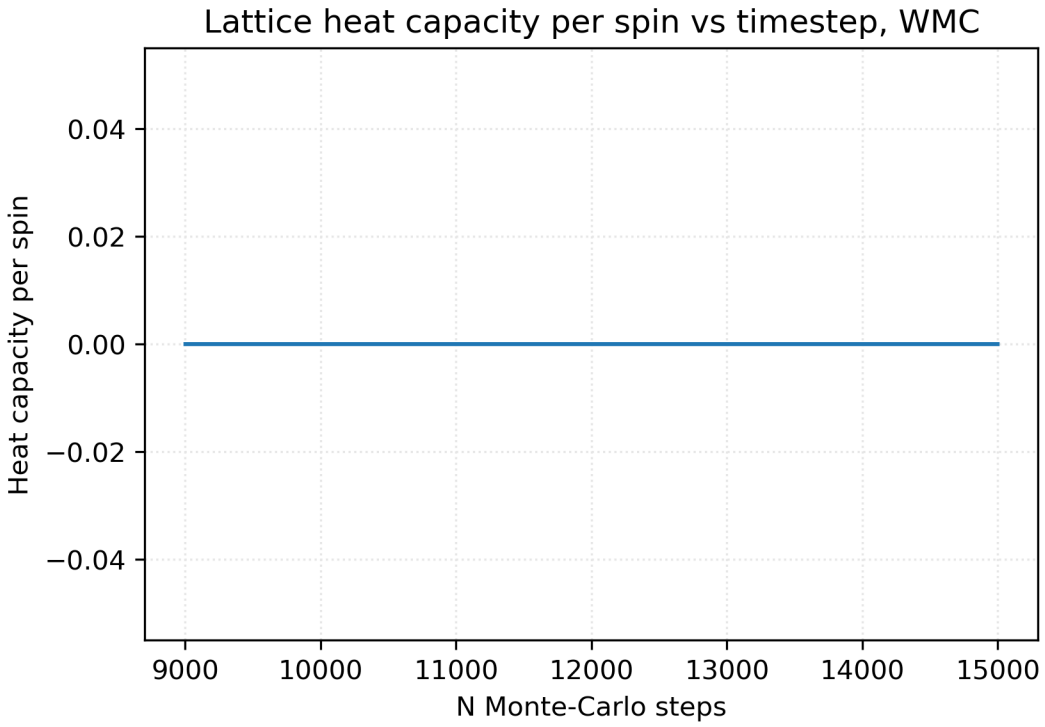

2D Ising Model Lattice Plot, WMC
0
10
20
30
40
50
60
y
0
20
40
60
x
Spin = 1


Lattice energy per spin vs timestep, WMC
−2.5
−5.0
−7.5
−10.0
−12.5
−15.0
−17.5
−20.0
Energy per spin / J
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute magnetisation per spin vs timestep, WMC
Absolute magnetisation per spin
1.0
0.8
0.6
0.4
0.2
0.0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute internal magnetic field per spin vs timestep, WMC
Absolute internal magnetic field per spin
800000
700000
600000
500000
400000
300000
200000
100000
0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps

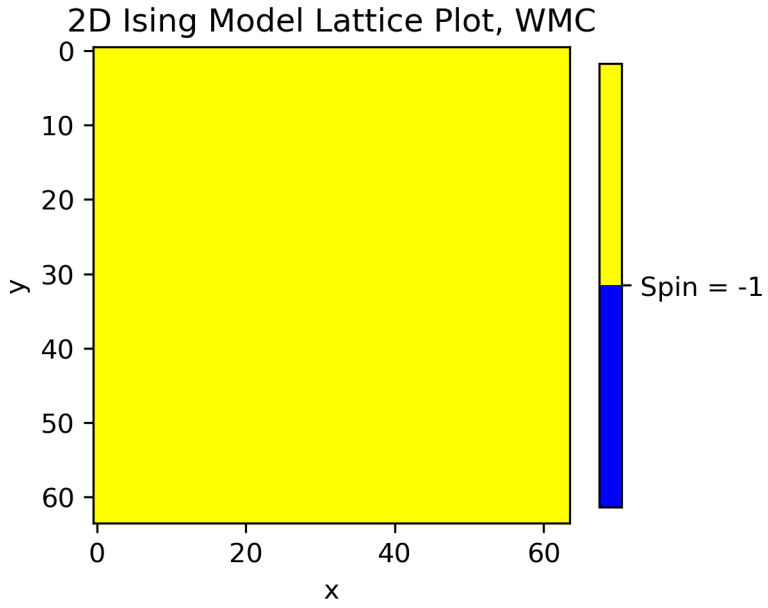

Lattice magnetic susceptibility per spin vs timestep, WMC
Magnetic susceptibility per spin
0.04
0.02
0.00
−0.02
−0.04
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

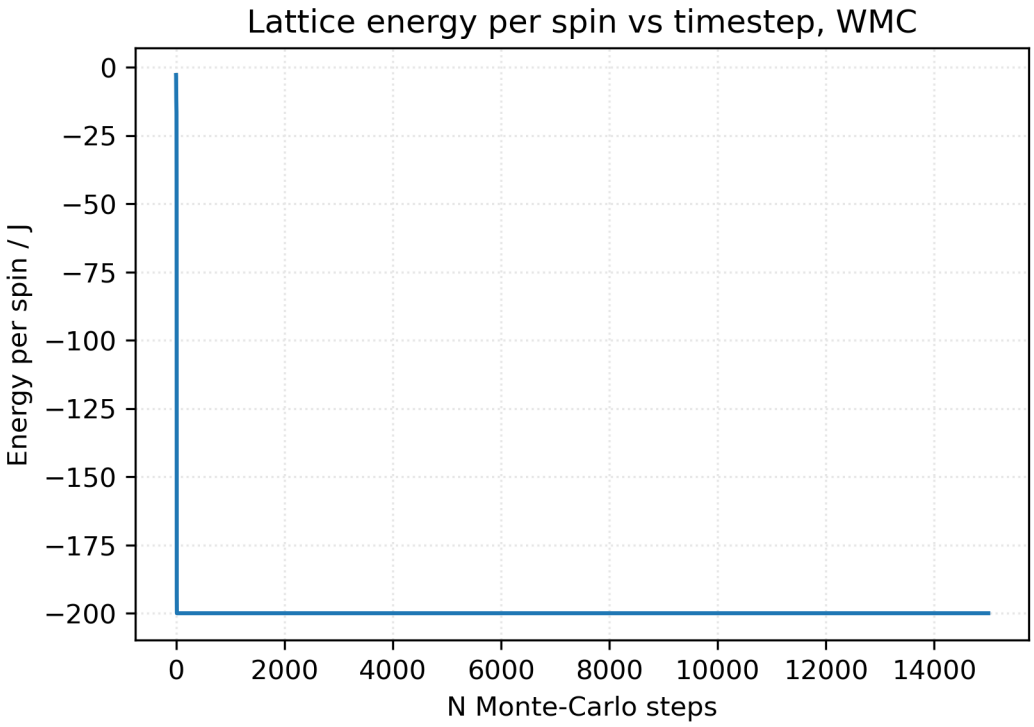

Lattice heat capacity per spin vs timestep, WMC
Heat capacity per spin
0.04
0.02
0.00
−0.02
−0.04
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps

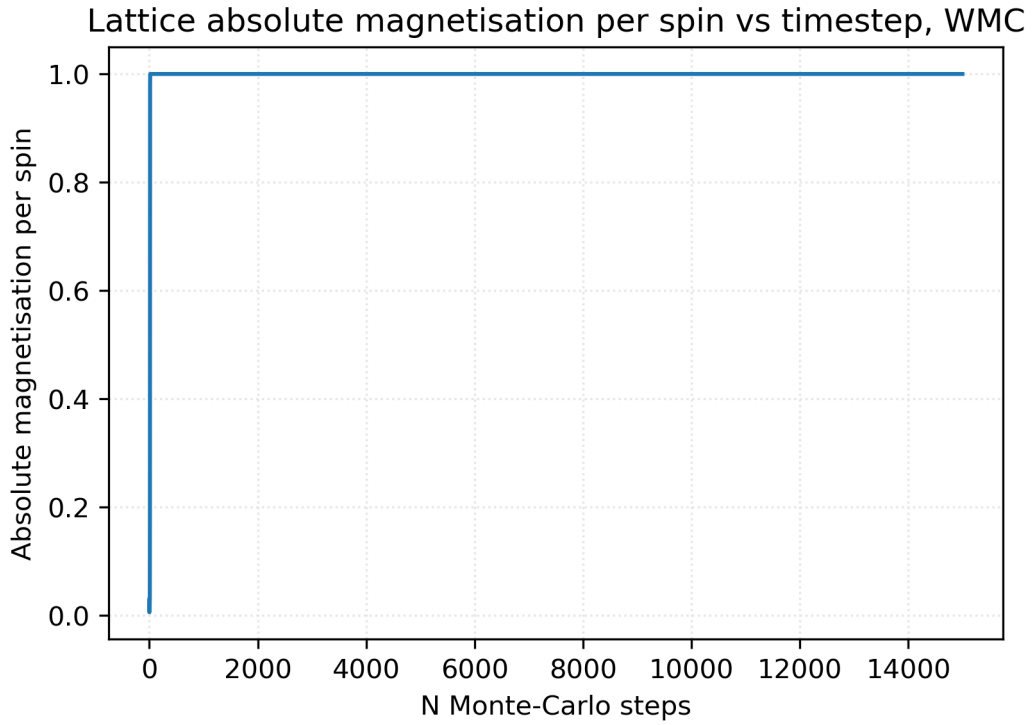
[52]:
```
(<Figure size 1200x1200 with 2 Axes>,
 <Axes: title={'center': '2D Ising Model Lattice Plot, WMC'}, xlabel='x',
ylabel='y'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice energy per spin vs timestep, WMC'}, xlabel='N
Monte-Carlo steps', ylabel='Energy per spin / J'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute magnetisation per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute magnetisation per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice absolute internal magnetic field per spin vs
timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute internal
magnetic field per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep,
WMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
 <Figure size 1800x1200 with 1 Axes>,
 <Axes: title={'center': 'Lattice heat capacity per spin vs timestep, WMC'},
xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)
```

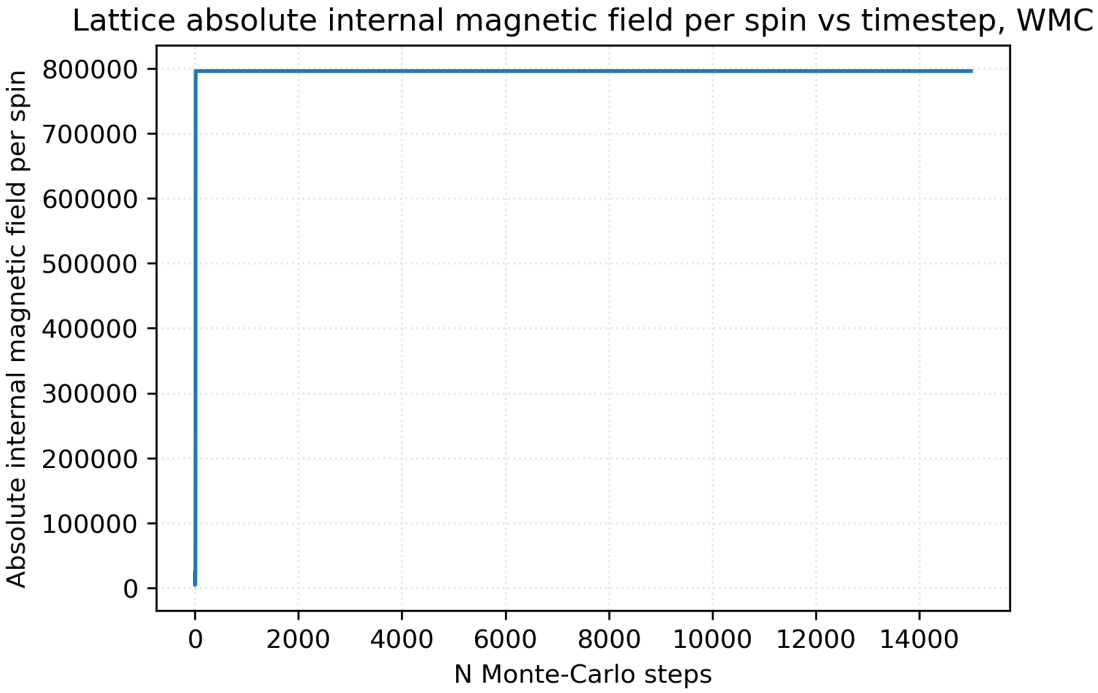
[53]:
```
# J = 100
parameter_file = 'ising_single_sim_J_100_WMC_ICS.txt'
run_single_ising_model_simulation_with_lattice_visualisation_WMC(parameter_file)
```

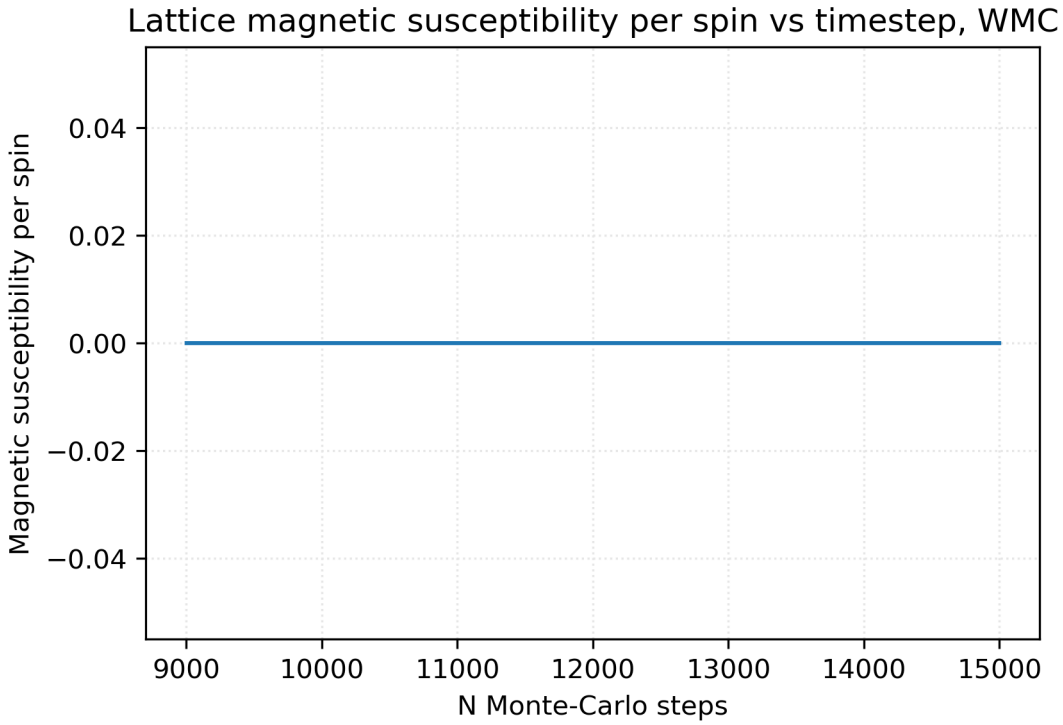
```
Running single Wolff Algorithm simulation with parameters:
temperature=2.269185K, lattice_side_length=64, magnetic_field=0.0,
exchange_constant_J=100.0, k_b=1.0, u_b=0.0, nsteps=15000


Running WMC Ising Model::   0%|          | 0/15000 [00:00<?, ?it/s]
```

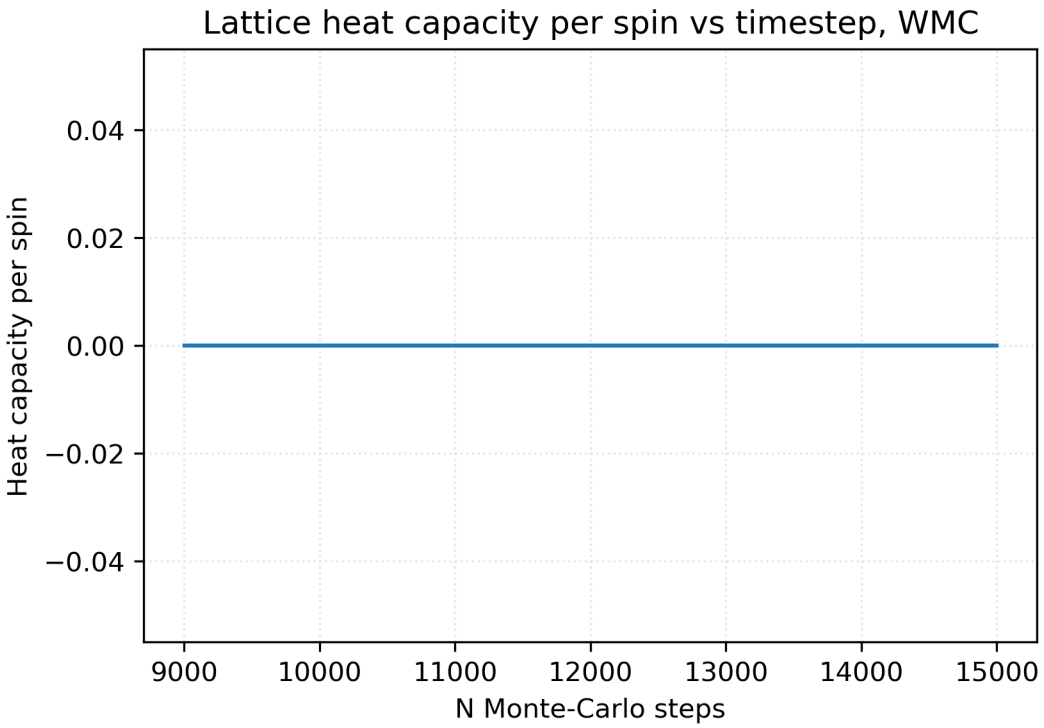



Lattice energy per spin vs timestep, WMC
Energy per spin / J
0
−25
−50
−75
−100
−125
−150
−175
−200
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute magnetisation per spin vs timestep, WMC
Absolute magnetisation per spin
1.0
0.8
0.6
0.4
0.2
0.0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice absolute internal magnetic field per spin vs timestep, WMC
Absolute internal magnetic field per spin
800000
700000
600000
500000
400000
300000
200000
100000
0
0
2000
4000
6000
8000
10000
12000
14000
N Monte-Carlo steps


Lattice magnetic susceptibility per spin vs timestep, WMC
Magnetic susceptibility per spin
0.04
0.02
0.00
−0.02
−0.04
9000
10000
11000
12000
13000
14000
15000
N Monte-Carlo steps



[53]: (<Figure size 1200x1200 with 2 Axes>,
<Axes: title={'center': '2D Ising Model Lattice Plot, WMC'}, xlabel='x', ylabel='y'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Lattice energy per spin vs timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Energy per spin / J'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Lattice absolute magnetisation per spin vs timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute magnetisation per spin'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Lattice absolute internal magnetic field per spin vs timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Absolute internal magnetic field per spin'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Lattice magnetic susceptibility per spin vs timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Magnetic susceptibility per spin'>,
<Figure size 1800x1200 with 1 Axes>,
<Axes: title={'center': 'Lattice heat capacity per spin vs timestep, WMC'}, xlabel='N Monte-Carlo steps', ylabel='Heat capacity per spin'>)

34 The Wolff Algorithm is not designed to account for magnetic fields, it is only designed to mitigate critical slowing and escape from local optima. Therefore the Metropolis Algorithm only is used to investigate the effects of changing the external B-fields.